PawMatch: Animal Adoption and Maintenance Tool

OCR A Level Computer Science (H446) Programming Project

Sienna Sisodia

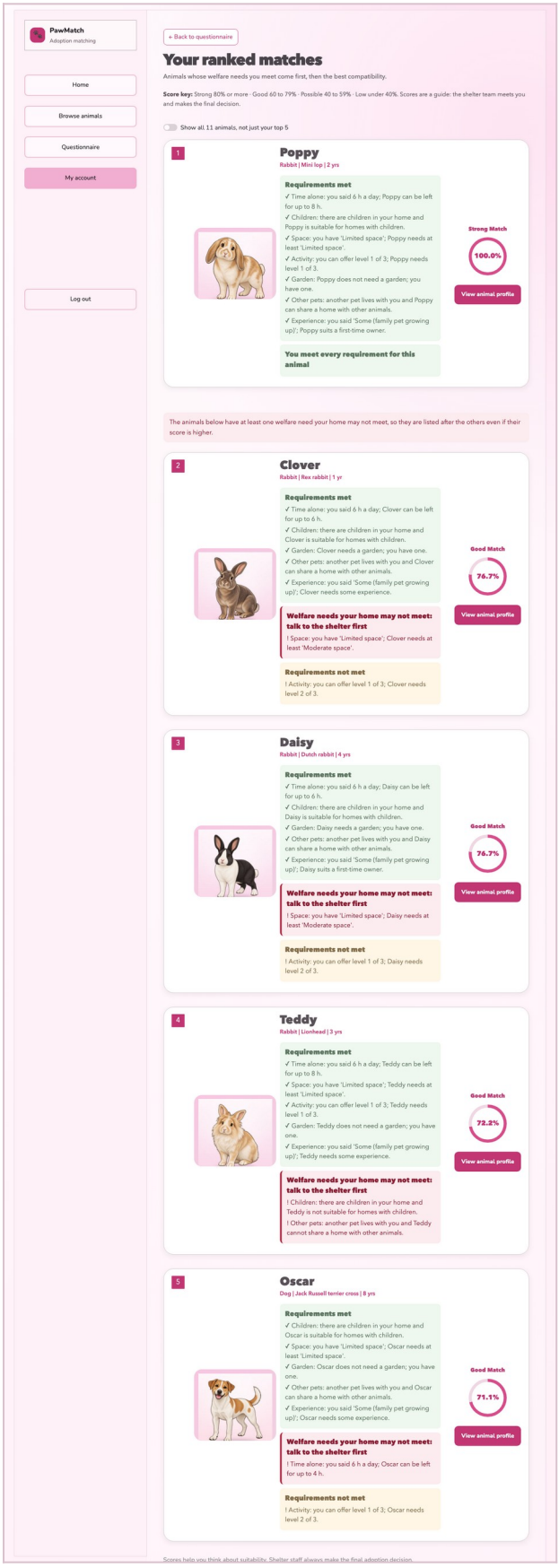


Figure I1: PawMatch: the ranked matches page from the finished system

Item	Detail
Qualification	OCR A Level Computer Science (H446), Component 03/04 Programming Project

Item	Detail
Project	PawMatch: an adoption matching and animal-records tool for one small rescue
Language and tools	Python 3, Streamlit, SQLite (sqlite3), pytest
Source code	Branch `opus-rebuild` of the project repository, folder `rebuild/`
Tests at hand-in	208 automated tests, 0 failing (pytest, saved in `docs/evidence/`)

> **About this version**
>
> This is a rebuilt model write-up of the PawMatch project, made after the original was handed in, to show how the same project can be planned, built, tested and evaluated against the OCR marking criteria. It keeps the original research, interview, survey and design work, marked where it is reused, and redesigns the weaker parts. Where AI tools were used (to role-play the client in reviews, to simulate usability testers and to help with checking), this is stated in the text and logged in Appendix A.

Contents

1 Analysis A1

1.1 Problem definition and stakeholders A1

The client, made smaller A2

Stakeholders and how each will use the system A2

Why this problem is amenable to a computational approach A3

1.2 Research A7

Existing solution 1: Dogs Trust A7

Existing solution 2: RSPCA Find a Pet A11

Existing solution 3: Petfinder A14

Existing solutions 4 to 8: wider research A18

Features selected from all eight solutions A19

1.3 Client interview A19

1.4 Stakeholder survey A22

1.5 Proposed solution A28

The solution in specific terms A30

1.6 Hardware and software requirements A31

Minimum requirements for running PawMatch A33

1.7 Limitations A33

1.8 Requirements and success criteria A33

Success criteria A36

1.9 References A36

Glossary A37

2 Design D1

2.1 Decomposition of the problem D1

Structure of the solution D4

2.2 Database design D5

Changes from the first data dictionary D7

Example records D7

2.3 Component 1: accounts and login D8

Pseudocode: registration and login D13

2.4 Component 2: staff animal management D18

Pseudocode: add, edit and delete an animal D21

2.5 Component 3: home page and browsing D29

2.6 Component 4 and 5: questionnaire and compatibility algorithm D42

Pseudocode: scoring and ranking D47

Weights D54

Final question list D55

2.7 Validation rules D57

2.8 Usability features D58

2.9 Development test plan D59

2.10 Post-development test plan D60

Usability tasks D61

Robustness tests D61

3 Development and testing to inform development S1

3.1 How development was organised S1

Program files S2

3.2 Prototype 1: the engine with a plain interface S2

3.3 Stage 1: accounts and login S4

Tests for stage 1 ..... S6

Client feedback 1 ..... S6

Changes made after feedback 1 ..... S7

3.4 Stage 2: staff animal management S8

Tests for stage 2 ..... S10

Client feedback 2 ..... S11

Changes made after feedback 2 ..... S11

3.5 Stage 3: home page, browse and profile S12

Tests for stage 3 ..... S15

Client feedback 3 ..... S15

Changes made after feedback 3 ..... S16

3.6 Stage 4: questionnaire S16

Tests for stage 4 ..... S17

Client feedback 4 ..... S17

Changes made after feedback 4 ..... S18

3.7 Stage 5: matching and results S19

Design re-check before building S19

Final weights S20

Worked example: checking the program by hand S20

Tests for stage 5 ..... S22

Client feedback 5 ..... S23

Changes made after feedback 5 ..... S24

3.8 Stage 6: accessibility and final checks S24

Tests for stage 6 ..... S26

Client feedback 6: final review S26

Changes made after feedback 6 ..... S27

3.9 Stage 7: changes from the reviews, and staying signed in S27

Tests for stage 7 ..... S28

4 Testing to inform evaluation E1

4.1 Post-development tests E1

Capacity and speed (SC10, SC14) E1

Results against each success criterion E1

Agreement with hand-written expectations and weight sensitivity E3

Phone and tablet render check E4

4.2 Robustness E4

4.3 Usability testing E4

What the usability testing showed E5

Stakeholder survey after the build (planned, not yet run) E6

5 Evaluation E6

5.1 Success criteria E6

5.2 How the partly met criteria would be fully met E7

5.3 Usability features evaluated E8
5.4 Maintenance E8
Privacy and personal data E9
5.5 Limitations E9
5.6 Future development E10

Appendices X1

Appendix A: Use of AI tools X1
Appendix B: References X1
Appendix C: Evidence files X2
Appendix D: How to install and run X2
Appendix E: Program listing X2

How to read this document

Pages are numbered by section: A (Analysis), D (Design), S (Development and testing to inform development), E (Testing to inform evaluation, and Evaluation), X (Appendices). A reference such as "see D14" means page 14 of the Design section. Source codes in tables: **Q** = client interview question, **S** = stakeholder survey question, **E** = existing solution, **SC** = success criterion, **T** = development test, **P** = post-development test, **U** = usability task, **R** = robustness test.

1 Analysis

1.1 Problem definition and stakeholders

Problem definition + Stakeholders

According to BBC On average, the RSPCA received 374 reports of animal cruelty every day in June to August 2024. The Royal Society for the Prevention of Cruelty to Animals (RSPCA) received 34,401 cruelty reports between June and August 2024, up from 25,887 in the same period the year before (BBC, 2024). These statistics show us how the number of mistreated animals is increasing each year. I want to help reduce the likelihood of an animal receiving mistreatment from its owner.

My client is a friend at St Dominics called Matilde, who has a part time job looking after animals when their owners are away. She looks after a range of different animals from dogs and cats to chickens. We have various conversations about her work where she has made me aware how common it is for animals to be in unsafe environments, where signs of neglect are seen from the owners.

I aim to make an adoption management system which is for one adoption agency, which will allow the staff to manage the animals up for adoption and their personal details and will allow any potential rescuers to see if they are suitable to adopt an animal.

Adoption agency staff are a key stakeholder for this project. This application will have a private part using an admin login for staff and requires an interface to allow them to interact with this software. This will contain details of all the animals, and it will allow the staff to delete, modify, edit and add different animals. This system should reduce the amount of manual administration required and allows for a consistent form of assessing potential applicants.

Another key stakeholder is potential adopters who are the primary users of this system. Their experience with animals may vary significantly, ranging from first time pet owners to experienced animal carers. They will be using the public side accessed by a user's own login details stored within a large database, with hashing in order to keep the passwords safe if the database is breached. My system will also require a simple and easy to use interface allowing them to browse available animals, view information about each animal and complete an adoption questionnaire. This system will gather user information which will be processed by a compatibility scoring algorithm which will rank the right animals according to their suitability.

Another possible stakeholder is the animals. Although animals do not directly interact with the system, they are an important stakeholder as they are affected by the outcome of adoption decisions. This system's key aim is to improve animal welfare by encouraging more informed adoption choices and reducing likelihood of animals being placed in unsuitable homes.

Due to my client's experience caring for a variety of animals, she is skilled and knowledgeable about the environments required for an animal to live happily and safely and has asked me to incorporate the compatibility matcher with questions and a set criteria to examine whether they are suitable. Examples of the set criteria may include: the amount of available living space, access to a garden, experience with animals, income and more. Her feedback will be used throughout development to ensure the system meets the needs of animal welfare professionals.

Although many adoption websites allow users to browse animals, they often rely on their own judgement whether or not they are fit to adopt. This may lead to adoptions where the owner does not fully understand the commitment required which may lead to neglect. In some cases, unsuitable adoptions may result in animals being returned to shelters, creating additional pressure on adoption agencies and causing unnecessary stress for animals involved. This application should provide guidance whether a particular animal is compatible with their lifestyle, living arrangements, financial situation or previous experience. Additionally, current adoption agencies may use manual assessments and application forms by staff to which this may be time consuming and influenced by subjective judgement. My system will differ as it provides a consistent method of evaluating applicants through a scoring algorithm.

The client, made smaller

To keep the project to a size that one person can build and test properly, the scope is fixed as follows.

Decision	What it means for PawMatch
One client organisation	PawMatch is built for **one small, single-site rescue**: about 10 to 20 animals at any time (dogs, cats and rabbits), run by two or three staff or volunteers. My client does not work for a rescue, but her pet-care job means she has looked after a wide range of animals and knows what each needs. She therefore acts as the rescue's welfare adviser: she defines the welfare rules and approves each stage.
Adopters are users, not the client	Members of the public use PawMatch to browse and to see how well they suit each animal. Their needs come from the survey (27 responses) and from usability testing, but the client decides the rules.
Not in scope	Several branches, online applications or payments, fostering, vet records, or any species other than dogs, cats and rabbits. These are explained under Limitations (A33).

Stakeholders and how each will use the system

Stakeholder	How they will use PawMatch	Why PawMatch is appropriate for them	Evidence
Rescue staff and volunteers (the client's side)	Sign in with a staff account; add, edit and delete animal records with photos; change an animal's status to Reserved or Adopted; see an overview of records that need attention.	Without a shared system, assessments are manual and records easily go out of date (Q5, Q15). One validated form per animal keeps records consistent, and the same rules are applied to every adopter, which removes the subjective judgement the client described.	Q5, Q6, Q12, Q15

Stakeholder	How they will use PawMatch	Why PawMatch is appropriate for them	Evidence
Potential adopters (primary users)	Create an account, browse available animals with filters, open profiles, answer a 10-question lifestyle questionnaire once, then see every animal ranked by suitability with reasons.	Many choose on looks (S3: only 48.1% choose on lifestyle). They need a quick, friendly way to see which animals fit their home before they contact the rescue, and an explanation they can act on. Experience ranges from first-time owners to experienced carers, so the language must be plain.	Q1, Q3, Q7, S3, S5, S6, S7
The animals (indirect)	Never use the system, but are affected by its output.	Every scoring rule protects an animal's welfare: the client named space, allergies and young children as the most important things to check (Q10).	Q1, Q10, S9
The client	Reviews each development stage and signs off the final system.	She has the animal-care knowledge to judge whether a match is sensible.	Q1 to Q15

Why this problem is amenable to a computational approach

Use of Computational Methods

The most suitable solution for a problem like this is to create a computer based adoption matching system. This is because the problem includes storing animal data, adopter information, comparing different factors and producing recommendations. A manual system may require staff to compare each adopter with each animal individually, which could take significantly longer and will require more effort. Using a manual way of storing data may also lead to inconsistencies, missing information and errors, whereas using a database allows data to be searched efficiently and stored consistently. A computational solution allows this problem to be solved more efficiently by using algorithms, data validation, automated scoring and ensuring consistent data handling

Thinking Abstractly

The process of abstraction is removing unnecessary details in order to simplify the approach of a problem, making it easier to understand and solve. Ignoring insignificant details will reduce the amount of unnecessary coding and make the system easier to maintain. Within my system, it will be used in order to focus specifically on matching an adopter with an animal.

For example, the system does not need to store every detail about an adopter's personal life. It will only need to collect information that affects animal suitability such as the living space, garden access, experience with animals. Similarly, with the animal profiles it will only store useful information. It won't include its nicknames, or minor habits or favourite toy as this is not as important for the matching system.

This will make the system easier to design and implement as irrelevant details are ignored. This will also make the system more efficient by focusing on the most important factors that affect suitability.

Thinking Ahead

During the development of this system, I will need to think ahead in order to identify the requirements of my user and relate to the issues they might face or be facing.

I have to anticipate how users would like their information to be presented in order to attract them to adopting an animal, it must also be fully accessible and easy to use.

I will also need to plan how the data will be stored before implementation. For example, animal records and adopter records must be stored persistently so information is not lost when the program closes. I will also need to consider validation as users should not be able to leave empty fields or enter invalid data. This will help ensure that the matching algorithm is using accurate and correct information.

I will also need to think about the structures and designs that can be reused on different pages before I start developing my project. I also have to consider the technologies I will use to develop my system, as there are many different technologies I can use with each their own pros and cons.

Another key part of my system to which I will need to think ahead is the compatibility scoring system. This system should not simply recommend animals based on experience or breed. Instead, it should compare multiple factors such as living space, experience, time available, children and animal requirements.

Thinking Concurrently

Processing multiple tasks at once will reduce the processing time of my app, reduce waiting time for my user and increase the overall efficiency of my system. When the user completes the adoption questionnaire, the system may need to retrieve animal records, compare the adopter answers with requirements, calculate compatibility scores and display results.

It is also important within the login page, where the credentials that are entered need to be validated and sanitised to prevent any injection attacks, whilst finding user profile. After this stage is cleared, multiple elements will need to be loaded at the same time.

The system also needs to be able to handle different users carrying out tasks at the same time. For example, an adopter could be viewing suitable animal matches while an admin user updates or adds an animal profile. I will need to design the system, so these processes are independent, and the data is able to remain consistent.

Concurrency may not be a main focus of my project, but it will allow me to design a system which is responsive and efficient as more animal records and adopter applications are added.

Thinking Procedurally with Decomposition

Decomposition is the process of splitting a large process into small more manageable sections. This will make the system easier to design, implement, test and improve. Instead of trying to build the whole system at once, I can separate it into different components.

The main components which build up my system will include animal records with their different cards, the adopter questionnaire with the matching algorithm, compatibility score calculation, results page, validation and the admin control page. The animal records section will store details about each animal and the adopter's questionnaire collects information about users home and lifestyle. The matching algorithm will compare adopter answers with each animal's requirements and calculate compatibility score.

I will also split the application into the front end and back end sections. The front end is what the user or the admin user will experience, which will have the login forms and present the different features of the application. The backend is where all the logic will be processed and will include a relational database for secure storage of user data.

By using decomposition on my system, it will allow each section to be developed and tested separately. For example, I can first test whether the animal records are being stored and added correctly, and if all the information is shown in the correct format. I can then test whether the adopter questionnaire collects all the information required and validates the inputs. Then I can test whether the matching algorithm calculates scores correctly. This way of testing will make errors easier to identify and fix.

Procedural thinking will allow me to further separate each function of front and backend and allow me to design my application using modules of reusable and readable code. The different subprograms will be kept in their own areas so I can work on them without affecting other areas. I will use this method in both the front and back ends of this project.

Thinking Logically

My program needs to be designed to make it simple to use but also logical to follow. This is important as users are guided through the adoption process clearly, from logging in to receiving suitable animal recommendations.

My system should hopefully follow a sequential process, for the user to view:

1. User creates an account or logs in using username and password
2. Once logins, users directed to main page, with the animals
3. The adopter will then complete a questionnaire
4. A compatibility score is shown for animals and ranked from suitable to least suitable and the user can browse.

For the admin to view:

1. Admin user will login in using their username and passwords
2. The user will be directed to the admin control page
3. The animal is able to add, edit or delete records similar to a CRUD interface.
4. The system will validate the animal record to make sure required fields are completed and data in correct format and if it is valid will be saved to the database.
5. The updated animal information is then available for adopters to view and matching algorithms to use.

The system will also use logical decision making. For example, if the username or password is incorrect, the user cannot access their account. If required questionnaires fields are left blank, the system should display an error message. In the compatibility scoring algorithm, if an animal cannot live with children and the adopter has children then the compatibility score should decrease. If an animal requires a garden and the adopter has one, then the score should increase.

This also shows that my system will use sequencing, selection, validation, iteration and sorting. It is not simply just displaying the animal profiles but processing data logically to produce suitable recommendations.

The table below ties each computational method to a specific part of PawMatch, so it is clear why a program solves this problem better than the current paper process.

Feature of the problem	Why a computer handles it well	Where it appears in PawMatch
The suitability of every adopter for every animal must be judged the same way each time	A fixed algorithm applies identical weighted rules to every pair, which a busy volunteer cannot do consistently by hand	`score_animal` and `rank_animals` (pseudocode on D42)
20 animals × 7 factors = 140 comparisons for every adopter, repeated whenever an animal is added	Repetitive arithmetic and sorting are instant for a computer (500 animals scored in 0.098 s at worst, measured in section 4.1, page E1)	Questionnaire submit: every Available animal is scored and stored
The information is structured (yes/no, numbered levels, ranges)	It can be stored in typed database fields and validated on entry, so the algorithm never receives impossible values	Four SQLite tables with CHECK constraints (D5)
Records change often (new arrivals, adoptions)	A database with an admin form updates everything at once; old match results are recalculated on the next submission	Staff dashboard and status field
The output must explain itself	The program can list exactly which rules passed and failed for each animal	Met / not met / warning lists on each match
Passwords and staff powers must be protected	Hashing and role checks are things software does reliably	scrypt hashing, role checked on every staff action

What the computer should **not** decide is also clear from the interview: the final adoption decision stays with people (Q4), so PawMatch ranks and explains but never approves or refuses anyone.

1.2 Research

Research - Approach and Objectives

I will take a structured approach to my research by using both primary and secondary research in order to understand the problem from different perspectives and ensure my final system meets user needs and expectations:

1. I will inspect existing adoption websites, analyse their design, structure, features and adoption process, while looking at features I may want to include or avoid.
2. I will conduct an interview with my client (Matilde) to get a better understanding of what she wants within this project and reflect this in the design of my system.
3. I will carry out a stakeholder survey to find out more about their preferences. I will use google forms and this will help me see if my problem is important and worth solving and will help me in developing parts of the system such as the questionnaire.
4. I will assess the different technologies that can be used to produce my application.
5. I will identify software and hardware requirements for my project to run effectively.

My research objectives are:

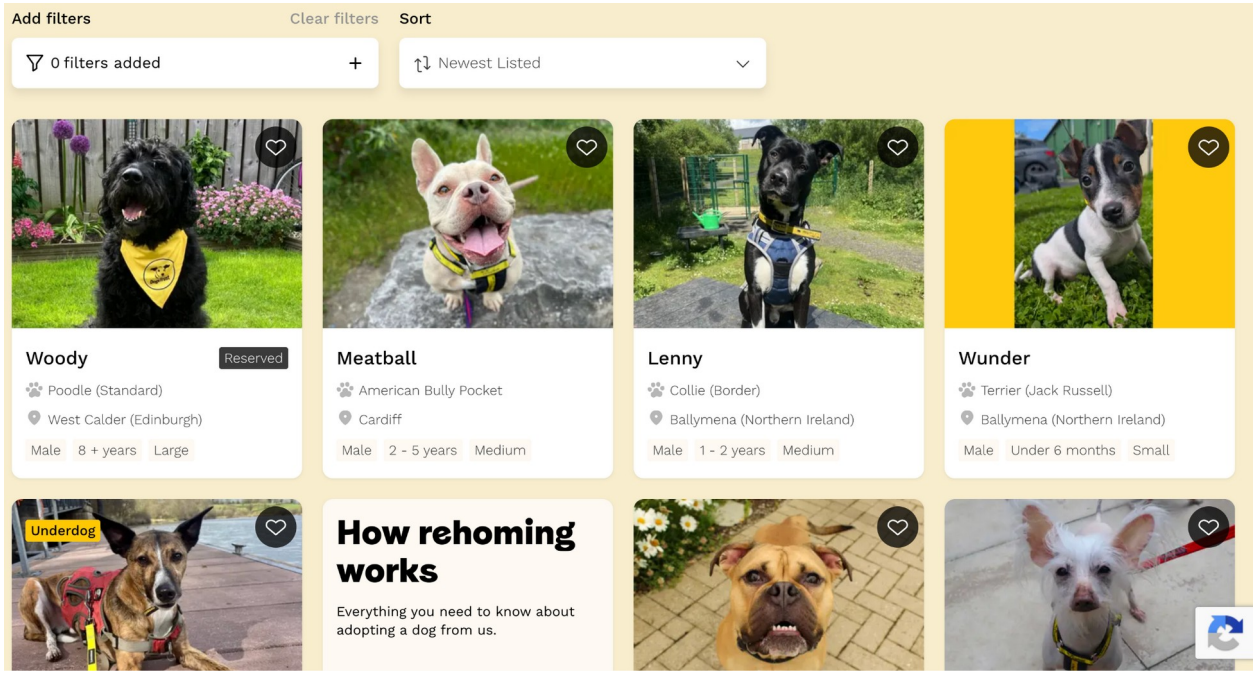
1. Understand current problems with adoption systems.
2. Identify which information needs to be stored about each animal
3. Identify which information needs to be collected from adopters
4. Find out what factors are most important when matching an adopter to an animal
5. Understand and take inspiration from existing adoption websites
6. Understand user expectations for the interface
7. Use the research findings to help decide the final features and requirements of my system

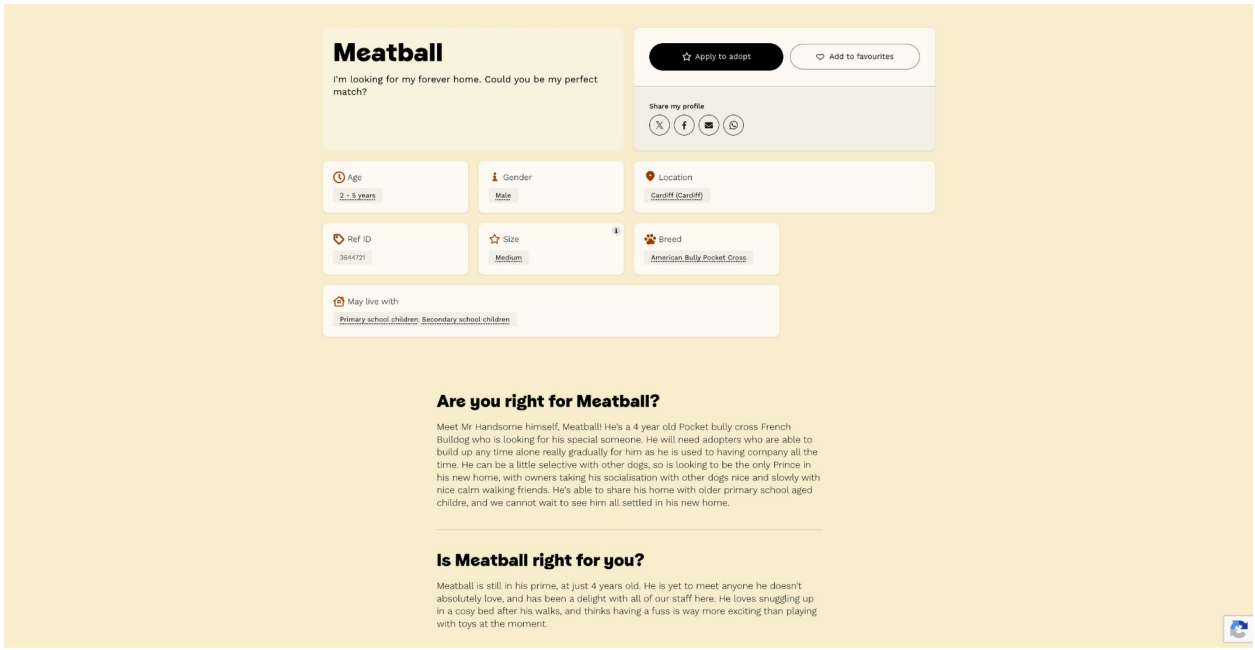
Existing solution 1: Dogs Trust

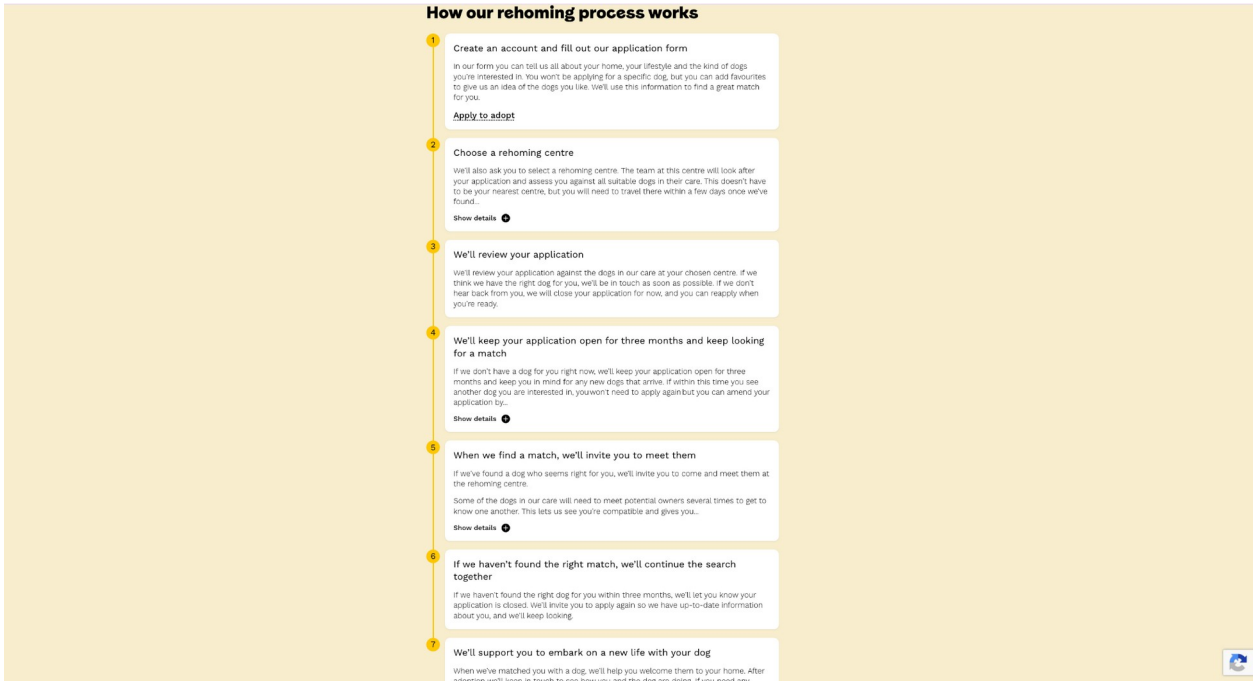
Research of Competitors

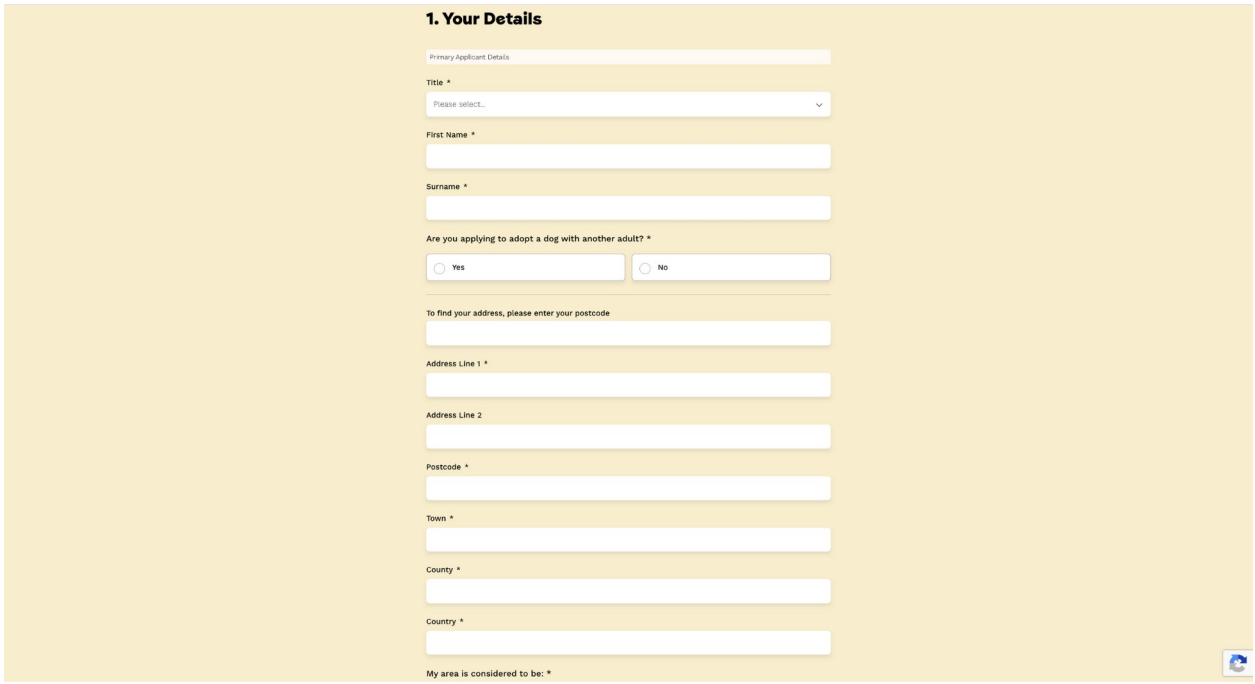
Example 1 - Dogs Trust

Dogs trust ‘offer practical solutions and support to help owners and dogs stay together wherever possible’ (Dogs Trust, 2026). They allow users to complete an application form which describes their home, lifestyle and preferences. Staff at the adoption home then compare applications with suitable dogs.


Add filters
Clear filters
Sort
0 filters added
Newest Listed
Woody
Reserved
Poodle (Standard)
West Calder (Edinburgh)
Male
8 + years
Large
Meatball
American Bully Pocket
Cardiff
Male
2 - 5 years
Medium
Lenny
Collie (Border)
Ballymena (Northern Ireland)
Male
1 - 2 years
Medium
Wunder
Terrier (Jack Russell)
Ballymena (Northern Ireland)
Male
Under 6 months
Small
Underdog
How rehoming works
Everything you need to know about adopting a dog from us.


Meatball
I'm looking for my forever home. Could you be my perfect match?
Apply to adopt
Add to favourites
Share my profile
Age
2 - 5 years
Gender
Male
Location
Cardiff (Cardiff)
Ref ID
3644721
Size
Medium
Breed
American Bully Pocket Cross
May live with
Primary school children, Secondary school children
Are you right for Meatball?
Meet Mr Handsome himself, Meatball! He's a 4 year old Pocket bully cross French Bulldog who is looking for his special someone. He will need adopters who are able to build up any time alone really gradually for him as he is used to having company all the time. He can be a little selective with other dogs, so is looking to be the only Prince in his new home, with owners taking his socialisation with other dogs nice and slowly with nice calm walking friends. He's able to share his home with older primary school aged childre, and we cannot wait to see him all settled in his new home.
Is Meatball right for you?
Meatball is still in his prime, at just 4 years old. He is yet to meet anyone he doesn't absolutely love, and has been a delight with all of our staff here. He loves snuggling up in a cosy bed after his walks, and thinks having a fuss is way more exciting than playing with toys at the moment.





Positives	Negatives
Dog profiles display clear information about the dog such as the age, gender, breed, size, location and whether the dog can live with children.	The system only supports dogs so people interested in different animals must use another service.

Positives	Negatives
The profile also includes the dog's personality, behaviour and required home environment. This helps the user with making a decision on how the dog acts rather than from looks.	Users are not immediately told how well they match. Especially as suitability is assessed manually, so users have to wait.
Users only have to complete the application once rather than for each dog	The system does not explain which individual requirements the adopter meets or fails to meet
Users can add up to four favourite dogs.	The user must select a particular rehoming centre and be able to travel there when a match is found
Applications remain active for three months, so the user can be considered for dogs that arrive later.	Not every available dog is shown on the website meaning users cannot browse the complete set of possible matches themselves
Dogs are assessed by behavioural and veterinary staff before being matched, making the final decision more credible.	The bright yellow colour may be distracting and make it harder to read.
There is a strong yellow, black and white colour scheme making their branding recognisable.	The large photographs and sections mean that only a small amount of information is visible at once, so users must scroll frequently.
Large photographs are used for each dog to make it more appealing to adopt	Some pages are very long and contain several sections such as the 'About us' page, which may feel overwhelming.
There is plenty of white space between sections, preventing pages from appearing cluttered	The similar appearance of every dog card may make results look repetitive.
Important buttons such as apply to adopt and add to favourites stand out.	On smaller screens, the large images and vertically arranged content could require significantly more scrolling.

IMPACT ON MY SOLUTION

Dogs trust has influenced me to collect information about the adopter's home, lifestyle and preferences only once and reuse it when comparing with different animals. I will also create

similar animal cards which have detailed information about each animal. It will also contain another section with personality, behaviour and home requirements.

However, my system will differ as it allows for many different species and will automatically give partial feedback whether they are suitable but ultimately leave the final decision to staff.

They have also influenced me to use a consistent colour scheme, clear headings with organised animal cards. I will make buttons easy to identify.

I will avoid bright colours, long pages and make mine less overwhelming and simple to use.

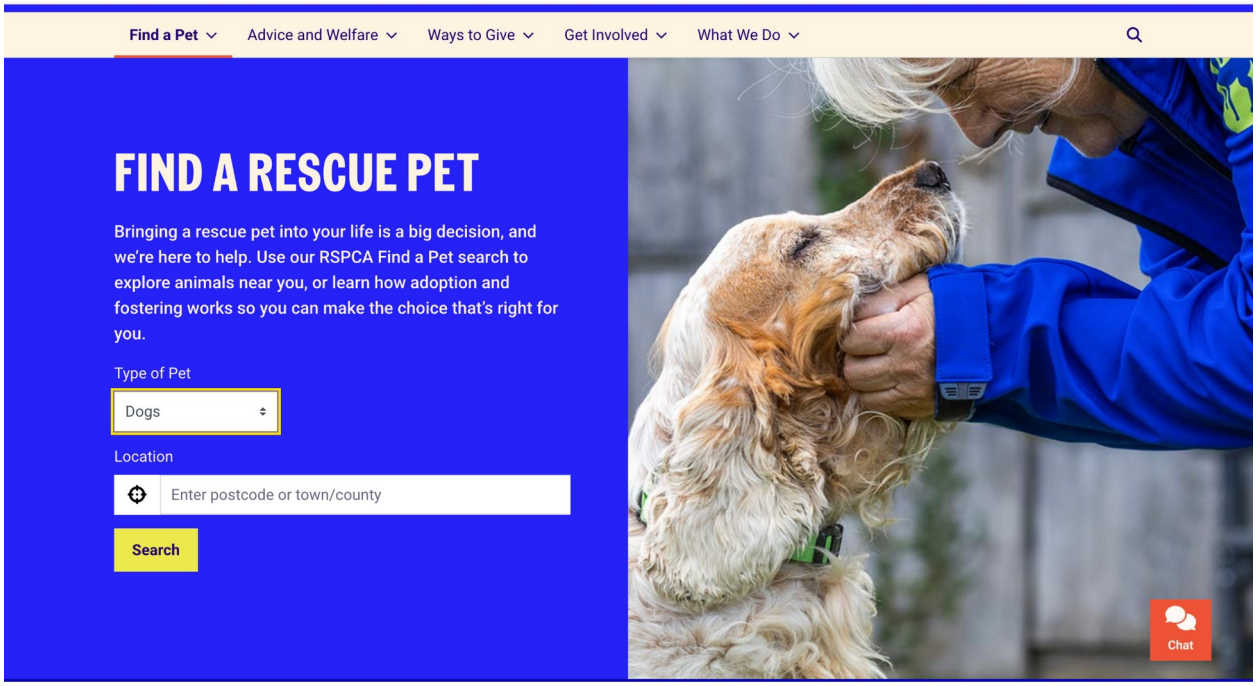
Features I will take from Dogs Trust (E1)

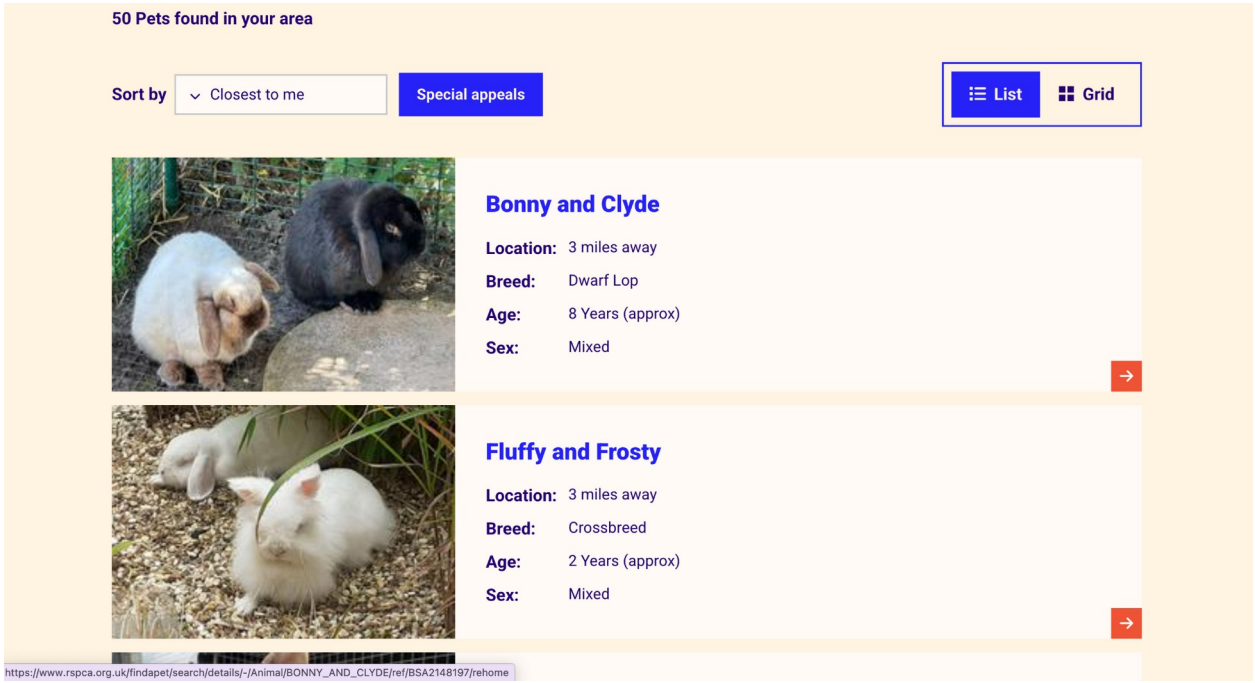
- **Adopt:** collecting information about the adopter's home and lifestyle before matching; a calm, consistent colour scheme with clear headings.
- **Adapt:** its matching is done by phone with staff. PawMatch automates the first sift so staff time goes on the best-matched adopters.
- **Reject:** dog-only scope and long pages.

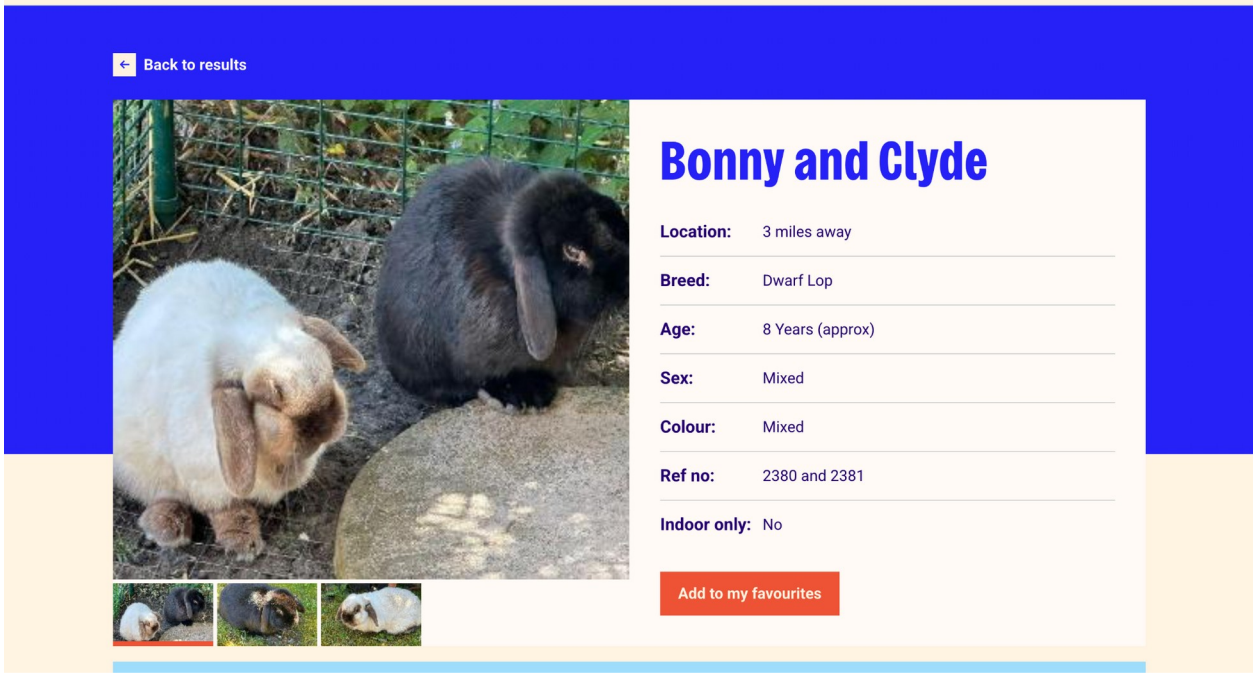
Existing solution 2: RSPCA Find a Pet

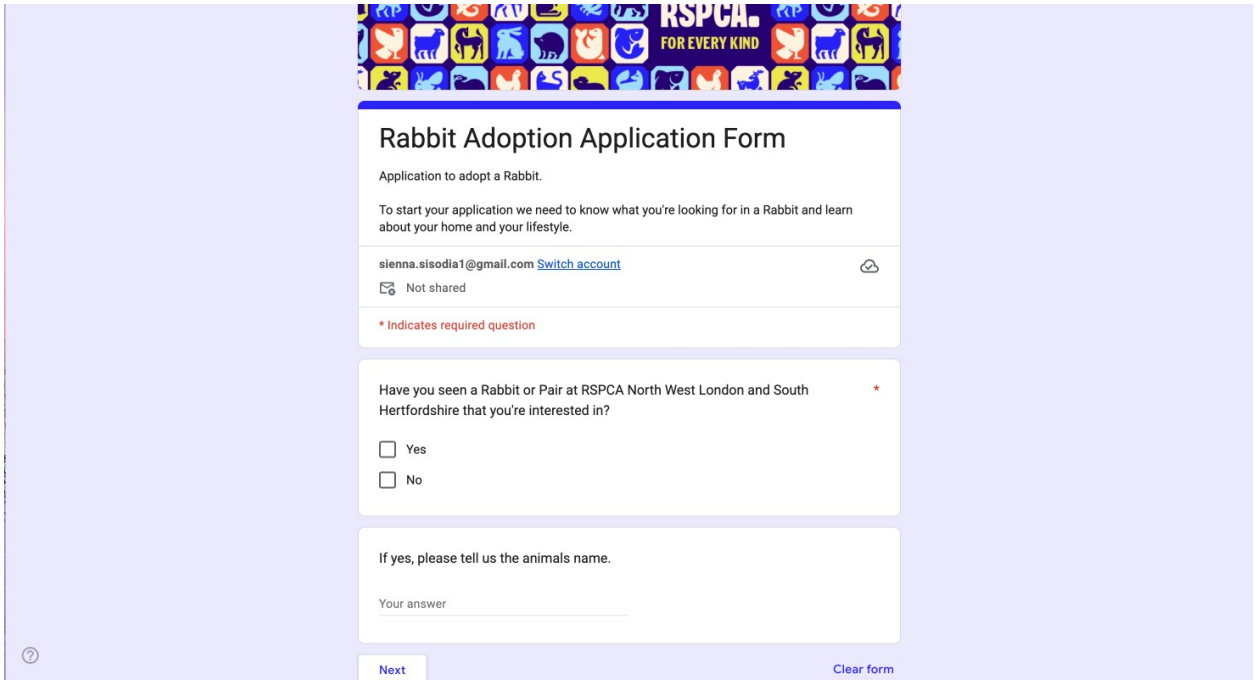
Example 2 - RSPCA

The RPSCA Find a pet website allows a user to search for several different types of animals. Its pages use photographs, clear heading and separate content sections to present animal information.




50 Pets found in your area
Sort by
Closest to me
Special appeals
List
Grid
Bonny and Clyde
Location: 3 miles away
Breed: Dwarf Lop
Age: 8 Years (approx)
Sex: Mixed
Fluffy and Frosty
Location: 3 miles away
Breed: Crossbreed
Age: 2 Years (approx)
Sex: Mixed
https://www.rspca.org.uk/findapet/search/details/~/Animal/BONNY_AND_CLYDE/ref/BSA2148197/rehome


Back to results
Bonny and Clyde
Location: 3 miles away
Breed: Dwarf Lop
Age: 8 Years (approx)
Sex: Mixed
Colour: Mixed
Ref no: 2380 and 2381
Indoor only: No
Add to my favourites



Positives	Negatives
Large animal photographs make the website visually appealing and immediately attract users' attention.	Applications are all reviewed manually by staff. The user does not receive an immediate result and should normally wait up to 48hours for a response. (Although this may vary between branches)
The colour scheme fonts and button styles remain consistent which allows the website to appear professional	The website contains many navigation options as it covers more than adoption, which may overwhelm some users. The number of filters and controls may make the search area appear crowded.
They support a wide range of species, rabbits, hamsters, guinea pigs. This is more similar to what I want within my system as it doesn't just support cats and dogs.	Some sections which are important and not important have a similar appearance which may make it hard to show what is important and allow it to stand out.
Each animal has an individual profile explaining needs, allowing adopters to consider suitability before applying. Applications also are reviewed by trained staff meaning factors for a computer to assess still can be considered.	Some important details are only available after opening the full animal profile.

Positives	Negatives
They also have a feature for support after adoption, including behaviour services and pet care advice. This also could reduce chances of adoption failing after the animal entered a new home.	It may be hard to objectively analyse if they are suitable for the animal due to the lack of numeracy.
RSPCA may suggest alternative animals when the selected animal is unsuitable.	Procedures and costs may differ between RSPCA centres and branches, which does not allow for consistency which may confuse the user.
Search controls and filters are clearly labelled helping users understand what information they need to enter.	The published process requires applications to be started from an animal's profile. This may cause users to repeat parts of the process when interested in another animal.
Information is divided into sections with headings, helping users distinguish between different types of contexts.	Some centres only allow people to view animals by appointment after choosing a pet, this reduces the ability to visit freely and compare animals in person before applying.

IMPACT ON MY SOLUTION

The RSPCA has influenced me to support a wide range of animal species and organise them into clear categories. I may include simple species categories, but the main method of finding suitable animals will be the compatibility questionnaire and ranked recommendations. It has also inspired me with its layout and visual design. I like its consistent colour scheme with clear headings.

However, unlike the RSPCA, my system will calculate a numerical suitability score, explaining which requirements have and have not been met. It will also use a simpler navigation system focused solely on adoption, preventing users from becoming overwhelmed.

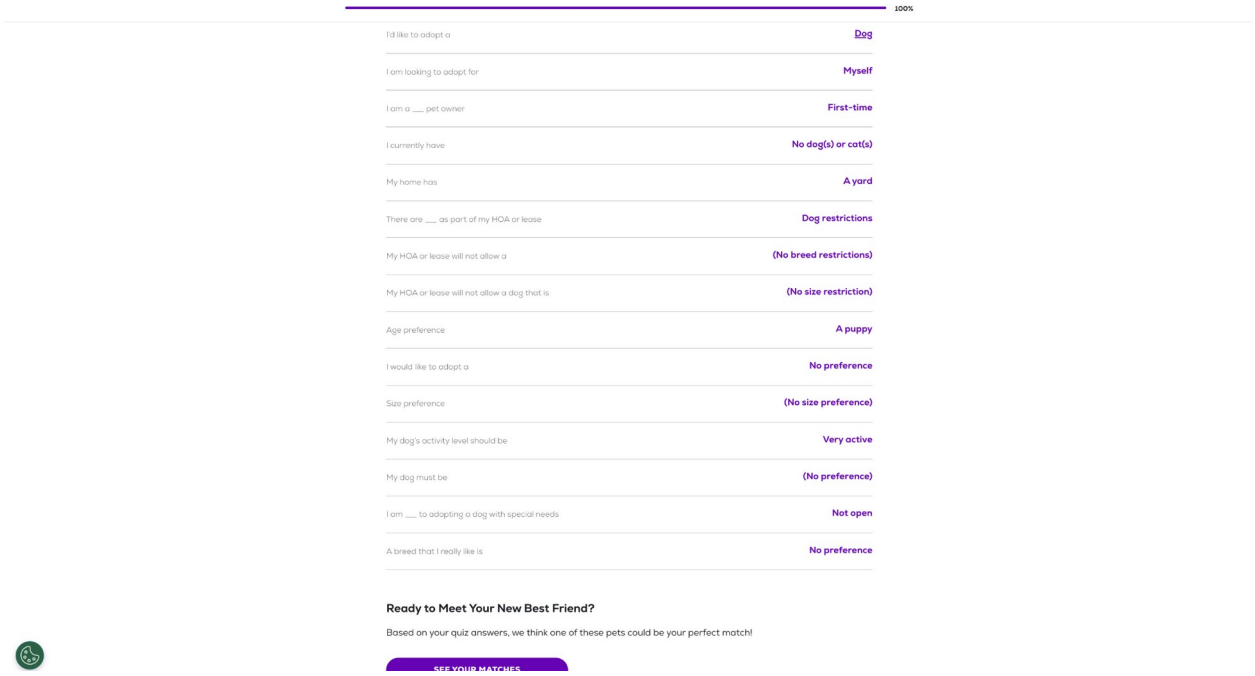
Features I will take from the RSPCA (E2)

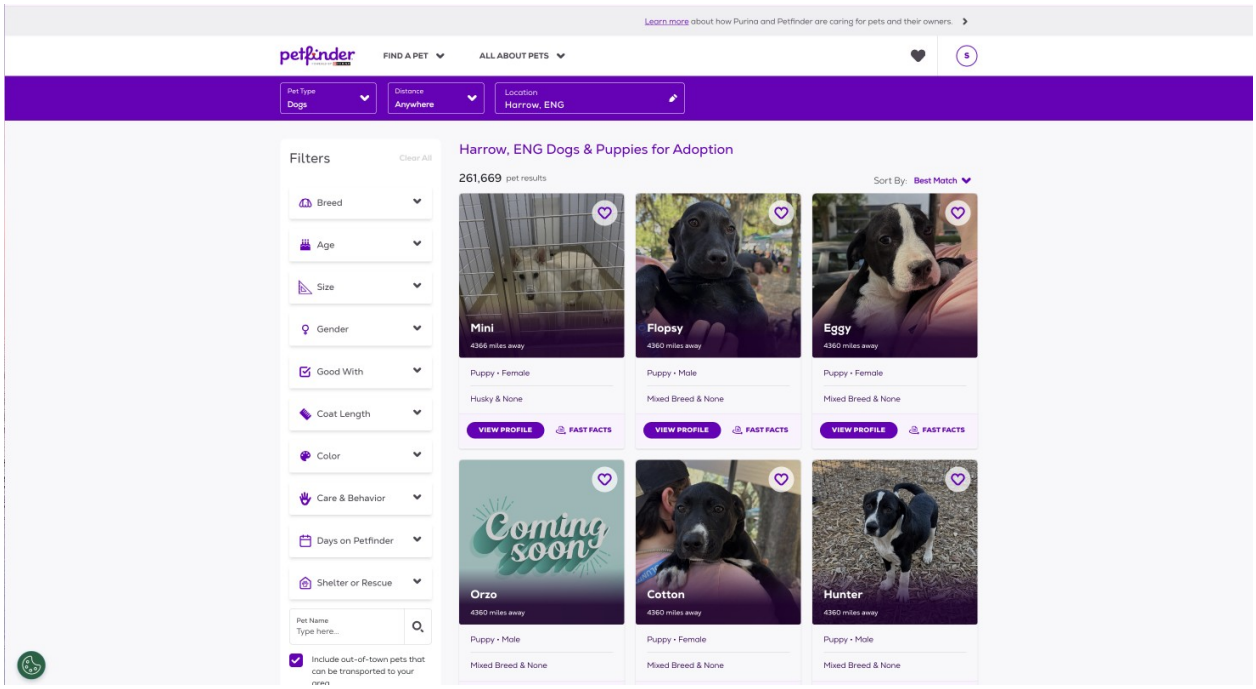
- **Adopt:** several species organised into clear categories; large animal photos on compact cards; a species filter.
- **Adapt:** the RSPCA shows suitability notes as free text. PawMatch turns these into structured fields (child-safe, good with pets, hours alone) so they can be scored.
- **Reject:** the per-pet application form, which the client does not want (Q4).

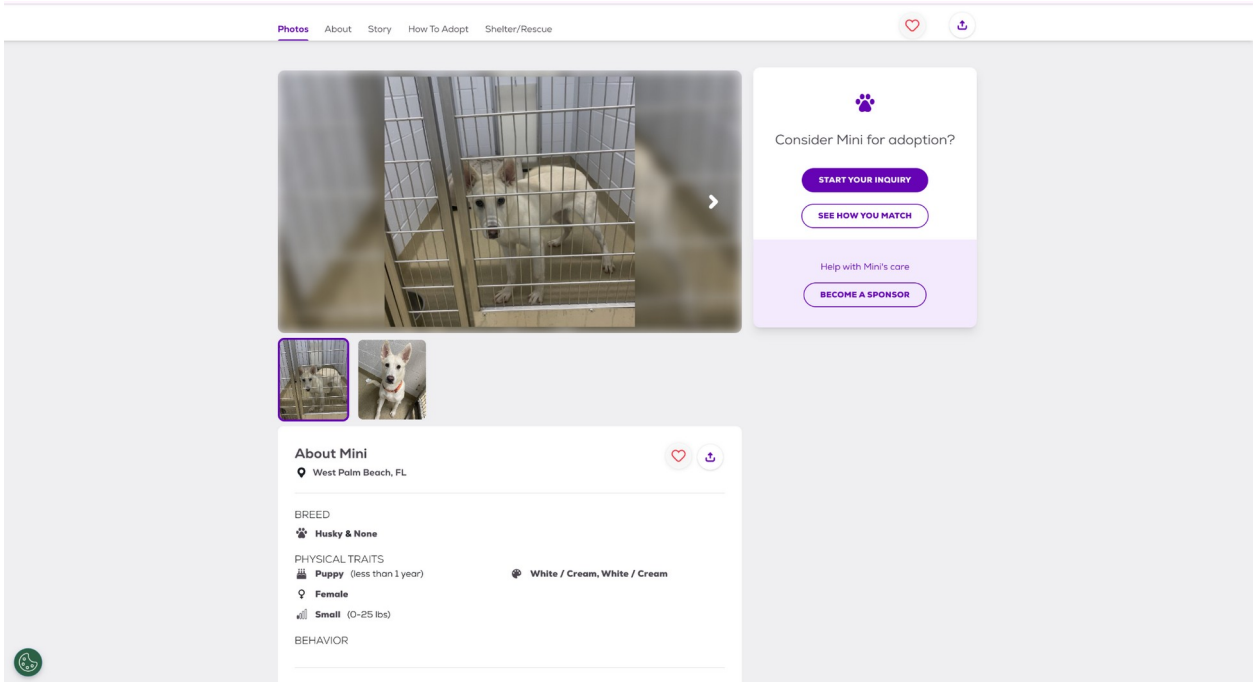
Existing solution 3: Petfinder

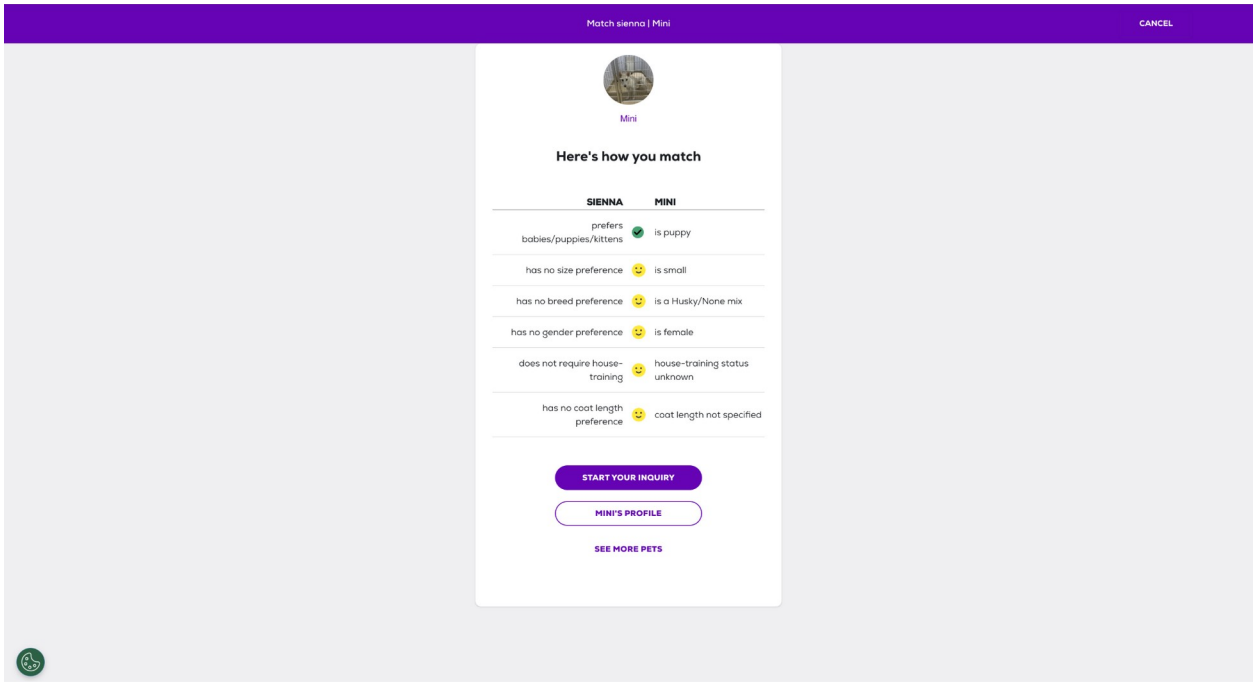
Example 3 - Petfinder

Pet finder is my closest competitor to my proposed system as it provides a short adopter questionnaire, and the system displays compatible pets. This website also includes a “See how you match” option. Animal listings contain information of the dogs such as age, housetrained ability and whether it can live with other animals. Users can also favourite animals.









Positives	Negatives
The search filters allow users to quickly narrow down the large number of available animals	The large number of filters and listings could overwhelm some users.
Animal cards are very clear and display a photo and important information such as the name, age and breed	It may not be as colourful and bright as the other examples.

Positives	Negatives
The animal profiles are detailed and show information about the personality, medical needs and compatibility with children or other pets.	The automated matching appears to focus mainly on cats and dogs rather than equal support for every species.
Users can save animals as favourites so they can return to them later.	A visible suitability percentage is not provided making it harder for users to compare matches
The adopter questionnaire can provide more personalised recommendations then a basic search.	Information can be incomplete or marked as ‘unknown’ as each shelter is responsible for entering its own animal data
Important actions such as ‘start your enquiry’ and ‘see how you match’ are clearly labelled and easy to identify, allowing ease of use for the user.	The rescue organisation may ask the user to complete another official application, causing information to be entered more than once. This also may mean the information provided may be inconsistent as different rescue organisations manage their own listings.
Pet finder brings listings from thousands of different shelters and rescues into one website.	Several actions appear on the same profile, including enquiring, matching and sponsoring which may compete for users’ attention.
Data can be saved by creating a login.	Some less important promotional content may distract users from main adoption features.
Bold headings divide profiles into clear sections such as behaviour, health compatibility and adoption information.	As individual shelters manage their own listings, an animal's availability may not be updated immediately. Users might need to contact the shelter to confirm whether an animal is still available which may be an inconvenience for the user.

IMPACT ON MY SOLUTION

Petfinder’s system has strongly influenced the design of my system. I am inspired how the survey is divided into clear manageable questions rather than displaying a long form at once. I will use a similar step by step structure with clearly labelled answer options and progress indicator, so the user knows how much of the questionnaire remains. I also like how it compares the adopters' preferences with animal characteristics in a clear table with visual symbols to show if each factor is compatible.

However, my system will provide more detailed feedback by displaying a numerical compatibility score and explaining which animal requirements have and have not been met. This will allow the several animals to be compared more objectively.

Features I will take from Petfinder (E3)

- **Adopt:** an adopter questionnaire; filters for species and size.
- **Adapt:** Petfinder uses answers to filter, not to score. PawMatch gives a percentage and a reason for every animal.
- **Reject:** the very large national listing, which is the opposite of a single small rescue.

Overall Conclusion

My research shows that each competitor contains useful features, but none fully have all the requirements I want for my own system. Considering the design of dogs' trust, it has a recognisable and consistent interface where the information is presented very clearly. Dogs trust has a thorough application process even though it is only for dogs and does not provide immediate feedback. However, the frequent bright yellow may be visually distracting and long pages which can require excessive scrolling.

The RSPCA supports several species and organises animals into clear categories, but users must browse and select an animal before suitability is assessed. Their professional colour scheme makes it very recognisable and trustworthy, which may make people more intrigued to adopt. However, because the website contains services beyond adoption, its navigation can appear crowded and some important information may not stand out. Users will not be given instant or numerical feedback as they browse for an animal then begin the application. Only after staff are able to manually assess the suitability.

Petfinder is the closest competitor to my proposed system as it uses an adopter questionnaire and compares the user’s preferences with individual animals. However, it does not provide a clear numerical score making it harder to compare with many animals. Information may also be inconsistent because individual rescue organisations maintain their own animal records.

Existing solutions 4 to 8: wider research

Three existing solutions do not show enough of the range of approaches, so I looked at five more, chosen to cover the three parts of PawMatch: the adopter questionnaire (E4, E6), the staff back office (E7), the public listing (E5) and weighted matching outside animal adoption (E8).

Solution	What it does (checked October 2026)	Lesson for PawMatch
E4 ASPCA Meet Your Match (US)	An 18-question adopter survey for dogs, combined with a staff behaviour assessment, puts each dog into one of nine colour-coded types. Counsellors use the result as guidance.	The closest idea to PawMatch, and it confirms the client's view that a score should guide staff, not decide. But it classifies dogs into types rather than ranking every animal with reasons.
E5 Battersea rehoming (UK)	Keyword search and a "best match" claim with no stated method; clear "Meet the dogs" route; accessibility policy (skip link, alt text).	An unexplained match is not trusted. PawMatch must say why. Battersea's accessibility commitments set the bar for contrast and text size.

Solution	What it does (checked October 2026)	Lesson for PawMatch
E6 Woodgreen Pets Charity (UK)	An online questionnaire on home, lifestyle and type of dog that stays valid for six weeks; staff make contact only on a potential match.	Answer once and reuse the answers: PawMatch stores the questionnaire so adopters do not repeat it, and lets them update it later.
E7 Shelterluv (shelter software)	Staff suite: medical records, fosters, adoption checkout. Staff log in; fosters have limited rights. No public matching.	Separate staff and public roles with different rights. Its feature list is far bigger than a small rescue needs, which is why a simple tool is justified.
E8 OkCupid (dating analogue)	Each question has an importance weight; a percentage is calculated over shared answers and used to rank people.	Weighted scoring normalised to a percentage is a proven way to rank. PawMatch avoids its weakness (the percentage looks high when few questions are shared) because every adopter answers every question.

Features selected from all eight solutions

Feature	E1	E2	E3	E4	E5	E6	E7	E8	Decision
Lifestyle questionnaire before matching	✓		✓	✓		✓		✓	Adopt
Weighted score shown as a percentage				~				✓	Adopt
Every animal ranked, best first, with reasons					~			~	Adopt (gap none fill)
Warning for a welfare risk instead of blocking	✓			✓					Adopt
Species and size filters on browse		✓	✓		✓				Adopt
Answers stored and reusable						✓		✓	Adopt
Staff add / edit / delete animal records							✓		Adopt
Separate staff and adopter roles							✓		Adopt
Apply or pay online		✓					✓		Reject (Q4)
Staff behaviour assessment feeding the score				✓					Adapt: staff enter needs as fields
Location search across many centres		✓	✓		✓				Reject (one site)

✓ = has it, ~ = partly. Sources E1 to E8 are listed in Appendix B.

1.3 Client interview

Client Interview

1. **What factors do people fail to consider before adopting an animal?**

"I think a lot of people choose animals based on how cute they are without thinking how much work they can be. They often forget to consider things like how much space they have, how much time they can spend with their animal, how much it costs to look after, and whether they have experience caring for that type of animal. Some people also do not think about whether their home is suitable for the animal's needs."

2. **What information should be stored about each animal?**

“Each animal should have basic information such as its name, age, breed, species, sex and a photo of the animal. It can also contain an extra information section such as personality, medical conditions, exercise needs, whether it gets along with children or other pets and special requirements other adopters should know about.”

3. **How should the system respond if the user is not suitable for the particular animal?**

“I don't think the system should completely stop someone from adopting an animal, but it should warn them they might not be a good match. It should explain which requirements they don't meet and recommend other animals. This helps people make better decisions and reduce chances of animals ending up in the wrong home, but also not diverting people from adopting.”

4. **Do you think users should be able to apply for adoption directly through the system?**

“No, I don't think users should be able to apply for adoption directly through the system. Although the system can help identify suitable animals, I still think adoption agencies should still carry out further checks in person. If staff are able to meet the adopter face-to-face they can assess personality, commitment and understanding of responsibility in owning a pet. It should aid the original process not completely take over that decision”

5. **What key tasks should the staff have on their side of the system?**

“The key features the staff should have is being able to add, edit and remove animal records, update animal information, upload photos and mark animals as available and unavailable for adoption by deleting. The database should also be kept up to date.”

6. **What features would make the system easy to use for both staff and adopters?**

“I think the main thing is that it should be simple and not confusing. People should be able to find what they are looking for quickly without clicking through loads of pages. It should be easy to browse animals and use the questionnaire, and for staff it should be straightforward to update animal information. Clear buttons, a clean layout and simple instructions.”

7. **What features should be displayed on the homepage?**

“I think the homepage should show information about how the adoption process works and a button to complete the questionnaire. The homepage should be welcoming and make it clear what the website is for.”

8. **Should users be able to search for animals, if so, what filters should be available?**

“Yes, I think users should be able to search for animals as it makes it much easier to find a suitable pet. Filter categories could be the type of animal, breed, age, size and activity level, but mainly type of animal and size.”

9. **What information should users provide when creating an account?**

“Basic information such as name, email, address, phone number and a password when creating an account. You could also include the age to ensure they are old enough to adopt the animal.”

10. **Should some factors have a greater impact on the compatibility algorithm than others and if so which factors?**

“Yes, I think some factors should matter more than others. Things like available space, allergies and whether there are young children in the house should have a bigger impact. For example, if someone is allergic to cats, they should not be strongly matched with a cat, even if the rest of their answers are suitable. Less serious factors such as preferred age or breed, should not be considered as important as the animal’s safety and wellbeing.”

11. **Do you want the website to look a specific way? Any colours, fonts etc. It would also help if you gave specific examples of apps, you like**

“I would like the website to have a cute and friendly look rather than it being too serious. Soft pink and pastel colours would make it feel more welcoming and enjoyable to use. I think the design should be simple but visually appealing, with lots of photos of the animals and clear buttons. The main thing is that it feels approachable and encourages people to engage with the adoption process.”

12. **Do you have any security concerns regarding the app?**

“Yes, I think security is important because the system will store users' accounts and animal records. Passwords should be protected, and only staff should be able to add, edit or delete animal information. Regular users should only be able to browse animals and complete questionnaires, not change anything in the database.”

13. **Which devices do you want it to work on?**

“I want the system to work on both laptop and desktop computers, as these will be mostly used by adoption agency staff.”

14. **What should happen after a user receives their compatibility results?**

“I think the system should show users the results in a clear way and provide them with more information about any animals that are a good match. What happens after that would depend on how the adoption agency wants to use the system.”

15. **Are there any features that you believe are essential for the success of this system?**

“The compatibility scoring system is probably the most important feature as it helps match animals with suitable owners. Additionally, the animal profiles need to be detailed and up to date so the users will have all the information they need before making a decision”

Overall, the client interview provided valuable insight into the function and design requirements for this system. The client identified that many people fail to consider factors such as available

space, time commitment, allergies and previous experience before adopting an animal. Therefore, these factors will be incorporated into the compatibility questionnaire and scoring algorithm. The compatibility algorithm should use weighted factors so that important requirements such as allergies, children, other pets and living space have a greater impact than softer preferences such as preferred breed or age. The client also highlighted the importance of maintaining and up to date detailed animal profiles and providing staff with ability to edit, add and remove animals through an administrative interface. For the design, the client wants a simple, easy to use interface with clear navigation, animal photographs and soft pastel colours to make it a friendly interface. The client also is very focused on security ensuring only staff can change details within the database regarding the animals.

1.4 Stakeholder survey

Stakeholder Survey

This stakeholder survey received 27 responses, which gave me feedback from a range of potential users.

QUESTIONS:

1. **Have you ever owned a pet before?**

A. Yes

B. No

2. **If you did own a pet, did you adopt it from a centre?**

A. Yes

B. No

C. N/A

3. **How would you normally choose a pet?**

A. Appearance

B. Personality

C. Breed

D. Lifestyle Compatibility

E. Advice from experts

4. **How important do you think it is to research an animal before adopting?**

A. Very Important

B. Important

C. Neither

D. Not Important

E. Not very important

5. Which factors do you think should be considered when matching an animal to an adopter? (Select many)

A. Living Space

B. Garden Access

C. Income

D. Experience with animals

E. Time available

F. Presence of Children

G. Other pets

6. How important is it that an adoption website is easy to use?

A. Very important (1)

B. Slightly important (5)

7. Would a compatibility score influence your decision when adopting an animal?

A. Yes

B. No

8. Do you think adoption agencies should use a compatibility scoring system?

A. Yes

B. No

9. How concerned are you about the welfare of adopted animals?

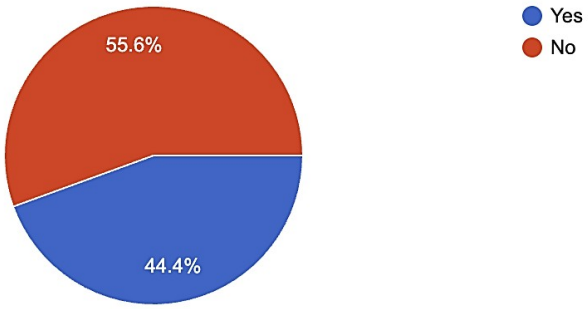
A. Concerned (1)

B. Not Concerned (5)

RESPONSES:

Have you ever owned a pet before?

27 responses

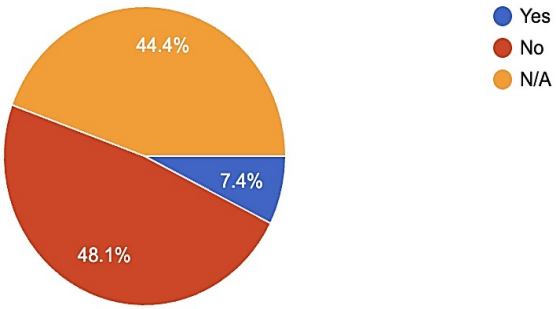


Analysis:

44.4% of respondents said yes, while 55.6% said no. This shows that my survey consists of a mixture of people with and without pet ownership experience. This is useful as my system needs to be able to support experienced and inexperienced adopters. This means the system should not assume that users will already understand animal care or adoption requirements.

If you did own a pet, did you adopt it from a centre?

27 responses

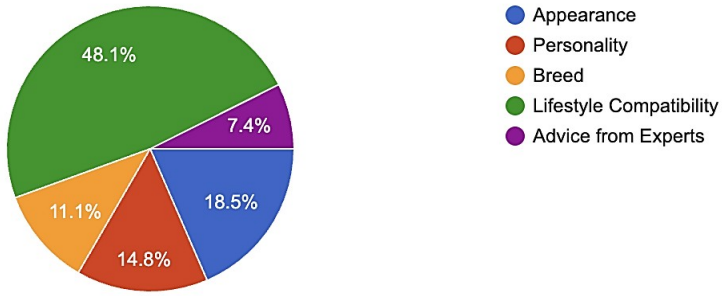


Analysis:

These results show that very few respondents have experience adopting from an adoption centre. This means my solution should clearly explain the adoption process and avoid making it confusing for users who have no experience.

How would you normally choose a pet?

27 responses



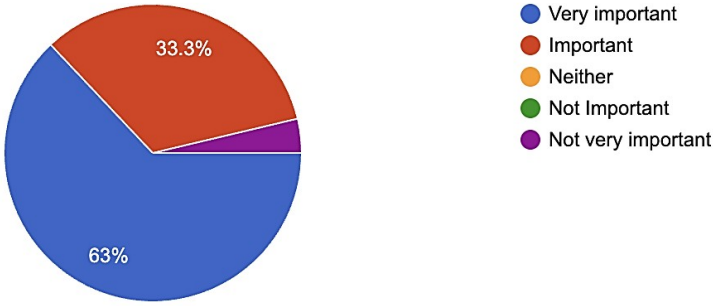
Analysis:

The most common answer was lifestyle compatibility, chosen by 48.1% of respondents. This shows most users already do care about which animal fits their lifestyle rather than just how the animal looks, even though a few care about the aesthetics of the animal. This will allow my system to be suitable for those looking to adopt.

How important do you think it is to research an animal before adopting?

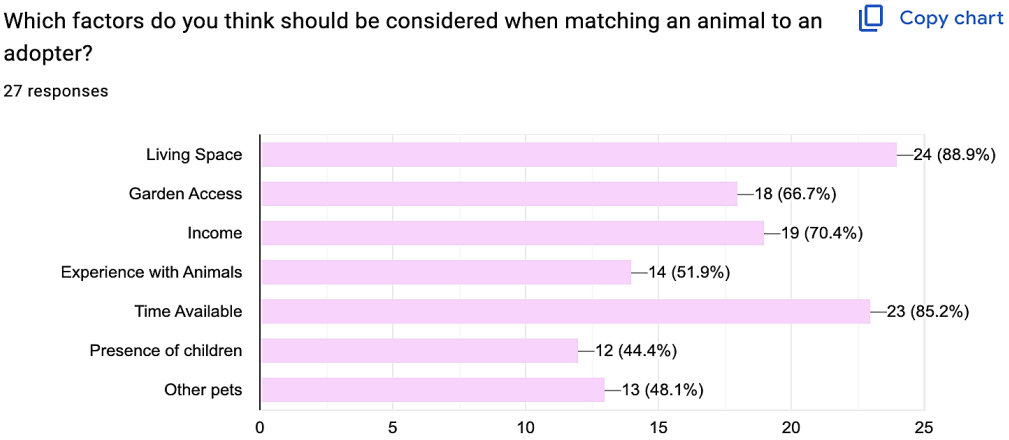
Copy chart

27 responses



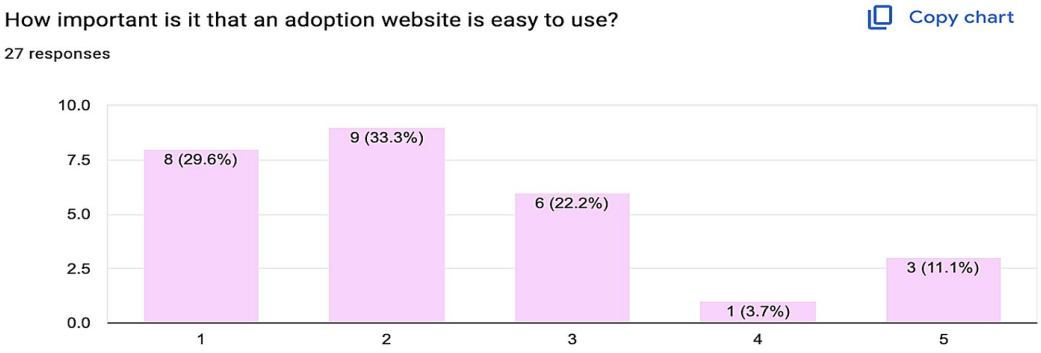
Analysis:

63% of respondents said it was very important, and 33.3% said it was important. This means that 96.3% of respondents agreed that research is important before adopting an animal. This supports my idea that websites should include detailed animal information.



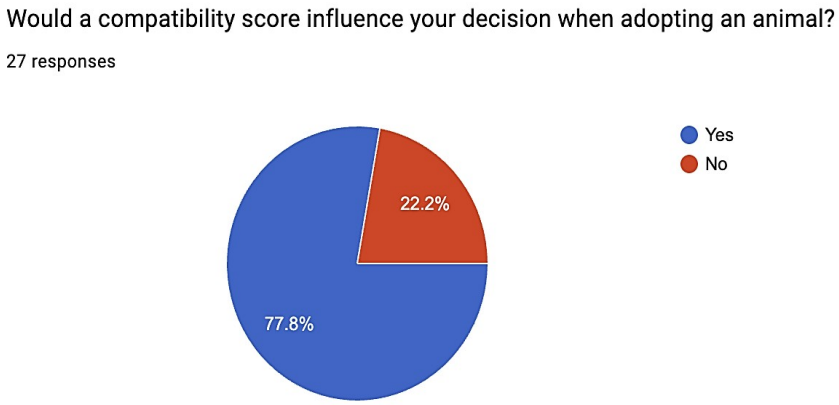
Analysis:

The most selected factors were living space at 88.9% and time available at 85.2% showing that respondents think an adopter's environment and lifestyle are the most important parts of a successful match. Income and garden access were also highly chosen showing that practical care needs should be considered primarily.



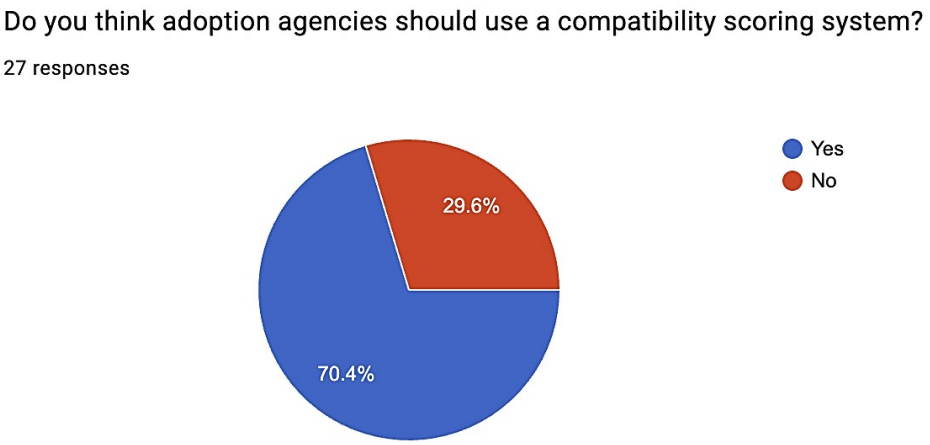
Analysis:

Most responses were grouped around the higher end of the scale, which suggests that ease of use is important to users. As a result, I would like my website to have a simple layout, clear navigation, readable text and easy process. I aim to make it as accessible as possible.



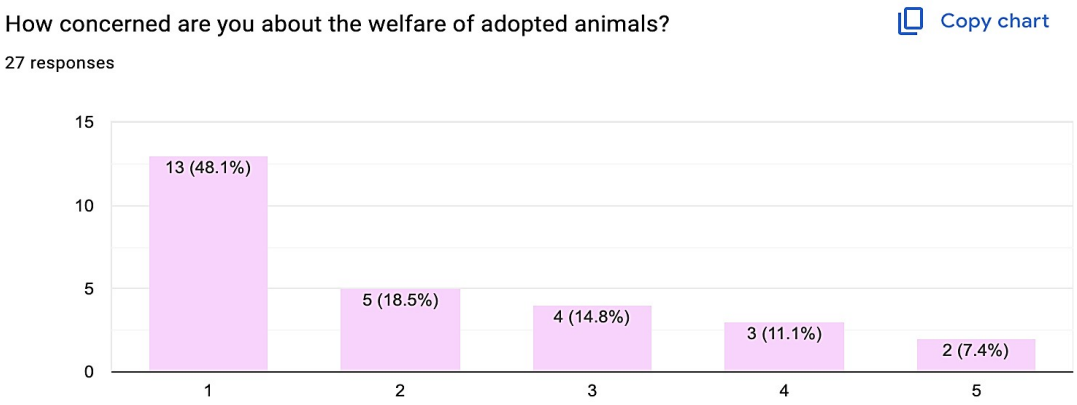
Analysis:

77.8% of respondents said yes and 22.2% said no. This shows that most users may find a compatibility score helpful when deciding whether an animal would be suitable for them.



Analysis:

70.4% of respondents said yes, while 29.6% said no. This shows that respondents support the idea of adoption centres using this scoring system, but not everyone agrees. Therefore, the compatibility score should be used as a guidance but not as the deciding factor.



Analysis:

The largest group of respondents selected 1, with 48.1% and 18.5% selected 2. This shows that many respondents are concerned about animal welfare. This supports the purpose of this system as it aims to reduce unsuitable adoptions by helping users find animals and reduce chances of animals being placed in unsuitable homes.

Overall:

Overall, the survey shows that there is a clear need for a pet adoption system that helps users make informed decisions. Most respondents have shown that they agree that research is important before adopting an animal and would be influenced by the compatibility score. This

survey also shows that the important factors include living space, time available, income, garden access and experience with animals.

My solution should include detailed animal profiles, a user questionnaire with a compatibility scoring system, where lifestyle compatibility will be the most important and popular factor in choosing an animal. This system should also be easy to use as users will have different levels of experiences with pet ownership and adoption.

However, one limitation of this survey is that it only received 27 responses, so it may not represent all potential adopters, and some respondents may have limited experience with adoption centres. It still provided me with guidance about common user expectations, especially importance of ease of use, animal welfare and compatibility scoring.

This survey was insightful to help me understand what potential users would expect from an animal adoption system, especially the need for clear information, lifestyle based matching and a compatibility score.

1.5 Proposed solution

Prefixes: E—Examples | Q—Client Interview |S—Stakeholder Survey

Features

#	Feature	Notes	Justification	Source
1	Clean user interface	Calm colour scheme, clear headings, identifiable buttons and a clear layout	Makes system easier to understand and use, especially for those not confident with technology	S6, E3, E1, E2
2	Navigation bar across different pages	Allows users to move between pages easily	Competitor websites used simple navigation, which helps reduce confusion and makes system more user friendly	E1, E2, E3, S6
3	User Home page	This system will include a clear home page introducing the purpose of adoption matching system	Simple starting point, and helps user understand system is focused on adoption suitability rather than browsing animals	E1, E2, E3
4	Animal preview Page	Users will be able to view available animals using compact animal cards. Each card will show key information such as name, species, breed, age and image.	Compact cards will make it easier for users to compare animals without needing to open every profile individually	E1, E2, E3

#	Feature	Notes	Justification	Source
5	Animal Profile Page	Each animal will have a detailed profile page containing information such as name, species, breed, age, sex, personality, medical needs, exercise need and suitability requirements	The client stated that the animal profiles should show key details about the animal, which is also required for the matching algorithm	Q2, E1, E2
6	Multi Species Categories	Animals will be organised by species such as dogs, cats and rabbits	The RSPCA also has different animal categories, whereas dogs trust is just based around dogs.	E1, E2
7	Adopter Questionnaire	Users complete a questionnaire about their lifestyle and animal experience and will include many other answers which will contribute to the compatibility scoring algorithm.	The client stated that adopters often fail to consider important factors before adopting, this will allow them to reflect on their suitability before decisions are made.	Q1, S4, S5
8	Step by step questionnaire layout	This questionnaire will be split question by question instead of being displayed as one long form, with dropdown options.	This makes the process easier to complete and reduces the chance of users feeling overwhelmed.	E3, S6
9	Input Validation	To check if data that is required is not left blank and the data is valid.	Preventing the incomplete or incorrect data being used by the matching algorithm, recommendations more reliable	Q15, S8
10	Compatibility scoring algorithm	The system will compare the adopter's answers with each animals requirements and calculate a numerical score.	The main computational feature, allows animals to be recommended on suitability	E3, S7, Q10
11	Ranked animal recommendations	After the questionnaire is submitted, animals will be displayed in order from most suitable to least suitable	Removes need for user to manually search through every animal	Q14
12	Requirements Breakdown	The system will show which requirements the adopters meet and which requirements they do not meet for each animal.	Helps users understand why an animal has received a particular score.	E3, Q3

#	Feature	Notes	Justification	Source
13	Unsuitable match warning	If the user does not meet the requirements, the system will display a clear warning	The client said the system should not completely stop a user from adoption but warn them if they are unsuitable and why	Q3
14	User registration system	Users can create an account by registering with personal details such as name, email and password	Users can save progress, access their questionnaire and personalise their own experience	Q9
15	User and Admin Login and Logout system	Registered users will be able to securely login and logout their accounts and staff can login securely to access management side of the system	User login ensures data is only accessible to the correct user and admin login prevents normal user from changing animal records and protects accuracy over database	Q12
16	Admin animal management	Admins should be able to add, edit and delete animal records	Animal information should be kept up to date when new animals arrive, details change or animals are adopted	Q5
17	Database Storage	System will need to have a database alongside to store animal records, user accounts, admin login details and adopter questionnaire details	So data can be stored, searched, updated and used by the matching algorithm	Q15
18	Responsive streamlit design	I will use Streamlit layout tools so pages remain clear on different screen sizes.	This makes system easier to use on different laptops or tablets without information being cut off or difficult to read	Q6, E1, E2, E3, S6

The solution in specific terms

Part	Exactly what will be built
Accounts	Register with name, email, age (18 or over) and a password of at least 8 characters, typed twice. Passwords stored as salted scrypt hashes. One login form for everyone; the role stored in the database decides what each person sees. Optional "Continue with Google".
Browse	Cards for every Available animal (photo, name, species, breed, age, size), filtered by species, size and a text search; a profile page per animal.
Questionnaire	10 questions, one per screen, in four sections (home, household, lifestyle, preferences); card buttons for choices, a number box for hours alone; Next is disabled until the question is answered.
Matching	Weighted welfare factors (time alone, children, space, activity, garden and the others the client named), normalised to a percentage: points earned ÷ total points × 100. An unmet welfare requirement raises a warning rather than hiding the animal (Q3); an allergy can never give a strong match (Q10). Species and size preferences change the order only. Weights are designed on D54 and finalised on S20.
Results	At least 4 animals (top 5 shown, "show all" toggle), animals without warnings first, each with met and not-met reasons and a statement that staff make the final decision.

Part	Exactly what will be built
Staff dashboard	Overview of records, a table of all animals, and add / edit / delete forms with the same validation as the database; photos PNG or JPG up to 2 MB; delete needs a tick-box confirmation.

1.6 Hardware and software requirements

Hardware and Software Requirements

The technology used to build this system is based on the features that my client requires. The list of requirements is as follows:

- Clean user interface
- User registration and login
- Admin login
- Database storage for animal records, user accounts and questionnaire answers
- CRUD functionality for admins
- Input validation
- Compatibility scoring algorithm
- Ranked animal recommendations

For this project, I will develop my system on my MacBook, which has the required hardware and software capabilities to develop, run and test my system.

Developer

Hardware:

Device: MacBook Air, 13 Inch, M3 2024

Processor: Apple M3 Chip

RAM: 8GB

Display: I will use the 13.6 inch built in retina display alongside a 13 inch monitor

Input: Keyboard and Trackpad

Internet Connection: This is required in order to install packages, access documentation and test deployment.

This hardware is suitable as my system will not require advanced graphics processing or a powerful server. The main processing will be form input, database queries, validation and the algorithm, which is perfectly suited for my laptop.

Software:

Operating system: MacOS Sequoia 15.0

IDE: Visual studio code

Programming Language: Python

User interface framework: Streamlit

Database: SQLite

Database module: sqlite3

Browser: Chrome

I will use some additional libraries such as pandas which may be used to organise and display data in tables.

I have chosen python as my main programming language as it is the one I am most confident in, therefore I will not have to spend as much time trying to learn another language when python is suitable enough. It is suitable to create the matching algorithm, validating user inputs and all the functionalities my system requires. It allows me to be more efficient and focus on the computational parts of the system such as the comparison between adopter answers and animal requirements producing compatibility scores.

For my user and admin interface I will be using streamlit. It is simpler to use than using tkinter and it provides built in tools such as buttons, forms, dropdowns, columns, containers and page navigation. This allows me to create the pages without using a full HTML, CSS and JS website.

For data storage I will use SQLite as it is what I have the most experience in. It is suitable and easy to use to store data required for my system such as animal records, user accounts, admin login details, questionnaire answers and compatibility results.

User + Admins

My product will be a streamlit application, so users and admins will only require a device that accesses the system through any web browser., If deployed online, users will require an internet connection and they do not need to install any modules. They could access from a laptop, desktop computer or tablet, which may require a mouse, keyboard and a monitor to present the graphical user interface or even touchscreen if using a tablet.

Minimum requirements for running PawMatch

The section above describes the computer I develop on. The requirements for the users and for the machine that runs the system are below, with the reason for each.

Who	Minimum	Justification
Adopter or staff device	Any laptop, desktop or tablet (screen at least 1024 px wide) with a current browser (Chrome, Edge, Safari or Firefox); internet connection	Streamlit runs in the browser, so nothing is installed. 1024 px is a landscape tablet, the smallest screen tested (section 4.1, page E1).
Input	Mouse or trackpad and keyboard, or touch	Card buttons are large enough to tap; every control can also be reached with the Tab key.
Server (runs the app)	Python 3.11 or later; 1 CPU core, 1 GB RAM; 100 MB disk for the program and database plus up to 2 MB per animal photo	Measured on 500 animals: a submission takes under 0.1 s (section 4.1, page E1). Photos are limited to 2 MB each, so 20 animals need at most 40 MB.
Software (server)	`streamlit 1.66.1`, `pillow 13.0.0`, `Authlib 1.9.1`, `httpx 0.28.1` (pinned in `requirements.txt`); SQLite is built into Python	Exact versions mean every install behaves like the tested one.
Developer tools	Visual Studio Code, pytest, Git	pytest runs the 208 automated tests; Git records each stage as a commit.

1.7 Limitations

Each limitation from my original list is shown with its cause, its effect, and how PawMatch reduces the risk.

Limitation	Cause	Effect	How it is reduced
One rescue only	Built for a small single-site client	Cannot be shared by several branches	Data model has no branch field; adding one later is listed as future development
Dogs, cats and rabbits only	Matching rules researched for these three	Other species cannot be listed	Species are a fixed list checked by validation and the database; adding one is a single change
The score is guidance, not a decision	A short questionnaire cannot capture everything (Q4)	An adopter might over-trust a high score	Every results page says staff make the final decision; warnings are shown first
No online adoption process	Client wants staff to meet adopters (Q4)	Adopters still contact the rescue themselves	"What happens next" text on the home and results pages
Accuracy depends on honest answers	Answers cannot be checked by the system	A dishonest adopter can score highly	Staff home check remains part of the real process
Rule-based, not machine learning	No past adoption data to learn from	Weights are set by judgement, not proven by data	Weights come from the client and survey and are easy to change in one place (D54)
Records are updated by hand	Staff enter every change	Out-of-date records give wrong matches	Overview flags records missing a photo or with a short description
No live chat or animations	Time available and Streamlit's design	Less engaging than large charity sites	Friendly pastel style and photos instead (Q11)

1.8 Requirements and success criteria

My research produced the detailed requirements below (numbered by feature area, 1 to 9). The list is long and its items cannot each be measured, so it is kept as the **requirements list**, and the **success criteria** that follow combine them into 16 measurable statements. Each success criterion states how it will be measured and links to the post-development tests on D60.

Prefixes: E—Examples | Q—Client Interview|S—Stakeholder Survey

#	Feature	Requirements	Source
1	Navigation and Clean user interface	Pages must use a calm colour scheme, clear headings and identifiable buttons	**S6, E1, E2, E3**
		The interface must remain readable on laptop and tablet sized screens	**Q6, S6**
		The system must include a navigation menu to access home page, animal preview page, profile page, and login page	**E1, E2, E3, S6**
		The menu and interface style must remain consistent across all the pages	**E1, E2, E3, S6**
2	Home Page	Should explain the purpose of the animal adoption matching system, especially including that the system focuses on suitability rather than only appearance	**Q1, Q3, E1, E2, E3**
		Should have a clear button or navigation option that directs users to complete the adoption questionnaire	**Q1, S5, S6**
3	Animal preview page	The page should display every available animal using compact animal cards	**E1, E2, E3**
		Each card should show the species, name, breed, age and image	**Q2, E1, E2**
		The system must support at least 3 animal species including dogs, cat and rabbits	**E2**
		Users must be able to open a full animal profile from the preview page	**E1, E2, E3**
4	Animal Profile Page	Each profile must display the animals name, species, breed, age and sex	**E1, E2, Q2**
		Each profile also must include personality, medical needs, exercise needs and suitability requirements	**Q2**
		Animal profile information must be retrieved from the database	**Q15**
		Users must be able to return to the animal preview page or results page	**S6, E1, E2, E3**
5	User and Admin registration and login page	Users must be able to create an account, using name, email and password and it can include other features such as age to ensure they are at a suitable age	**Q9, Q12**
		The system must reject registration if required fields are empty and the email format is invalid, by validating inputs	**Q12, Q15**
		Registered users must be able to login and logout using their correct account details	**Q12**
		The admin login details should allow admins to access their own section with the CRUD interface	**Q12, Q5**
		Incorrect Login details must display an error message and prevent access to the account	**Q12, S6**

#	Feature	Requirements	Source
6	Adopter Quest'naire Page	The questionnaire must collect information about the user's home, lifestyle, garden access, children, other pets, time available and previous animal experience	**Q1, S5**
		The questionnaire must be split into clear sections or individual questions rather than appearing as one long form	**E3, S6**
		The questionnaire must use suitable input methods such as dropdowns, radio buttons or selection boxes but preferably dropdowns.	**E3, S6**
		The system must prevent the questionnaire from being submitted if important questions are unanswered	**Q15, S8**
7	Results and recomm'ns page	The system must compare adopter's questionnaire with each animal's requirements	**Q10, Q1, S5**
		The system must calculate a numerical compatibility score for each animal	**Q10, S7**
		The top 4 animals must be displayed from highest compatibility score to lowest at a minimum	**Q14, S7**
		The results page must show which key requirements the user meets and does not meet for each animal	**Q3, E3**
		The system must display a warning if the user does not meet important requirements for an animal	**Q3**
		The results page must explain that the compatibility scores are guidance, and the adoption staff have the final say	**Q3**
8	Admin manage't page	This page should be accessed separately with a separate admin login	**Q12**
		Admin users must be able to add a new animal record with all the information required	**Q5, Q2**
		Admin users must be able to edit existing animal records and save the updated information to the database	**Q5, Q15**
		Admin users must be able to delete animal records when animals adopted or no longer available	**Q5**
		Normal users must not be able to access the admin management page	**Q12**
9	Database system	The database must store animal records, user account details, admin login details and questionnaire answers	**Q15**
		The database must allow animal records to be added, viewed, edited and deleted through the admin system	**Q5, Q15**
		Questionnaire answers must be stored or processed so that they can be used by the compatibility scoring algorithm	**Q10, Q15**
		Data must be retrieved correctly so animal profiles and recommendations display accurate information	**Q15, Q2, Q10**

Success criteria

SC	Success criterion	How it will be measured (target)	Requirements	Source
SC1	The interface uses the client's soft pink palette, clear headings and obvious buttons	Every text colour reaches a contrast of 4.5:1 and no text is below 12 px, on every screen (automated audit of all screens); client approves the look	1	Q11, S6
SC2	Every page can be reached from one consistent menu	Each page reachable in 1 click from the menu; staff items never shown to adopters	1, 8	Q6, E1 to E3
SC3	The home page explains that PawMatch is about suitability and that staff decide	Text present; a button opens the questionnaire in 1 click	2	Q1, Q3, Q7
SC4	Adopters can browse every Available animal and narrow the list	All Available animals shown as cards with photo, name, species, breed, age; filters for species and size and a search; at least 3 species	3	Q8, E2, E3
SC5	Each animal has a full profile loaded from the database	Profile shows all stored fields and needs; back link returns to the list	4	Q2
SC6	Registration rejects every invalid input with a message beside the field	Each of name, email, age, password and confirm tested with normal, boundary and invalid data; duplicate email refused	5	Q9, Q12
SC7	Accounts are secure	Passwords stored only as salted hashes; one message for a wrong email or a wrong password; sign-out ends the session	5	Q12
SC8	The questionnaire collects all 10 answers, one question per screen, and cannot be skipped	10 screens in 4 sections; Next disabled until answered; completed in under 3 minutes	6	Q1, S5, S6
SC9	The score is calculated correctly from the weighted factors	Matches a hand calculation exactly (S20); each factor changes the score by its weight; allergy caps at 39%	7	Q10, S5
SC10	Results are ranked quickly and safely	At least 4 shown; animals with warnings never above those without; 500 animals scored in under 1 s	7	Q3, Q14
SC11	Each result explains itself	Every match lists requirements met, not met and welfare warnings, with a statement that staff decide	7	Q3, E5
SC12	Staff can add, edit and delete animals with full validation	Each field rejects invalid data; photo PNG/JPG up to 2 MB; delete needs confirmation	8, 9	Q5
SC13	Adopters can never reach or use staff functions	Every staff action refused for adopter and signed-out users (automated tests)	8	Q12
SC14	Data is stored reliably and scales beyond the rescue's size	4 tables with constraints; works with 500 animals and 100 users; browse page loads in under 3 s	9	Q15
SC15	Users find the system easy to use	Average System Usability Scale score of 68 or more (the average for websites: Brooke, 1996, Appendix B); client signs off the final stage	1, 6, 7	Q6, S6
SC16	Works on laptop and tablet screens	No clipped text or controls at 1440 px and 1024 px widths	1	Q6, Q13

1.9 References

References

BBC News (2024) RSPCA receives record number of animal cruelty reports. Available at: https://www.bbc.co.uk/news/articles/cwyrnlj5632o (Accessed: 19 June 2026).

Dogs Trust (n.d.) About us. Available at: https://www.dogstrust.org.uk/about-us (Accessed: 25 June 2026).

Dogs Trust (n.d.) How to adopt a dog. Available at: https://www.dogstrust.org.uk/rehoming/how-to-adopt (Accessed: 25 June 2026).

Petfinder (n.d.) Adopter profile: Finding your perfect pet. Available at: https://www.petfinder.com/pet-adopter-profile/ (Accessed: 25 June 2026).

Petfinder (n.d.) Pet Finder Quiz. Available at: https://www.petfinder.com/user/profile/create/ (Accessed: 25 June 2026).

RSPCA (n.d.) Find a pet. Available at: https://www.rspca.org.uk/findapet (Accessed: 25 June 2026).

RSPCA (n.d.) Pet adoption process guide. Available at: https://www.rspca.org.uk/findapet/adopt (Accessed: 25 June 2026).

The sources for E4 to E8 are listed in Appendix B (X1).

Glossary

Term	Meaning in this project
CRUD	Create, read, update and delete: the four things staff can do to an animal record
Hash	A one-way scrambled version of a password. The program can check a typed password against it, but the password cannot be worked back out from it
Salt	Random data added to each password before hashing, so two people with the same password get different hashes
scrypt	The hashing method used for passwords (Python's `hashlib.scrypt`). It is deliberately slow, which makes guessing many passwords expensive
Streamlit	The Python library that turns the program's pages into a website in the browser
SQLite	The database. It keeps all tables in one file and comes with Python, so nothing extra has to be installed
Primary key / foreign key	A primary key is the unique ID of a row (for example animal_id). A foreign key is a column that stores another table's primary key, which links the two rows
Session token	A long random code created at sign-in and stored in the database. It keeps the user signed in when the page is reloaded, and ends at sign-out or after 12 hours
WCAG	Web Content Accessibility Guidelines, the standard for readable and accessible websites (contrast, text size, focus)
SUS	System Usability Scale: ten statements, each rated 1 to 5, that give a usability score out of 100 (Brooke, 1996)

2 Design

2.1 Decomposition of the problem

The system is broken down top-down until each box is small enough to design, build and test on its own. Each leaf of the chart is one module or one page, and each is built in one development stage (the stage number is shown on the chart and in the table).

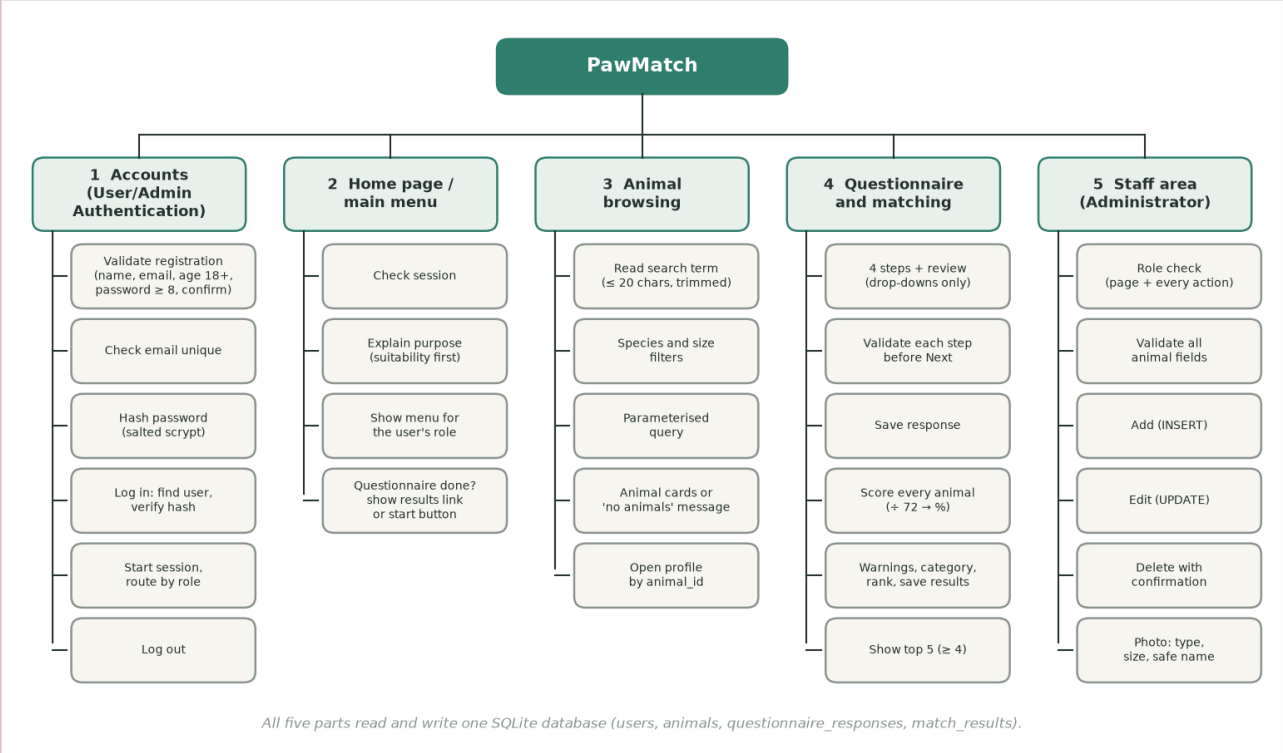


Figure D1: Hierarchy chart: PawMatch decomposed into five components and their sub-tasks

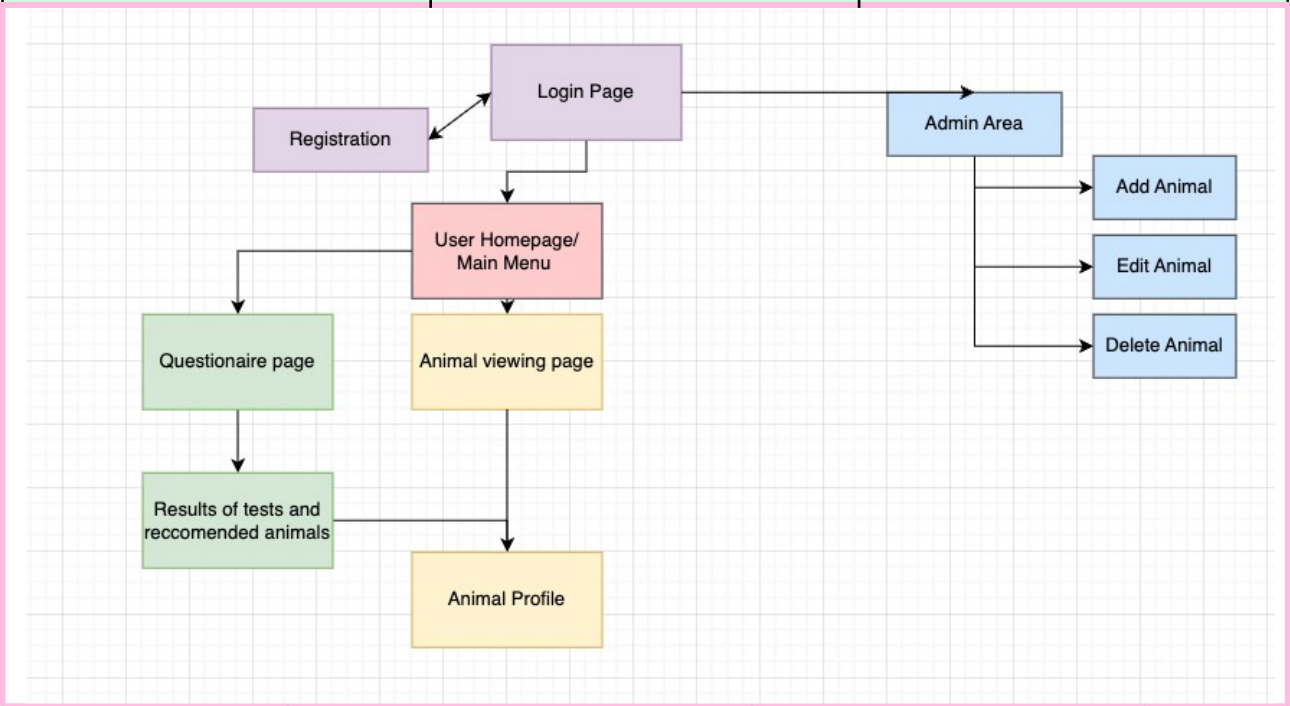
Proposed Layout

The system can be decomposed into five main components. Separating the system into smaller components makes it easier to design, develop, test and maintain whilst each section performs a specific function.

Section	Front End Justification	Backend Justification

User/Admin Authentication	Will allow users to register and existing users to login, and allow admins to access their page safely and securely. Registration will collect the users name, email, age and password. The interface should provide clear feedback when details are missing or forgotten from its validation system. It will also show an error message if login details are incorrect	The system will validate entered details, such as: - email cannot be blank and must be valid and unique - Password minimum 8 characters - Age must be a valid number - Login fields cannot be blank The passwords will be hashed in order to be kept secure and safe. During login, enter details will be compared against records in the users table. A session will then be created when authentication is
User Homepage/Main Menu	Will provide the main navigation area after the user logs in. It will allow the user to access the questionnaire, browse available animals and view their account. It will also explain the purpose of the adoption	The system will check the user's session to make sure they are logged in. It may retrieve the email and recent questionnaire information from the database so that personalised information can be displayed.
Administrator Menu	Provides staff with a dashboard to manage existing animals and their records. Admins will be clearly able to add new animals, edit existing details and remove records. To add animals it will	The system will check that the logged in account has the admin role before access is allowed. Parameterised SQL commands will insert, update and delete animal
	children compatible, garden required, medical notes and an image. Forms will be clearly Labelled and display confirmation or	out function will clear the admins session and return them to the login page, preventing another person using the same device which will
Animal Viewing	Allows users to browse available animals using clearly presented animal cards. Each card will show information such as the animal name, image, species,	Animal records will be retrieved from the animals table. Search and filter selections will be used to change the database query or filter the retrieved records. Selecting an animal will use its

Questionnaire and Recommendations	Will use a form in order to allow users to answer questions about their home, lifestyle, experience and preferences. The questionnaire will have questions such as the housing type, access to garden, animal loneliness times, children, pets, experience, activity labels, what species and size preferred. These questions would not appear all at one time to not overwhelm the user and the input type would be drop downs or radio buttons.	Responses will be validated and compared with the requirements of each available animal. A weighted compatibility algorithm will calculate a percentage score for every animal. The animals will then be sorted from highest to lowest score so the top recommendations can be displayed. The questionnaire responses and match results may also be stored within the database with the



Component	Sub-tasks	Module(s)	Stage
Accounts and login	Register, validate, hash password, log in, keep session, log out	`pawmatch/auth.py`, `validation.py`, `webui/views/login.py`, `registration.py`	1
Staff animal management	Role check, add / edit / delete, validate every field, photo upload	`pawmatch/services.py`, `webui/views/admin.py`, `webui/images.py`	2
Home and browse	Home page, cards, filters and search, profile	`webui/views/home.py`, `browse.py`, `webui/animals.py`	3
Questionnaire	One question per screen, validate each answer, store response	`webui/questions.py`, `webui/views/questionnaire.py`	4
Matching and results	Score each animal, rank, store and show results with reasons	`pawmatch/matching.py`, `webui/views/matches.py`	5
Database (shared)	Create tables, read and write records, constraints	`pawmatch/db.py`, `models.py`, `seed.py`	1 to 5

The analysis named six parts (animal records, questionnaire, score calculation, results, validation and the admin page). In the design these become five components: score calculation and results are one

component because the results page only shows what the scoring produces, and validation is not a component of its own but a shared module (`validation.py`) used by every form.

Structure of the solution

The program is split into two layers. The **domain layer** (`pawmatch/`) contains all rules: validation, scoring, accounts and the database, and has no screen code, so every rule can be tested automatically without opening a browser. The **interface layer** (`webui/`) draws the pages and calls the domain layer. This separation is what lets 208 tests run in a few seconds.

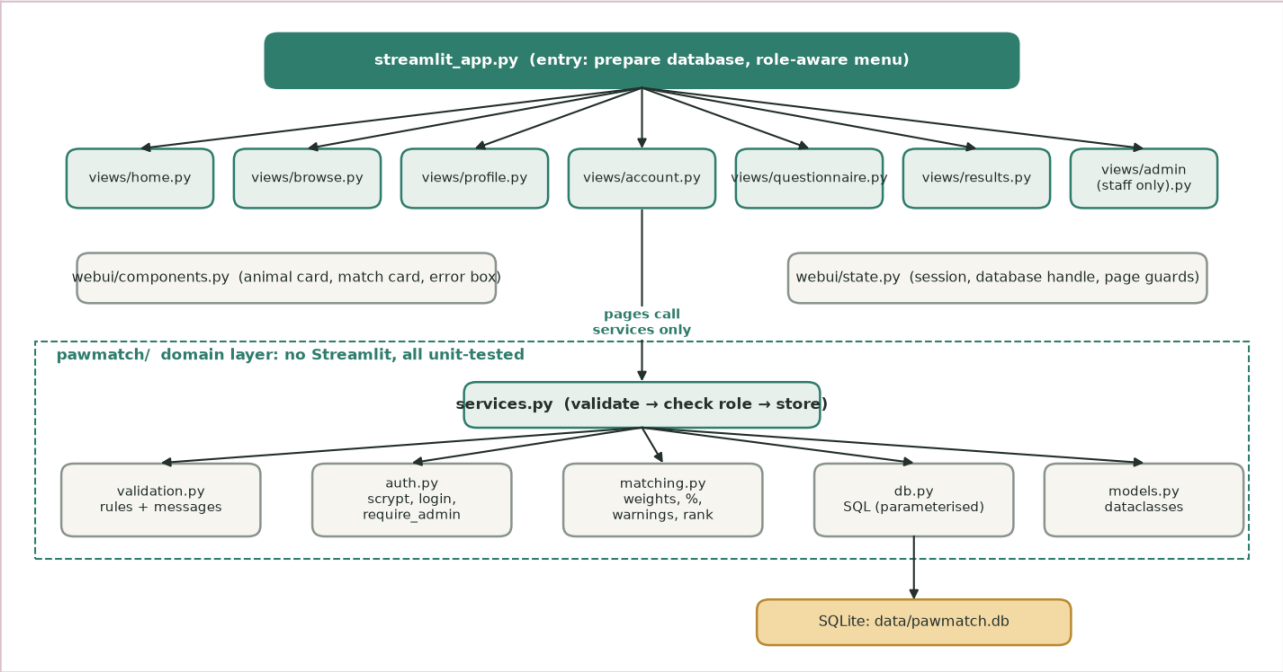


Figure D2: Structure: the interface layer calls the domain layer, which alone touches the database

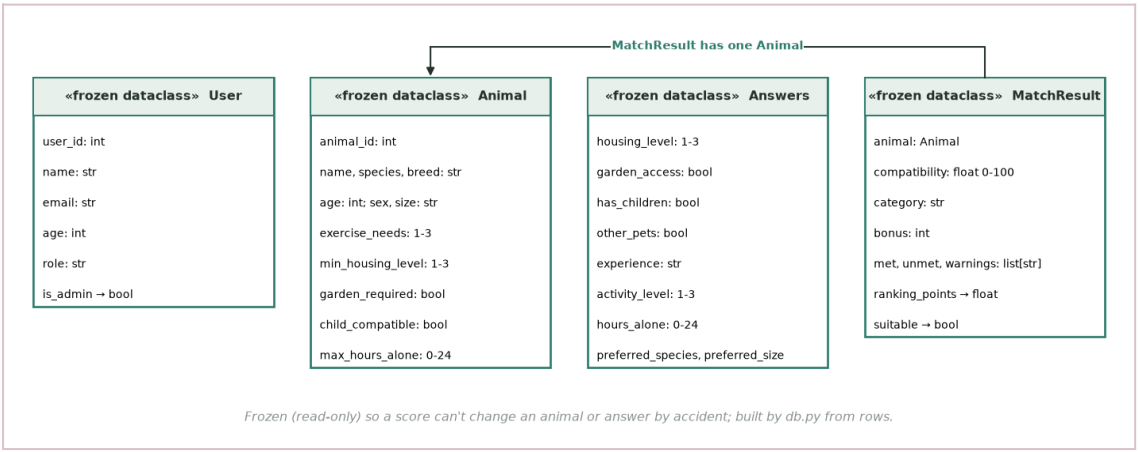


Figure D3: Data classes used between the layers (Animal, Answers, MatchResult, User)

2.2 Database design

Database structure

Tables	Purpose
Users	Stores registered user and administrator account details
Animals	Stores user profiles, care needs and matching requirements
Questionnaire responses	Stores answers submitted by users
Matchresults	Stores calculated compatibility scores between responses and animals

One user can complete multiple questionnaire responses, each questionnaire can produce multiple compatibility results because it is compared to every available animal. Each animal can therefore appear in multiple match results belonging to different users or questionnaire attempts.

-Users 1 → Many Questionnaire responses

-Questionnaire responses 1 → Many matchresults -animals 1 → Many matchresults

Validation

All user entered data should be validated before it is processed or stored in the SQLite database. Streamlit controls such as dropdowns radio buttons and number inputs will restrict the range of values users can enter. However, the backend will still validate the submitted values because interface controls alone should not be relied on.

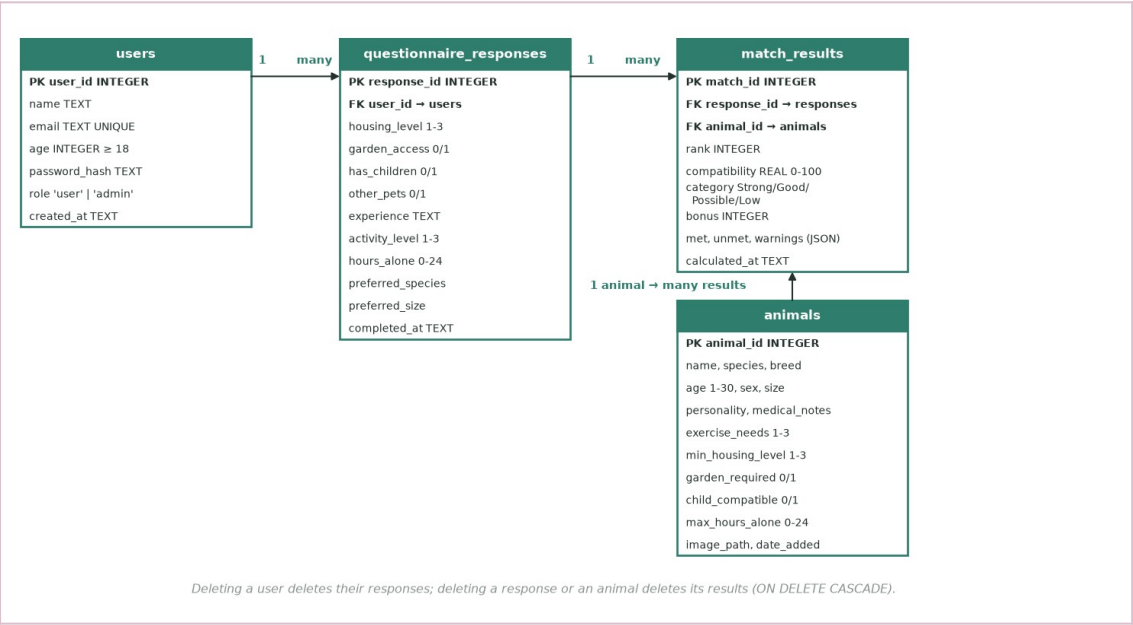


Figure D4: Entity relationship diagram: users 1 to many responses 1 to many match results; animals 1 to many match results

The final table definitions are below. Every rule that validation enforces is repeated as a database constraint (CHECK, NOT NULL, UNIQUE), so even a programming mistake cannot store an impossible value. This is tested on purpose (D59, T2.11).

Table . field	Type	Key / constraint and why
users . user_id	INTEGER	Primary key, auto-increment
users . email	TEXT	NOT NULL, UNIQUE, case-insensitive (so Sam@x.com and sam@x.com are the same account)
users . age	INTEGER	CHECK age ≥ 18 (adopters must be adults)
users . password_hash	TEXT	scrypt hash with a random salt; the password itself is never stored
users . role	TEXT	'user' or 'admin'; decides what each person can see and do
animals . animal_id	INTEGER	Primary key, auto-increment
animals . species / sex / size / status	TEXT	Each CHECKed against its fixed list (Dog, Cat, Rabbit; Male, Female, Unknown; Small, Medium, Large; Available, Reserved, Adopted)
animals . age	INTEGER	CHECK 1 to 30
animals . exercise_needs, min_housing_level	INTEGER	CHECK 1 to 3 (the same scale the adopter answers on)
animals . max_hours_alone	INTEGER	CHECK 0 to 24
animals . garden_required, child_compatible, good_with_pets	INTEGER	CHECK 0 or 1 (Boolean)
animals . experience_needed	INTEGER	CHECK 0 to 2 (first-time owner, some, experienced)
questionnaire_responses . response_id	INTEGER	Primary key; user_id is a foreign key, deleted with its user
questionnaire_responses . (10 answers)	INTEGER / TEXT	Each CHECKed to the same range as the question
match_results . match_id	INTEGER	Primary key; response_id and animal_id are foreign keys (deleted with the response or animal)
match_results . compatibility	REAL	CHECK 0 to 100
match_results . category	TEXT	Strong, Good, Possible or Low
match_results . met, unmet, warnings	TEXT	The reasons, stored so a result can be shown again without recalculating

Changes from the first data dictionary

The table definitions above are the ones built and tested. Where they differ from the first data dictionary (reproduced later in this section), the change and the reason are listed here, so every name in the pseudocode, the code and the tests is the same.

First data dictionary	Final definition	Why
Users . user_role	users . role ('user' or 'admin')	One short name used everywhere (pseudocode D3, `require_admin`)
Users age: 18 or over	Validation 18 to 120; CHECK age ≥ 18	The upper limit catches typing mistakes such as 1999 (T1.4)
Animals age: "cannot be zero"	1 to 30, in validation and as a CHECK	A range can be tested at both ends (T2.3: 0 and 31 refused)
Animals table described as storing user profiles	Stores one record per animal	A slip in the first description
(not in the first table)	animals . status, good_with_pets, experience_needed	Browse shows only Available animals; other pets and experience became scored factors in the design re-check (S21)
Responses . preffered_species	preferred_species, preferred_size; plus other_pets, allergies, experience	Spelling corrected; three questions added (see the final question list on D55)
Match results . total_score, maximum_score (INTEGER)	compatibility REAL 0 to 100, and category	The activity and experience factors can award half their weight, so scores are not whole numbers; the maximum is the constant 90, so it is not stored on every row
unmet_requirements; compatibility_score	met, unmet, warnings; compatibility	Three lists are needed to show met, not met and warnings separately (SC11)
Animals . minimum_housing_level	animals . min_housing_level	The same short name as the code and the CHECK constraint
Variables total_score, maximum_score, animal_scores	earned (a running total), the constant MAX_POINTS = 90, and a list of results	See D6a and D7; the maximum never changes while the program runs, so it is a constant, not a variable
Browse variable sort_option	(removed)	One fixed order, species then name (D29)

Example records

One example row from each of the two main tables, in the form the finished program stores it (the password hash is shortened here; the real salt and hash are 32 and 128 hexadecimal characters).

users field	Example value
user_id	2
name	Sam Lee
email	sam@example.com (always stored in lower case)
age	30
password_hash	scrypt$16384$8$1$9f2c...e1$5b7d...a0 (method, cost settings, salt, hash)
role	user

animals field	Example value
animal_id	1
name / species / breed	Luna / Dog / Labrador cross
age / sex / size	4 / Female / Large
exercise_needs / min_housing_level	3 / 2

animals field	Example value
garden_required / child_compatible / good_with_pets	1 / 1 / 1
max_hours_alone / experience_needed	4 / 1
personality	Luna is an affectionate and energetic dog who enjoys long walks, training games and spending time with people.
medical_notes / image_path / status	(blank) / assets/animals/luna.png / Available

Pseudocode D8: open the database, upgrade an older file, and save safely (R2, R5)

```
FUNCTION connect(path)
    conn ← OPEN path;  TURN ON foreign keys
    TRY
        RUN the caller's statements
        COMMIT                              // every change saved together...
    EXCEPT any error
        ROLLBACK                            // ...or none of them (R2)
        RAISE the error
    FINALLY
        CLOSE conn
ENDFUNCTION

PROCEDURE init_schema(conn)
    CREATE TABLE IF NOT EXISTS for every table
    FOR EACH (table, column, definition) IN NEW_COLUMNS      // a fixed list, never user input
        IF column NOT IN columns of table THEN
            ALTER TABLE table ADD COLUMN column definition    // old files gain new fields (R5)
        ENDIF
    NEXT
ENDPROCEDURE
```

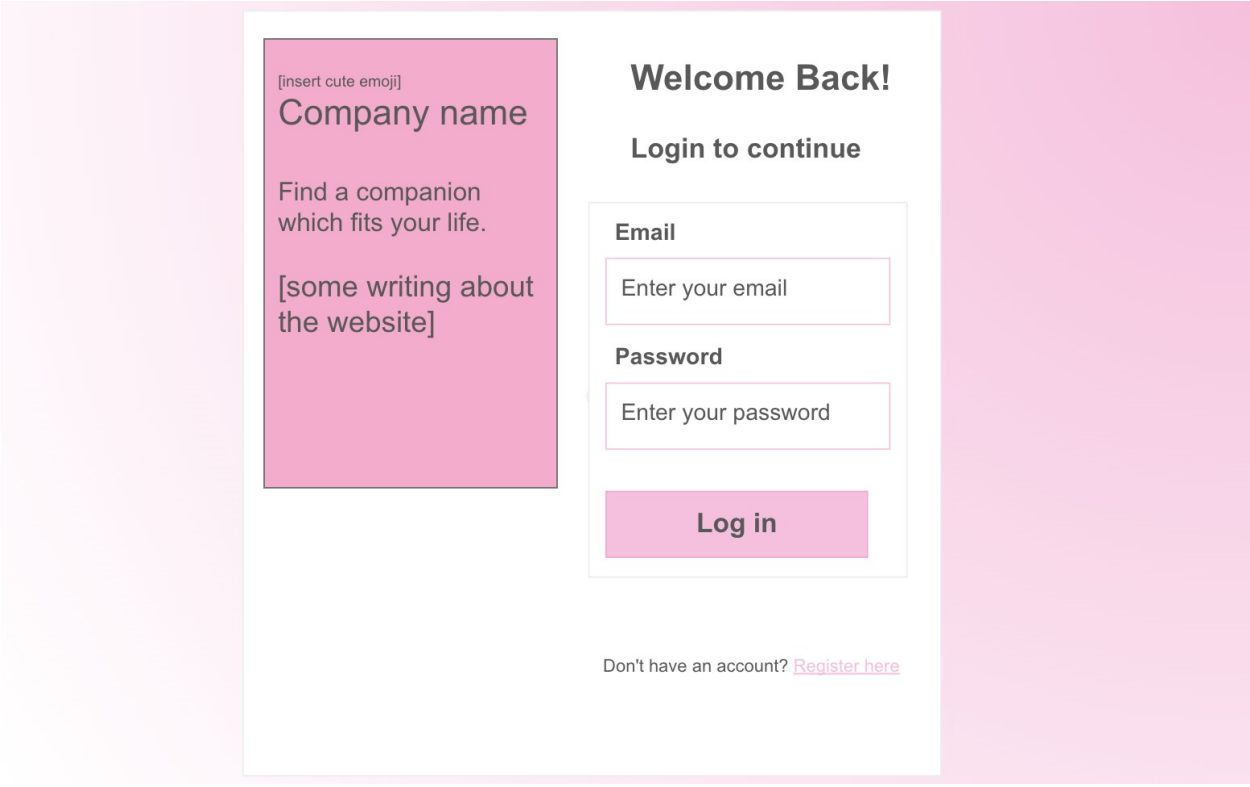
2.3 Component 1: accounts and login

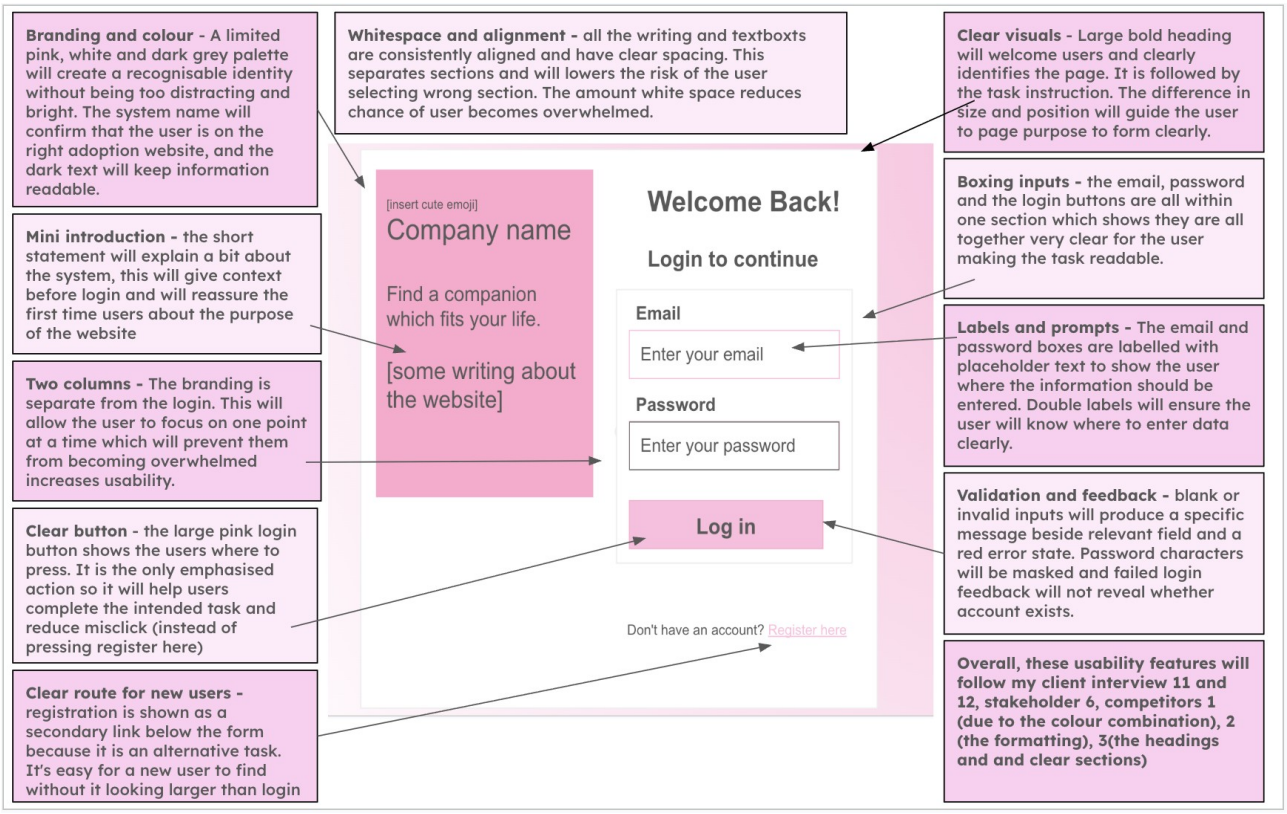
Login + Adminitration

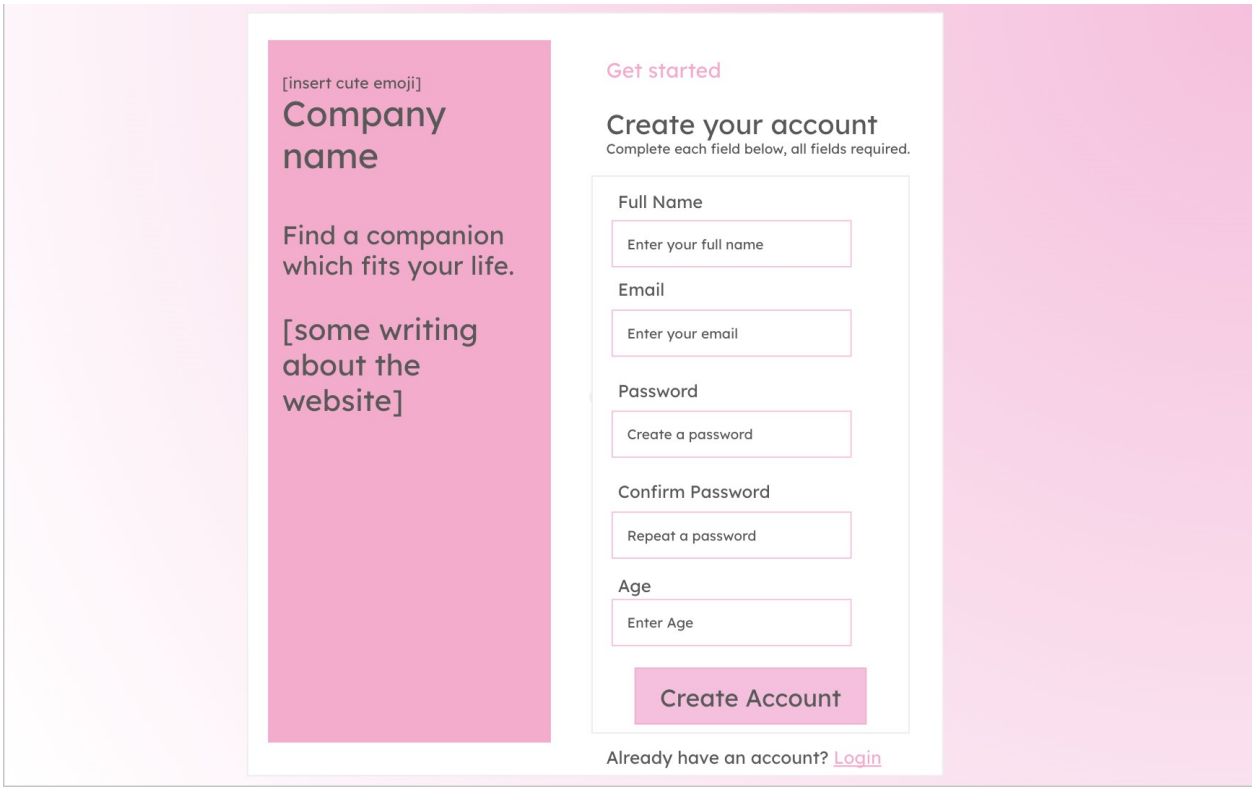
Login and Registration

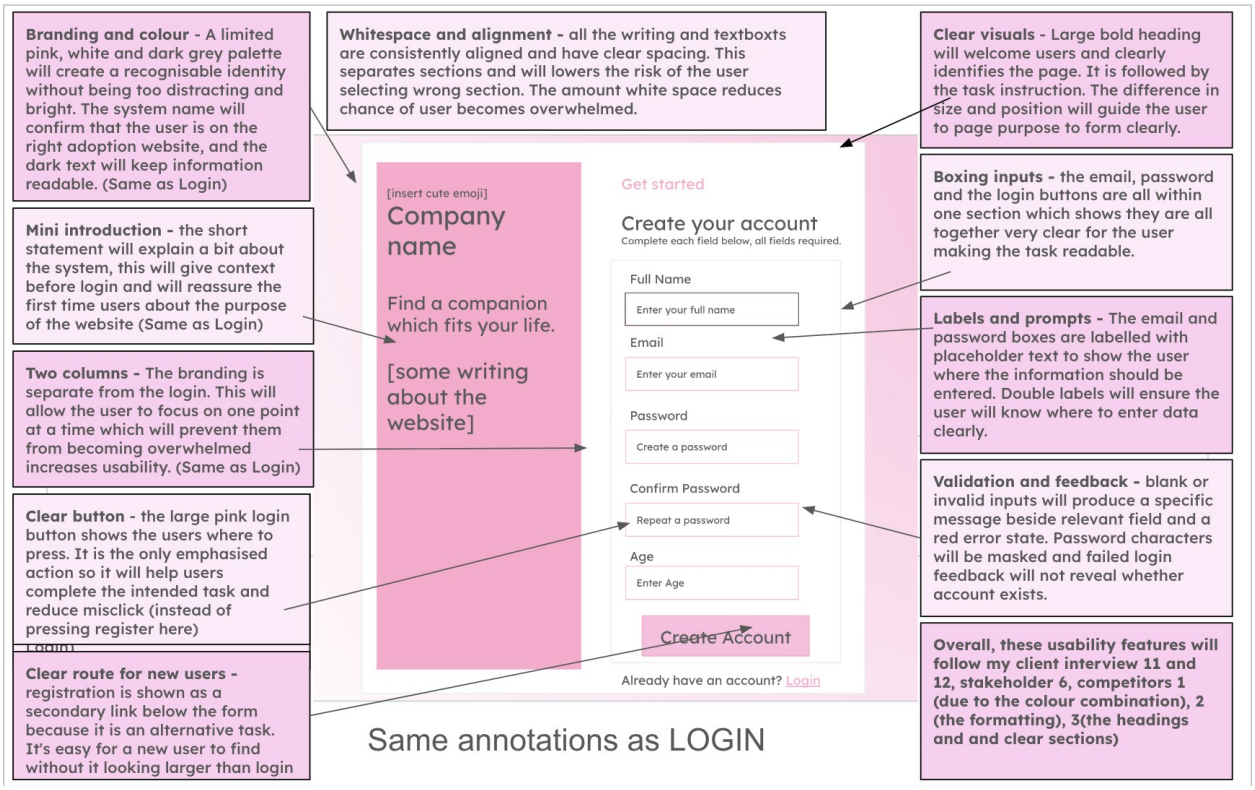
User/Admin Authentication	Will allow users to register and existing users to login, and allow admins to access their page safely and securely. Registration will collect the users name, email, age, address and password. The interface should provide clear feedback when details are missing or forgotten from its validation system. It will also show an error message if login details are incorrect	The system will validate entered details, such as: - Username cannot be blank - Password minimum 8 characters - Required questions cannot be skipped - Age must be a valid number - Duplicate usernames not allowed The passwords will be hashed in order to be kept secure and safe. During login, enter details will be compared against records in the users table. A session will then be created when authentication is

Interface Design

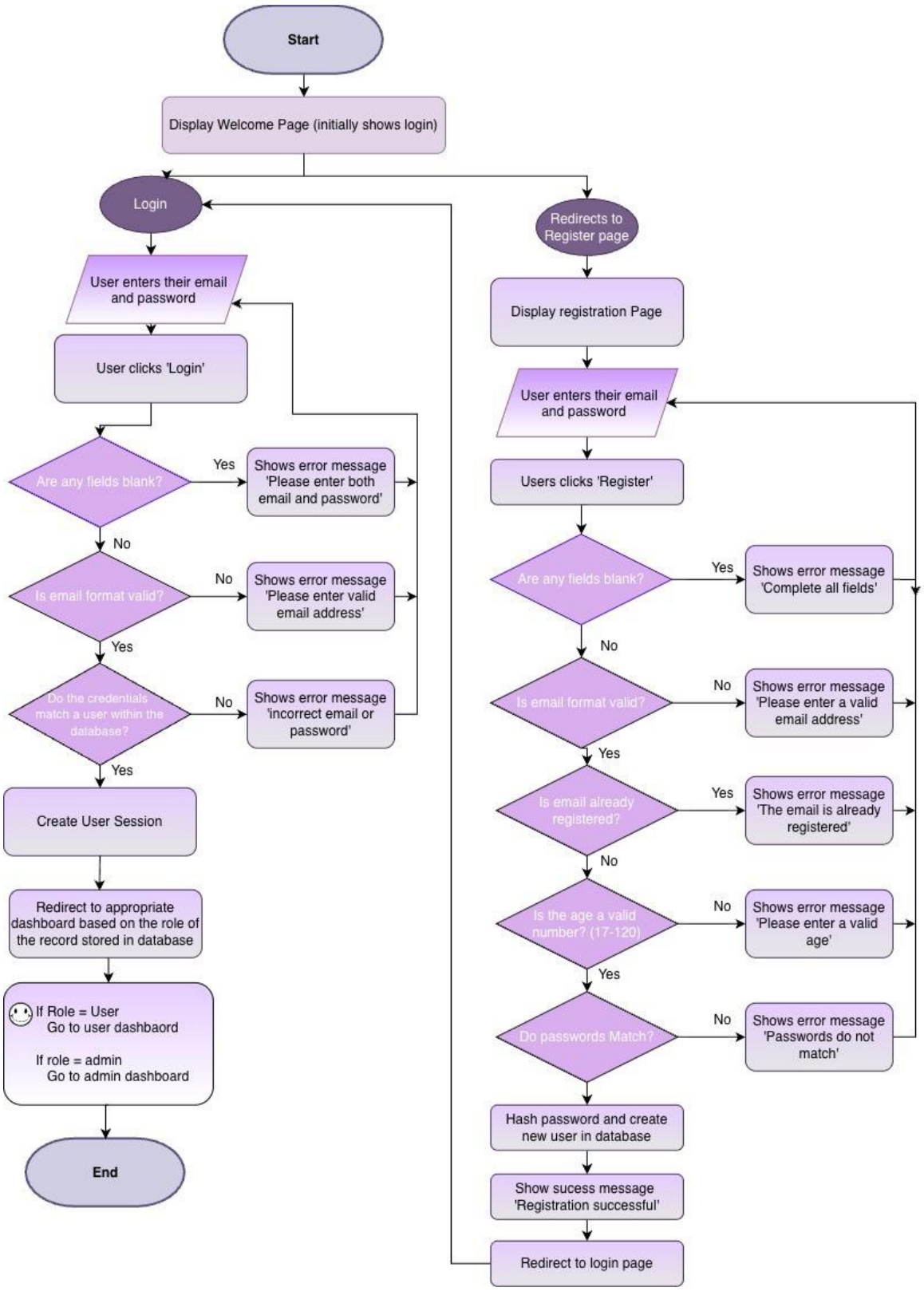



Branding and colour - A limited pink, white and dark grey palette will create a recognisable identity without being too distracting and bright. The system name will confirm that the user is on the right adoption website, and the dark text will keep information readable.
Whitespace and alignment - all the writing and textboxts are consistently aligned and have clear spacing. This separates sections and will lowers the risk of the user selecting wrong section. The amount white space reduces chance of user becomes overwhelmed.
Clear visuals - Large bold heading will welcome users and clearly identifies the page. It is followed by the task instruction. The difference in size and position will guide the user to page purpose to form clearly.
Mini introduction - the short statement will explain a bit about the system, this will give context before login and will reassure the first time users about the purpose of the website
Boxing inputs - the email, password and the login buttons are all within one section which shows they are all together very clear for the user making the task readable.
Two columns - The branding is separate from the login. This will allow the user to focus on one point at a time which will prevent them from becoming overwhelmed increases usability.
Labels and prompts - The email and password boxes are labelled with placeholder text to show the user where the information should be entered. Double labels will ensure the user will know where to enter data clearly.
Clear button - the large pink login button shows the users where to press. It is the only emphasised action so it will help users complete the intended task and reduce misclick (instead of pressing register here)
Validation and feedback - blank or invalid inputs will produce a specific message beside relevant field and a red error state. Password characters will be masked and failed login feedback will not reveal whether account exists.
Clear route for new users - registration is shown as a secondary link below the form because it is an alternative task. It's easy for a new user to find without it looking larger than login
Overall, these usability features will follow my client interview 11 and 12, stakeholder 6, competitors 1 (due to the colour combination), 2 (the formatting), 3(the headings and and clear sections)
[insert cute emoji]
Company name
Find a companion which fits your life.
[some writing about the website]
Welcome Back!
Login to continue
Email
Enter your email
Password
Enter your password
Log in
Don't have an account? Register here


[insert cute emoji]
Company name
Find a companion which fits your life.
[some writing about the website]
Get started
Create your account
Complete each field below, all fields required.
Full Name
Enter your full name
Email
Enter your email
Password
Create a password
Confirm Password
Repeat a password
Age
Enter Age
Create Account
Already have an account? Login



Algorithm (Flow chart, pseudocode)


Start
Display Welcome Page (initially shows login)
Login
User enters their email and password
User clicks 'Login'
Are any fields blank?
Yes
Shows error message 'Please enter both email and password'
No
Is email format valid?
No
Shows error message 'Please enter valid email address'
Yes
Do the credentials match a user within the database?
No
Shows error message 'incorrect email or password'
Yes
Create User Session
Redirect to appropriate dashboard based on the role of the record stored in database
If Role = User
Go to user dashbaord
If role = admin
Go to admin dashboard
End
Redirects to Register page
Display registration Page
User enters their email and password
Users clicks 'Register'
Are any fields blank?
Yes
Shows error message 'Complete all fields'
No
Is email format valid?
No
Shows error message 'Please enter a valid email address'
Yes
Is email already registered?
Yes
Shows error message 'The email is already registered'
No
Is the age a valid number? (17-120)
No
Shows error message 'Please enter a valid age'
Yes
Do passwords Match?
No
Shows error message 'Passwords do not match'
Hash password and create new user in database
Show sucess message 'Registration successful'
Redirect to login page

Pseudocode

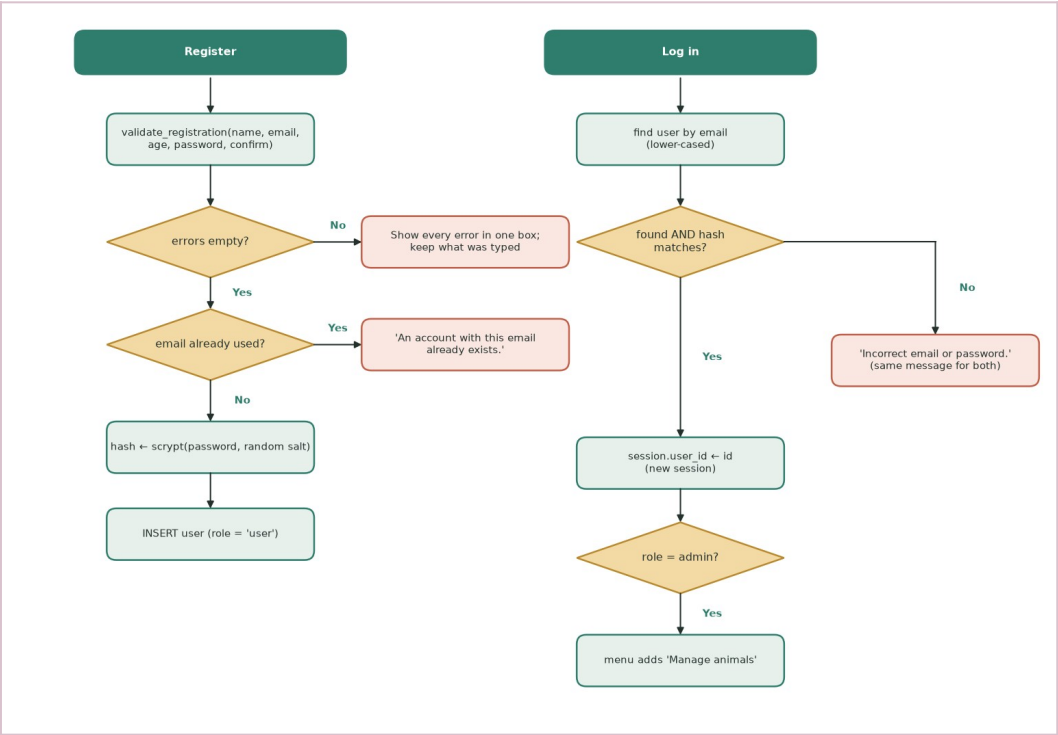


Figure D5: Flowchart: login and registration

Pseudocode: registration and login

Pseudocode D1: register a new adopter

```
FUNCTION register(name, email, age, password, confirm)
    errors ← validate_registration(name, email, age, password, confirm)
    IF errors is not empty THEN
        RETURN (null, errors)               // each message shown under its own box
    ENDIF
    IF a user with LOWER(email) already exists THEN
        RETURN (null, {email: "An account with this email already exists."})
    ENDIF
    salt ← 16 random bytes
    hash ← scrypt(password, salt)
    INSERT INTO users (name, email, age, hash, role = 'user', now)
    RETURN (new user, no errors)             // signed straight in
ENDFUNCTION
```

Pseudocode D2: log in (the same message for every failure)

```
FUNCTION login(email, password)
    IF email is blank THEN RETURN "Enter your email address."
    IF password is blank THEN RETURN "Enter your password."
    user ← SELECT * FROM users WHERE email = LOWER(TRIM(email))
    IF user is null OR verify(password, user.password_hash) = false THEN
        RETURN "Incorrect email or password."  // never says which part was wrong
    ENDIF
    session.user_id ← user.user_id
    IF user.role = 'admin' THEN open staff dashboard ELSE open home page
ENDFUNCTION
```

Database (Table structure)

Field	Data type	Key/validation
user_id	INTEGER	Primary key which will auto increment
name	TEXT	required
email	TEXT	Required and unique
age	INTEGER	required , valid range
password_hash	TEXT	required
user_role	TEXT	‘User’ or ‘admin’
date_created	DATE	required

Variables

Variable Name	Data Type	Purpose	Justification	E.g or Validation
user_name	String	Stores the name entered during registration	This will allow the system to identify and personalise the	Cannot be blank
			alphabetic characters and may	
email	String	Will store users email address	It will be used as a unique identifier for logging in and contacting the user. It will contain	Must follow a valid email format and be unique
age	integer	Stores the users age	It will be used to ensure the user meets any minimum age requirements. An	Must be a valid positive number.

password	string	Temporarily stores the password entered by the password	It is needed to authenticate the user securely. Passwords usually contain letters,	Minimum of eight characters.
confirm_passwo rd	string	Stores the repeated password entered during registration	This allows the system to check that the user has mistyped their password before	It will have to match password
password_hash	string	Stores the hashed version of the password	Passwords should not be stored as plain text. Storing the hash reduces the risk of	Generated using a secure hashing function
login_email	string	Stores the email entered on the login page	The variable is required so that the system can search for the	Cannot be blank
login_password	string	Temporarily stores the password entered during login	The entered password will be checked against the stored password hash in	Cannot be blank
user_id	integer	Uniquely	An integer is	It will be retrieved
		identifies the logged in user	database primary key, it will allow records such as questionnaires and	from the database after login
user_role	String	Stores whether the account is an administrator or a user	This variable is required for access control, so that normal users cannot access the administrator pages Controls access to the administrator	Will be stored as ‘user’ or ‘admin’

error_message	string	Stores the error message which will appear when the input or login details are invalid	My system will provide clear feedback and help the user understand what has gone wrong and how to correct	'Incorrect email or password, please retry'
is_authenticated	booleen	Records whether login attempt was successful	A boolean data type is suitable as authentication only	True or False

Validation

1.Name

- Presence check: Field cannot be empty or contain only spaces
- Length check: name must be between 2 and 50 characters
- Whitespace check: spaces at beginning and end will be removed

2.Email address

- Presence check: Field cannot be empty or contain only spaces
- Format check: value must contain a @ and domain
- Uniqueness check: a database query will check

3.Age

- Presence check: Field cannot be empty or contain only spaces Type check: data must be a whole number

4.Password

- Presence check: Field cannot be empty or contain only spaces
- Length check: password must be more than 8 characters
- Confirmation check: the confirmation field must exactly match original
- Security processing: the valid password will be hashed before it is inserted database and the original password will never be stored.
- The password will be checked against the stored password hash rather as plain text.

Registration password into the Login than compared

Registration outcome

- The account will be created if every field passes validation, individual messages will be shown if one or more fields are invalid.

Session check

- A session will only be created after successful authentication. Invalid information will leave users on the login page and will not allow them to access the user or admin page.

> **Final names and rules for this component**
>
> The variables and validation lists above are from the first design. In the build the login uses the email address (there is no username and no postal address), `user_role` is called `role` (D7), a password needs **at least** 8 characters rather than "more than 8", and a person's age must be 18 to 120. The exact rules are V1 to V5 in the validation table on D57.

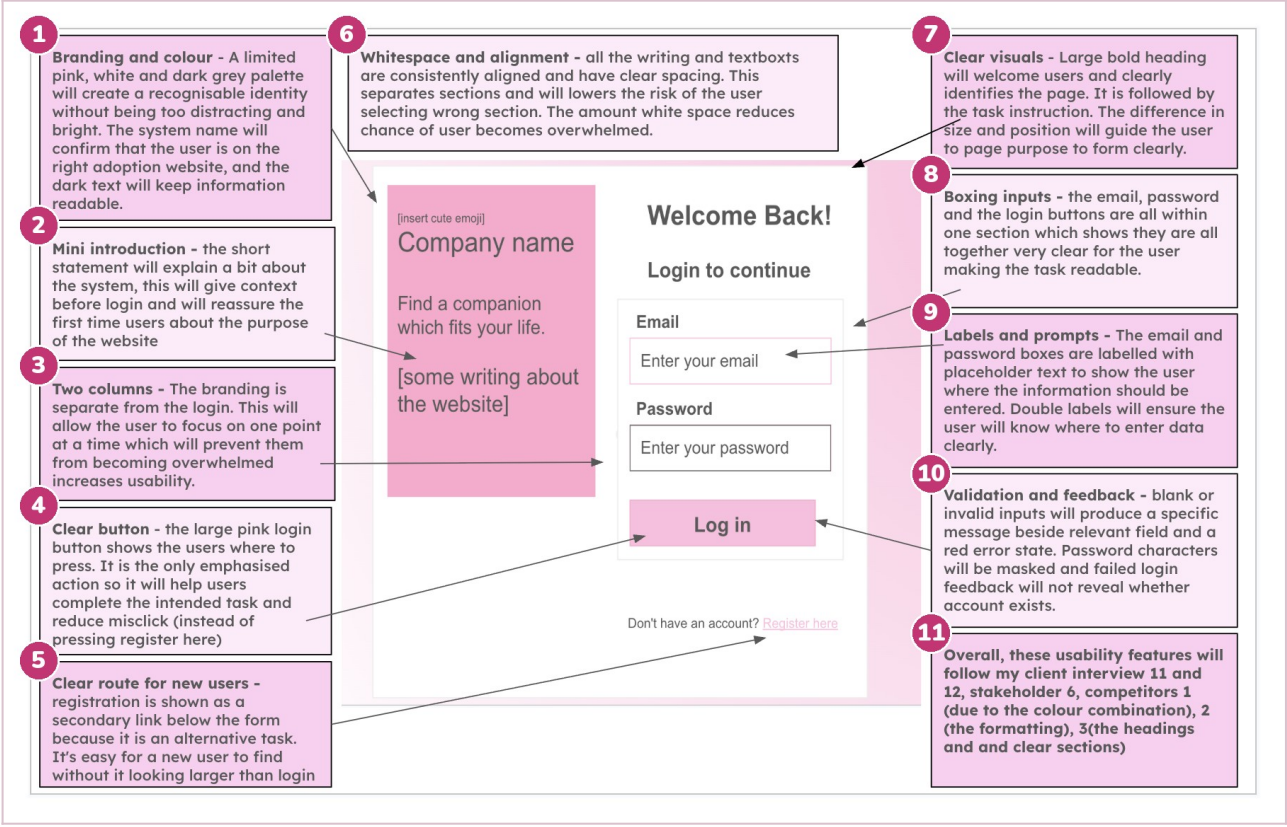


Figure D6: Annotated design: login screen (her layout and colours)

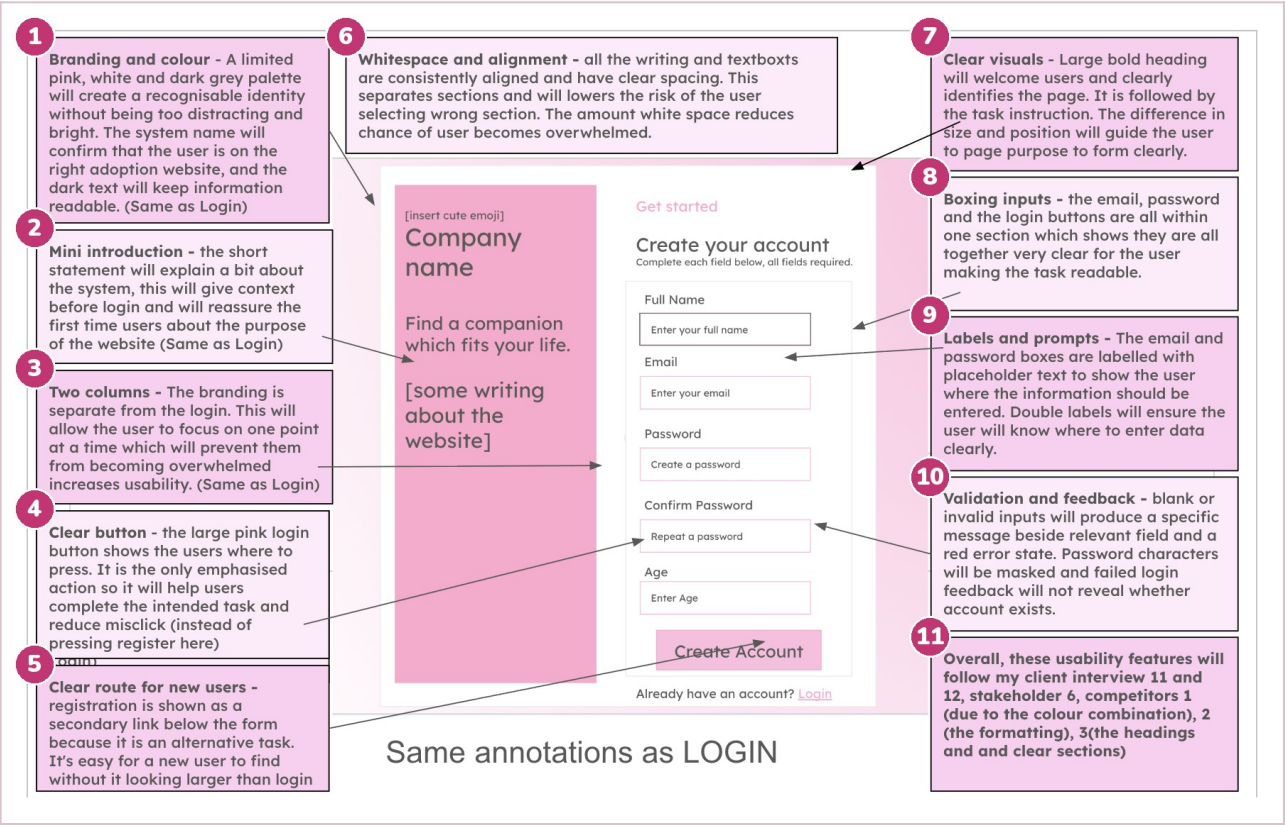


Figure D7: Annotated design: registration with a message under each field

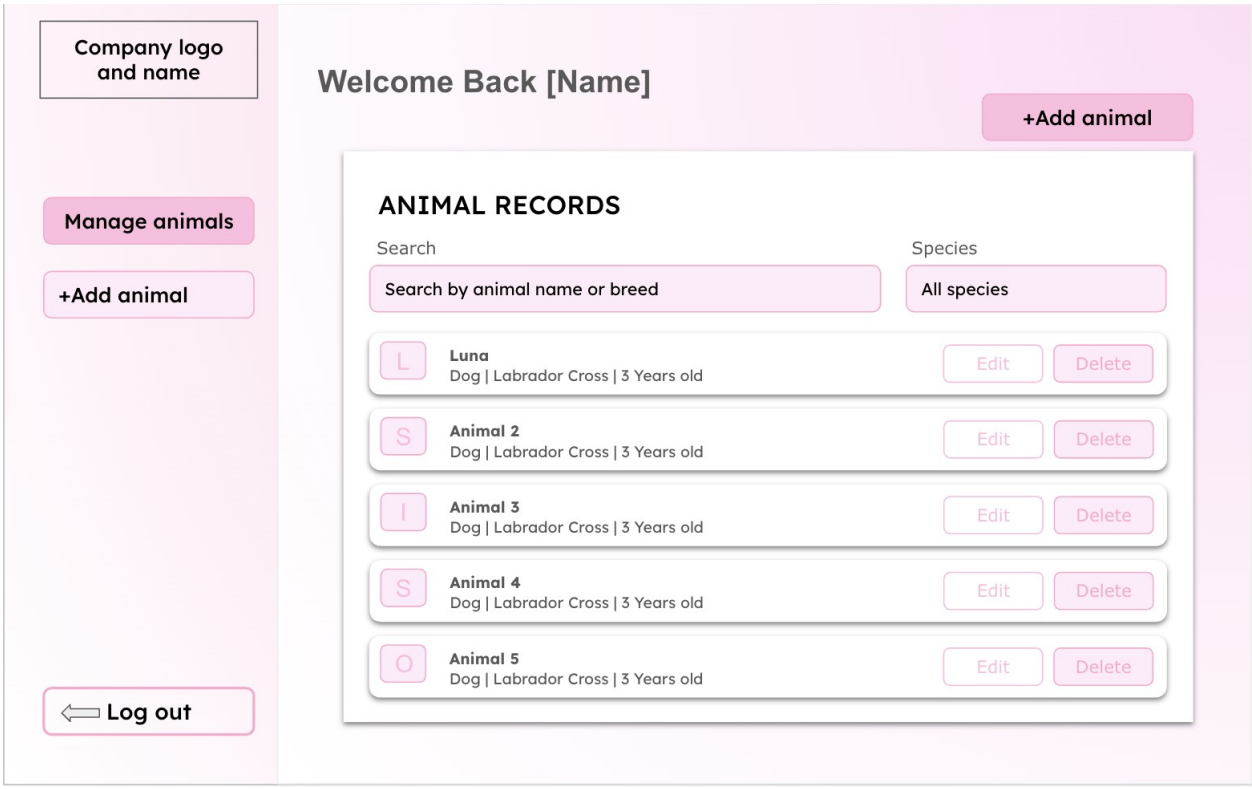
2.4 Component 2: staff animal management

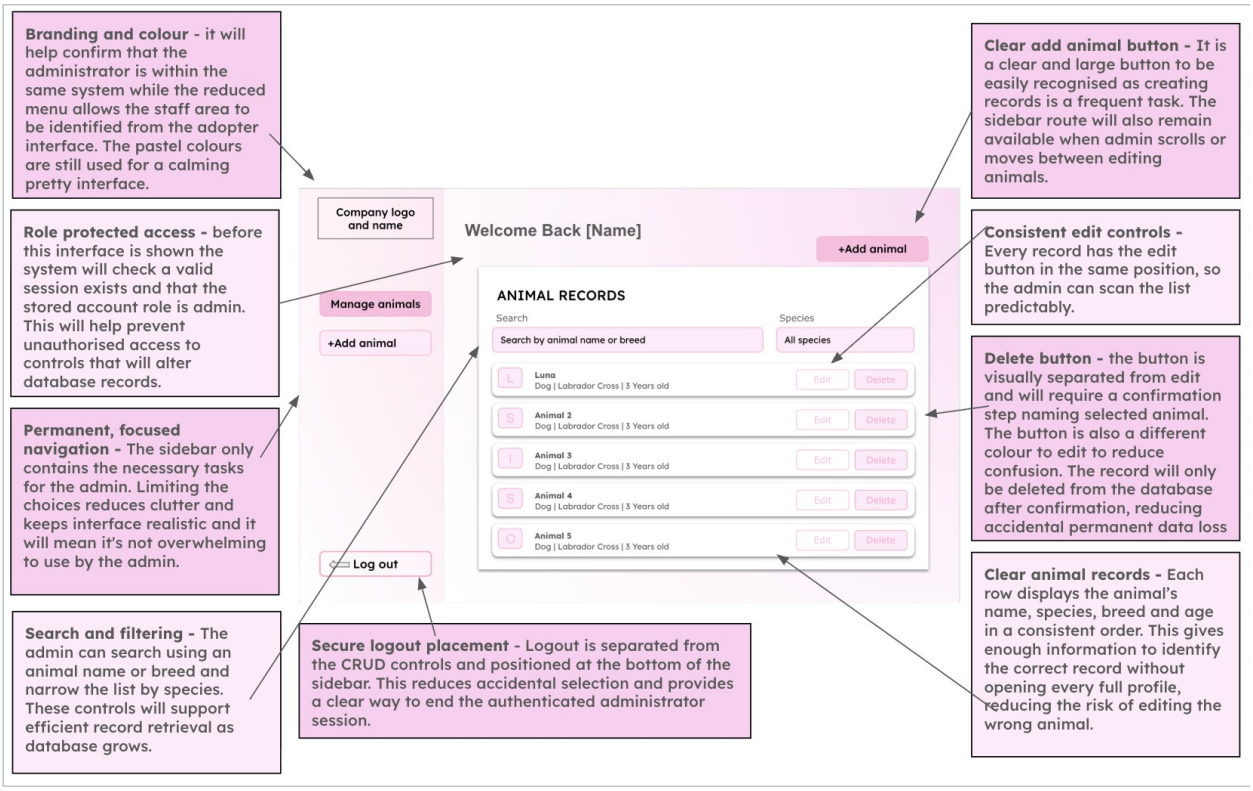
Admin Page

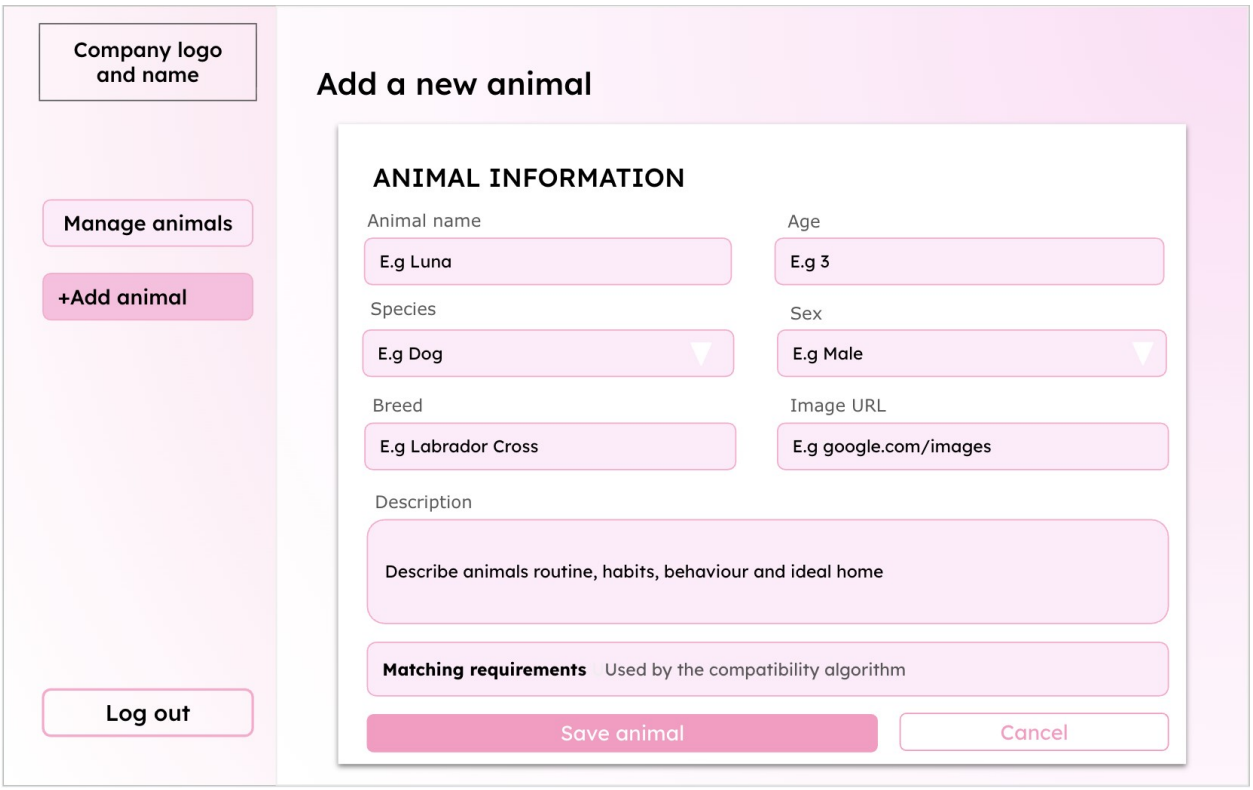
Admin Page

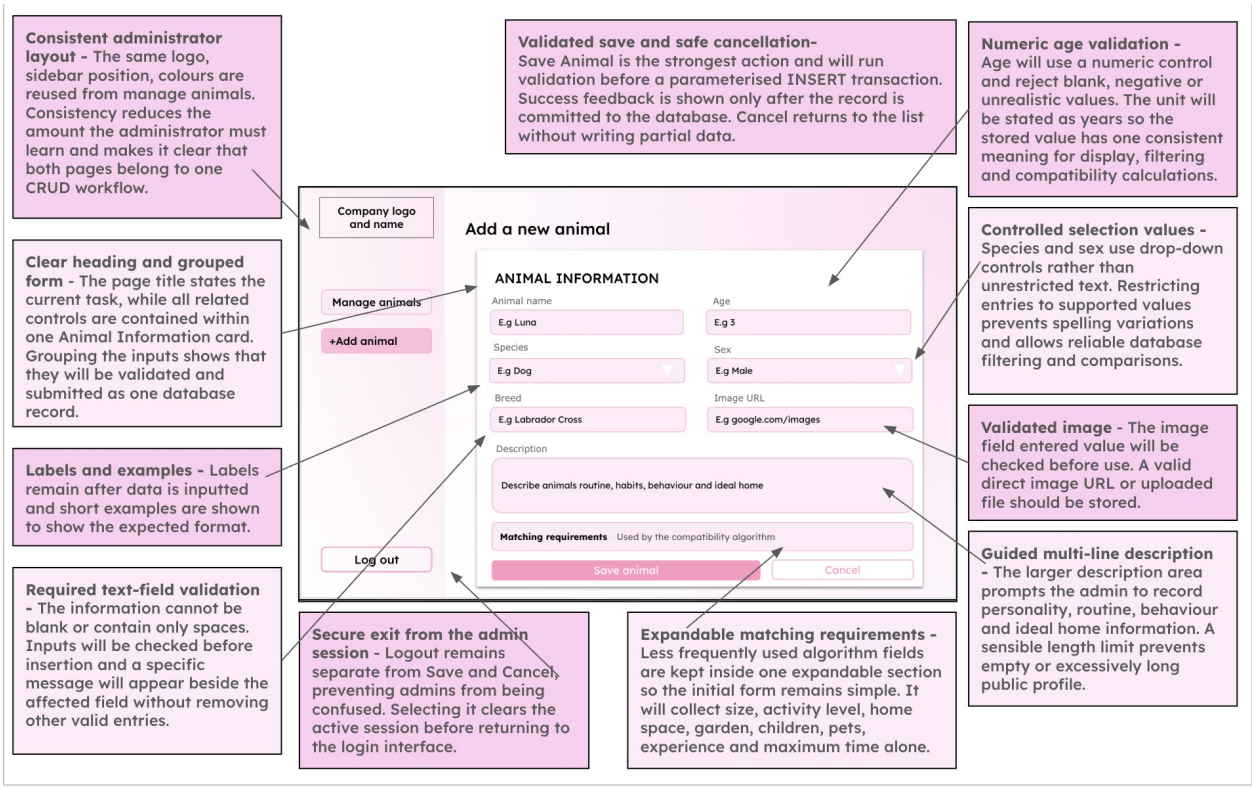
Administrator Menu	Provides staff with a dashboard to manage existing animals and their records. Admins will be clearly able to add new animals, edit existing details and remove records. To add animals it will store: Name, species, age, breed, sex, personality and exercise needs, children compatible, garden required,	The system will check that the loginned in account has the role before access is allowed. Parameterised SQL commands will insert, update and delete animal records. All inputs will be validated before changes are made to the database. Will also include a logout function, to

Interface




Branding and colour - it will help confirm that the administrator is within the same system while the reduced menu allows the staff area to be identified from the adopter interface. The pastel colours are still used for a calming pretty interface.
Role protected access - before this interface is shown the system will check a valid session exists and that the stored account role is admin. This will help prevent unauthorised access to controls that will alter database records.
Permanent, focused navigation - The sidebar only contains the necessary tasks for the admin. Limiting the choices reduces clutter and keeps interface realistic and it will mean it's not overwhelming to use by the admin.
Search and filtering - The admin can search using an animal name or breed and narrow the list by species. These controls will support efficient record retrieval as database grows.
Company logo and name
Welcome Back [Name]
+Add animal
Manage animals
+Add animal
ANIMAL RECORDS
Search
Search by animal name or breed
Species
All species
Luna
Dog | Labrador Cross | 3 Years old
Animal 2
Dog | Labrador Cross | 3 Years old
Animal 3
Dog | Labrador Cross | 3 Years old
Animal 4
Dog | Labrador Cross | 3 Years old
Animal 5
Dog | Labrador Cross | 3 Years old
Edit
Delete
Log out
Secure logout placement - Logout is separated from the CRUD controls and positioned at the bottom of the sidebar. This reduces accidental selection and provides a clear way to end the authenticated administrator session.
Clear add animal button - It is a clear and large button to be easily recognised as creating records is a frequent task. The sidebar route will also remain available when admin scrolls or moves between editing animals.
Consistent edit controls - Every record has the edit button in the same position, so the admin can scan the list predictably.
Delete button - the button is visually separated from edit and will require a confirmation step naming selected animal. The button is also a different colour to edit to reduce confusion. The record will only be deleted from the database after confirmation, reducing accidental permanent data loss
Clear animal records - Each row displays the animal's name, species, breed and age in a consistent order. This gives enough information to identify the correct record without opening every full profile, reducing the risk of editing the wrong animal.


Company logo and name
Manage animals
+Add animal
Log out
Add a new animal
ANIMAL INFORMATION
Animal name
E.g Luna
Age
E.g 3
Species
E.g Dog
Sex
E.g Male
Breed
E.g Labrador Cross
Image URL
E.g google.com/images
Description
Describe animals routine, habits, behaviour and ideal home
Matching requirements
Used by the compatibility algorithm
Save animal
Cancel

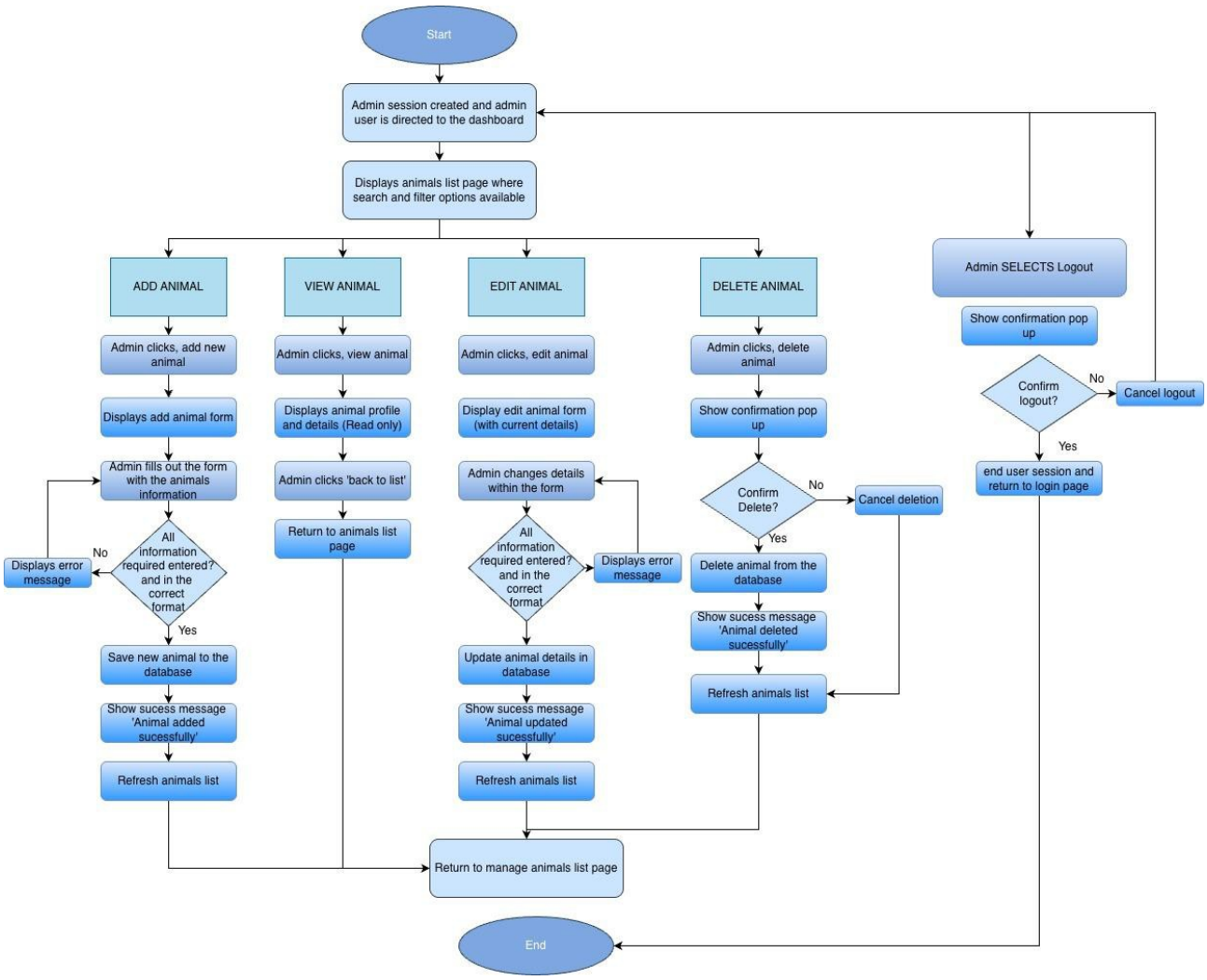


Pseudocode: add, edit and delete an animal

Pseudocode D3: every staff action checks the role first

```
FUNCTION save_animal(current_user, form, animal_id = null)
    IF current_user is null OR current_user.role ≠ 'admin' THEN
        STOP with an access error (require_admin)
    ENDIF
    (values, errors) ← validate_animal(form)      // 14 field checks, below
    IF form.photo is present THEN
        problem ← validate_image(form.photo.name, form.photo.size)
        IF problem ≠ null THEN errors.image ← problem
    ENDIF
    IF errors is not empty THEN RETURN errors          // nothing is saved
    IF animal_id = null THEN
        INSERT INTO animals VALUES (values)            // parameterised query
    ELSE
        rows ← UPDATE animals SET values WHERE animal_id = ?
        IF rows = 0 THEN RETURN "That animal no longer exists."
    ENDIF
    RETURN "Saved"
ENDFUNCTION

FUNCTION delete_animal(current_user, animal_id, confirmed)
    IF current_user.role ≠ 'admin' THEN STOP with an access error (require_admin)
    IF confirmed = false THEN RETURN "Tick the box to confirm."
    DELETE FROM animals WHERE animal_id = ?            // its stored match results go too
ENDFUNCTION
```



Animal Table

Field	Data type	Key/validation
animal_id	INTEGER	Primary key, autoincrement
name	TEXT	required
species	TEXT	Required - dog, cat or rabbit
breed	TEXT	Required
age	INTEGER	Required, cannot be zero
sex	TEXT	required- male, female or
size	TEXT	Required - small medium or

personality	TEXT	required
Exercise_needs	INTEGER	Required - 1 to 3
minimum_housing_level	INTEGER	Required - 1 to 3
garden_required	BOOLEAN	Required 0 or 1
child_compatible	BOOLEAN	Required 0 or 1
max_hours_left_alone	INTEGER	required
Medical_notes	TEXT	optional
image	TEXT	Required, an image path
date_added	DATE	required

Variables

Variable Name	Data Type	Purpose	Justification	Validation or e.g
animal_id	INTEGER	Uniquely identifies	A unique integer	Automatically
		each animal record	stops confusion between animals with similar names and allows the correct to be updated and	generated

name	string	Stores the animals name	The name is required to identify the animal throughout the interface and allows the user to remember the animal with a specific name. A string is suitable as	Can’t be blank
species	string	Stores the type of animal	Species is required for searching, filtering and matching	dog, cat or rabbit
breed	string	Stores the animals breed	Breeds provides users with important information and may help describe likely	Can't be blank or might be ‘unknown’
age	INTEGER	Stores the animals age	Age affects care needs. An integer is suitable as it allows the value to be validated sorted	cannot be zero
sex	string	Stores the animals sex	This is relevant profile information and may be considered by the	male, female
size	string	Stores the animals	Size affects space	small medium or
		size and is used in filtering	requirements and is needed by the filtering and compatibility	
personality	string	Stores a description of the animals behaviour and personality	A text field will allow the admin to explain the animals personality and background in more detail to help attract possible	Cant be blank but a character limit may be applied

Exercise_needs	INTEGER	Stores whether the animal has low, medium or high exercise needs	This can be compared with the adopters lifestyle to produce a more appropriate compatibility score. An integer is suitability for	1 to 3 (low medium or high)
minimum_housing_level	INTEGER	Stores the amount of living space required	This can be compared with adopters lifestyle to aid the algorithm when creating the	1 to 3
garden_required	boolean	Records whether the animal requires access to a garden	This may act as a compulsory suitability rule. A boolean value is useful as it is	True or false
child_compatible	boolean	Records whether the animal is suitable for a household with children	There are only two options which make boolean values suitable. This is very important for the safety and welfare	True or false
max_hours_left_alone	INTEGER	Stores value for which the animal can be left home alone for, and representing if more independent or clingy	This is important if a user works long hours and the animal requires a lot of attention it may cause neglect of the animal which is a key issue	Cant be blank
Medical_notes	string/list	Stores any medical conditions, treatments or special requirements	Potential adopters need to understand any medical needs before making a decision. A string will allow multiple to be described.	May be blank or ‘N/A’

image	string	Stores the location of the animals image	This allows the correct photo to be displayed without storing the full image directly in	Must use an accepted file extension
form_action	string	Stores the operation the admin is performing	The same form may be used for adding and editing variables. This variable will	‘Add’ or ‘edit’ or ‘delete’ or ‘view’
delete_confirmation	boolean	Records whether the administrator has confirmed deletion	Deletion is permanent so confirmation reduces risk of animal being removed accidentally and	True or false
record_found	boolean	Records whether the database has retrieved the animal record	This prevents the program from attempting to edit or delete a record	True or false
			present in the database	

Validation

1.Animal name

-Presence check: Cannot be empty
-Length check: Between 1 and 50 characters
-Format check: Letters, spaces, apostrophes,

allowed

2.Species

-Presence check: Must be selected.
-Lookup check: Must be Dog, Cat or Rabbit.
-A drop-down menu will restrict the choices.

3.Breed

-Presence check: Cannot be empty
-Length check: For example, between 2 and 50
-Format check: Must not consist entirely of

4.Age

-Presence check: Must be provided

-Type check: Must be a whole number
-Range check: Cannot be negative and must be age

5.Sex

-Lookup check: Must be Male, Female or
-A drop-down menu prevents invalid values

6.Size

-Lookup check: Must be Small, Medium or Large

7.Personality

- Presence check: A description must be provided
- Length check: For example, between 20 and 1,000 characters
- Whitespace check: A value consisting only of spaces will be rejected

8.Exercise need

-Presence check: An option must be selected
-Range check: Must be between 1 and 3

9.Minimum housing requirement

- Presence check: Must be selected
- Range check: Must be between 1 and 3

10.Maximum hours alone

- Type check: Must be an integer
- Range check: Must be between 0 and 24

11.Garden required

- Type check: Must be stored as a Boolean value 0 or 1

12.Child compatibility

- Type check: Must be stored as a Boolean value 0 or 1

13.Medical notes

- This field may be optional
- Type check: it will have a maximum length
- Text will be stored as plain text rather than interpreted as code

14.Animal image

- Presence check : An image must be uploaded if every profile requires one
- File-type check : Only approved formats such as JPG, JPEG and PNG will be accepted
- File-size check : The file must not exceed the chosen maximum size

- Filename handling : A safe, unique filename will be generated to avoid overwriting existing images

The animal record will only be inserted after all required fields pass validation. The SQL statement will be parameterised and committed only after successful validation. A success message will then confirm that the record has been created.

Final rules for the animal form

The first validation list above is kept as the plan; the built rules are V6 to V14 on D57. The changes: an animal's name is 2 to 50 characters and may contain digits (V6); age is a whole number 1 to 30 (V8); personality is required and at most **500** characters, not 20 to 1,000 (one limit for both text boxes is simpler to explain and test, and 500 characters is about five sentences) (V13); medical notes are optional, also at most 500 (V13); a photo is optional, because a new arrival may not have one yet, and the overview lists animals without one (T2.10); if given it must be PNG or JPG and at most 2 MB (V14). `minimum_housing_level` is called `min_housing_level` (D7).

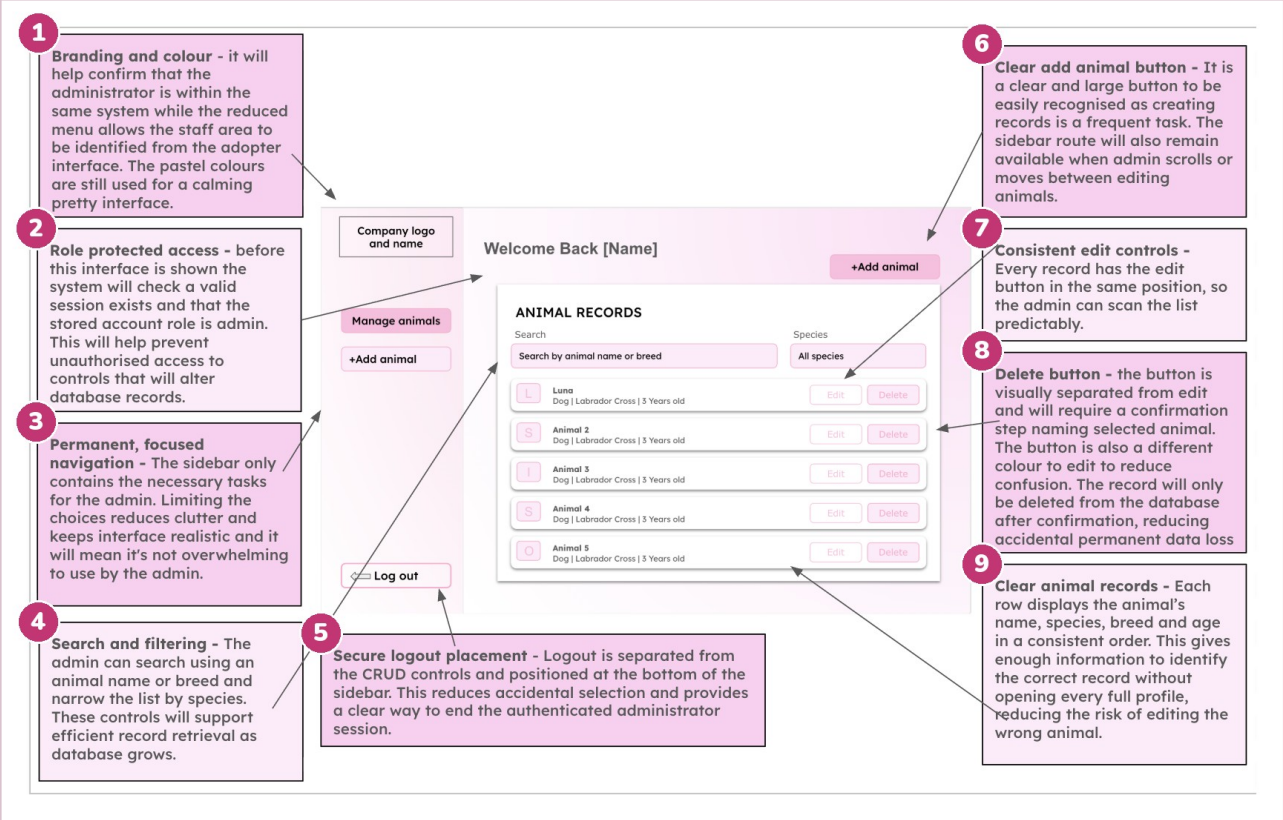


Figure D8: Annotated design: staff table of animals

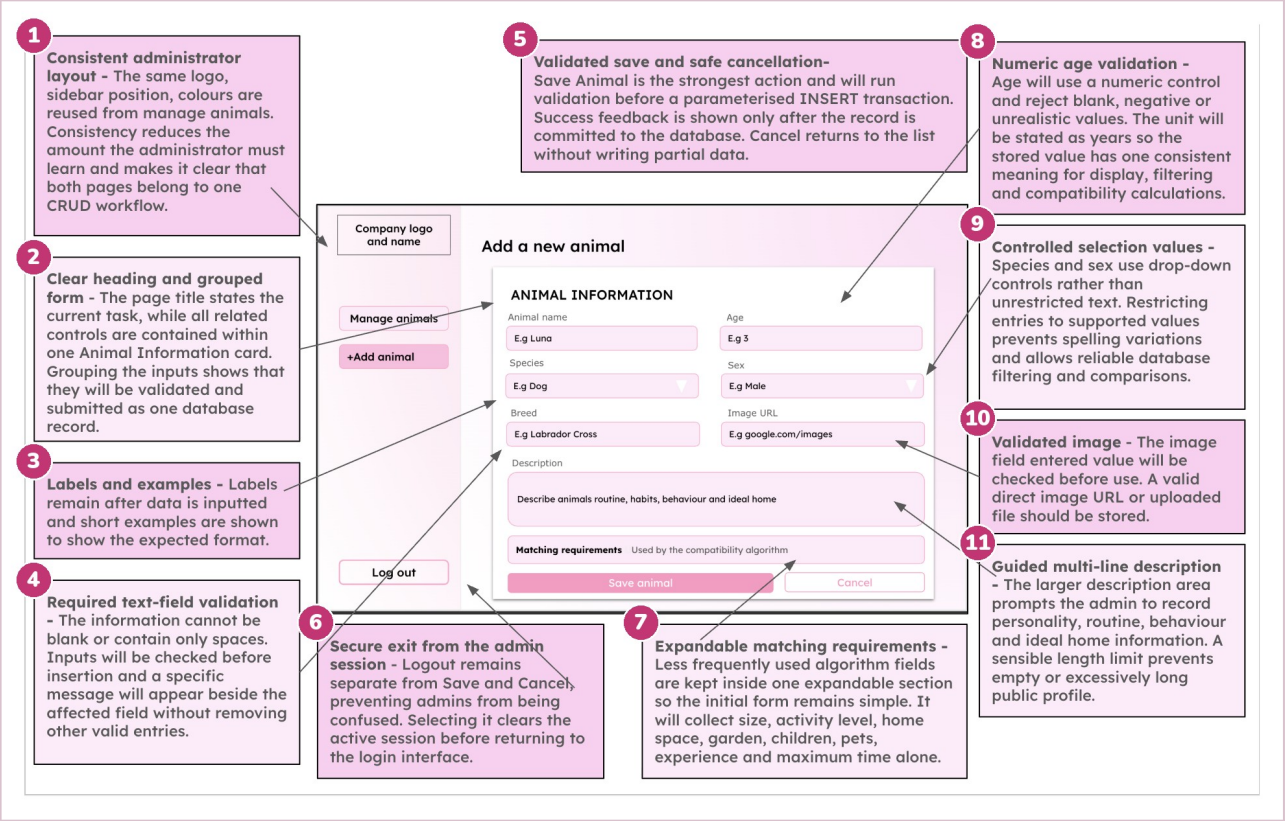


Figure D9: Annotated design: add-animal form with grouped sections

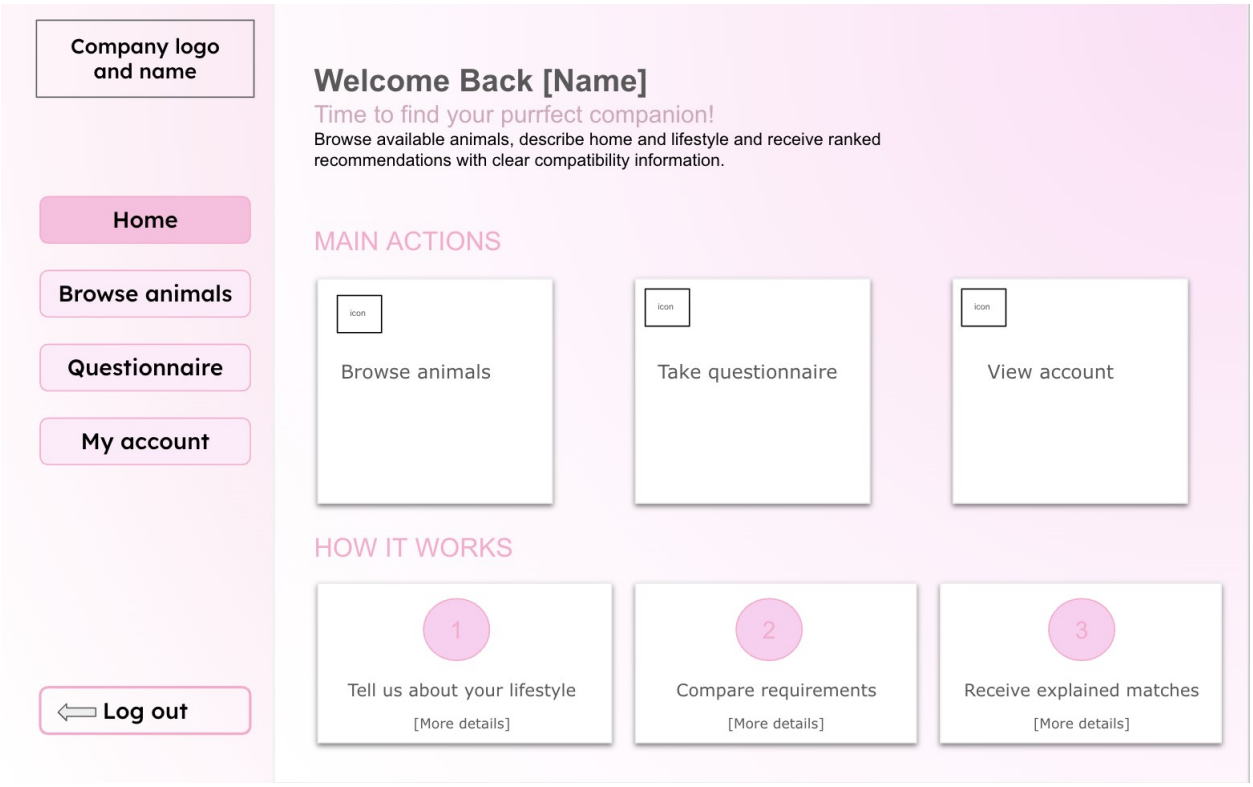
2.5 Component 3: home page and browsing

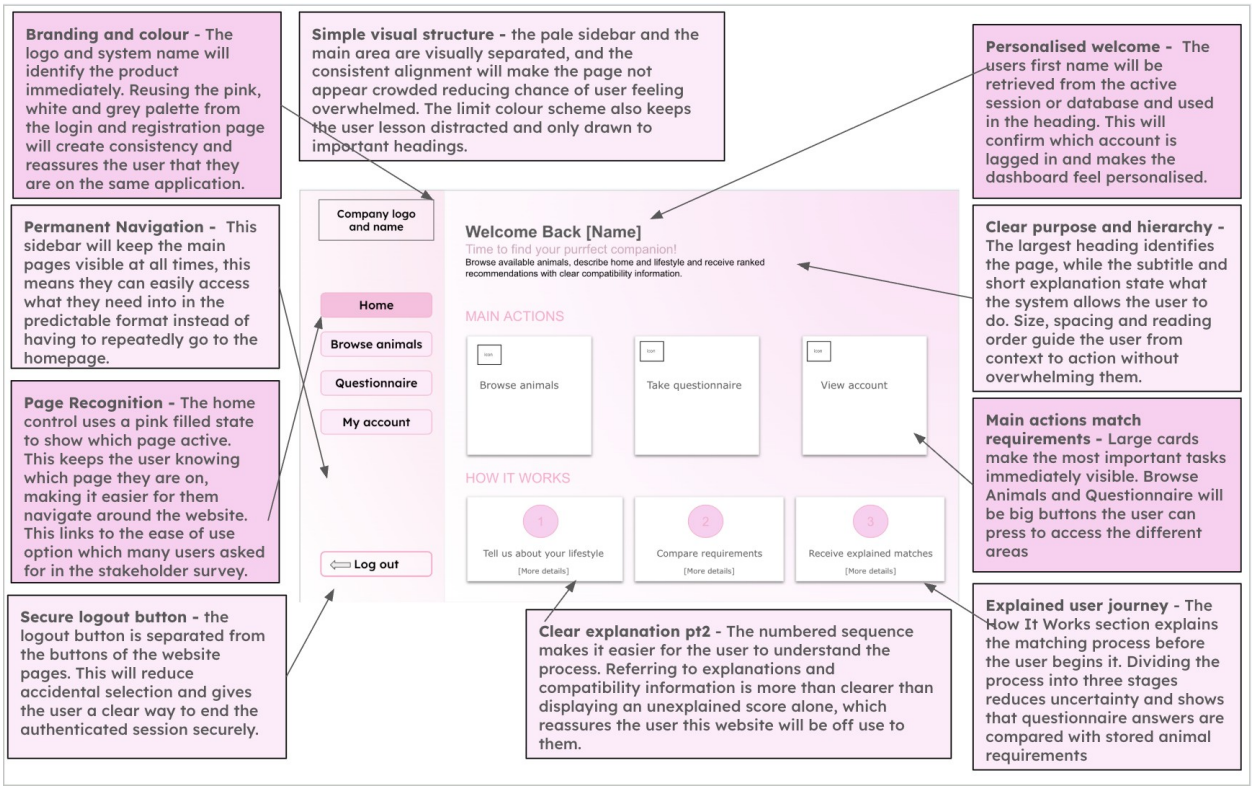
Main Menu/INterface

Main Menu

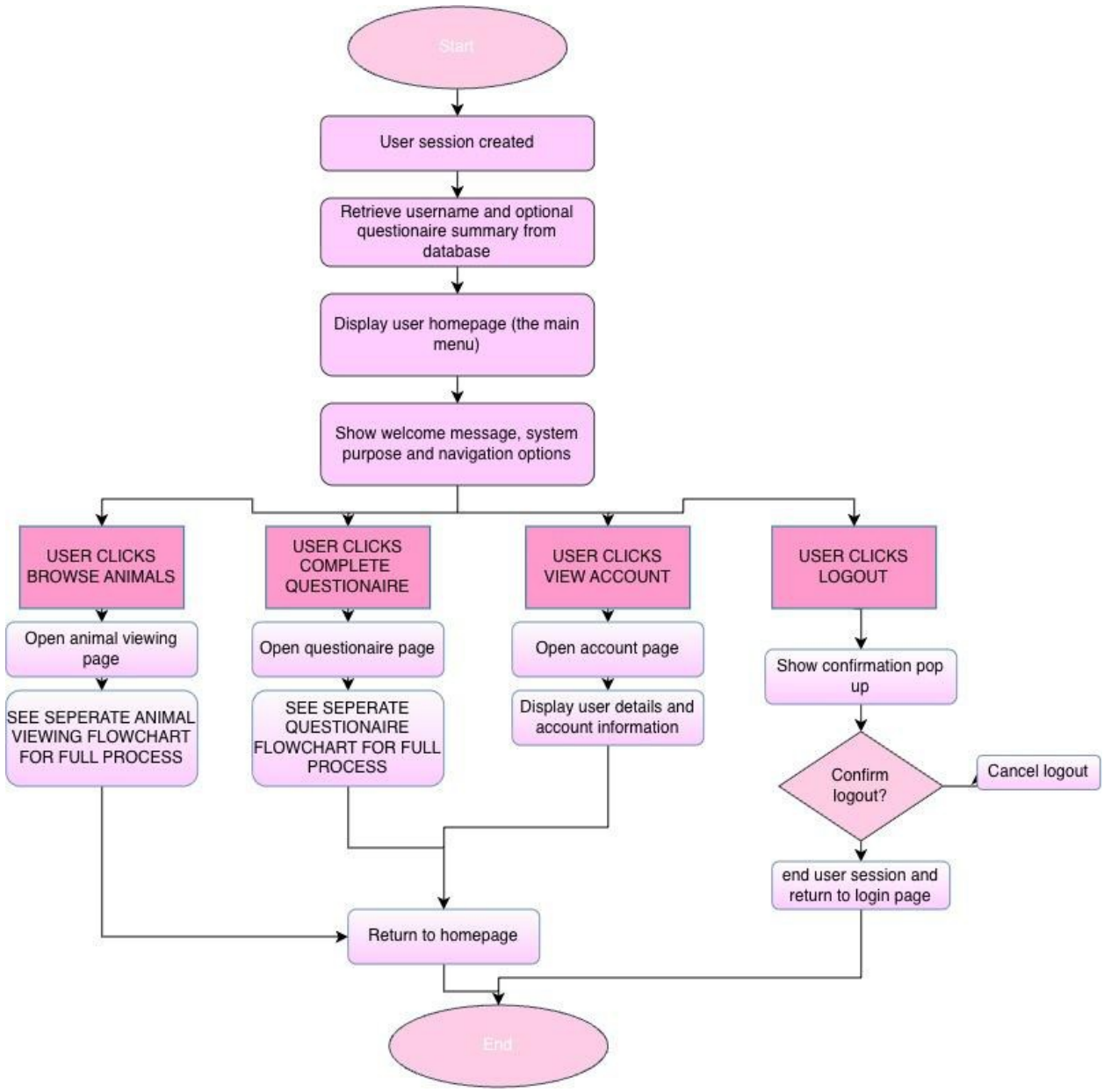
User Homepage/Main Menu	Will provide the main navigation area after the user logs in. It will allow the user to access the questionnaire, browse available animals and view their account. It will also explain the purpose of the adoption	The system will check the user's session to make sure they are logged in. It may retrieve the user's name and recent questionnaire information from the database so that personalised information can be displayed.

Interface Design





Algorithm (Flow chart, pseudocode)



Pseudocode D4: home page (main menu)

```
IF session.user_id is null THEN go to login page
user ← SELECT name, role FROM users WHERE user_id = session.user_id
SHOW "Welcome, " + user.name, the three "how it works" steps and "staff make the final decision"
IF the user has a stored questionnaire THEN show "See my matches" ELSE show "Take the questionnaire"
MENU ← [Home, Browse animals, Questionnaire, My matches, My account]
IF user.role = 'admin' THEN MENU ← [Dashboard, Manage animals, Add animal]
```

Variables

Variable Name	Data Type	Purpose	Justification	E.g or validation

current_user_id	INTEGER	Identifies user currently accessing dashboard	Ensures that the dashboard displays information belonging to the correct account	Retrieved from session
current_user_name	STRING	Stores the logged in users name	This will allow the interface to display personalised information and reassures user they are accessing correct account	‘Welcome sienna’
animal_count	INTEGER	Stores the number of animals currently available	It will provide a useful summary of information on the dashboard. An integer is a suitable data type as the number of animals is a whole number	Must be zero or larger
questionaire_compl eted	BOOLEAN	Records whether the user has completed the questionnaire	This variable allows the system to display recommendations or prompt the user to complete the questionnaire. Boolean is suitable as there is only	True or False
reccomended_animals	LIST	Stores the animals recommended to the users	Multiple animals will need to be displayed in ranked order so a list is a suitable data type	Ordered from lowest to highest compatibility
selected_page	STRING	Stores the page selected from navigation menu	This allows the program to choose which page or section should be selected	Animals or questionaire

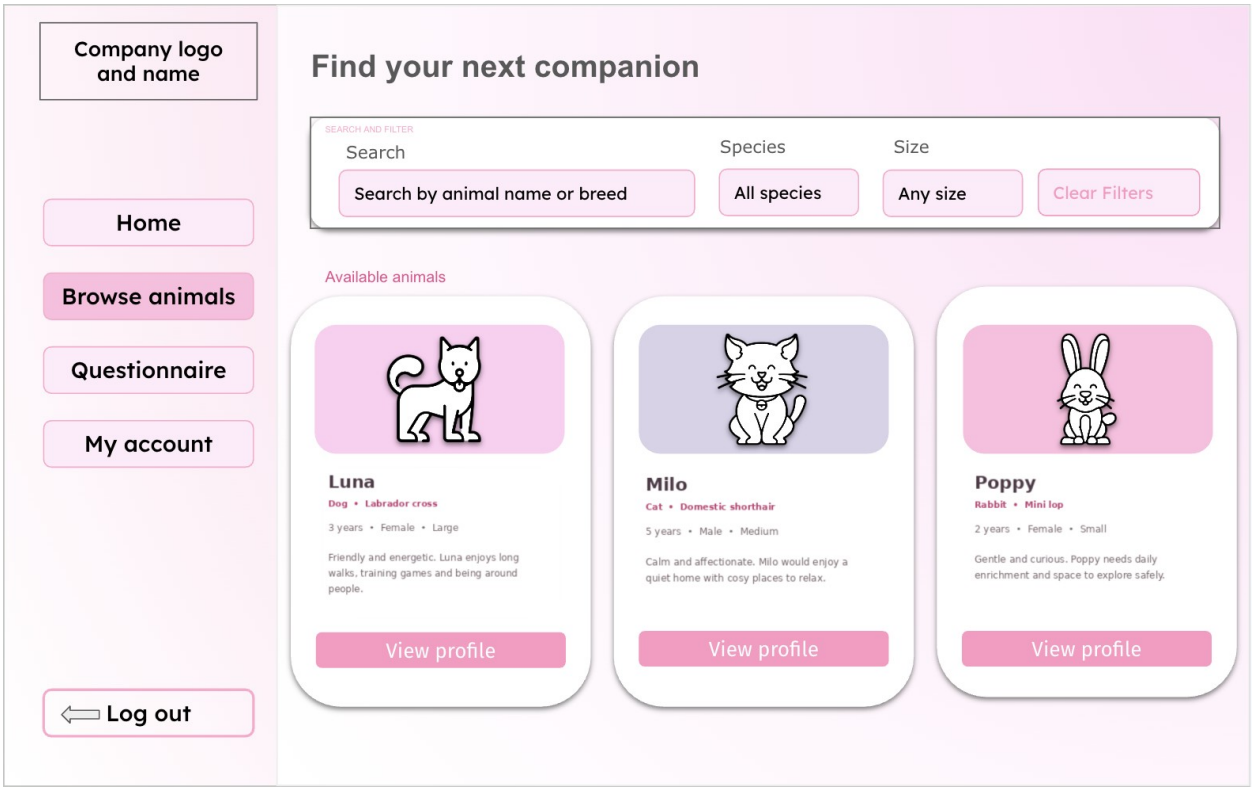
notification_messa g e	STRING	Stores confirmation or status feedback	The user will need clear feedback after completing actions such as submitting	‘Questionaire submitted successfully’

Animal Viewing Page

Animal Viewing Page

Animal Viewing	Allows users to browse available animals using clearly presented animal cards. Each card will show information such as the animal name, image, species,	Animal records will be retrieved from the animals table. Search and filter selections will be used to change the database query or filter the retrieved records. Selecting an animal will use its

Interface Design



Consistent branding and permanent navigation - Reusing these elements gives the application a recognisable identity as it matches the other pages. The main sections are in a predictable sidebar so users don't need to remember page routes or repeatedly return home. Text labels are used rather than relying on icons alone, improving clarity for different users.

Current page feedback - browse animals uses the filled pink active state while the other navigation controls are outlined. This immediate feedback confirms the users current location.

Secure logout placement - logout is separated from ordinary navigation and placed at the bottom of the sidebar. This reduces accidental selection and gives the user an obvious way to clear the authenticated session before returning to the login page.

Filtering - The species and size use controlled choices that align with the database fields and clear filters will restore the default view, when the selected criteria return too few results.

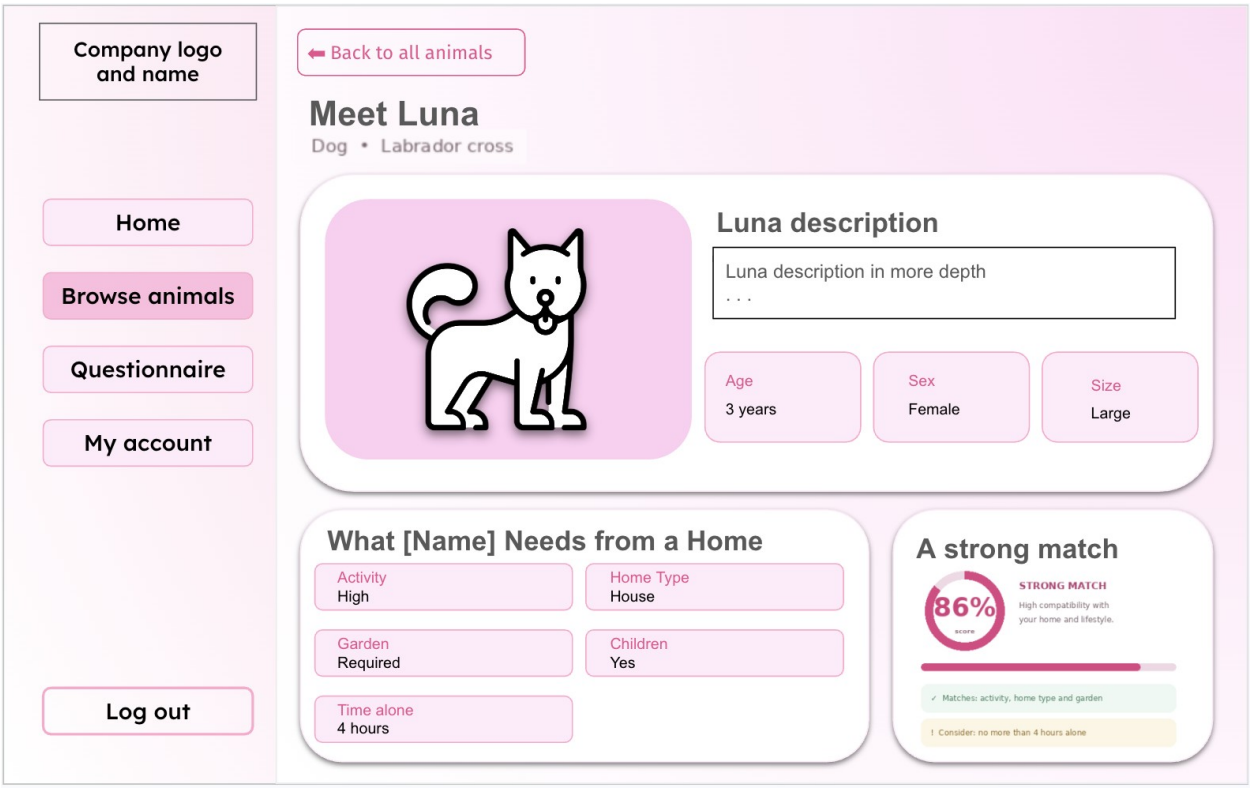
Page purpose is clear - The largest heading states the task before controls are shown. The position and size of the title are larger than the rest of the text so there is a clear visual hierarchy,

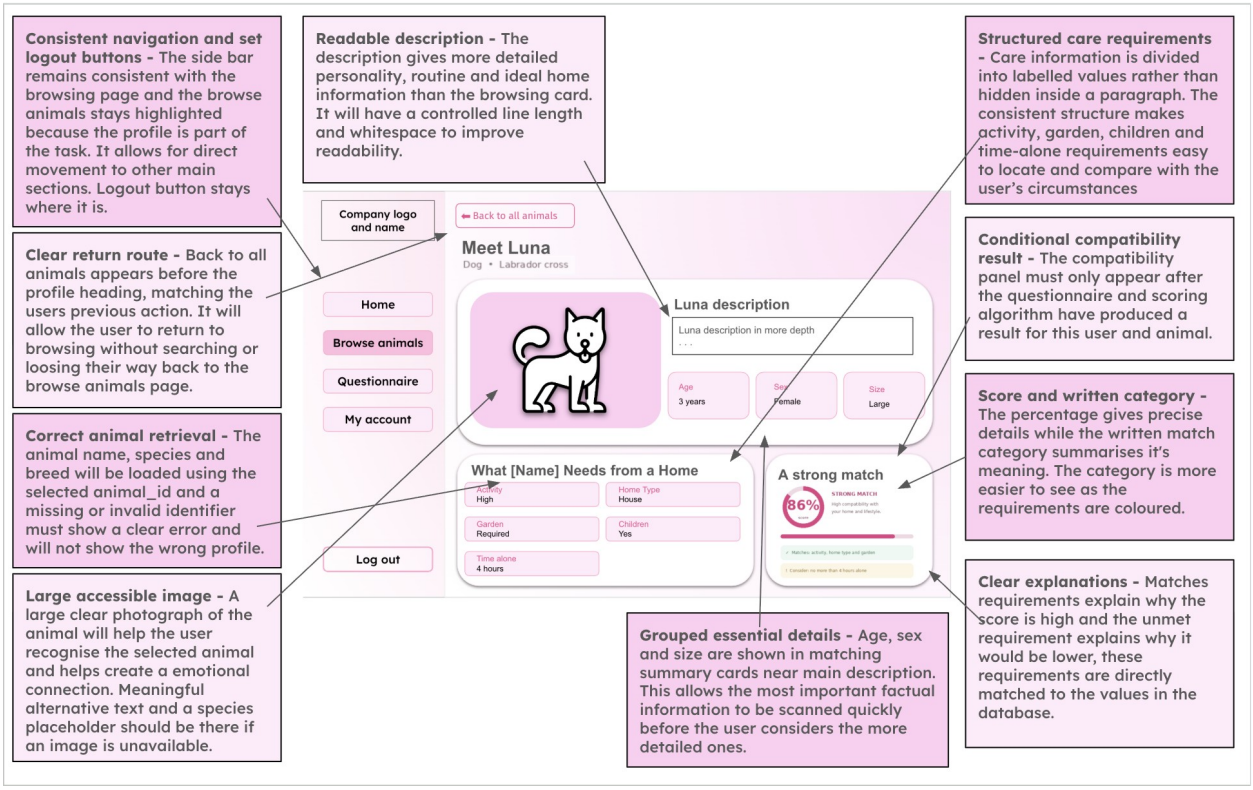
Consistent profile cards - every animal uses the same image area, information order, spacing and action position. This makes it easier for the user to navigate the different animals especially as they don't have to learn a different layout for each animal and makes coding the website easier.

Essential information first - Name, species, breed, age, sex, size and a short description provide enough information for an initial decision without overcrowding the page. The values should be read directly from the selected SQLite animal record.

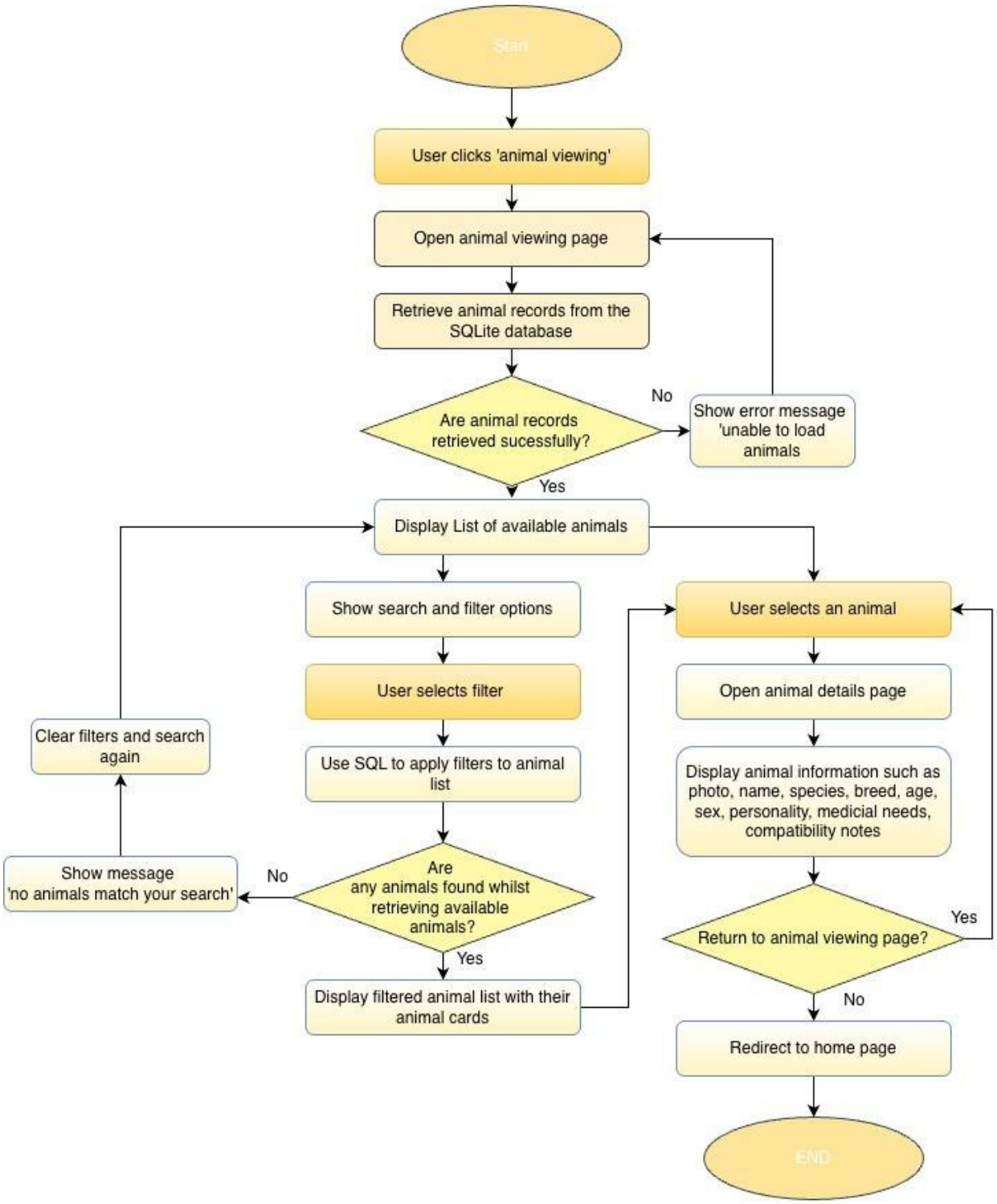
Clear route to full details - View profile is the only strongly emphasised action on each card and has a large click target. Selecting it should store animal id and opening the matching animal profile preventing the wrong record from being displayed.

Flexible search - The search field allows records to be found by animal name and breed. The implementation should have validation so users can find a record effectively.





Algorithm (Flow chart, pseudocode)



Pseudocode D5: browse with filters (search text is data, never SQL)

```
FUNCTION browse(species, size, search)
    sql ← "SELECT * FROM animals WHERE status = 'Available'"
    params ← []
    IF species ≠ "All species" THEN sql ← sql + " AND species = ?"; ADD species TO params
    IF size ≠ "All sizes" THEN sql ← sql + " AND size = ?"; ADD size TO params
    term ← TRIM(search)                    // the search box accepts at most 20 characters
    IF term ≠ "" THEN
        sql ← sql + " AND (name LIKE ? OR breed LIKE ?)"
        ADD "%" + term + "%" TO params TWICE
    ENDIF
    RETURN rows of sql + " ORDER BY species, name", using params
ENDFUNCTION
```

The first browse design had a sort option. It is replaced by one fixed order (species, then name A to Z): with 10 to 20 animals and two filters plus a search, a fixed order is simpler to use and to test (T3.1 to T3.3).

Variables

Variable	Data Type	Purpose	Justification	E.g or
search_term	String	Stores text entered into the search bar	A string allows users to search using names, breeds or	Leading and trailing spaces should be removed
species_filter	String	Filters animals by species	This is required to reduce the displayed records to animals matching the users	'Dog', 'cat', 'rabbit' are the specified filters
size_filter	String	Filters animals by size	Size may affect whether an animal is suitable for the users home so it is suitable for narrowing the	'Small', 'medium', large'
sort_option	String	Determines the order of results	Users may wish to order the results by suitability or animal information. The value determines which sorting	'Compatibility'
animal_results	list	Holds the records returned from the database	A list is suitable because a search may return multiple animal records. It can be iterated through to generate animal	May be empty is no matching results

selected_animal_id	integer	Identifies the animal whose profile is opened	Using the unique id ensures the correct animal is retrieved	Must match an existing record
			even when animals have the same	
compatibility_score	float	Displays or sorts by compatibility	A float is a more suitable data type than an integer as it allows the result to include decimal values, which will produce a more precise % than an integer.	Between 0 and 100
no_results	boolean	Feedback displayed if there is any animals which meet the filters	Without feedback an empty page could appear to be an error, so the message informs the user	'No animals match your search'

Validation

1.Search input

- Length check: the text will not exceed 20 characters
- Whitespace check: leading and trailing spaces will be removed
- Empty input handling: an empty search will be treated as a request to display all available animals rather than as an error

2.Filter inputs

Drop down menus or selection controls will be used for:

-Species

-Dog

-Cat

-Rabbit

-Size

-Small

-Medium

-Large

Final browse variables

`sort_option` in the variables table above is not used in the final build, because the order is fixed (see above). The search term is trimmed and limited to 20 characters (V16).

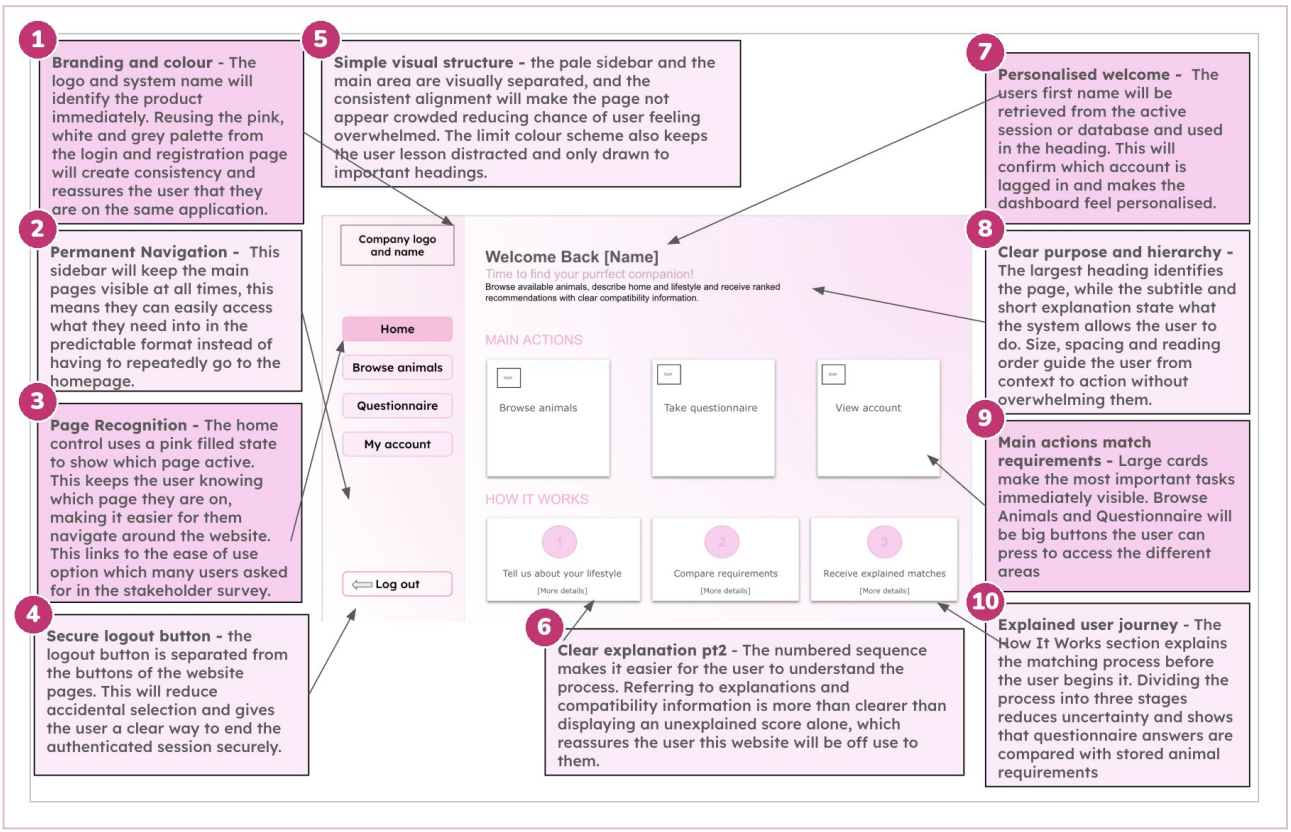


Figure D10: Annotated design: home page

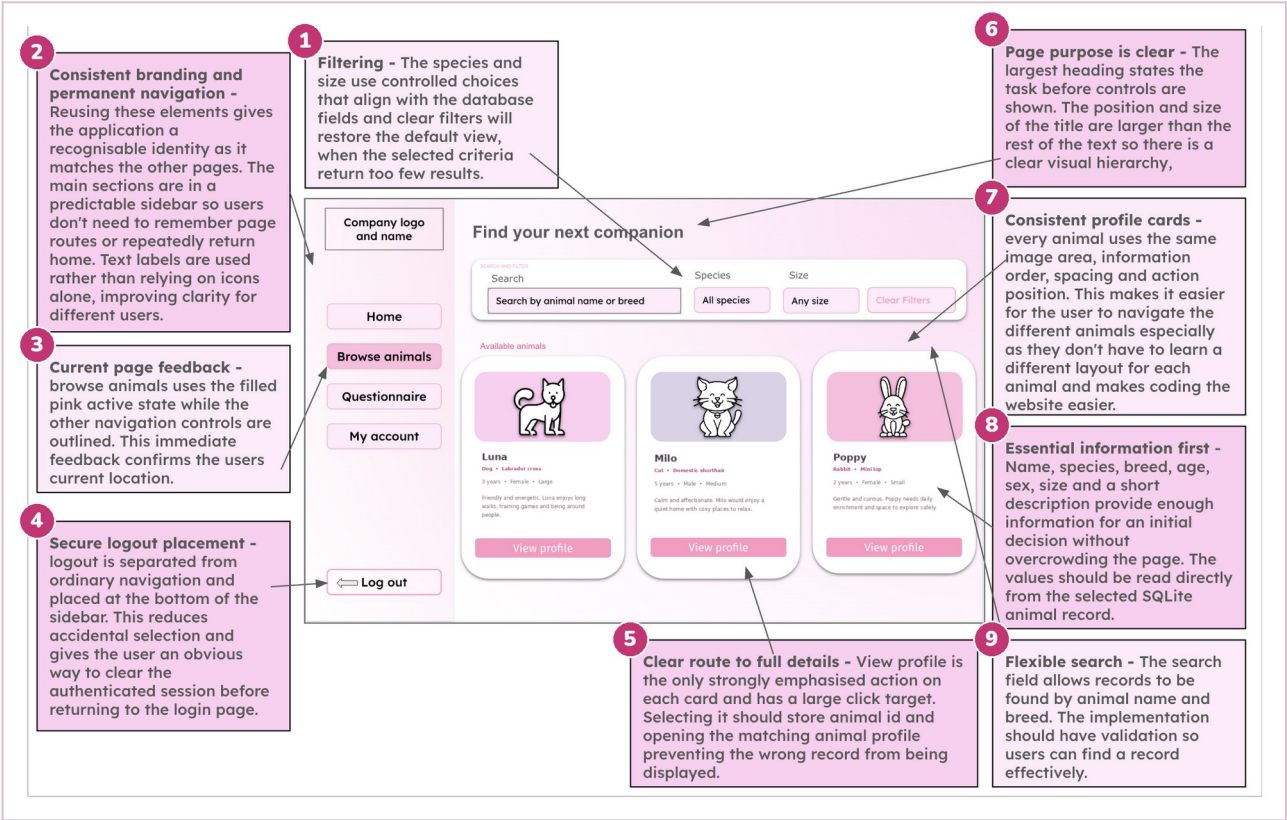


Figure D11: Annotated design: browse cards with species and size filters

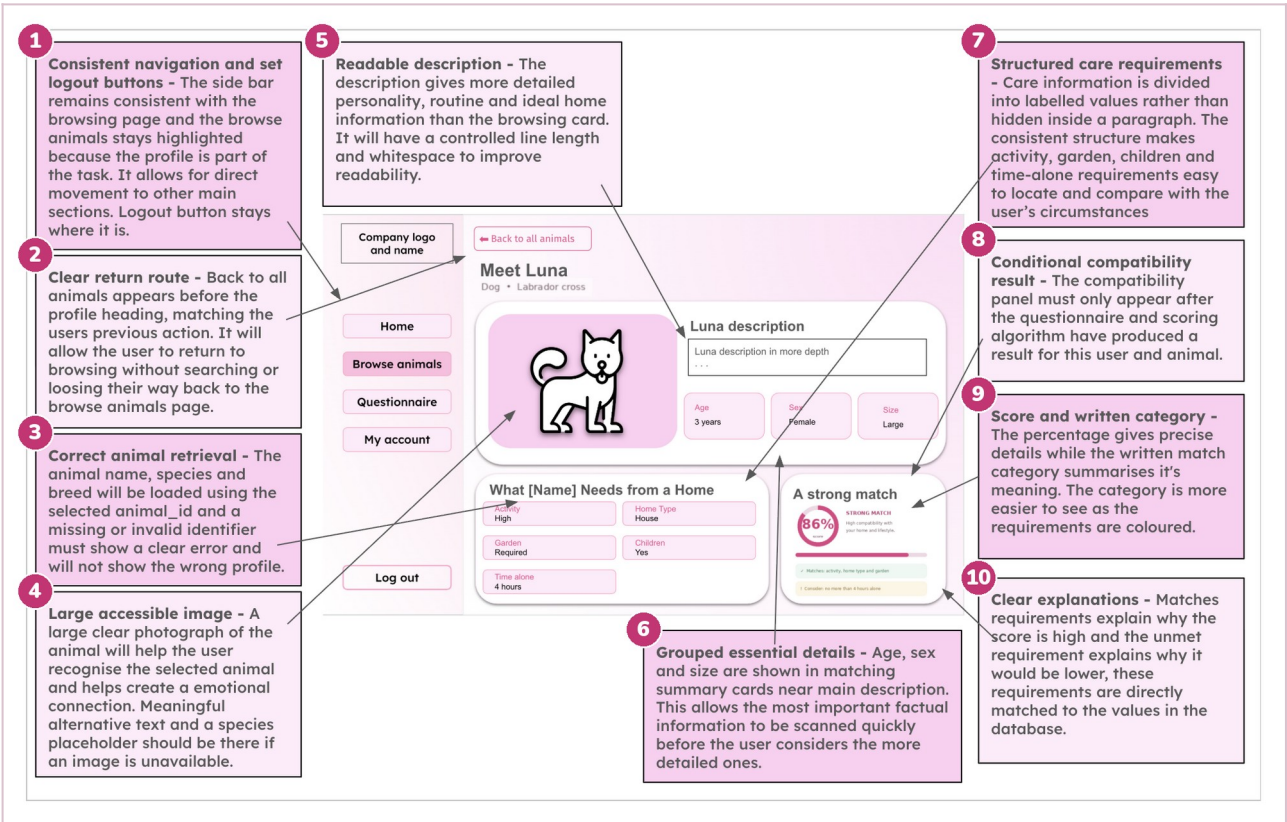


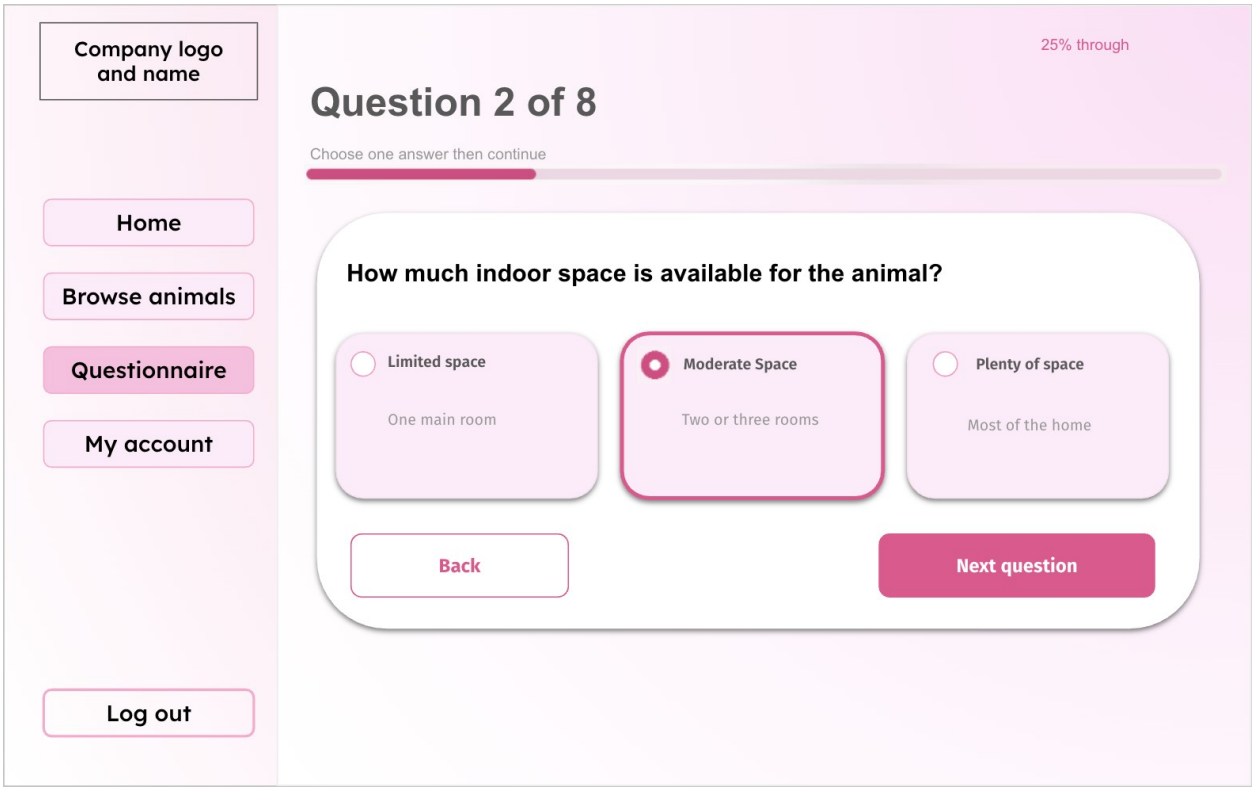
Figure D12: Annotated design: animal profile with its needs

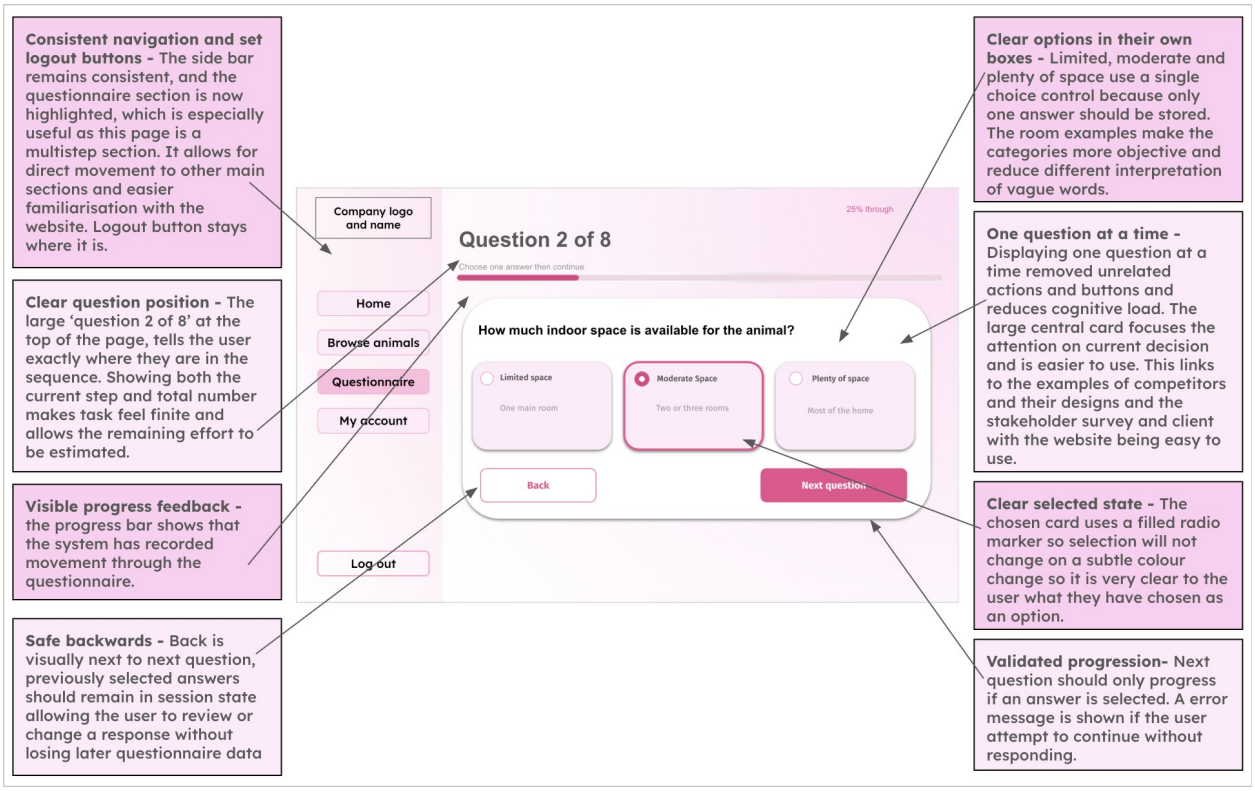
Compatibility algorithms

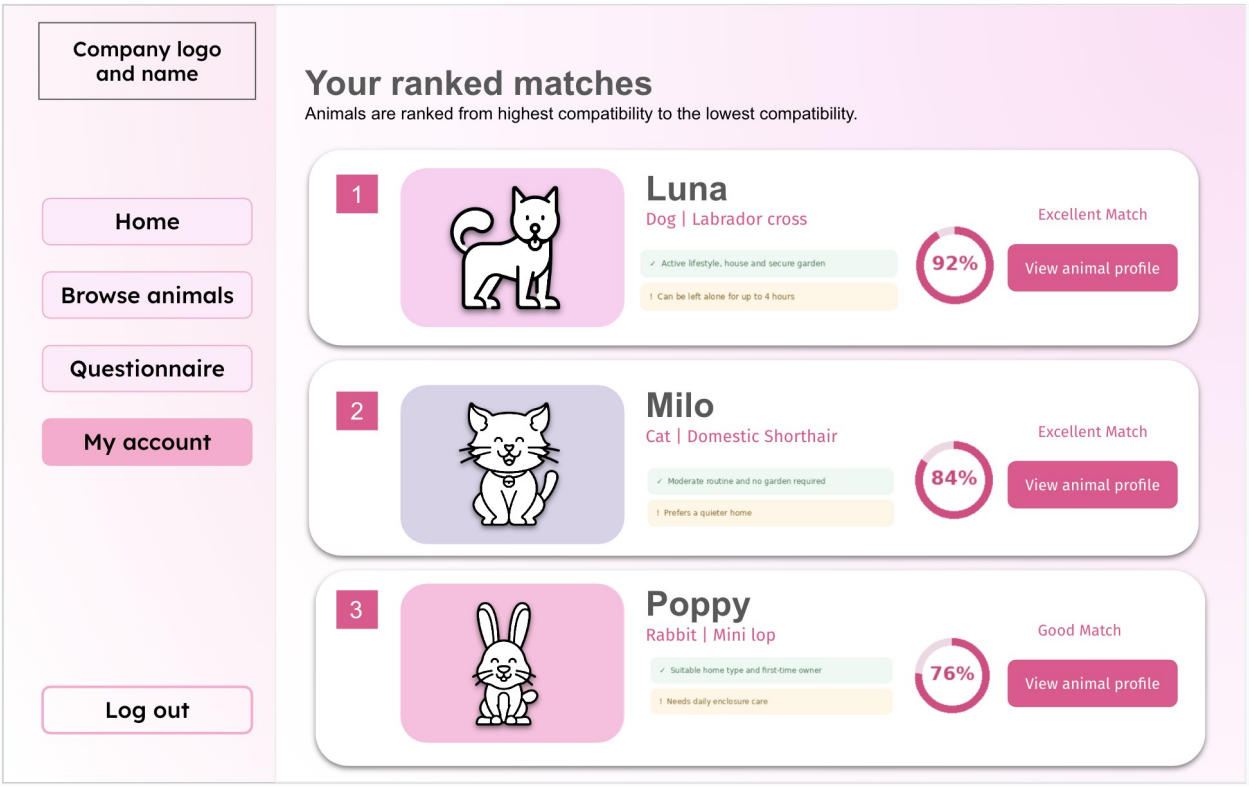
Compatibility algorithm and questionnaire

Questionnaire and Recommendations	Will use a form in order to allow users to answer questions about their home, lifestyle, experience and preferences. The questionnaire will have questions such as the housing type, access to garden, animal loneliness times, children, pets, experience, activity labels, what species and size preferred. These questions would not appear all at one time to not overwhelm the user and the input type would be drop downs or radio buttons. After	Responses will be validated and compared with the requirements of each available animal. A weighted compatibility algorithm will calculate a percentage score for every animal. The animals will then be sorted from highest to lowest score so the top recommendations can be displayed. The questionnaire responses and match results may also be stored within the database with the

Interface Design


Company logo and name
25% through
Question 2 of 8
Choose one answer then continue
Home
Browse animals
Questionnaire
My account
How much indoor space is available for the animal?
Limited space
One main room
Moderate Space
Two or three rooms
Plenty of space
Most of the home
Back
Next question
Log out


Consistent navigation and set logout buttons - The side bar remains consistent, and the questionnaire section is now highlighted, which is especially useful as this page is a multistep section. It allows for direct movement to other main sections and easier familiarisation with the website. Logout button stays where it is.
Clear question position - The large 'question 2 of 8' at the top of the page, tells the user exactly where they are in the sequence. Showing both the current step and total number makes task feel finite and allows the remaining effort to be estimated.
Visible progress feedback - the progress bar shows that the system has recorded movement through the questionnaire.
Safe backwards - Back is visually next to next question, previously selected answers should remain in session state allowing the user to review or change a response without losing later questionnaire data
Clear options in their own boxes - Limited, moderate and plenty of space use a single choice control because only one answer should be stored. The room examples make the categories more objective and reduce different interpretation of vague words.
One question at a time - Displaying one question at a time removed unrelated actions and buttons and reduces cognitive load. The large central card focuses the attention on current decision and is easier to use. This links to the examples of competitors and their designs and the stakeholder survey and client with the website being easy to use.
Clear selected state - The chosen card uses a filled radio marker so selection will not change on a subtle colour change so it is very clear to the user what they have chosen as an option.
Validated progression- Next question should only progress if an answer is selected. A error message is shown if the user attempt to continue without responding.
Company logo and name
Question 2 of 8
Home
Browse animals
Questionnaire
My account
Log out
How much indoor space is available for the animal?
Limited space
Moderate Space
Plenty of space
Back



Consistent navigation - The ranked-results page reuses the same sidebar, palette and card styling as the other user interfaces. This provides continuity after questionnaire submission and reassures the user that the results belong to this website

Clear ranking explanation - The heading and subtitle state that results are ordered from highest to lowest compatibility. This explains the purpose of the page before the user examines individual scores.

Unique rank numbers - The animals are labelled 1 2 3 are in order so they can easily see the clear order and ensure its the right order.

Consistent result cards - Each animal uses the same sequence of rank, image, identity, explanations, score and action. A repeated layout allows rapid comparison and ensures that there is no animal stands out more and differently.

Connected full profile - View animal profile stores selected animal_id and opens the corresponding full page. The detailed page then can show every care requirements without making the ranked result too long or difficult to compare.

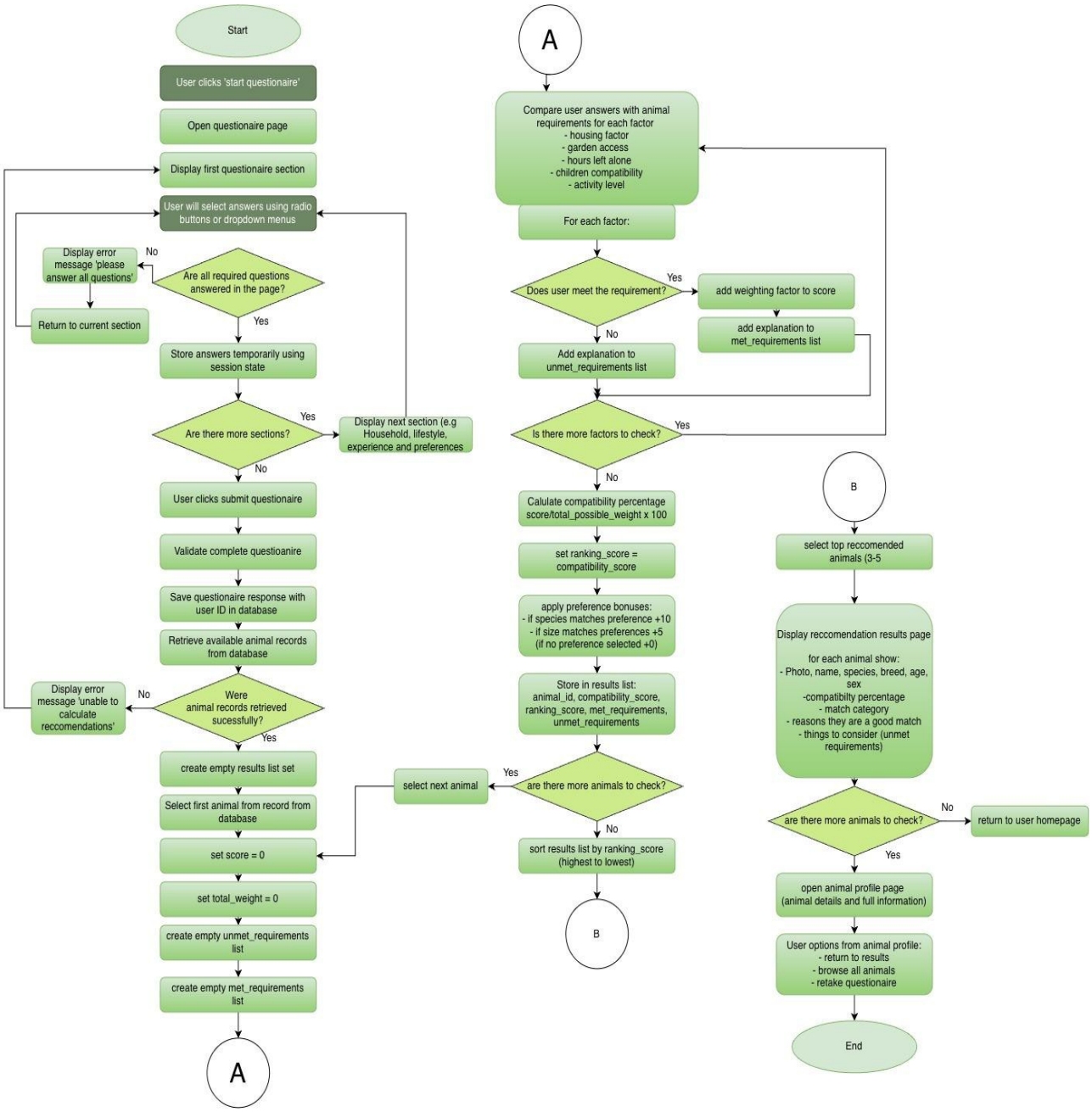
Animal identity remains clear - The image, name, species and breed make each result identifiable before the user interprets the score. These values should be retrieved from the available-animal records rather than copied into the results interface.

Sorted algorithm output - compatibility scores should be calculated for every available animal, stored against its animal_id and sorted in descending order before cards are displayed.

Consistent score bands - A quick way of seeing if its suitable instead of looking at the percentages and being upset if its a bit less than another animal.

Matched requirements explained (and unmatched) - The green explanation identifies the strongest reasons for the recommendation, such as activity, home space or garden access. This provides evidence for the score rather than asking the user to trust an unexplained percentage.

Algorithm (Flow chart, pseudocode)



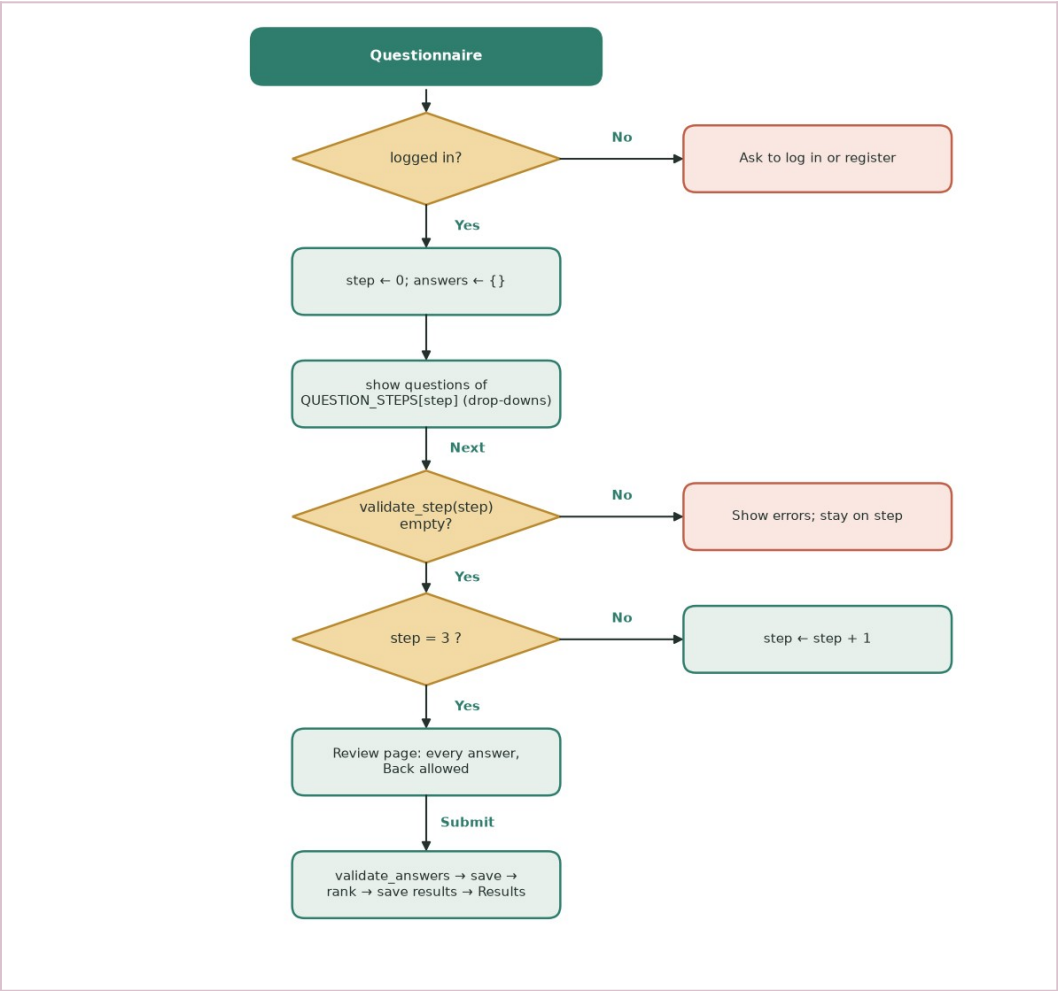


Figure D13: Flowchart: one question per screen, Next only when answered

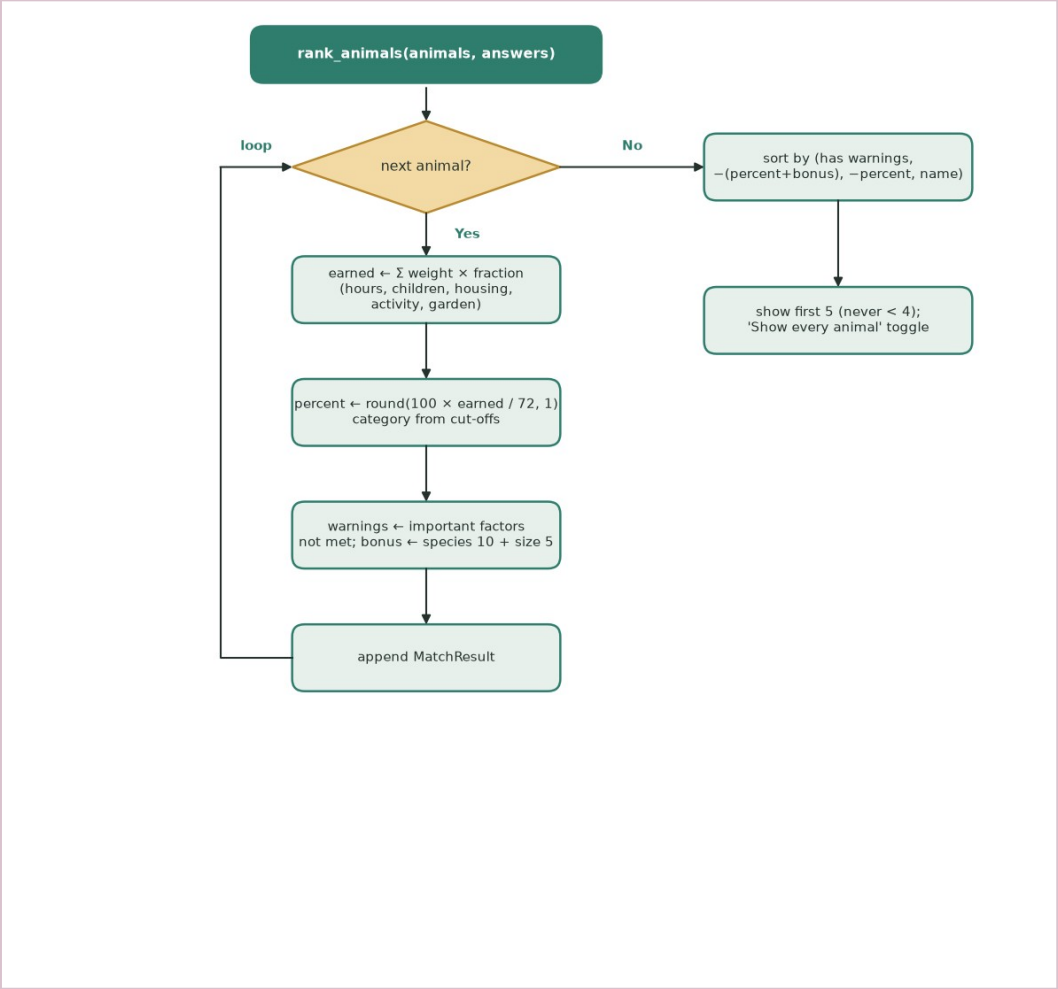


Figure D14: Flowchart: scoring and ranking every available animal

Pseudocode: scoring and ranking

The score is a weighted sum normalised to a percentage, so the scale stays 0 to 100 however many factors there are or however the weights are changed.

Pseudocode D6a: the constants and the check for each factor

```
CONSTANT WEIGHTS ← {hours_alone, children, housing, activity, garden, other_pets, experience}
                   // values: first design on D51, final values after the re-check on S21
CONSTANT IMPORTANT_FACTORS ← {hours_alone, children, housing, other_pets}   // unmet → warning
CONSTANT ALLERGY_CAP ← 39.0                 // below "Possible", so never a strong match (Q10)
CONSTANT PREFERENCE_BONUS ← {species: 10, size: 5}   // ranking only, never the percentage
CONSTANT SHOWN ← 5                          // results shown before "show all" (at least 4, req. 7)

FUNCTION check(factor, animal, answers)     // returns 1 (met), 0 (not met) or 0.5
    CASE factor OF
        hours_alone: RETURN 1 IF answers.hours_alone ≤ animal.max_hours_alone ELSE 0
        children:    RETURN 1 IF animal.child_compatible OR NOT answers.has_children ELSE 0
        housing:     RETURN 1 IF answers.housing_level ≥ animal.min_housing_level ELSE 0
        activity:    gap ← MAX(0, animal.exercise_needs − answers.activity_level)
                     RETURN MAX(0, 1 − gap ÷ 2)           // one level short earns half
        garden:      RETURN 1 IF answers.garden_access OR NOT animal.garden_required ELSE 0
        other_pets:  RETURN 1 IF animal.good_with_pets OR NOT answers.other_pets ELSE 0
        experience:  gap ← MAX(0, animal.experience_needed − answers.experience)
                     RETURN MAX(0, 1 − gap ÷ 2)
    ENDCASE
ENDFUNCTION
```

Pseudocode D6: score one animal

```
FUNCTION score_animal(animal, answers)
    earned ← 0;  met ← [];  unmet ← [];  warnings ← []
    FOR EACH factor IN WEIGHTS
        fraction ← check(factor, animal, answers)   // 1 = met, 0 = not met, 0.5 = one level short
        earned ← earned + WEIGHTS[factor] × fraction
        IF fraction = 1 THEN
            ADD reason TO met
```

```
        ELSE
            ADD reason TO unmet
            IF factor IN IMPORTANT_FACTORS THEN ADD reason TO warnings
        ENDIF
    NEXT factor
    compatibility ← ROUND(100 × earned ÷ SUM(WEIGHTS), 1)
    IF animal.species IN answers.allergies THEN
        ADD allergy reason TO unmet AND TO warnings
        compatibility ← MIN(compatibility, ALLERGY_CAP)
    ENDIF
    category ← "Strong" IF compatibility ≥ 80, "Good" IF ≥ 60, "Possible" IF ≥ 40, ELSE "Low"
    bonus ← 10 IF animal.species = answers.preferred_species, + 5 IF animal.size = answers.preferred_size
    RETURN (compatibility, category, bonus, met, unmet, warnings)
ENDFUNCTION
```

Pseudocode D7: rank every available animal

```
FUNCTION rank_animals(animals, answers)
    results ← [score_animal(a, answers) FOR EACH a IN animals]
    SORT results BY
        1. has no warnings first           // a welfare risk is never above a safe match (Q3)
        2. compatibility + bonus, highest first
        3. compatibility, highest first
        4. name A to Z, then animal_id     // identical animals always come out in the same order
    RETURN results
ENDFUNCTION
```

Pseudocode D9: submit the questionnaire and store the results

```
PROCEDURE submit(user, raw_answers)
    answers, errors ← validate_answers(raw_answers)       // every question again, not only the screens
    IF errors ≠ {} THEN SHOW errors; RETURN
    WITH connect(DB_PATH) AS conn                        // one transaction (D8)
        response_id ← save_response(conn, user.user_id, answers)
        results ← rank_animals(Available animals, answers)
        save_results(conn, response_id, results)         // with calculated_at = now
    SHOW the matches page: the first SHOWN results, "show all" for the rest
ENDPROCEDURE
```

Sorting uses Python's built-in sort (Timsort, O(n log n)), which is stable and far faster than needed for 20 animals; the measured time for 500 animals is on E1.

Validation - questionnaire

1.Housing type or housing level

-Presence check: The user must select an answer
-Lookup check: The answer must be one of the
-A radio button or drop-down will prevent

1 = Flat or limited space
2 = Small or medium house
3 = Large house

2.Garden access

-Presence check: Yes or No must be selected
-Boolean check: The stored result must be 1 or 0

3.Hours the animal will be left alone

-Presence check : a value must be provided
-Type check : a whole number
-Range check : between 0 and 24 hours

4.Children in the home

-Presence check: Yes or No must be selected

-Boolean check: The stored result must be 1 or 0

5.Preferred activity level

-Presence check: an option must be selected
-Boolean check: The stored result must be 1, 2 or

After

6.Preferred species

-Presence check: The user must select an answer
-Lookup check: The answer must be a dog, cat,

7.Preferred size

-Presence check: The user must select an answer
-Lookup check: answer must be small, medium,

Validation is necessary within this compatibility algorithm to ensure that the recommendations produced are reliable and that invalid data does not cause incorrect results.

Questionnaire Response table

Field	Data type	Key/validation
response_id	INTEGER	Primary key, auto increment
user_id	INTEGER	Foreign key linked to
housing_level	INTEGER	Required range 1-3
garden_access	BOOLEAN	Required 0 or 1
hours_left_alone	INTEGER	Required valid range
has_children	BOOLEAN	Required 0 or 1
preferred_activity_level	INTEGER	Required range 1 - 3
preffered_species	TEXT	Required dog, cat, rabbit or no
preferred_size	TEXT	Required small, medium, large or no preference
completed_at	DATE	required

Match Results Table

Field	Data type	Key/validation
match_id	INTEGER	Primary key autoincrement
response_id	INTEGER	Foreign key linked to QuestionaireReponses.response
animal_id	INTEGER	Foreign key linked to
compatibility_score	REAL	Required range 0 to 100
match_category	TEXT	Required strong, good, possible
unmet_requirements	TEXT	optional
Calculated_at	DATE	required

Adoption Questionnaire Variables

Variable Name	Data Type	Purpose	Justification	Validation or e.g
housing_level	integer	Stores amount of living space available	The available space may affect whether or not the home is suitable for different sizes and	Required range 1-3
garden_access	boolean	Records whether user has access to a garden	Some animals may require regular access to an outdoor space. A boolean value is suitable as the answer has	0 or 1

hours_left_alone	integer	Stores how long the animal would normally be left alone	Time availability can be numerically compared with the animals required attention and exercise time	Required valid range such as 0-24
has_children	boolean	Records whether children live in the household	Some animals may not be suitable for homes with children making this an important safety	0 or 1
preferred_activity_level	integer	Stores the users preferred animal activity	Matching activity level helps ensure the adopter can	range 1 - 3
		level	the animals exercise	
preffered_species	string	Stores the users preferred species	This ensures that the recommendations remain relevant to the type of animal the user is	Required ‘dog’, ‘cat’, ‘rabbit’ or ‘no preference’
preferred_size	string	Stores the users preferred size	The user may have practical or personal reasons for preferring a	‘small’, ‘medium’, ‘large’ or ‘no preference’

Compatibility scoring algorithm variable names

Variable Name	Data Type	Purpose	Justification	Validation or e.g

animal_id	integer	Stores the unique identifier of the animal being assessed	It ensures that each calculated score is linked to the correct animal, even when animals have the	Must match an existing record in the animals table e.g 12
maximum_score	integer	Stores the highest possible number of points that can be awarded by the compatibility algorithm	This is required so that the user's raw score can be converted into a percentage. An integer is suitable because the algorithm awards whole number points for each matching	Must be greater than 0
total_score	integer	Stores the total compatibility points awarded for the animal currently being assessed	The value begins at zero and increases whenever the user's answers from the questionnaire match an animal's requirements. It must be reset	Begins at 0 and cannot exceed maximum_score
animal_scores	list	Stores the ID and compatibility score for all available animals	This will allow the results to be sorted from highest to lowest compatibility. A list is suitable because the algorithm	[{"animal_id": 3, "score": 92.5}, {"animal_id": 7, "score": 78.0}]
compatibility_score	float	Stores the calculated compatibility percentage	A percentage is easier for users to understand and compare than a raw points total, a float will allow decimal results to	Must be between 0-100 and can have decimals

match_category	string	Stores an understandable category for the calculated score	A category makes the numerical result easier for the user to interpret. It is selected according to predefined	strong, good, possible or low
unmet_requirements	list	Stores explanations of requirements the user did not meet	There may be several reasons why an animal is not fully suitable so a list will allow multiple explanations to be	Example: "Requires a secure garden", "Not suitable with cats". It may be empty when all essential requirements are

Questionnaire Plan

Section 1: House environment

- Amount of living space
- Garden access?

Section 2:

- Children in the home?

Section 3:

- Preferred activity level / time available for exercise and care
- Hours the animal would be left alone

Section 4:

- Preferred species
- Preferred animal size

Section 5:

- Submit

Where it links

- Amount of living space → minimum housing
- level Garden access → garden required
- Children in the home → child compatible
- Preferred activity level → exercise need
- Hours left alone → maximum hours left alone
- Preferred species → animal species
- Preferred size → animal size

Final scoring variables

In the build, `total_score` is the running total `earned`, `maximum_score` is the constant MAX_POINTS = 90 (the sum of the weights), and `compatibility_score` is `compatibility`, a percentage with one decimal place (D6a, D7 and D7).

Design decision: no animal is removed from the results

The first plan above removed animals with low suitability and showed the top 3 or 5. Removing animals could leave fewer than the 4 that requirement 7 asks for, and would hide an animal the adopter might ask staff about. Instead every Available animal is ranked (D7): animals with a welfare warning go to the bottom, the first 5 are shown, and a "show all" switch lists the rest. T5.9 tests this.

Weights

First Matching part

Factor	Suggested weight (can be changed)	Justification
Hours Left alone	20	Strongly will affect whether the adopter can meet the animals needs
Child compatibility	15	Important for safety and suitability
Housing space	15	Ensures the home provides enough room
Activity	12	Matches exercise needs with users lifestyle and what they can provide
Garden access	10	Important for animals requiring outdoor access, but can even use a park so not as important factor

-	Calculate the suitability score
-	Remove animals with low suitability
-	Prioritise animals matching users preferred species
-	Prioritise animals matching users preferred size
-	Sort remaining animals by suitability score

Factor	Suggested weight
Species	10
Size	5

However, these weights will not add to the compatibility percentage as their preferences do not reflect animal suitability. It will influence the order in which suitable animals are displayed. This will prevent an unsuitable animal from receiving a misleading high compatibility percentage as it matches the users preferences, but still ensuring preferred animals are presented as users top choices as this will make them more inclined to adopt. If the user picks 'no preference' no animal will receive a bonus.

> **Design decision: preferences do not change the percentage**
>
> Species and size preferences only add to a separate ranking total. If they were added to the percentage, an unsuitable animal of the preferred species could show a misleadingly high score, which is exactly what the client wanted to avoid (Q10). The final weights, after the design re-check in development stage 5, are on S20.

Final question list

The questionnaire plan above has seven questions. The design re-check against the interview added three, because the client named allergies as a deciding factor (Q10) and the animal records already held whether an animal can live with other pets and how much experience it needs. The final questionnaire has **10 questions in 4 sections**, one per screen:

Section	Question (stored as)	Input	Validation	Used by
Your home	Living space (housing_level)	3 cards	1, 2 or 3	housing
Your home	Garden access (garden_access)	Yes / No cards	0 or 1	garden
Your household	Children at home (has_children)	Yes / No cards	0 or 1	children
Your household	Other pets at home (other_pets)	Yes / No cards	0 or 1	other_pets
Your household	Allergies (allergies)	Tick dog, cat, rabbit, or none	Each species at most once	allergy cap
Your lifestyle	Activity you can offer (activity_level)	3 cards	1, 2 or 3	activity
Your lifestyle	Hours alone on a normal day (hours_alone)	Number box, starts empty	Whole number 0 to 24	hours_alone
Your lifestyle	Experience (experience)	3 cards	first-time, some, experienced	experience

Section	Question (stored as)	Input	Validation	Used by
Your preferences	Preferred species (preferred_species)	4 cards	Dog, Cat, Rabbit or No preference	ranking bonus
Your preferences	Preferred size (preferred_size)	4 cards	Small, Medium, Large or No preference	ranking bonus

Card buttons are used instead of the dropdowns preferred in requirement 6, because every choice is visible at once and large enough to tap on a tablet (SC16); requirement 6's aim, a suitable input for each question, is kept.

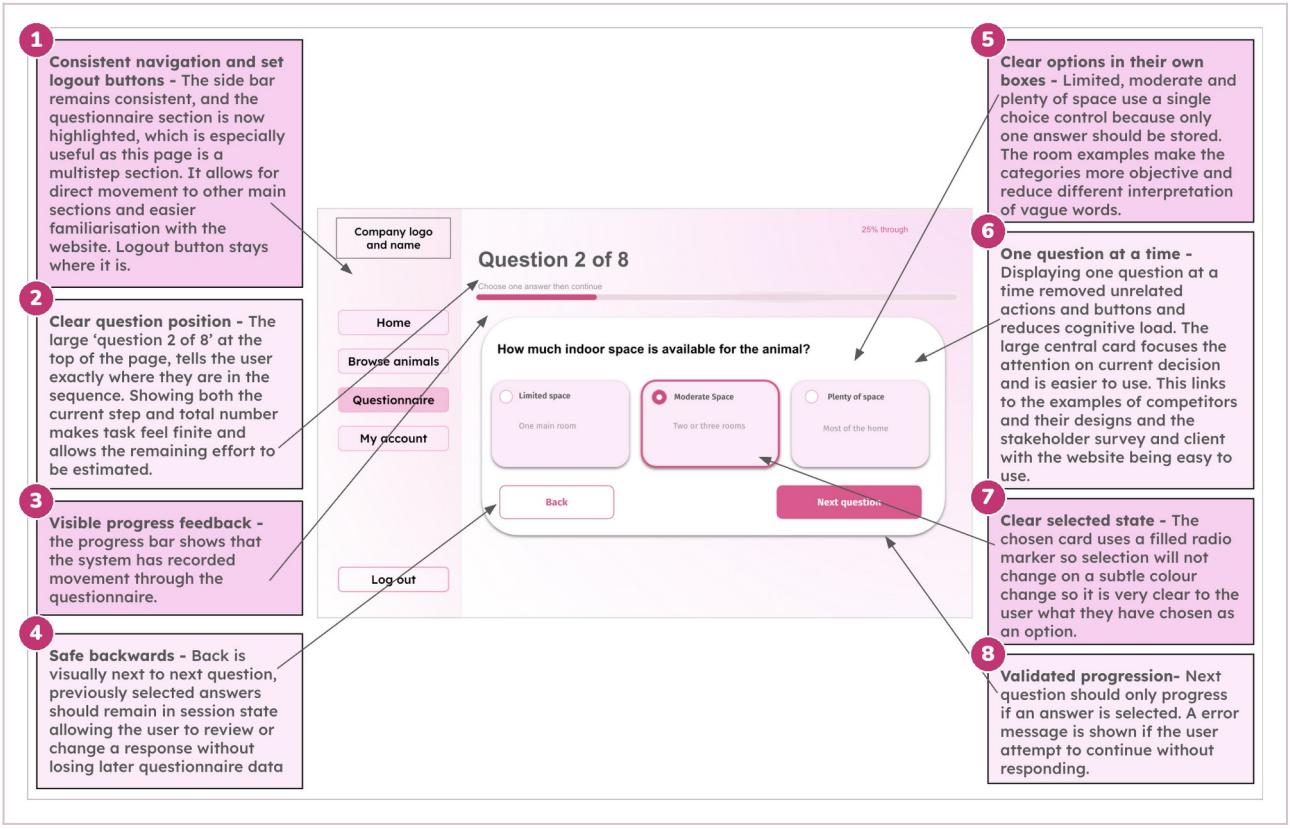


Figure D15: Annotated design: one question per screen with card answers and progress

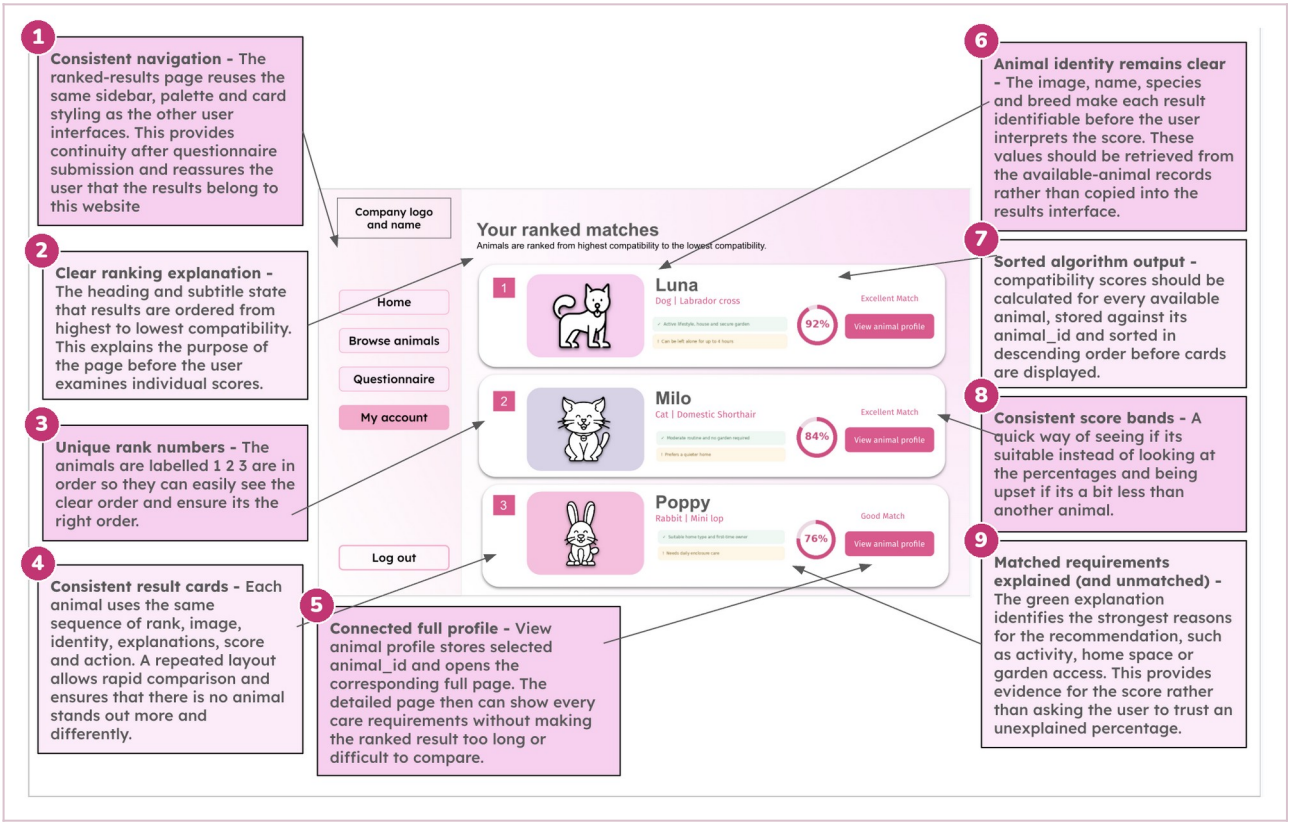


Figure D16: Annotated design: ranked matches with met and not-met reasons

2.7 Validation rules

Every input is checked in one place (`validation.py`) before it reaches the database, and the same rule is repeated as a database constraint. The rules below are exact; each is tested with normal, boundary and invalid data in the development test plan.

ID	Input	Rule	Message shown	Tests
V1	Name (person)	Required; 2 to 50 characters; letters, spaces, hyphens and apostrophes only	"Enter a name." / "Name must be 2 to 50 characters." / "Name can only use letters, spaces, hyphens and apostrophes."	T1.2, T1.8
V2	Email	Required; the form name@domain.xx; stored in lower case; not already used	"Enter a valid email address, like name@domain.com." / "An account with this email already exists."	T1.3, T1.7
V3	Age (person)	Whole number 18 to 120	"You must be 18 or over to adopt." / "Age must be between 18 and 120."	T1.4
V4	Password	At least 8 characters; confirm box filled and equal	"Password must be at least 8 characters." / "Type your password again to confirm it." / "The two passwords do not match."	T1.5, T1.6
V5	Login	Both boxes filled (presence only); then one message for any wrong detail	"Enter your email address." / "Enter your password." / "Incorrect email or password."	T1.9, T1.10
V6	Animal name	As person names, but digits allowed ("R2")	As above, "(and numbers)"	T2.2
V7	Breed	Required; 2 to 50 characters; contains a letter	"Enter a breed (use 'Mixed' if unknown)."	T2.2
V8	Animal age	Whole number 1 to 30	"Age must be between 1 and 30."	T2.3
V9	Species, sex, size, status	One of the fixed lists	"Choose a species." (and so on)	T2.4

ID	Input	Rule	Message shown	Tests
V10	Exercise needs, minimum housing	1 to 3	"... must be between 1 and 3."	T2.4
V11	Hours alone (animal and adopter)	Whole number 0 to 24	"Hours alone must be between 0 and 24."	T2.3, T4.2
V12	Experience needed	0 to 2	"... must be between 0 and 2."	T2.4
V13	Personality; medical notes	Personality required; both at most 500 characters	"Describe the animal's personality." / "... at most 500 characters."	T2.2, T2.6
V14	Photo	PNG or JPG, at most 2 MB	"Images must be PNG or JPG." / "Images must be 2 MB or smaller."	T2.5
V15	Questionnaire answers	Each answer in its range (final question list); Next disabled until answered	"Choose ..." / "Answer yes or no for ..."	T4.1 to T4.6
V16	Search box	At most 20 characters; used only as data	none (no results)	T3.3, R1

Pseudocode D10: the shared checks every validator uses

```
FUNCTION check_range(value, low, high, field, label, errors)
    number ← whole number in value, or NULL
    IF number = NULL THEN
        errors[field] ← label + " is required." IF value is blank ELSE label + " must be a whole number."
    ELSE IF number < low OR number > high THEN
        errors[field] ← label + " must be between " + low + " and " + high + "."
    ENDIF
    RETURN number
ENDFUNCTION

FUNCTION validate_registration(name, email, age, password, confirm)
    errors ← {}
    check_name(name, errors);  check_email(email, errors);  check_range(age, 18, 120, "age", "Age", errors)
    IF password is blank THEN errors["password"] ← "Enter a password."
    ELSE IF LENGTH(password) < 8 THEN errors["password"] ← "Password must be at least 8 characters."
    IF confirm is blank THEN errors["confirm"] ← "Type your password again to confirm it."
    ELSE IF confirm ≠ password THEN errors["confirm"] ← "The two passwords do not match."
    RETURN errors                              // empty means valid; each message shows under its box
ENDFUNCTION
```

`validate_animal` (staff form) and `validate_answers` (questionnaire) follow the same pattern with the rules in the table.

2.8 Usability features

Usability feature	Where it appears	Why (source)	Evaluated
Soft pink palette, Nunito font, large photos	Every page	Client asked for a cute, friendly, pastel look with lots of photos (Q11)	SC1, U5
Contrast at least 4.5:1 and text at least 12 px	Every page	Readable for older and low-vision users (WCAG 2.2 AA, E5)	SC1
One consistent menu; staff see only staff items	Sidebar	Find things quickly without clicking through many pages (Q6)	SC2
One question per screen with progress "Question N of 10"	Questionnaire	Less overwhelming than a long form (E3, S6)	SC8, U2
Card buttons instead of dropdowns	Questionnaire	Fewer clicks and a bigger target than a dropdown; the selected card is highlighted	U2
Next disabled until the question is answered	Questionnaire	Cannot submit an incomplete questionnaire (Q15, S8)	SC8
Error message under the box that caused it	Register, login, staff forms	The user sees exactly what to fix	SC6, U1
One generic login error	Login	Security: does not reveal which emails have accounts (Q12)	SC7
Met / not met / warning lists on each match	Matches, profile	Explains the score (Q3, E5)	SC11, U3

Usability feature	Where it appears	Why (source)	Evaluated
"Staff make the final decision" wording	Home, matches	Sets the right expectation (Q4)	SC3
Tick-box confirmation before delete	Staff manage page	Stops an accidental delete	SC12, U4
Keyboard focus outline	Every control	Usable without a mouse	SC1

2.9 Development test plan

These tests are run during development, stage by stage, to inform it. Each has an ID used in the development section, and almost all are automated with pytest so they can be re-run after every change (regression testing). Types: **N** = normal, **B** = boundary, **I** = invalid / erroneous.

ID	Test	Type	Test data	Expected result
Stage 1	**Accounts and login**			
T1.1	Register a valid adopter	N	Sam Lee, sam@example.com, 30, password1 twice	Account created, password stored as a hash, signed straight in
T1.2	Required fields blank (V1 to V4)	I	Every box empty	A message under each of the five boxes; nothing saved
T1.3	Email format (V2)	I	no-at-sign.com, a@b, two@@x.com, spa ce@x.com	"Enter a valid email address..."
T1.4	Age limits (V3)	B	17, 18, 120, 121, 2.5, abc	18 and 120 accepted; others refused with a reason
T1.5	Password length (V4)	B	7 characters, 8 characters	7 refused, 8 accepted
T1.6	Confirm password (V4)	I	different second password; blank second box	"The two passwords do not match." / "Type your password again..."
T1.7	Duplicate email, different case (V2)	I	SAM@example.com after sam@example.com	"An account with this email already exists."
T1.8	Name format (V1)	B/I	Mary-Jane, Siobhan O'Neill / Sam3, 12345, Sam!	First two accepted, the rest refused
T1.9	Correct login, both roles (V5)	N	demo adopter; staff account	Adopter to home page, staff to dashboard
T1.10	Wrong password and unknown email (V5)	I	right email + wrong password; unknown email	The same single message for both
Stage 2	**Staff animal management**			
T2.1	Add, edit, delete as staff	N	A complete valid animal	Saved, changed, then removed
T2.2	Required fields blank (V6, V7, V13)	I	Empty form	A message for each missing field; nothing saved
T2.3	Animal age and hours alone (V8, V11)	B	age 0, 1, 30, 31; hours −1, 0, 24, 25	Values inside the range accepted, outside refused
T2.4	Fixed lists (V9, V10, V12)	I	species Hamster, size Huge, sex ?, exercise 4, experience 3	Refused with a reason
T2.5	Photo type and size (V14)	B/I	a.png 100 B, b.JPG 2 097 152 B / c.gif, d.jpg 2 097 153 B	First two accepted, last two refused
T2.6	Text length (V13)	B	Medical notes of 501 characters	Refused (limit 500)
T2.7	Edit an animal deleted meanwhile	I	Edit animal_id that no longer exists	"That animal no longer exists."
T2.8	Adopter or signed-out user tries every staff action	I	role = user; no user	Every add, edit and delete refused
T2.9	Delete removes stored results too	N	Animal with match results	Animal and its results removed
T2.10	Overview flags weak records	N	Animal without a photo, short description	Listed as needing attention

ID	Test	Type	Test data	Expected result
T2.11	Database rejects a bad row inserted directly	I	INSERT with species Hamster, bypassing validation	sqlite3.IntegrityError
Stage 3	**Home and browse**			
T3.1	Every Available animal has a card	N	Demo data	One card per Available animal; Reserved and Adopted hidden
T3.2	Species, size and search filters	N	Cat; Small; "lab"	Only matching animals shown
T3.3	Search text is data, not SQL (V16)	I	`' OR 1=1 --`	No results; every animal still in the table
T3.4	Profile shows the animal's needs	N	Open Milo	All fields and needs shown, back link works
T3.5	Signed-out visitor opens a page	I	No session	Sent to the login page
Stage 4	**Questionnaire**			
T4.1	Next waits for an answer (V15)	I	No card chosen	Next disabled
T4.2	Hours alone limits (V11)	B	−1, 0, 24, 25	0 and 24 accepted, −1 and 25 refused
T4.3	Allergies (V15)	N/I	none; Dog and Cat / Ferret; Dog twice	Valid answers stored in a fixed order; others refused
T4.4	"No allergies" clears the others	N	Choose Cat, then No allergies	Only No allergies selected
T4.5	Complete questionnaire	N	10 valid answers	Every Available animal scored and stored
T4.6	Incomplete submission (V15)	I	9 of 10 answers	Nothing saved
Stage 5	**Matching and results**			
T5.1	Perfect match	N	Answers meeting every need	100%, Strong, no warnings
T5.2	Each factor loses exactly its weight	N	Change one need at a time	Score drops by that weight (half for one level short)
T5.3	Boundaries count as met	B	hours alone = animal's maximum	Met
T5.4	Category thresholds	B	80, 79.9, 60, 59.9, 40, 39.9	Strong, Good, Good, Possible, Possible, Low
T5.5	Important factor unmet	N	Children at home, animal not child-safe	Warning raised; garden or activity shortfall does not warn
T5.6	Allergy	N	Allergic to cats, perfect cat	Score capped at 39%, warning listed first
T5.7	Ranking order	N	Mixed animals	No-warning animals first; preference changes order, not percentage
T5.8	Hand calculation	N	Worked example (S20)	Program equals hand calculation
T5.9	Results page	N	Completed questionnaire	Top 5 shown, "show all" shows the rest
T5.10	Empty and identical lists	B	No animals; two identical animals	No crash; same order every time
Stage 6	**Accessibility and final checks**			
T6.1	Text contrast	N	Every text colour in the stylesheet	At least 4.5:1
T6.2	Text size	N	Every font size	At least 12 px
T6.3	Rendered audit	N	Every screen at 1440 and 1024 px	0 low-contrast texts, 0 small texts, focus visible

2.10 Post-development test plan

After development is finished, the complete system is tested against **each success criterion** (P1 to P16). This plan is linked to the development tests: the third column shows which T tests are re-run as

part of each P test, so the evidence on E1 is the final build passing them all again, plus the extra checks that only make sense on the finished system.

P	Success criterion	Re-runs	Extra evidence on the finished system
P1	SC1 look, contrast, text size	T6.1 to T6.3	Rendered audit of all 18 screens; client sign-off
P2	SC2 one menu	T3.5, T2.8	Screenshots of the adopter and staff menus
P3	SC3 home explains	T3.4	Home screenshot; button opens questionnaire
P4	SC4 browse and filters	T3.1 to T3.3	Browse screenshot with each filter
P5	SC5 profile	T3.4	Profile screenshot
P6	SC6 registration validation	T1.1 to T1.8	Error screenshot
P7	SC7 secure accounts	T1.9, T1.10	Database row shows a hash, not a password
P8	SC8 questionnaire	T4.1 to T4.6	Timed run by testers (U2)
P9	SC9 correct score	T5.1 to T5.4, T5.6, T5.8	Worked example against the screen
P10	SC10 ranking and speed	T5.7, T5.9, T5.10	Timed with 500 animals and 100 users
P11	SC11 explanations	T5.5, T5.6	Matches screenshot
P12	SC12 staff CRUD	T2.1 to T2.7, T2.9	Add-form error screenshot
P13	SC13 access control	T2.8	Adopter opening the staff page
P14	SC14 data and capacity	T2.11, T4.5	Timed browse page with 500 animals
P15	SC15 usability	n/a	Usability tasks U1 to U5 and SUS questionnaire; client final review
P16	SC16 laptop and tablet	T6.3	Screenshots at 1440 and 1024 px

Usability tasks

Each tester is given the task only, with no help, then answers the ten-item System Usability Scale (SUS). A SUS score of 68 is the average for websites, so SC15 needs an average of at least 68.

U	Task	Success looks like
U1	Create an account and log in	Done without help; every error understood
U2	Complete the questionnaire	Done in under 3 minutes without help
U3	Find your top match and explain why it suits you and what any warning means	Correct explanation in the tester's own words
U4	(Staff) Add an animal, edit it, then delete it	Done without help
U5	Browse, open a profile and return to the list	Done without help

Robustness tests

R	What could go wrong	Test	Expected
R1	Malicious text typed into search	`' OR 1=1 --`	Treated as plain text (T3.3)
R2	A database error halfway through saving	Force an error inside a transaction	Whole change rolled back
R3	A row inserted without the app	Bad row inserted directly	Refused by CHECK constraints (T2.11)
R4	Two staff members: one deletes while the other edits	Edit a deleted animal	Clear message, no crash (T2.7)
R5	An older database file from an earlier version	Open an old database	Missing columns added automatically
R6	Far more data than the rescue holds	500 animals, 100 users	Every page still works and is fast

3 Development and testing to inform development

3.1 How development was organised

PawMatch was built iteratively, one component at a time, in the order of the decomposition on D1. Every stage follows the same cycle, and every stage ends with a working, tested program saved as a Git commit:

1. **Aim:** what the stage builds and which design pages and success criteria it covers.
2. **Build:** the code, with annotated excerpts and screenshots.
3. **Test to inform development:** the T tests from the plan on D59, run with pytest. Any failure is shown with its cause, the fix and the retest.
4. **Client feedback:** the client reviews the stage's screens and answers my questions.
5. **Changes:** what I changed because of the feedback, and what I decided not to change and why.
6. **Development assessment:** what now works, what does not yet, and what the next stage must do.

Stage	Built	Commits	Saved test run at the end of the stage
P1	First working prototype on the tested engine	`ffd0ad5`, `10e84e7`, `b76527c`	73 engine tests (`stage1-pytest.txt`), then 117 with the first page tests (`stage3-pytest.txt`)
1	Accounts and login in her design	`847920a`, `f3d44f9`	103 (`stage2-pytest.txt`: accounts, database and staff access added)
2	Staff animal management	`10e84e7`, `847920a`	103 (same run: the staff rules were built with the accounts)
3	Home page, browse and profile	`847920a`	117 (`stage3-pytest.txt`: page tests added)
4	Questionnaire	`e2f5c6a`, `69b2648`	run together with stage 5
5	Matching and results, design re-check	`fdef5a0`, `e2f5c6a`	134 (`stage5-pytest.txt`)
6	Accessibility audit and final checks	`5c09de3`	browser audit (`rendered-audit-*.json`) and 4 style tests
7	Changes from all the reviews; staying signed in after a reload	`77ef20a`, `b0e302a`	191 (`review-pytest.txt`), then 208 (`pytest-junit.xml`)

The engine and the database were built and tested first, and her interface was then ported onto them in one commit (`847920a`). That is why stages 1 to 3 share a commit: each stage's own tests, failures and fixes are in its section.

Program files

The finished program is split into the five components from the decomposition on D1. The rules (`pawmatch/`) never import Streamlit, so they can be tested without a browser; the pages (`webui/`) call the rules but contain none.

File	Lines	What it does
`streamlit_app.py`	28	Starts the app, prepares the database and lists the pages each role may open
`pawmatch/models.py`	132	The records (User, Animal, Answers, MatchResult) and the fixed answer scales
`pawmatch/validation.py`	272	Every validation rule V1 to V16 (D57)
`pawmatch/auth.py`	109	Registration, password hashing with scrypt, login
`pawmatch/matching.py`	166	Weights, scoring, warnings, allergy cap and ranking (D6)
`pawmatch/db.py`	345	The SQLite tables, all queries, sign-in tokens
`pawmatch/services.py`	102	Submitting a questionnaire: validate, score, store (D9)
`pawmatch/seed.py`	146	Demo animals and the first staff account
`webui/state.py`	182	Who is signed in, access checks, sign in and out
`webui/views/*.py`	1,484	One file per page: login, registration, home, browse, questionnaire, matches, account, admin
`webui/pawmatch_style.py`	1,944	Her pink design as one stylesheet
`tests/*.py`	208 tests	Unit, database, access and page tests run with pytest

> **How the client reviews were carried out**
>
> The client's answers in this section were produced by an AI model role-playing the client: it was given only the client interview from the analysis and the screenshots of each stage, and answered my questions in her voice. This was done because the client was not available for six separate review sessions. It is a limitation, discussed in the evaluation (E9) and logged in Appendix A (X1). The reviews were all held after the build, one per stage, so the changes they asked for were made together in stage 7 (commit `77ef20a`). They are reported under the stage they belong to so each stage's story is complete.

3.2 Prototype 1: the engine with a plain interface

Aim. Before building the real pages, prove that the hardest parts work together: the database, validation, accounts and the weighted matching, with a plain Streamlit interface on top. The domain layer was written first with its tests (73 tests at commit `ffd0ad5`, 103 at `10e84e7`), then simple pages (117 tests at `b76527c`).

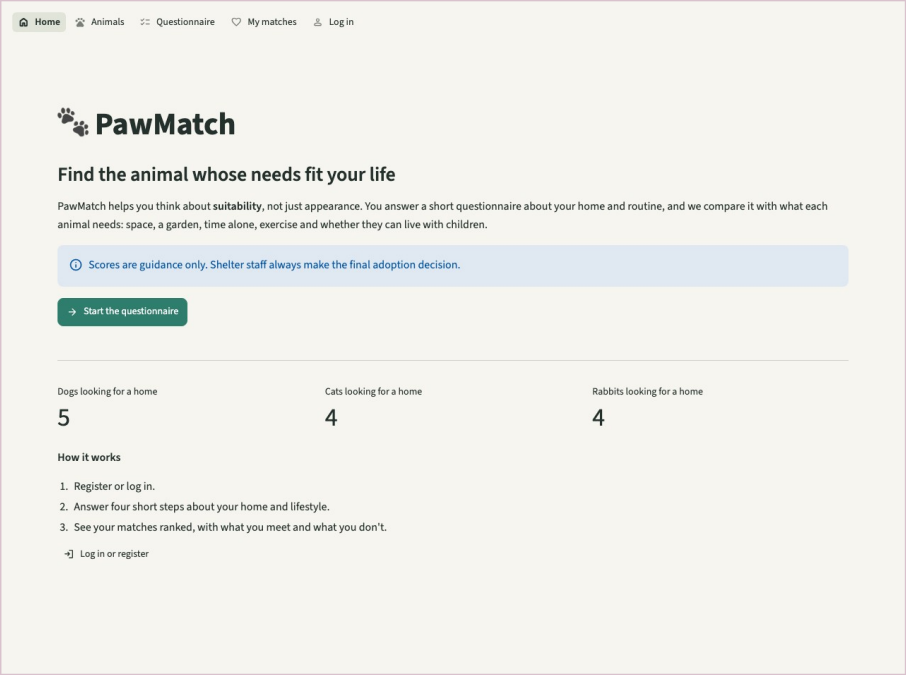


Figure S1: Prototype 1: plain home page in Streamlit's default style

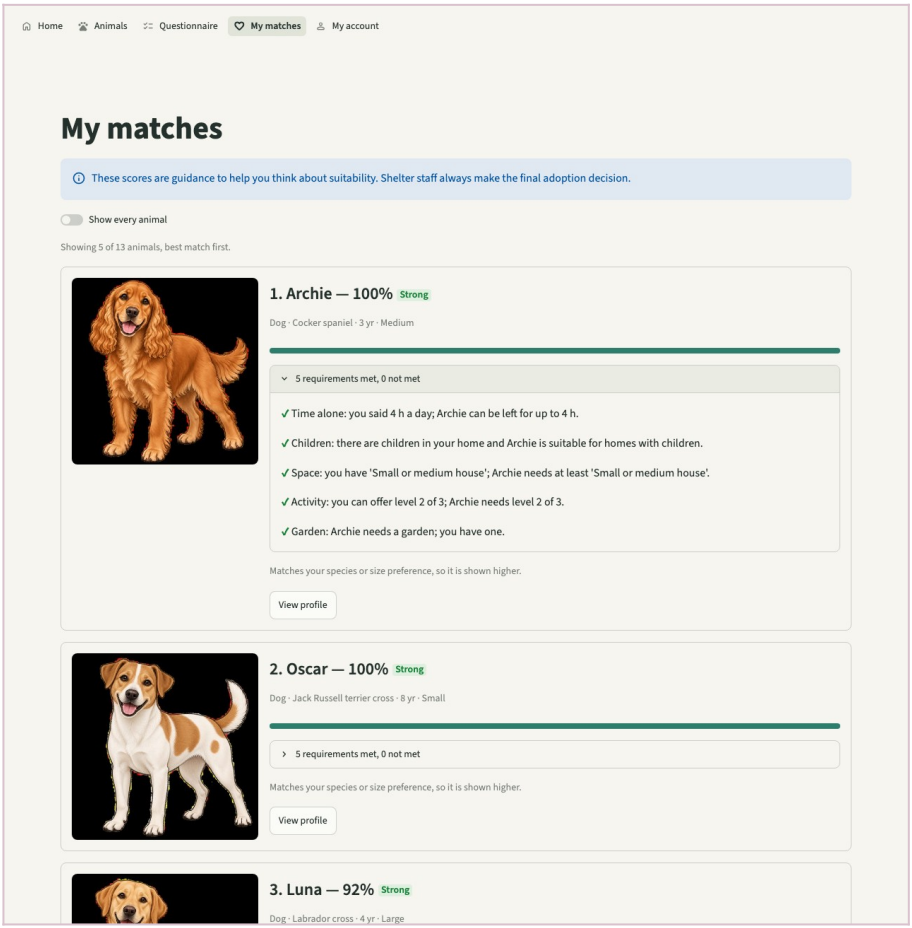


Figure S2: Prototype 1: ranked results with a reason for each requirement (five factors at this point)

Two failures in prototype 1

- **Every page test failed (9 of 9):** `FileNotFoundError: AppTest script not found at rebuild/tests/streamlit_app.py`. Cause: Streamlit's testing tool looks for the app file relative to the test file, not the folder the tests are run from.
- **A questionnaire test failed:** `assert (1 == 2)`. The test expected two error boxes for two unanswered questions; I had merged the messages into one box so the user sees one clear list.

Fixes and retest

- The tests now build the absolute path to `streamlit_app.py` from the test file's own location. Retest: 9 of 9 passed.
- The test was changed to check for one box containing both messages, because one box is the better design. This is a change to the design's expected result, recorded here: the questionnaire later moved to one question per screen (stage 4), so T4.6 checks that Next stays disabled instead. Retest: passed (evidence `docs/evidence/stage3-fail1-apptest-path.txt`, `stage3-fail2-error-box.txt`).

Development assessment P1

Working: accounts, validation, the database, scoring and ranking, all tested. **Not right:** the interface looks nothing like my design or the client's request for a soft pink, friendly look (Q11), and it only scored five factors. **Decision:** keep the tested engine and rebuild every page in my own design (pink palette, Nunito font, card layout) on top of it, starting with login. Re-check the scoring design against the interview before the matching stage.

3.3 Stage 1: accounts and login

Aim. Build the login and registration pages from my designs (D8) on the tested account functions. Covers SC6 and SC7.

Code S1: every registration field is checked and each error is stored against its own field · pawmatch/validation.py, lines 111–128

```
111  def validate_registration(
112      name: Any, email: Any, age: Any, password: Any, confirm: Any
113  ) -> tuple[dict[str, Any], Errors]:
114      """Validate a sign-up form (criteria 5.1 and 5.2)."""
115      cleaned, errors = validate_person(name, email, age)
116      raw_password = "" if password is None else str(password)
117      if not raw_password.strip():
118          errors["password"] = "Enter a password."
119      elif len(raw_password) < PASSWORD_MIN:
120          errors["password"] = f"Password must be at least {PASSWORD_MIN} characters."
121      raw_confirm = "" if confirm is None else str(confirm)
122      if not raw_confirm:
123          # Client review 1: a blank confirm box was not flagged when the password was also blank.
124          errors["confirm"] = "Type your password again to confirm it."
125      elif raw_password.strip() and raw_password != raw_confirm:
126          errors["confirm"] = "The two passwords do not match."
127      cleaned["password"] = raw_password
128      return cleaned, errors
```

Code S2: passwords are stored as salted scrypt hashes, never as text · pawmatch/auth.py, lines 33–37

```
33  def hash_password(password: str) -> str:
34      """Return 'scrypt$n$r$p$salt$hash' with a fresh 16-byte salt."""
35      salt = secrets.token_bytes(16)
36      digest = hashlib.scrypt(password.encode(), salt=salt, n=SCRYPT_N, r=SCRYPT_R, p=SCRYPT_P)
37      return f"scrypt${SCRYPT_N}${SCRYPT_R}${SCRYPT_P}${salt.hex()}${digest.hex()}"
```

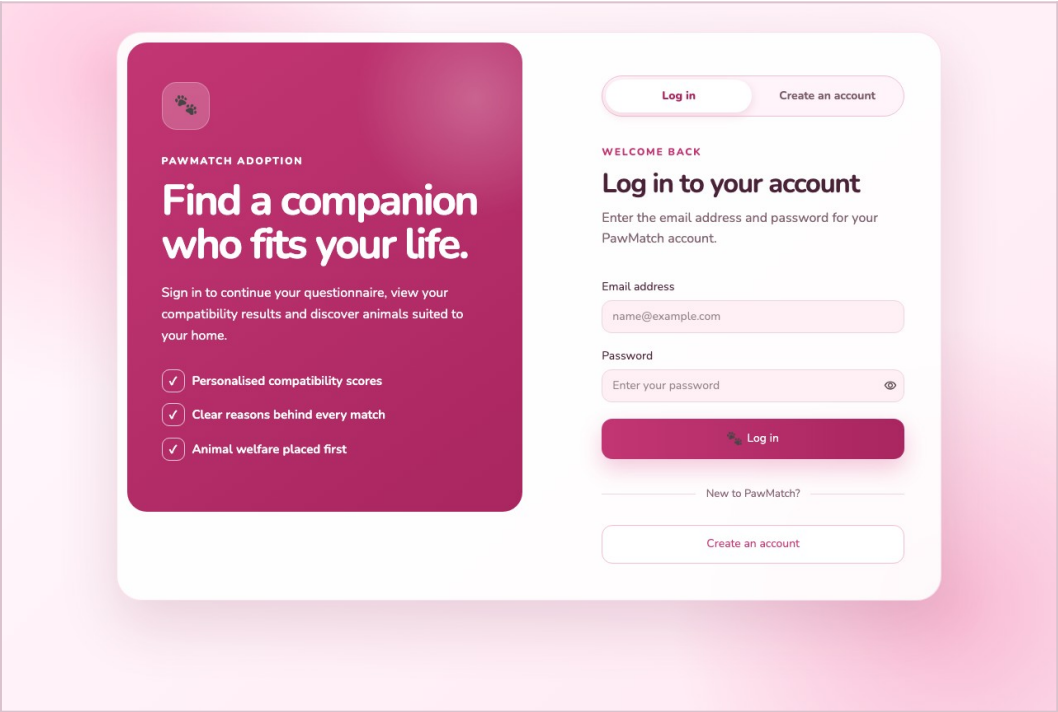


Figure S3: Login page in her design, with the Log in / Create an account switch

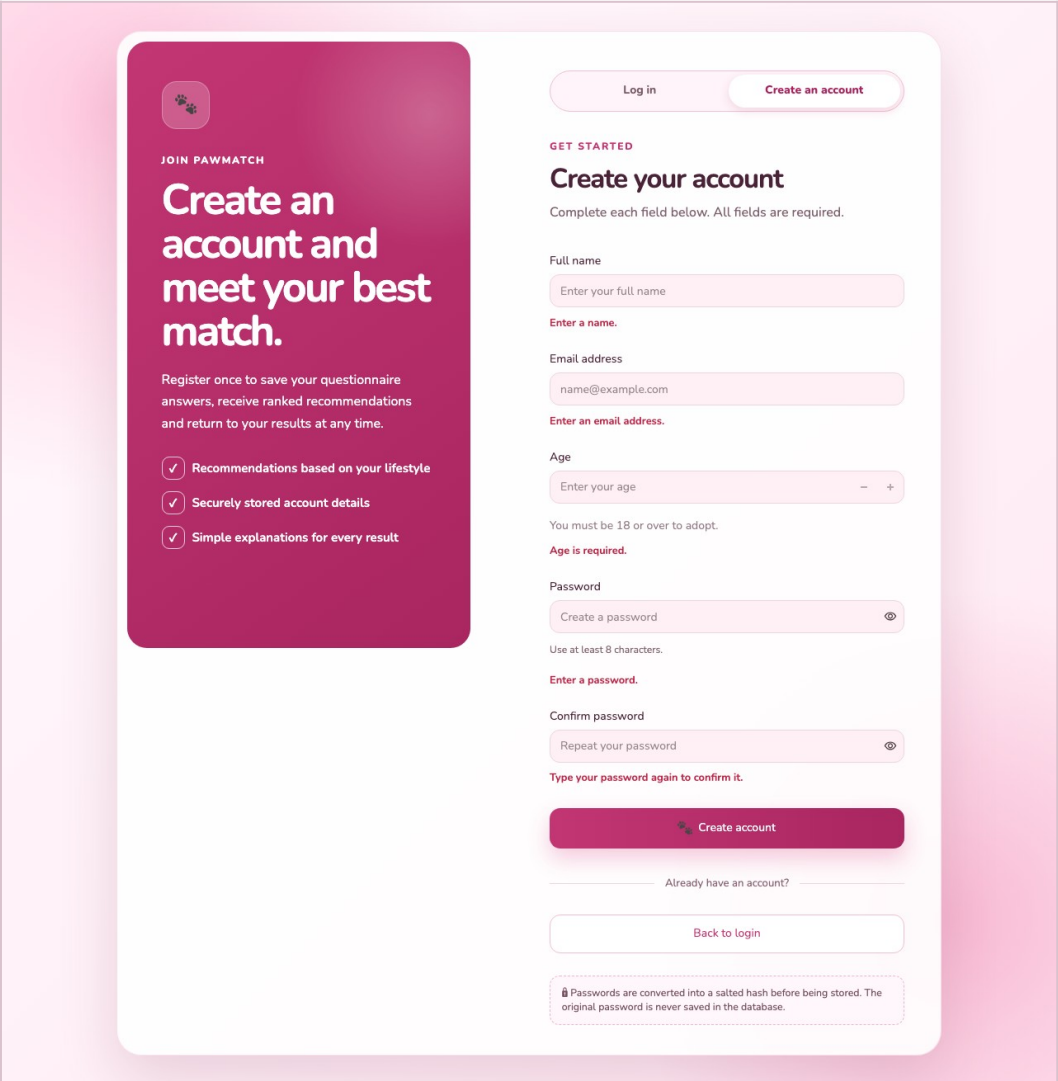


Figure S4: Registration with every box left empty: a message appears under each box

Tests for stage 1

ID	Test	Automated test (pytest)	Result
T1.1	Register a valid adopter	`test_valid_registration_is_cleaned`, `test_registration_creates_an_adopter_account_and_signs_it_in`	Pass
T1.2	Required fields blank	`test_registration_rejects_blank_required_fields` (4 cases), `test_registration_reports_every_missing_field`	Pass
T1.3	Email format	`test_registration_rejects_bad_email_format` (4 cases)	Pass
T1.4	Age 17, 18, 120, 121, 2.5, abc	`test_registration_age_boundaries` (6 cases)	Pass
T1.5	Password 7 and 8 characters	`test_password_minimum_length_boundary`	Pass
T1.6	Confirm password	`test_password_confirmation_must_match`, `test_registration_reports_a_password_mismatch_under_confirm`	Pass
T1.7	Duplicate email, different case	`test_register_rejects_duplicate_email_case_insensitively`	Pass after fix
T1.8	Name format	`test_registration_name_is_letters_spaces_and_hyphens` (6 cases)	Pass
T1.9	Correct login, both roles	`test_adopter_login_opens_the_home_page`, `test_staff_login_opens_the_dashboard`	Pass
T1.10	Wrong password and unknown email	`test_wrong_password_and_unknown_email_give_the_same_message` and two page tests	Pass

T1.7 failed when name checking was tightened

`KeyError: 'email'` in `test_register_rejects_duplicate_email_case_insensitively`. When the name rules were made stricter (letters, spaces, hyphens and apostrophes only), the test's sample name contained a digit, so registration stopped at the name error and never reached the duplicate-email check.

Fix and retest

The sample data was corrected to a valid name so the test checks what it is meant to check, the duplicate email. Retest: passed; the whole suite was re-run to check nothing else had changed (evidence `stage5-fail-regression-auth.txt`).

A real tester found a problem the tests did not

A person trying the live system created an account and was sent back to the login screen, so they thought sign-up had failed. The tests only checked that the account was stored.

Fix and retest

A new account is now signed in straight away and taken to the home page (commit `f3d44f9`). The test was extended to check this: `test_registration_creates_an_adopter_account_and_signs_it_in` now asserts the user is signed in. Retest: passed.

Client feedback 1

#	My question	Client's answer
1	Is the login error message clear enough?	Yes, "Incorrect email or password." is the right amount of information, because it doesn't tell someone whether the email exists, which matters for security. I'd still make it a bit bigger, as it's quite small under the password box and easy to miss.

#	My question	Client's answer
2	Would staff understand what the question mark next to Age on the register page means?	Not really. I can see a little question mark icon by the Age label, but there's no visible text when I look at it, so I'd guess most people would just ignore it. Either show the 18+ rule in plain words underneath, like the password hint does, or leave the icon out.
3	Do the validation messages on the register page tell people what to fix?	Mostly, yes. "Enter a name.", "Enter an email address." and "Age is required." are simple and appear under each box in red, which is easy to spot. But when I look at the Confirm password box, nothing is flagged even though the form was submitted with it empty, so people could think it doesn't matter.
4	Is the home page welcoming enough for someone who has never used it before?	I like it. The pink colours and the three step "How it works" boxes make it clear what the site is for, and the Browse animals and Open questionnaire buttons are clear. The three big cards have a lot of empty white space at the bottom though, so I'd make them shorter or put a photo of an animal in each one, as I asked for lots of photos.
5	Does the register page make people feel safe giving their details?	Yes, the dashed box at the bottom saying passwords are salted and hashed, and never saved in the database, gives me confidence. I would like the form to ask for the address and phone number too, because I said people should give those when they create an account. It's a small thing, but it's what staff will need when they follow up in person.

Client feedback 1: summary and next steps

Rating 7/10, not signed off yet. the client's view: the look is friendly and on-brief, the error and validation messages are mostly clear, and the security note helps trust. The main gaps are the unexplained Age help icon, the Confirm password box not being flagged, and missing address and phone fields that she asked for. Overall the stage is a solid start but not yet ready to use.

Next steps the client asked for:

- Flag the Confirm password box with its own message when it is empty or doesn't match, and make the login error text larger.
- Replace the Age question-mark icon with visible text, such as "You must be 18 or over", and add address and phone number fields on the register page.
- Shorten the three home page cards or add an animal photo to each so the page feels less empty.

Changes made after feedback 1

Changed

- **Blank confirm box now flagged.** If the confirm box is empty, it gets its own message, "Type your password again to confirm it." This changed the expected result of `test_registration_reports_every_missing_field`, which failed as expected (an extra `confirm_password` error, evidence `review-fail1-confirm-blank.txt`); the test was updated to the new requirement and passed.
- **Age rule written on the form.** The hidden "?" help icon was replaced with the visible line "You must be 18 or over to adopt." Test: `test_review1_age_rule_is_written_on_the_form`, passed.

Not changed, with reasons

- **Address and phone number fields.** The client mentioned these in the interview (Q9) and again here. I decided not to collect them: the matching never uses them, storing less personal data is safer (UK GDPR data minimisation), and the rescue collects them at the home visit. This is a partly met requirement in the evaluation.
- **Red outline on the box with the error.** Streamlit does not let a program colour a single input box, so the message under the box is used instead. Listed as future development.

Development assessment 1

Working: registration and login meet SC6 and SC7 in automated tests; new adopters are signed in straight away. **Next:** the staff side, because adopters need animal records to browse.

3.4 Stage 2: staff animal management

Aim. Give staff a dashboard to add, edit and delete animals (D18), with every field validated and every action limited to staff. Covers SC12 and SC13.

Code S3: the gate called first by every staff action · pawmatch/auth.py, lines 105–109

```
105  def require_admin(user: User | None) -> User:
106      """Gate for every staff action (criterion 8.5)."""
107      if user is None or not user.is_admin:
108          raise AccessDenied("Only shelter staff can manage animal records.")
109      return user
```

Code S4: an edit is validated, then saved only if the animal still exists · pawmatch/services.py, lines 31–38

```
 31  def edit_animal(conn: sqlite3.Connection, actor: User | None, animal_id: int, raw: dict[str, Any]) ->
Errors:
 32      require_admin(actor)
 33      values, errors = validate_animal(raw)
 34      if errors:
 35          return errors
 36      if not db.update_animal(conn, animal_id, values):
 37          return {"animal": "That animal no longer exists."}
 38      return {}
```

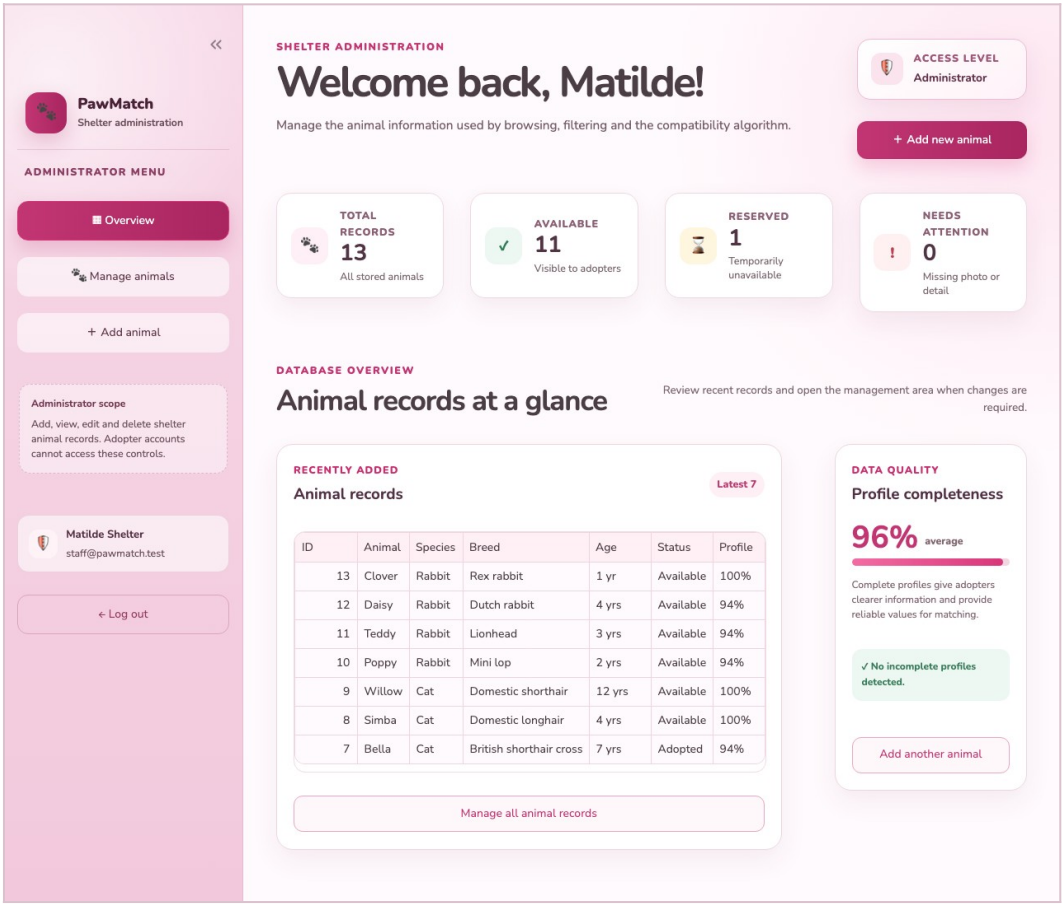


Figure S5: Staff dashboard: counts and records that need attention

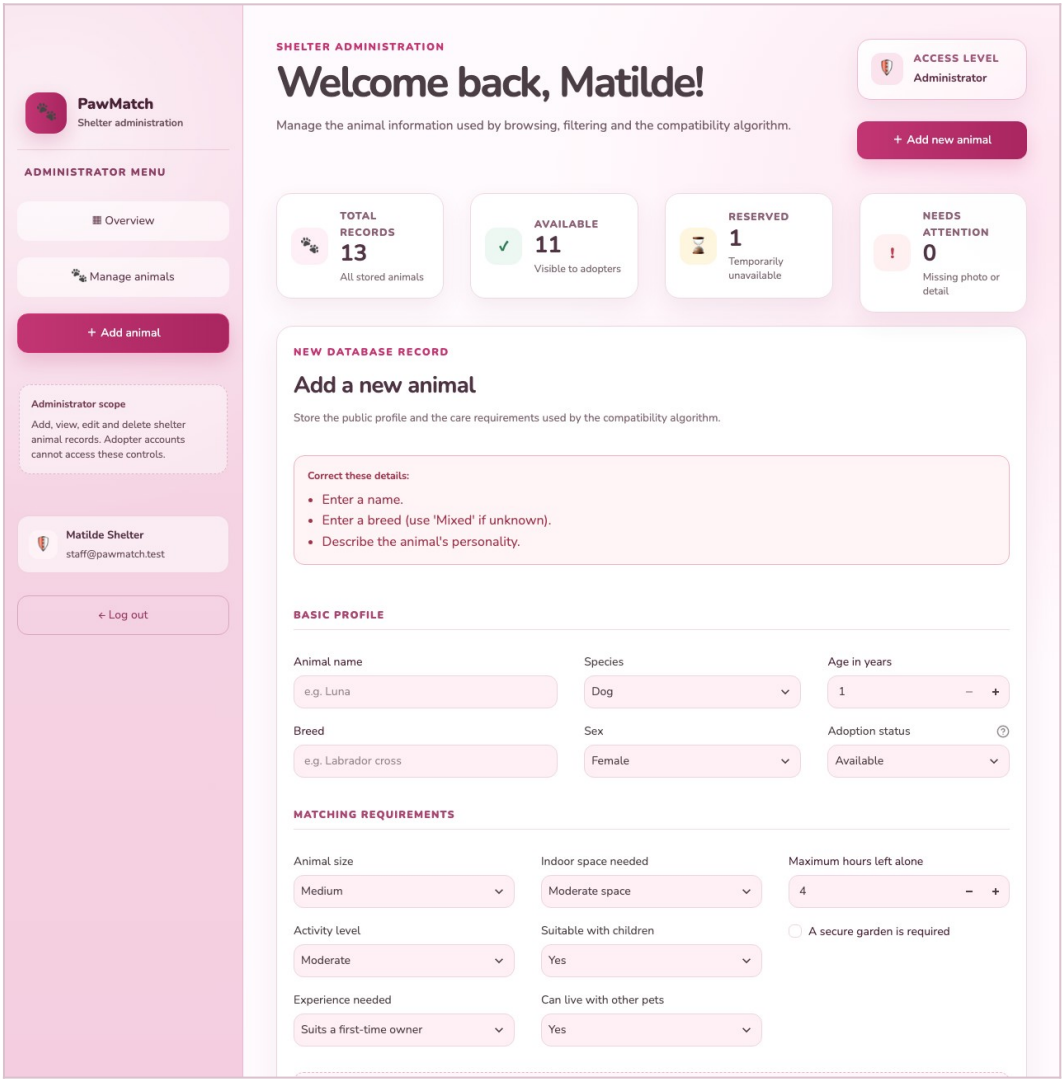


Figure S6: Add-animal form submitted empty: a message for each missing field

Tests for stage 2

ID	Test	Automated test (pytest)	Result
T2.1	Add, edit, delete as staff	`test_admin_can_add_edit_and_delete`, `test_staff_can_add_an_animal_and_then_delete_it`	Pass
T2.2	Required fields blank	`test_staff_add_form_reports_missing_fields`, `test_admin_add_still_validates`	Pass
T2.3	Age 0/31, hours −1/25	`test_invalid_animal_fields_are_rejected` (age, hours cases)	Pass
T2.4	Fixed lists	`test_invalid_animal_fields_are_rejected` (species, size, sex, exercise cases)	Pass
T2.5	Photo type and size	`test_image_type_and_size` (4 cases)	Pass
T2.6	Medical notes 501 characters	`test_invalid_animal_fields_are_rejected` (medical notes case)	Pass
T2.7	Edit a deleted animal	`test_edit_of_deleted_animal_reports_error`	Pass
T2.8	Adopter or signed-out user	`test_non_admins_are_refused_every_change` (2 cases), `test_admin_page_stops_adopters`	Pass
T2.9	Delete removes stored results	`test_delete_removes_animal_and_its_stored_results`	Pass
T2.10	Overview flags weak records	`test_profile_issues_flag_missing_photo_and_short_description`	Pass
T2.11	Database rejects a bad row	`test_database_constraints_back_up_validation`	Pass

Client feedback 2

#	My question	Client's answer
1	Is the error message on the add form clear enough?	Yes, the client would understand it straight away: 'Correct these details' with three plain bullets like 'Enter a name.' is much friendlier than a technical message. The one thing I would change is that the fields themselves don't go red, so I'd have to hunt for the blank boxes.
2	Would staff understand the overview numbers at the top (Total, Available, Reserved, Needs attention)?	Mostly, yes: 13 total, 11 available and 1 reserved add up, and the small captions help. 'Needs attention' showing 0 is handy, but I'd want it to say what it counts, such as missing photo or detail, which it does in small print, so that's good.
3	Is the Recently added table on the overview easy to read?	It's clear and the status and profile percentages are useful for a quick look. However, the 'Added' column is cut off at the right-hand edge, so the date reads '23 Oct' with the year missing, and that needs fixing so the table fits the laptop screen.
4	Is it obvious how to edit or delete an animal on the manage page?	Yes, you pick the animal from the 'Select a record to edit' dropdown and then use the Edit or Delete tabs, which is simple. I'd like a Cancel button next to 'Save changes' so staff can back out without changing anything, and the 'Read-only table' label is confusing next to a page that lets you edit, so please reword or remove it.
5	Does the add animal form explain the matching fields well enough for staff?	The 'Algorithm data' note and the help icon next to Adoption status are a good start, and the sections are well grouped. But the Description box says 'temperament, routine, care needs' while the error asks for 'personality', so please use the same word everywhere so staff know what to write.

Client feedback 2: summary and next steps

Rating 7/10, not signed off yet. the client likes the friendly pink look, the grouped sections and the clear error box on the add form, and she says the layout is easy to follow on a laptop. She wants the tables to fit the screen, the wording to be consistent, and error fields to be obvious. Score 7/10; not yet signed off because this is stage 2, not stage 6.

Next steps the client asked for:

- Make the table on the overview fit the screen so the Added column shows the full date, and check it at laptop width.
- Highlight the specific fields in red when validation fails, not only the summary box, and use one term for the description everywhere.
- Add a Cancel button beside Save changes on the edit form and reword the 'Read-only table' label on the manage page.

Changes made after feedback 2

Changed

- **Table columns sized to fit.** The ID, Age, Status, Profile and Added columns are now narrow so the table fits a laptop screen and "Added" is no longer cut off.
- **Clearer label.** "Read-only table" was replaced by "Pick a record below to edit or delete it", which tells staff what to do.

Not changed, with reasons

- **A Cancel button on the form.** Leaving the page already discards the form; listed as future development.
- **Red outline on the box with the error.** Same Streamlit limitation as stage 1.

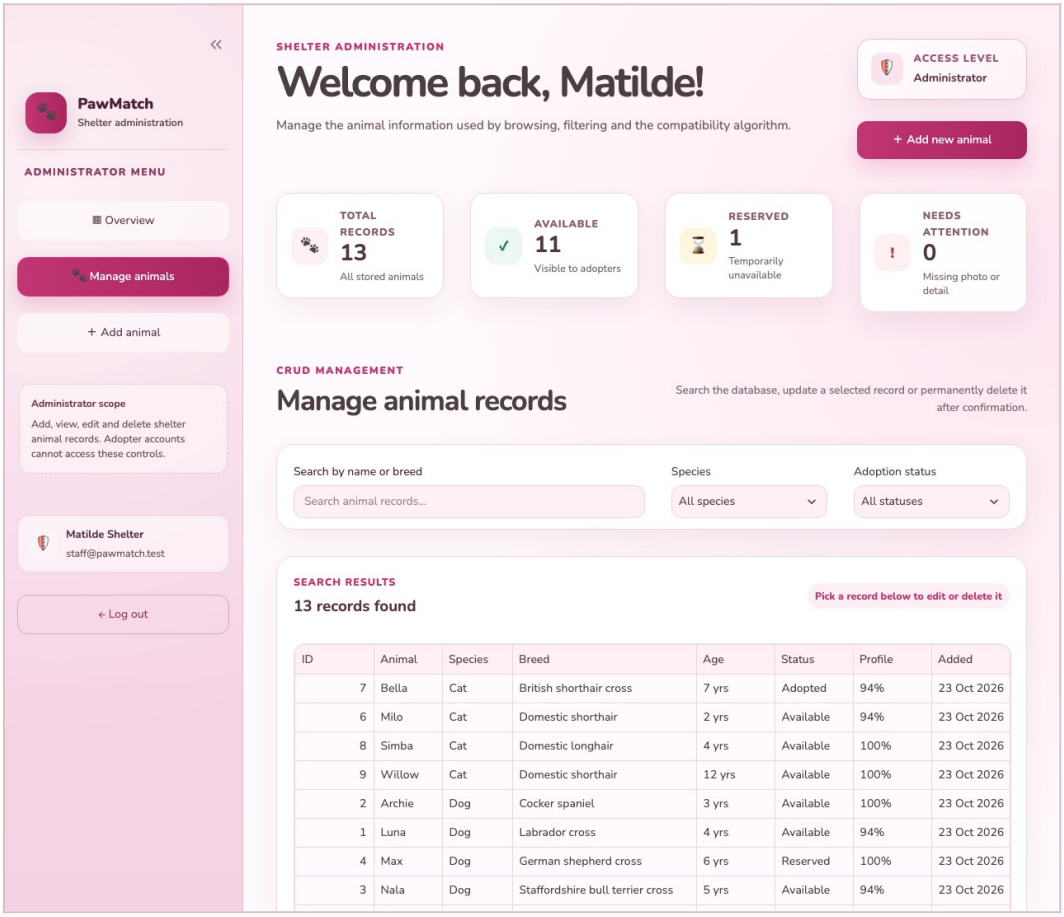


Figure S7: Staff table after the change: every column fits

Development assessment 2

Working: SC12 and SC13 pass their automated tests; adopters cannot reach any staff action even by calling the functions directly. **Next:** the adopter pages that show these records.

3.5 Stage 3: home page, browse and profile

Aim. Build the home page, the browse page with filters and the profile page (D29). Covers SC3, SC4 and SC5.

Code S5: filters are added as ? parameters, so search text can never run as SQL · pawmatch/db.py, lines 232–253

```
232 def list_animals(
233     conn: sqlite3.Connection, species: str | None = None, size: str | None = None, search: str = "",
234     status: str | None = None,
235 ) -> list[Animal]:
236     """Browse query with optional filters; the search term is bound, never concatenated."""
237     sql = "SELECT * FROM animals WHERE 1 = 1"
238     params: list[Any] = []
239     if status:
240         sql += " AND status = ?"
241         params.append(status)
242     if species:
243         sql += " AND species = ?"
244         params.append(species)
245     if size:
246         sql += " AND size = ?"
247         params.append(size)
248     if search.strip():
249         sql += " AND (name LIKE ? OR breed LIKE ?)"
250         term = f"%{search.strip()}%"
251         params += [term, term]
252     sql += " ORDER BY species, name"
253     return [_animal(r) for r in conn.execute(sql, params)]
```

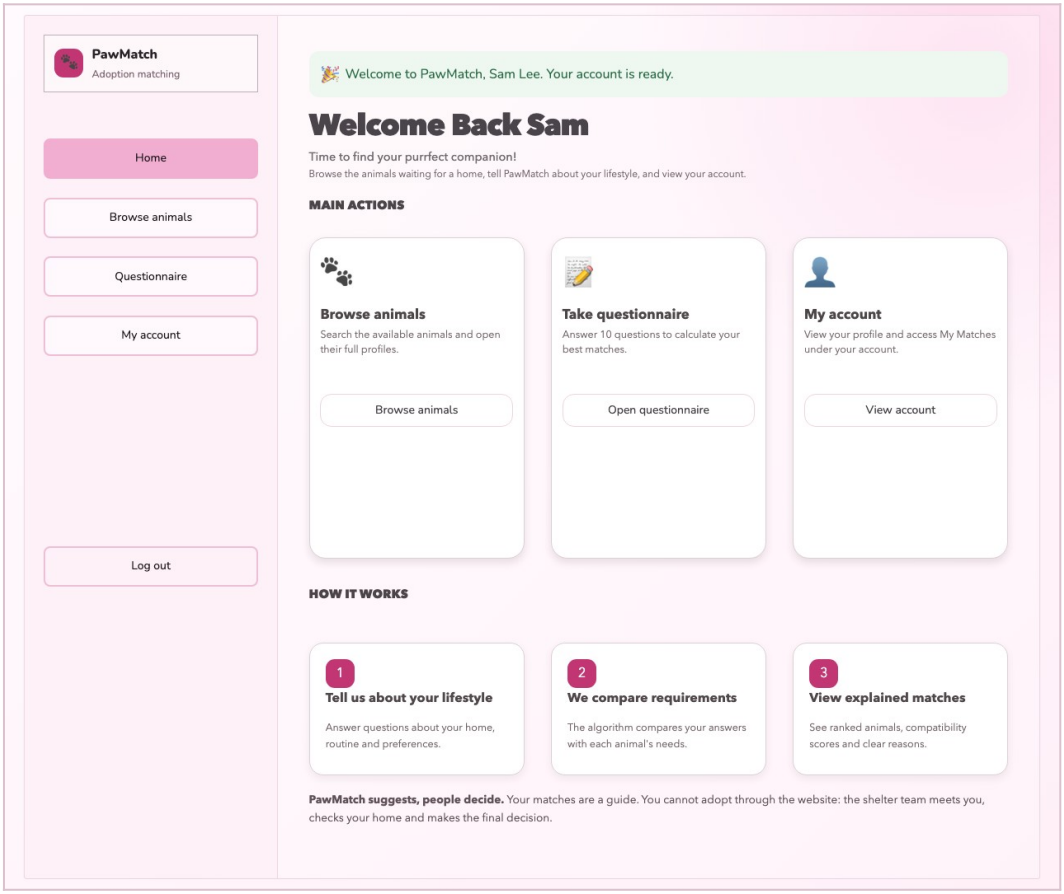


Figure S8: Home page: what PawMatch does and that staff make the decision

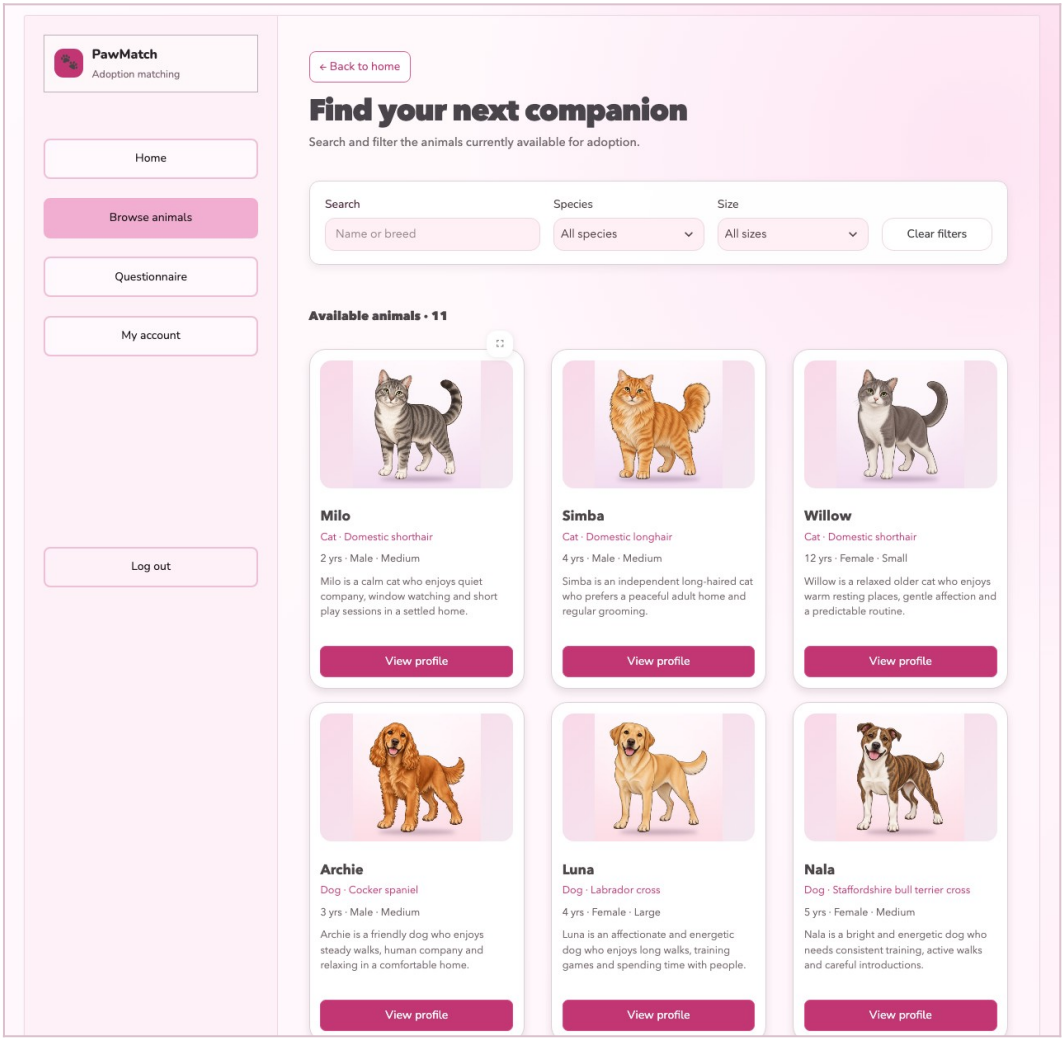


Figure S9: Browse: a card for each available animal, with species, size and search filters

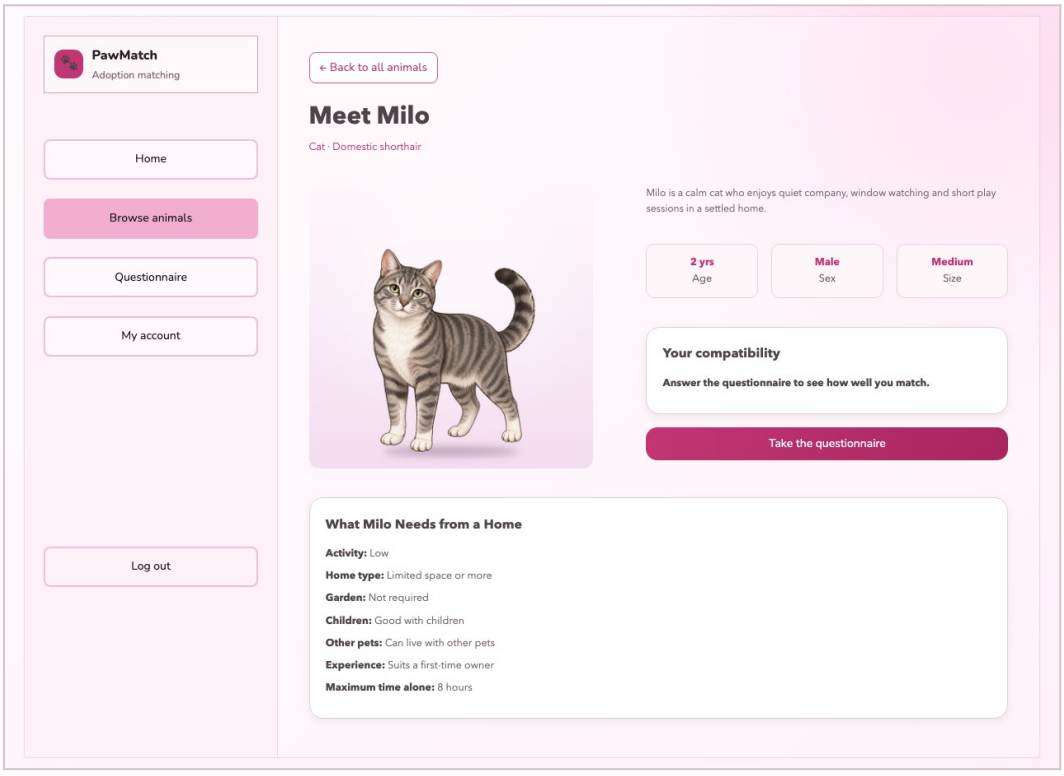


Figure S10: Profile: all of the animal's details and needs

Tests for stage 3

ID	Test	Automated test (pytest)	Result
T3.1	Every Available animal has a card	`test_browse_shows_a_card_for_every_available_animal`	Pass
T3.2	Species, size and search filters	`test_filters_by_species_size_and_search`, `test_review3_size_filter_narrows_the_list`	Pass
T3.3	Search text is data	`test_search_text_is_treated_as_data_not_sql`	Pass
T3.4	Profile shows needs	`test_view_profile_opens_the_animal_with_its_needs`	Pass
T3.5	Signed-out visitor	`test_adopter_pages_send_visitors_to_log_in`	Pass

Client feedback 3

#	My question	Client's answer
1	Is the Species filter on the Browse animals page clear enough for someone who has never used the site before?	Yes, I like that it is a simple dropdown that says All species, so people can pick cats, dogs or rabbits straight away. It sits next to the search box, which is where I would look for it. Keep the labels this plain.
2	Can a user filter by size on the Browse animals page?	Not from what I can see. I asked for size to be a filter, and the screenshot only shows Search, Species and Clear filters. Please add a Size dropdown beside Species, because I think people should be able to narrow it down by size as well as type.
3	Does the 'Your compatibility' box on the animal profile make it clear what the user should do next?	It says 'Complete the questionnaire to see compatibility', which tells them the score is missing. It would be much friendlier as a button that links straight to the questionnaire, so they do not have to hunt around the menu.
4	Is the 'What Milo Needs from a Home' section easy for a first-time owner to understand?	Yes, this is the bit I was most keen on, because each need is written as a label and a plain answer, like 'Garden: Not required'. The description text above it is quite small and light grey, though, so please make it a bit bigger and darker.
5	Does the Home page explain that staff make the final decision and that users cannot apply directly?	Not clearly. The 'How it works' steps talk about lifestyle, comparing requirements and explained matches, but nothing says the adoption team will meet and check people in person. Please add one short line saying that PawMatch helps you find a match and the shelter staff make the final decision.

Client feedback 3: summary and next steps

Rating 7/10, not signed off yet. Stage 3 looks warm, friendly and on-brief, with soft pink tones, big animal photos and clear buttons, which is what I asked for. The compatibility-led profile needs is the strongest part. The main gaps are the missing size filter and nothing on the Home page saying staff make the final call.

Next steps the client asked for:

- Add a Size filter beside Species on the Browse animals page.
- Turn the 'Complete the questionnaire' text on the profile into a button linking to the questionnaire.
- Add one line on the Home page saying shelter staff make the final decision after an in-person check.
- Make the profile description text larger and darker so it is easy to read on a laptop.

Changes made after feedback 3

Changed

- **Size filter added** next to species (the client asked for "type of animal and size" in Q8 too). Test `test_review3_size_filter_narrows_the_list`, passed.
- **"Take the questionnaire" button on the profile** in place of a sentence, so the next step is one click. Test `test_review3_profile_offers_the_questionnaire_as_a_button`, passed.
- **Home page says people decide:** "PawMatch suggests, people decide... the shelter team meets you, checks your home and makes the final decision." Test `test_review3_home_says_staff_make_the_decision`, passed.

Development assessment 3

Working: SC3 to SC5. **Next:** the questionnaire, which the browse and profile pages now point to.

3.6 Stage 4: questionnaire

Aim. One question per screen in four sections, with card answers and Next disabled until a question is answered (D42). Covers SC8. While building it, the design re-check (next stage) added three questions, so the questionnaire has ten.

Code S6: the whole questionnaire is checked again on submit, so nothing incomplete is ever stored · pawmatch/validation.py, lines 255–259

```
255  def validate_answers(raw: dict[str, Any]) -> tuple[Answers | None, Errors]:
256      """Validate the whole questionnaire; returns Answers only when every answer is valid."""
257      errors: Errors = {}
258      cleaned = {name: check(raw.get(name), errors) for name, check in QUESTION_VALIDATORS.items()}
259      return (None, errors) if errors else (Answers(**cleaned), errors)
```

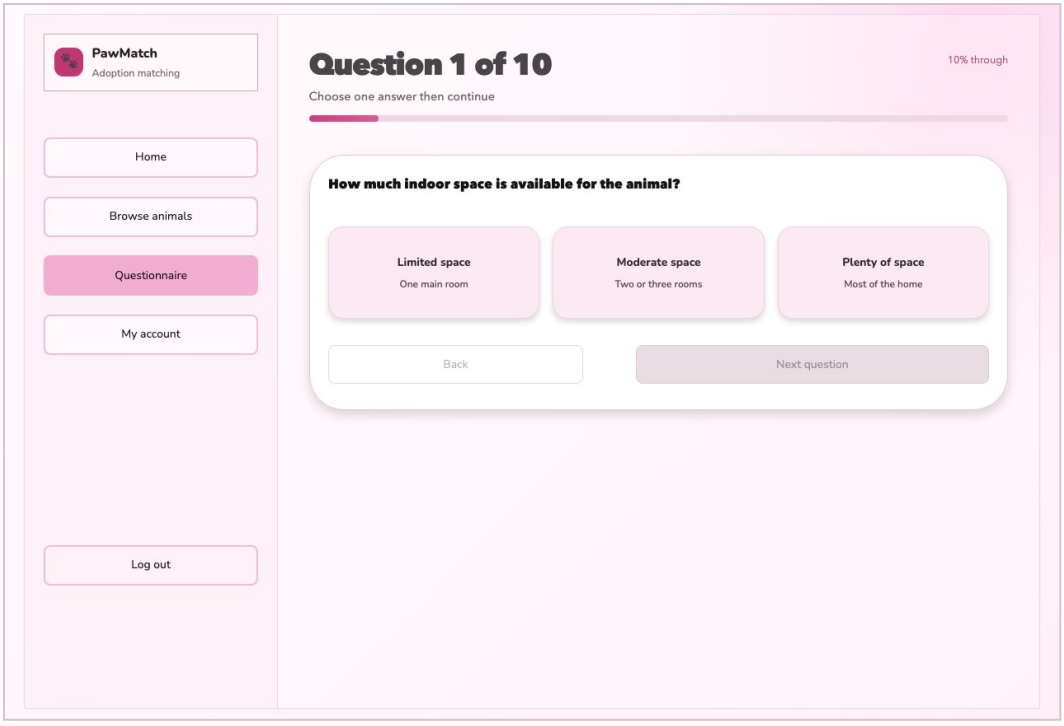


Figure S11: Question 1 of 10: card answers, progress and section name

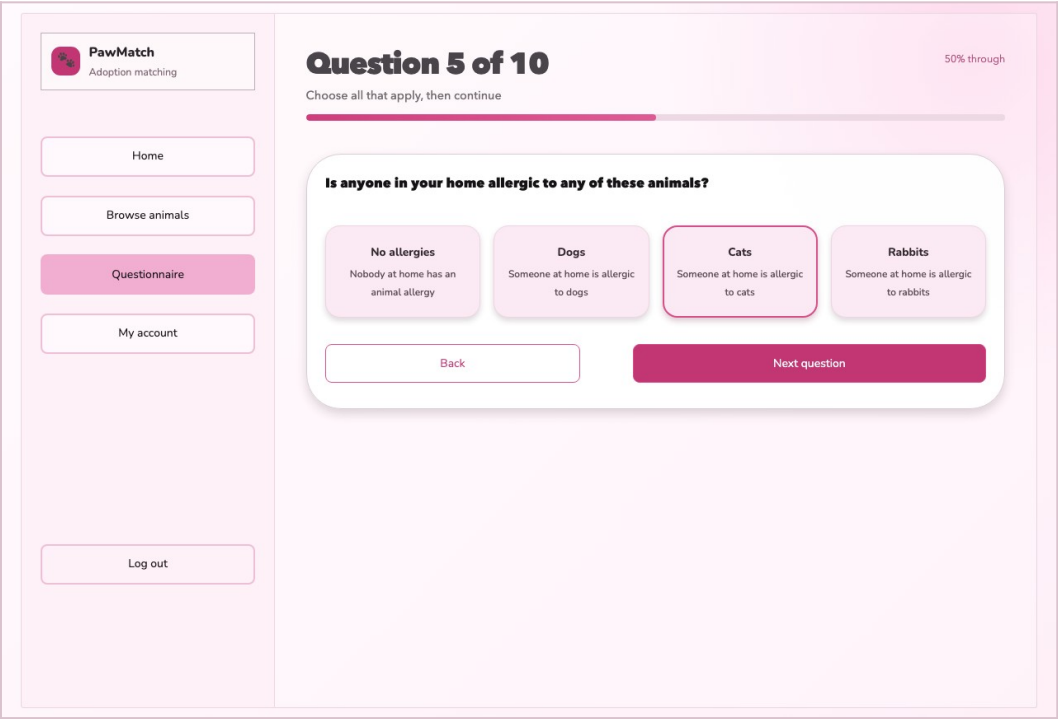


Figure S12: The allergies question allows several answers; "No allergies" clears the others

Tests for stage 4

ID	Test	Automated test (pytest)	Result
T4.1	Next waits for an answer	`test_questionnaire_waits_for_an_answer_before_next`, `test_step_validation_only_checks_that_step`	Pass
T4.2	Hours −1, 0, 24, 25	`test_hours_alone_range` (4 cases)	Pass
T4.3	Allergies	`test_allergies_accept_none_or_some_species_in_a_fixed_order` (5), `test_allergies_reject_…` (4)	Pass
T4.4	"No allergies" clears others	`test_allergy_cards_turn_on_and_off_and_no_allergies_clears_them`	Pass
T4.5	Complete questionnaire	`test_questionnaire_completes_and_ranks_every_available_animal`, `test_adopter_submission_stores_ranked_results`	Pass
T4.6	Incomplete submission	`test_unanswered_question_blocks_submission`, `test_incomplete_submission_saves_nothing`	Pass

Client feedback 4

#	My question	Client's answer
1	Q1 (indoor space): Are the three space options clear enough for someone who has never adopted before?	Yes, I like that each card says what the space looks like, like 'One main room' or 'Most of the home'. That's much easier than a plain dropdown. I would still like the question to say why you're asking, because people often don't think about space until it's too late.
2	Q5 (allergies): If someone ticks 'Cats' and then 'No allergies' by mistake, what happens?	In the screenshot I can see 'Cats' is outlined, but 'No allergies' doesn't look any different, so I can't tell whether it's still ticked. Please make 'No allergies' clear the other choices, and make the selected state much more obvious, because allergies have to be right for the matching to work.
3	Q7 (hours left alone): The box already says 6. Would someone understand they need to change it, or could they just click Next?	Pre-filling 6 is a problem, because a person could skip the question without thinking and the system would treat 6 as their real answer. Please leave it empty until they type something, and show a clear message if they try to continue without a number. The minus and plus buttons are nice, though.
4	Q11 (done screen): After 'Your questionnaire is complete', would I know what to do next and whether any animals matched?	The 'View matches' button is big and clear, which I like. But the message doesn't say how many matches there are, or that staff will look at things next, so people might not know what happens after this. Maybe add a line like 'You have 3 good matches' so they feel the result is useful.

#	My question	Client's answer
5	Q1 vs Q7 buttons: Are the Back and Next buttons easy to find and understand across the questions?	The layout is consistent and the pink look is soft and friendly, as I asked for. The 'Next question' button on question 1 is greyed out before anything is chosen, which is good, but the grey text is quite faint and hard to read on a laptop screen. Please make the disabled state a bit darker so it's still easy to see.

Client feedback 4: summary and next steps

Rating 7/10, not signed off yet. the client is generally happy with the stepped questionnaire: the card choices are friendly and the pink look matches what she asked for. The main concerns are the allergy multi-select not clearly clearing 'No allergies', the pre-filled 6 hours letting people skip the question, and a done screen that doesn't say how many matches there are.

Next steps the client asked for:

- Make 'No allergies' exclusive on question 5 and show a clearer selected state on the cards.
- Remove the pre-filled 6 on question 7 and show a validation message until a number is entered.
- Add a match count and a short 'what happens next' line on the 'Your questionnaire is complete' screen.

Changes made after feedback 4

New test failed: the hours question was already filled in

The client noticed the hours box already showed 0, so someone could press Next without thinking. I wrote the test first: `test_review4_hours_question_starts_empty` failed with `assert (0 is None)` (evidence `review-fail2-hours-prefilled.txt`). Cause: the function that sets up the answers gave the typed question a starting value of 0.

Fix and retest

Every answer now starts empty (`None`); the hours box shows the hint "Type a number from 0 to 24" and Next stays disabled until a number is typed. Retest: passed, and the full suite still passed.

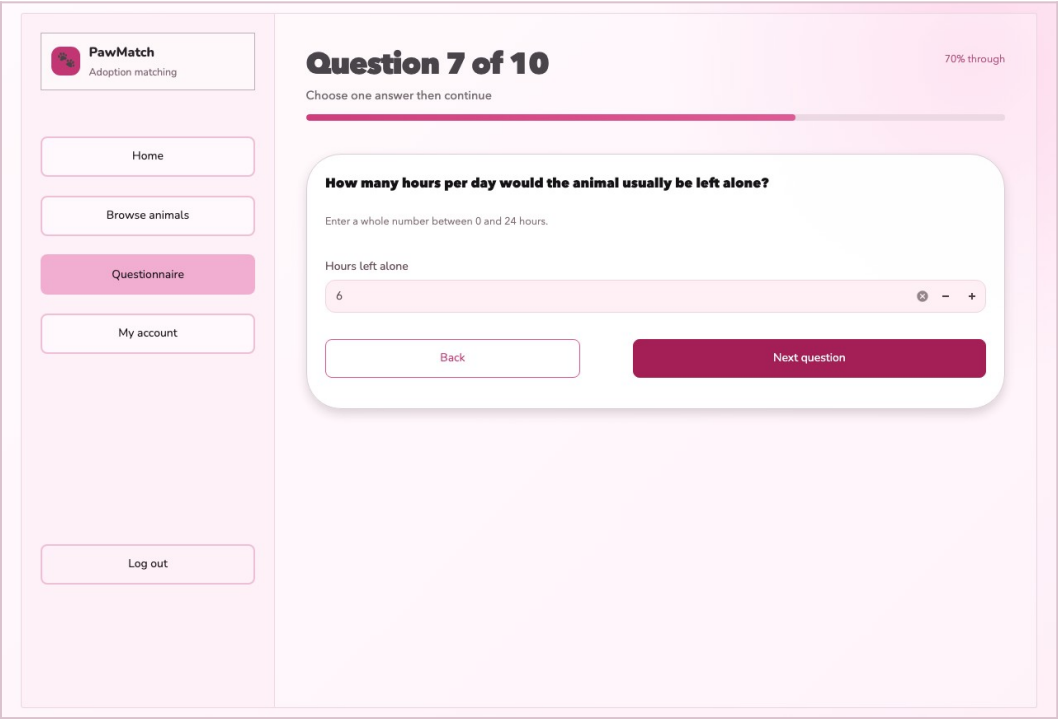


Figure S13: After the change: the hours box starts empty with a hint

Not changed, with reasons

- **"No allergies" behaviour.** It already cleared the other answers (T4.4); the review was looking at a screenshot taken before a card was clicked.
- **Number of matches on the "done" screen, and fainter disabled buttons.** Not done in time; both are in the evaluation.

Development assessment 4

Working: SC8 in automated tests (the timing part is measured in the evaluation). **Next:** use the answers to score and rank.

3.7 Stage 5: matching and results

Aim. Score and rank every available animal and show each result with its reasons (D47). Covers SC9, SC10 and SC11.

Design re-check before building

Before building this stage I checked the design against every interview answer again. Three things the client said were missing from my five-factor plan:

Client said	Missing from the plan	Change
"If someone is allergic to cats, they should not be strongly matched with a cat" (Q10)	No allergy question	New multiple-choice question; an allergy caps the score at 39% (Low) and adds a warning
Animals "get along with children or other pets" (Q2, Q10 summary)	Other pets not scored	New question and factor, weight 10, warns if unmet
People forget "whether they have experience" (Q1)	Experience not scored	New question and factor, weight 8, half points for one level short

The re-check also found two smaller gaps, each shown by a test written first that failed: match results did not record when they were calculated (`no such column: calculated_at`), and animal names accepted symbols (`assert ('name' not in {}) is False`). Both were fixed, with an automatic upgrade that adds the column to older database files (R5) (evidence `stage5-fail-design-recheck.txt`, then 134 passed).

Final weights

Factor	Weight	Warning if unmet?	Why this weight (source)
Hours alone	20	Yes	Biggest welfare risk named by the client (Q1, Q10) and by the survey (S5: time available 85.2%).
Children	15	Yes	Safety: a dog that is not child-safe in a family home is the outcome the client most wants to stop (Q10).
Housing	15	Yes	Living space was the top survey factor (S5: 88.9%).
Activity	12	No	An under-exercised animal is a common reason for returns (Q1). Half points for one level short.
Garden	10	No	Some animals need outdoor space; others do not (Q2, S5).
Other pets	10	Yes	Added in development stage 5 after a re-check against the interview: other pets are a key factor in the client's summary (Q2, Q10).
Experience	8	No	Added in stage 5 for the same reason: experience is a factor people forget (Q1). Half points for one level short.
Total	**90**		The percentage is points earned ÷ 90 × 100, so changing one weight cannot break the scale.

Worked example: checking the program by hand

An adopter in a flat with limited space and no garden, with children and another pet, some experience, activity level 1, out 6 hours a day, preferring a cat. Oscar is a dog who needs activity level 2 and can be left for at most 4 hours.

Factor	What the animal needs	What the adopter answered	Fraction	Weight	Points
Hours alone	Left alone at most 4 h	6 h a day	0	20	0
Children	Safe with children: Yes	Children at home: Yes	1	15	15
Housing	At least 'Limited space'	'Limited space'	1	15	15
Activity	Activity level 2 of 3	Can offer level 1 of 3	0.5	12	6
Garden	Needs a garden: No	Has a garden: No	1	10	10
Other pets	Good with other pets: Yes	Other pets at home: Yes	1	10	10
Experience	Needs some experience	Some (family pet growing up)	1	8	8
Total				**90**	**64**
Score = 64 ÷ 90 × 100, rounded to 1 decimal place					**71.1% (Good)**

So Oscar scores **71.1%** (Good) but has 1 welfare warning, because 6 hours alone is more than his 4. The ranking rule puts every animal without a warning above him, even though his percentage is higher than some of them: this is the client's rule that a welfare risk must not be presented as a top match (Q3). The automated test `test_worked_example_in_the_write_up_matches_the_program` checks that the program gives exactly this result (T5.8).

Code S7: each factor adds its weight times the fraction met; the allergy cap is applied last · pawmatch/matching.py, lines 121–150

```
121 def score_animal(animal: Animal, answers: Answers) -> MatchResult:
122     """Score one animal: percentage, category, met/unmet lists and warnings."""
123     earned = 0.0
124     met: list[str] = []
125     unmet: list[str] = []
126     warnings: list[str] = []
127     for factor, fraction, reason in _factor_checks(animal, answers):
128         earned += WEIGHTS[factor] * fraction
129         if fraction == 1.0:
130             met.append(reason)
131         else:
132             unmet.append(reason)
133             if factor in IMPORTANT_FACTORS:
134                 warnings.append(reason)
135     compatibility = round(100 * earned / MAX_POINTS, 1)
136     if animal.species in answers.allergies:
137         reason = (f"Allergy: someone in your home is allergic to {animal.species.lower()}s; "
138                   f"{animal.name} is a {animal.species.lower()}.")
139         unmet.insert(0, reason)
140         warnings.insert(0, reason)
141         compatibility = min(compatibility, ALLERGY_CAP)
142     return MatchResult(
143         animal=animal,
144         compatibility=compatibility,
145         category=category_for(compatibility),
146         bonus=preference_bonus(animal, answers),
147         met=met,
148         unmet=unmet,
149         warnings=warnings,
150     )
```

Code S8: the sort key: animals without warnings first, then points, then a fixed tie-break · pawmatch/matching.py, lines 153–161

```
153 def rank_key(result: MatchResult) -> tuple[int, float, float, str, int]:
154     """Suitable animals first, then ranking points, compatibility, name and id (all deterministic)."""
155     return (
156         0 if result.suitable else 1,
157         -result.ranking_points,
158         -result.compatibility,
159         result.animal.name.lower(),
160         result.animal.animal_id,
161     )
```

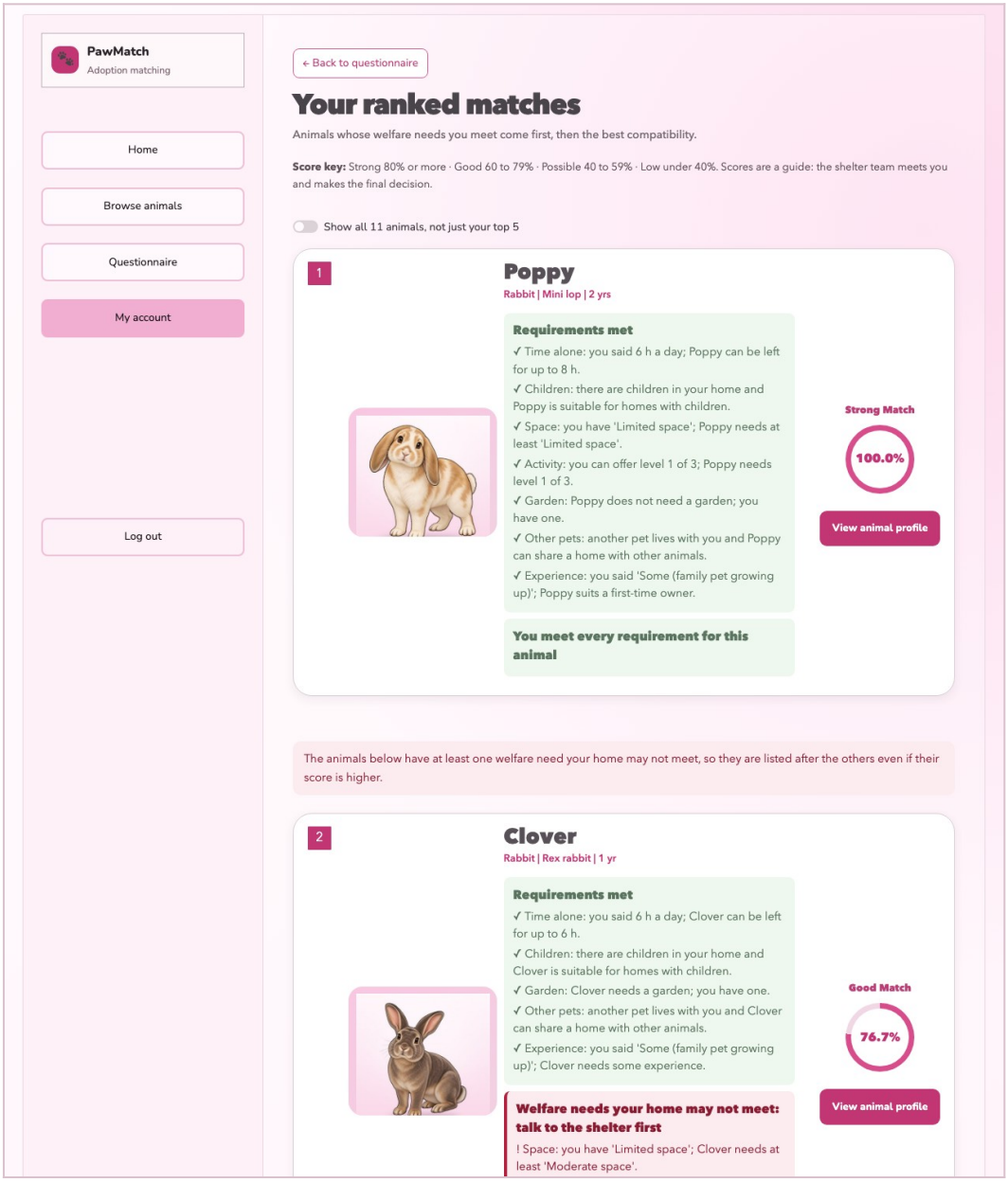


Figure S14: My matches: score key, ranked cards, met and not-met reasons, welfare warnings

Tests for stage 5

ID	Test	Automated test (pytest)	Result
T5.1	Perfect match	`test_perfect_match_scores_100`	Pass
T5.2	Each factor loses its weight	`test_each_factor_loses_its_weight` (10 cases)	Pass
T5.3	Boundaries count as met	`test_boundaries_count_as_met`	Pass
T5.4	Category thresholds	`test_category_thresholds` (8 cases)	Pass
T5.5	Important factor warns	`test_important_unmet_requirements_raise_warnings`, `test_garden_and_activity_shortfalls_do_not_warn`	Pass
T5.6	Allergy	`test_an_allergy_caps_the_score_in_the_low_band_and_warns_first` and 3 more	Pass
T5.7	Ranking order	`test_unsuitable_animals_always_rank_after_suitable`, `test_preferences_change_rank_but_not_percentage`	Pass
T5.8	Hand calculation	`test_worked_example_in_the_write_up_matches_the_program`	Pass
T5.9	Results page	`test_matches_page_shows_the_top_five_and_can_show_all`	Pass
T5.10	Empty and identical lists	`test_ranking_empty_list`, `test_ranking_is_deterministic_for_identical_animals`	Pass

Client feedback 5

#	My question	Client's answer
1	On Poppy's card (rank 1), the amber 'Requirements not met' box is shown but says 'No requirements are unmet for this animal'. Is that clear enough?	Not really, the client would be confused. If nothing is unmet, I'd rather that box was hidden or said something green like 'You meet every requirement', because an amber box looks like a warning.
2	Teddy (72.2%) is ranked 4th, above Oscar and Clover, even though Teddy fails two welfare needs (children and other pets). Would staff understand why?	No. The spec said animals with warnings go last, but on this screen almost every card has amber items and Teddy sits above Oscar, so it looks like the ranking ignores the warnings. Please make the order obvious, or put welfare-need animals below the rest and say so on screen.
3	On Milo's profile, is the allergy line ('! Allergy: someone in your home is allergic to cats; Milo is a cat.') clear enough for an adopter?	It is easy to miss. It is the last line in a plain list with only a small exclamation mark, and it is the one thing that really matters for a cat. Make it a bold red or amber warning at the top of the compatibility box, with the words 'not recommended'.
4	Does the 'Low match' label with 39.0% on Milo's profile make sense without any explanation?	Mostly, but the scale is not explained anywhere. A short line such as 'Strong 80%+, Good 60-79%, Low below 60%' under the score, and a note that staff make the final decision, would help people understand it.
5	On the My account page, can a user see how to redo the questionnaire or where the staff decision note is?	No. It shows 'Questionnaire: Complete' and a 'View my matches' button, but nothing lets them change their answers or see that the shelter decides. Add a 'Update questionnaire' button and a short line under 'My matches' saying that staff make the final decision.

Client feedback 5: summary and next steps

Rating 7/10, not signed off yet. the client likes the soft pink look, the clear buttons and the 'Requirements met / not met' split, which she says is the most useful part. She is not yet convinced the ranking respects welfare needs, and the allergy warning and the zero-unmet box need clearer styling. Overall the stage is close, but the ordering rule needs to be visible on screen before it is used.

Next steps the client asked for:

- Hide or recolour the 'Requirements not met' box when nothing is unmet (Poppy card).
- Make the ranking visibly follow the rule: animals with welfare-need failures go below those without, and show the allergy warning as a bold top-of-box warning on profiles.
- Add a one-line score key and a 'Update questionnaire' button on My account, then re-screenshot the three pages.

Changes made after feedback 5

Changed

- **Welfare warnings stand out.** Unmet welfare needs now appear first, in a red box headed "Welfare needs your home may not meet: talk to the shelter first". The amber "Requirements not met" box appears only when there is something in it; when everything is met, a green box says so.
- **The order is explained.** A line appears once, above the first animal with a warning, saying these animals are listed after the others even if their score is higher. This answers the client's worry that ranking "ignored" warnings: it did not, but it did not say so.
- **Score key** under the heading: "Strong 80% or more · Good 60 to 79% · Possible 40 to 59% · Low under 40%", with "the shelter team… makes the final decision".
- **Update answers from My account.** Test `test_review5_account_lets_answers_be_updated`, passed. Test for the first three: `test_review5_matches_explain_the_score_and_the_order`, passed.

Development assessment 5

Working: SC9 to SC11, including a hand-checked example. **Next:** check that every screen is readable for every user, which the client and my research (E5) both asked for.

3.8 Stage 6: accessibility and final checks

Aim. Make sure every screen meets SC1 and SC16: contrast of at least 4.5:1 (WCAG 2.2 level AA) and no text under 12 px, at laptop and tablet widths.

I wrote two automated tests that read the stylesheet, work out every text colour against the background behind it, and measure its contrast with the WCAG formula. A third tool opens every screen in a real browser at 1440 and 1024 px and measures the text actually drawn, including hover and keyboard focus.

Code S9: contrast ratio exactly as WCAG defines it (checked against WCAG's own examples by a test) · tests/test_style.py, lines 68–70

```
68  def contrast(first: str, second: str) -> float:
69      light, dark = sorted((_luminance(first), _luminance(second)), reverse=True)
70      return (light + 0.05) / (dark + 0.05)
```

The first run failed badly

`66 text colours below 4.5:1` and `37 font sizes below 12px` (evidence `style-test-before.txt`). The original pastel text colours looked soft but were too faint for older users (the survey and persona P3).

Fix and retest

Each failing colour was darkened to the nearest shade of the same pink that reaches 4.5:1, so the palette still looks like the design, and every size under 12 px was raised. Retest: both stylesheet tests pass.

The browser audit found problems the stylesheet tests had passed

Opening each page in a real browser showed faint text the stylesheet tests could not see: the side-menu text, text on see-through panels, the photo upload hint, the focus ring on the sign-in switch, and white labels on buttons while hovered.

Fix and retest

Each was fixed, and the stylesheet test was improved to model the side menu, see-through fills and hover states so these problems cannot return unnoticed (commit `5c09de3`). Retest: the audit of 18 screens at each width reports 0 low-contrast texts, 0 small texts and no focus problems (`rendered-audit-desktop.json`, `rendered-audit-tablet.json`).

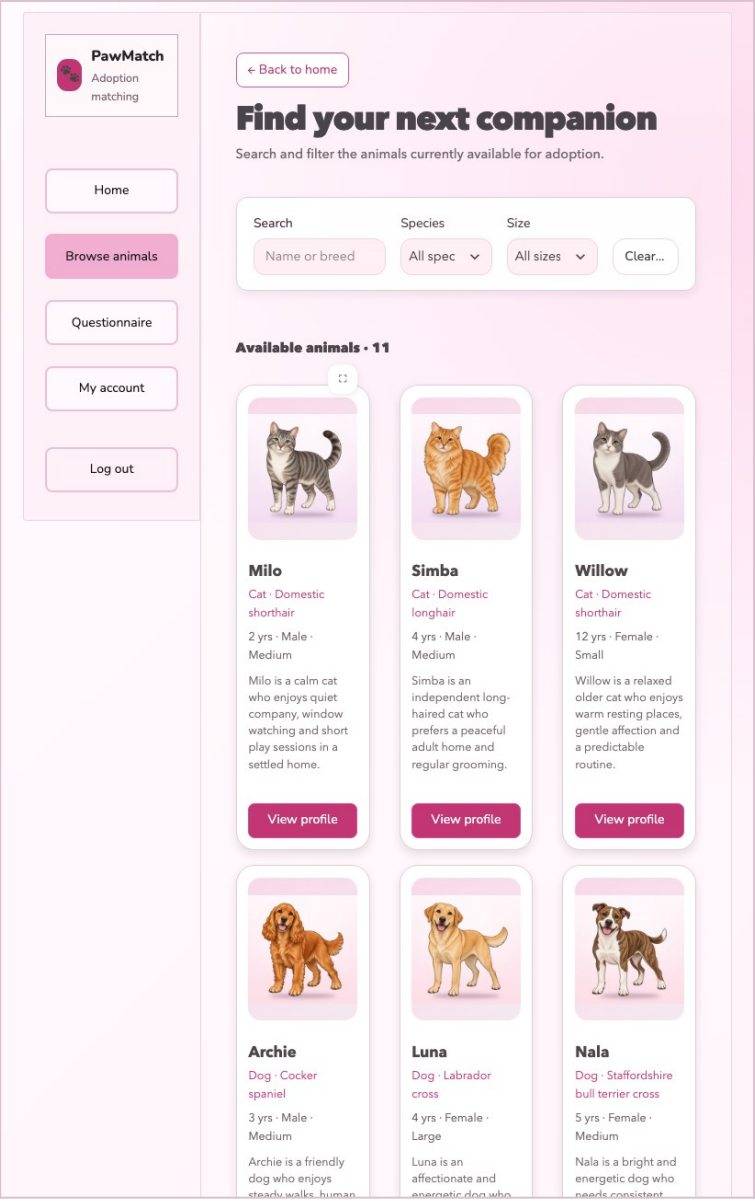


Figure S15: Browse at tablet width (1024 px): cards reflow to fit

Tests for stage 6

ID	Test	Automated test or tool	Result
T6.1	Text contrast	`test_every_text_colour_reaches_4_5_to_1`	Pass after fix
T6.2	Text size	`test_no_text_is_smaller_than_12px`	Pass after fix
T6.3	Rendered audit	`docs/tools/capture_screens.py`, both widths	Pass

Client feedback 6: final review

#	My question	Client's answer
1	Is the login page welcoming enough for adopters, and is the Log in / Create an account switch clear?	Yes, I like the soft pink and the big 'Find a companion' heading, and the switch at the top is easy to see. My worry is the placeholder text in the email and password boxes, which is very faint on the pale pink, so it is hard to read. Please make it darker.
2	Would an adopter understand the questionnaire, with the Back and Next buttons, on the first question?	The question and the three big answer cards are clear and simple, which is what I asked for. But on question 1 the Back and Next buttons look almost invisible because they are so pale, so people may not realise they are buttons. Please make the disabled state still readable.
3	Are the matches results clear enough for someone who has never adopted before?	This is the part I care most about, and it works well: each animal has a score, a green 'Requirements met' box and an amber 'Requirements not met' box with the welfare needs marked. But the percentages are shown as 100.0% and 76.7%, which is fussy, and the rabbits fill the whole top five, so please check the ranking gives a good spread.
4	On Milo's profile, would someone with a cat allergy notice the warning?	Not quickly enough. The allergy line is a small plain '!' sentence at the bottom of the compatibility box, under all the green ticks, and the score still says 39% without a strong warning. Allergies should be the first thing shown, in a clear warning colour.
5	Would staff understand the admin add-animal errors and the manage table?	The error list at the top ('Enter a name', 'Enter a breed', 'Describe the personality') is clear and matches what I asked for. But the fields themselves are not highlighted in red, so staff have to hunt for them, and the manage table says 'Read-only table' even though it is where they edit and delete. Also the last row is cut off and the Added column is clipped on the overview.

Client feedback 6: summary and next steps

Rating 7/10, signed off. the client's view of stage 6: the cute pastel look, clear navigation, separate staff area and the explained match results are what I asked for, and staff-only controls are clearly separated from adopter accounts. The main problems are faint placeholder and disabled-button text, a buried allergy warning, unhighlighted error fields and some clipped table content. These are fixable and need checking on a laptop screen. Note: the three extra image-dimension notes in the request arrived without visible images, so I based this answer on the 15 screenshots I could read.

Next steps the client asked for:

- Darken the placeholder and disabled-button text on login, register and questionnaire so it meets the 4.5:1 contrast rule.
- Show allergy conflicts as a red top-level warning on the profile and match cards, and mark invalid fields in red on the admin add/edit form.
- Fix the admin manage table (remove 'Read-only' label, stop clipping the last row and the Added column) and round scores to whole percentages.

Changes made after feedback 6

Changed

- **Allergy shown as a red warning.** It is part of the welfare box from stage 5, listed first.
- **Table clipping** fixed in stage 2's changes; retested at 1440 px in the browser audit and in the screenshot on page S12.

Not changed, with reasons

- **Fainter placeholder text and disabled buttons.** These are drawn by Streamlit itself, outside my stylesheet, so the audit's "0 low-contrast texts" covers my text and controls but not Streamlit's own placeholder hints. WCAG does not require contrast for disabled controls, but the client finds them faint, so this is in the evaluation.
- **Whole-number scores.** Scores keep one decimal place so two close animals are not shown as equal; the score key explains the bands.

Development assessment 6 and sign-off

The AI role-played client signed off the final stage (rating 7 out of 10). All 208 automated tests pass. The remaining requests are carried into the evaluation as partly met criteria and future development.

3.9 Stage 7: changes from the reviews, and staying signed in

Aim. Make every change the six client reviews and the four usability testers asked for (listed under each stage's "Changes made" above), then fix the last problem on my own list: a browser reload signed the user out.

What was built.

- All review changes in one commit (`77ef20a`), each with a test named after the review that asked for it.
- **Staying signed in.** Streamlit forgets everything when the page is reloaded. Signing in now creates a random session token (32 random bytes from `secrets`), stored in a new `sessions` table with an expiry 12 hours ahead and placed in the address as `?s=…`. On a reload the token is looked up and the user is signed back in; signing out deletes the token, and an expired or made-up token signs nobody in. Only the token travels, never the user's id, so it cannot be guessed or changed into someone else's (commit `b0e302a`).

Code S10: Making a session token · pawmatch/db.py, lines 140–147

```
140  def create_session(conn: sqlite3.Connection, user_id: int, method: str = "password") -> str:
141      """A random token for this browser tab; only the token travels, never the user id."""
142      token = secrets.token_urlsafe(32)
143      expires = (datetime.now(timezone.utc) + timedelta(hours=SESSION_HOURS)).isoformat(timespec="seconds")
144      conn.execute("DELETE FROM sessions WHERE expires_at < ?", (_now(),))
145      conn.execute("INSERT INTO sessions (token, user_id, method, expires_at) VALUES (?, ?, ?, ?)",
146                   (token, user_id, method, expires))
147      return token
```

Code S11: Looking a token up · pawmatch/db.py, lines 150–155

```
150  def session_user(conn: sqlite3.Connection, token: str) -> tuple[User, str] | None:
151      """The user and sign-in method for an unexpired token, or None."""
152      row = conn.execute(
153          "SELECT u.*, s.method FROM sessions s JOIN users u ON u.user_id = s.user_id "
```

```
154          "WHERE s.token = ? AND s.expires_at >= ?", (token, _now())).fetchone()
155      return (_user(row), row["method"]) if row else None
```

Tests for stage 7

Test	What it checks	Automated test	Result
T7.1	The age rule is written on the registration form (review 1)	`test_review1_age_rule_is_written_on_the_form`	Pass
T7.2	A blank confirm box gets its own message (review 1)	`test_registration_reports_every_missing_field`	Pass
T7.3	The size filter narrows the list (review 3)	`test_review3_size_filter_narrows_the_list`	Pass
T7.4	The profile offers the questionnaire as a button (review 3)	`test_review3_profile_offers_the_questionnaire_as_a_button`	Pass
T7.5	The home page says staff make the decision (review 3)	`test_review3_home_says_staff_make_the_decision`	Pass
T7.6	The hours question starts empty (review 4)	`test_review4_hours_question_starts_empty`	Pass
T7.7	The matches page explains the score and the order (review 5)	`test_review5_matches_explain_the_score_and_the_order`	Pass
T7.8	Answers can be updated from the account page (review 5)	`test_review5_account_lets_answers_be_updated`	Pass
T7.9	A reload with the token keeps the adopter signed in	`test_reload_keeps_the_adopter_signed_in`	Pass
T7.10	A made-up token signs nobody in	`test_signed_out_or_unknown_token_does_not_sign_in`	Pass
T7.11	A token stops working when signed out or expired	`test_session_token_finds_its_user_until_ended_or_expired`	Pass
All	Every earlier test still passes (regression)	Full suite	208 passed, 0 failed

The blank confirm box failed first (also shown under stage 1)

The new rule for an empty confirm box changed what `test_registration_reports_every_missing_field` expected, and it failed (an extra `confirm` message). Saved before the fix: `docs/evidence/review-fail1-confirm-blank.txt`.

Fix and retest

The test's expected list was updated to include the new message, because the extra message is the change the review asked for. Retest: passed, and the whole suite passed (`review-pytest.txt`).

Development assessment 7

Every change the reviews asked for that fits the design is built and tested, and a reload no longer signs the user out. Requests that were not built (address fields, a red outline on error boxes, a Cancel button) are explained under each stage and carried into the evaluation. Next: the post-development tests on E1.

4 Testing to inform evaluation

4.1 Post-development tests

The finished system (commit `77ef20a` plus the worked-example test) was tested against every success criterion using the plan on D60. First, the whole automated suite was run again on the final build: **208 tests, 0 failures** (`docs/evidence/pytest-junit.xml`). This re-runs every development test T1.1 to T6.3, so nothing built in an early stage was broken by a later one.

Test file	Validation	Matching	Accounts	Database	Staff access	Pages and style
Tests	76	41	16	16	10	32 + 4

Capacity and speed (SC10, SC14)

`docs/tools/measure_v2.py` builds a separate database far larger than the rescue needs, then times the slow operations.

Measurement	Result	Target	Verdict
Animals in the test database	500	at least 500	Met
Adopter accounts	100	at least 100	Met
Questionnaire submission: score, rank and store every Available animal (423 animals)	mean 0.041 s, worst 0.098 s	under 1 s	Met
Browse query returning 423 animals	0.007 s	n/a	Met
Browse page drawn in full by Streamlit's test runner (worst of 3; a browser adds network time)	1.76 s	under 3 s	Met

Results against each success criterion

P	What was tested on the finished system	Evidence	Result
P1	Contrast and text size on every screen	Audit of 18 screens at 1440 px and 16 at 1024 px (the two extra questionnaire screens were only captured at laptop width): 0 low-contrast, 0 small (T6.1 to T6.3)	Pass
P2	One menu; staff items hidden from adopters	Figures E1 and E2 below; T2.8, T3.5	Pass
P3	Home page explains suitability and that staff decide	Page S13; `test_review3_home_says_staff_make_the_decision`	Pass
P4	Browse with species, size and search	Page S14; T3.1 to T3.3	Pass
P5	Profile from the database	Page S14; T3.4	Pass
P6	Registration validation	Page S5; T1.1 to T1.8	Pass
P7	Hashed passwords; one login error	Figure E3 below; T1.9, T1.10, `test_register_stores_hash_not_password`	Pass
P8	Questionnaire: 10 screens, cannot skip	T4.1 to T4.6; timing not measured with real users	Partly
P9	Score equals the hand calculation	S20; T5.8	Pass
P10	Ranking order and speed	T5.7, T5.9 (`test_matches_page_shows_the_top_five_and_can_show_all`), T5.10; speed table above	Pass

P	What was tested on the finished system	Evidence	Result
P11	Every match explains itself	Page S22; T5.5, T5.6	Pass
P12	Staff add / edit / delete with validation	Pages S10, S12; T2.1 to T2.7, T2.9; photo type and size `test_image_type_and_size`	Pass
P13	Adopters cannot use staff functions	T2.8 (every action, both adopter and signed out)	Pass
P14	Storage and capacity	T2.11, T4.5; 500 animals / 100 users above	Pass
P15	Usability (SUS ≥ 68) and client sign-off	Section 4.3: mean SUS 73.8 from simulated testers; AI role-played client signed off	Partly
P16	Laptop and tablet widths	Page S25; audit of 18 screens at 1440 px and 16 at 1024 px	Pass

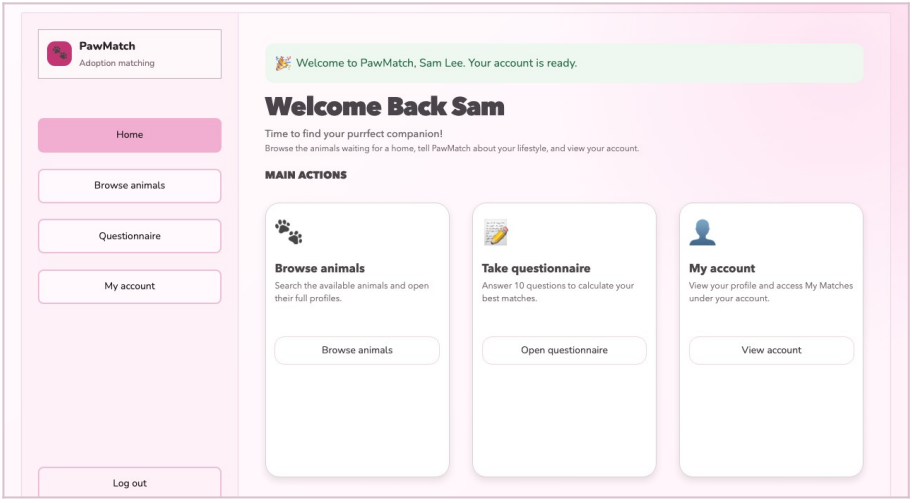


Figure E1: The adopter's menu: home, browse, questionnaire, matches, account (no staff items)

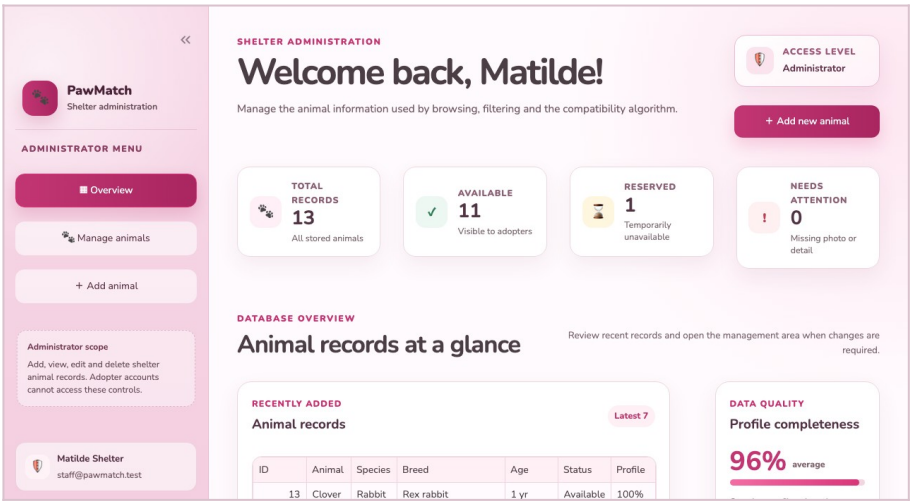


Figure E2: The staff menu after a staff login: dashboard, manage, add

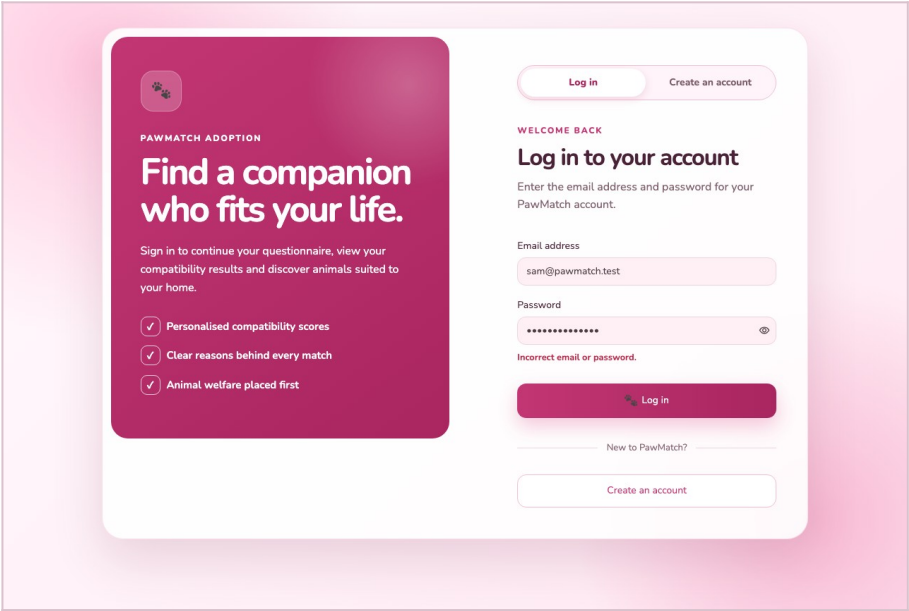


Figure E3: A wrong password and an unknown email give the same message

Agreement with hand-written expectations and weight sensitivity

The tests above show each rule works on its own. To check that the **whole ranking** gives the answers a person would expect, I wrote ten adopter profiles (A1 to A10) and, before running the program, wrote down for each one which demo animals must be in its top three and which must never be (for example, a dog must never be offered to someone out 8 hours a day). `tests/test_agreement.py` checks the program's top three against them; the results are saved in `docs/evidence/agreement-sensitivity.json`.

ID	Profile	Program's top three	Agrees
A1	Flat, out 8 h a day, first pet, low activity	Milo, Poppy, Teddy (all suitable)	Yes
A2	Family house with garden, wants a dog	Archie, Luna, Oscar	Yes
A3	Same family, allergic to dogs, wants a rabbit	Clover, Daisy, Poppy	Yes
A4	Older experienced owner, flat, out 10 h, likes cats	Willow; then Milo and Poppy with a warning	Yes
A5	Already has a pet, wants a second dog	Archie, Oscar, Luna	Yes
A6	First-time owner, out 6 h, wants a rabbit	Daisy, Poppy, Clover	Yes
A7	Experienced runner, large house, wants a large dog	Luna, Archie, Nala	Yes
A8	Flat with children and a cat, out 5 h	Milo, Poppy, Willow	Yes
A9	Allergic to cats and rabbits, house with garden	Archie, Oscar, Luna	Yes
A10	Out 12 h with children and a pet	Three animals, every one with a warning: none is called suitable	Yes

All ten agree. A4 shows the important-factor rule working: only one animal can safely be left 10 hours, so the next two are shown below it with a warning instead of being hidden.

Sensitivity. The weights in the design were my judgement, informed by the client interview, so I checked that the results do not depend on their exact values. Each of the seven weights was changed by 10% down and 10% up (14 runs), and the top three was recalculated for all ten profiles: **no top three changed in any run** (`test_ten_percent_weight_changes_do_not_change_any_top_three`). Even a 50% change to any one weight changed none of them, because the welfare factors decide which animals are suitable before the weights decide the order. This means the client can adjust a weight in `WEIGHTS` (section 5.4) without the matches changing unexpectedly. The limitation is that

many demo animals score 100% for a given profile, so ties are broken by name; with a larger shelter the order inside a tie would matter more.

Phone and tablet render check

After stage 7 the live system was opened in a headless browser at three sizes: a phone (390 px), a tablet upright (820 px) and a tablet on its side (1180 px), signed in as an adopter and as staff (`docs/evidence/render-phone-tablet.json`). On every page there was **no sideways scrolling and no text smaller than 12 px**, and the adopter's menu buttons could be reached at every size. Reloading the page kept the user signed in at all three sizes (T7.9 to T7.11). On the phone some controls were shorter than the 44 px touch target recommended by WCAG 2.2, so every input and button is now at least 44 px tall on screens narrower than 640 px. The staff sidebar takes most of a phone screen until it is closed, which is acceptable because staff use a laptop at the rescue (stakeholder analysis), but it is listed under future development (section 5.6).

4.2 Robustness

R	Test	Automated test	Result
R1	Search for `' OR 1=1 --`	`test_search_text_is_treated_as_data_not_sql`	Pass: no results, no rows lost
R2	Error halfway through saving	`test_connect_rolls_back_on_error`	Pass: nothing half-saved
R3	Bad row inserted directly into the database	`test_database_constraints_back_up_validation`	Pass: refused
R4	Edit an animal another staff member deleted	`test_edit_of_deleted_animal_reports_error`	Pass: clear message
R5	Open a database from an earlier version	`test_old_database_gains_calculated_at_column`, `test_old_database_gains_the_stage_9_columns_and_demo_values`	Pass: upgraded
R6	500 animals and 100 users	`measure_v2.py` (above)	Pass

4.3 Usability testing

Method. Four tester profiles were chosen to cover the stakeholders in the analysis: a young first-time adopter (P1), a parent (P2), an older user with low confidence online (P3, the hardest test of readability) and a staff volunteer (P4). Each was given the five tasks on D61 with screenshots of the system as it was before the stage 7 changes, and no help, rated each task out of 10, and answered the ten SUS statements.

Same starting data for every tester. The screenshots were taken by `docs/tools/capture_screens.py`, which starts PawMatch on a new temporary database each time; `prepare` in `seed.py` fills it with the same demo animals and one staff account. So every tester saw the same animals and the same results, and nothing left over from an earlier run could change what they saw.

Honest limitation

These four testers were **simulated by an AI model** given each profile, not real people (logged in Appendix A). Simulated testers can spot unclear wording and missing feedback, but cannot show how long tasks really take or where real people hesitate. One real person did use the live system during development, and found the sign-up problem fixed in stage 1 (S4); that is the only real-user evidence, so SC15 is treated as partly met below.

Tester	Who	U1	U2	U3	U4	U5	SUS
P1 Amira	19, student renting a small flat, no garden, out 6 to 7 hours a day, wants a cat, confident online	9/10	8/10	6/10	n/a	9/10	**82.5**
P2 Daniel	42, parent of children aged 6 and 9, house with a garden, works from home, wants a family dog	8/10	8/10	6/10	n/a	8/10	**72.5**
P3 Margaret	68, retired, lives alone in a bungalow with a small garden, has owned dogs, uses an iPad, low confidence online	7/10	7/10	5/10	n/a	8/10	**65**
P4 Josh	24, shelter volunteer with a staff account who updates the animal listings, in a hurry	8/10	7/10	7/10	6/10 (help)	8/10	**75**
Mean							**73.8**

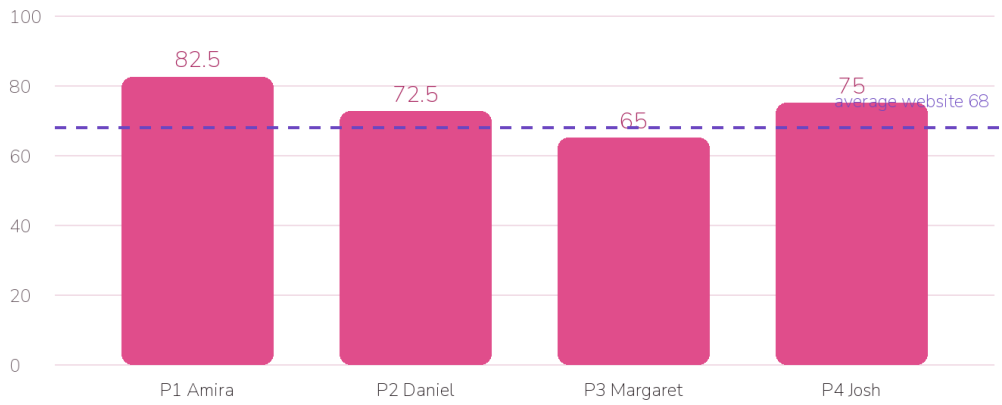


Figure E4: SUS score for each tester; the dashed line is the average website (68)

What the usability testing showed

Task	Screen	Criterion	What testers found	What happened
U1 register and log in	Register, login	SC6, SC7	All four completed it without help (P1 rated it 9/10). P1 noticed that an empty confirm box was not flagged.	Fixed in stage 7 (confirm box flagged)
U2 questionnaire	Questionnaire (10 screens)	SC8	Completed by all; the progress bar and "Question N of 10" were liked. The hours box started at 0.	Fixed: hours box starts empty
U3 explain the top match	Matches	SC11	The lowest-rated task (5 to 7 out of 10). Testers did not know what "welfare need" meant, could not see why a warned animal ranked lower, and wanted the warning to stand out.	Fixed: red welfare box in plain words, a line explaining the order, and a score key
U4 staff add, edit, delete	Staff manage, add animal	SC12	P4 completed add and edit, but rated it 6/10: the Delete button was not visible enough.	Partly met: the tick-box confirmation stays; making Delete more visible is future development
U5 browse and return	Browse, profile	SC4, SC5	Easy for everyone (8 to 9 out of 10); "Back to all animals" kept the list.	No change needed

The common request from all four testers was the same as the client's in review 5: say in plain words what a warning means. That is the strongest evidence that the stage 7 change to the matches page was the right one.

Stakeholder survey after the build (planned, not yet run)

The analysis survey of 27 adopters (A22) is the baseline for what adopters wanted before the build. No survey of real stakeholders has been run on the finished system yet, so there are **no results to report here**; this is limitation 1 (E9). The survey below is ready to send as soon as the client can share it with the rescue's adopters and volunteers.

	Plan
Who	At least 10 people who used the live system: adopters who completed the questionnaire, and two or more rescue staff or volunteers
When	After a week of using the live system at the rescue, before any further changes
Questions	1. Did the matches page explain why each animal suits you? (1 to 5) 2. Did any warning stop you from choosing an animal you would otherwise have chosen? (yes / no, why) 3. How long did the questionnaire take? (under 2 / 2 to 3 / over 3 minutes) 4. Was any question unclear? (which) 5. Staff only: did any match shown to an adopter look wrong to you? (which, why) 6. The ten SUS statements
Compared with	The baseline answers from the analysis survey and the simulated testers' SUS scores above

5 Evaluation

5.1 Success criteria

SC	Criterion	Verdict	Evidence and comment
SC1	Pink palette, readable	Met	0 low-contrast and 0 small texts on all 18 laptop screens and all 16 tablet screens (P1). The AI role-played client liked the look in every review, but the audit is the evidence. The client finds Streamlit's own placeholder text and disabled buttons faint, which my stylesheet does not control.
SC2	One consistent menu	Met	Every page one click from the menu; staff items never appear for adopters (P2).
SC3	Home explains suitability, staff decide	Met	The home page says what the score means and that staff make the final decision (page S13). `test_review3_home_says_staff_make_the_decision` opens the home page as an adopter and checks for that sentence, and `test_adopter_login_opens_the_home_page` checks an adopter lands there after logging in (P3).
SC4	Browse and filter	Met	All Available animals, three species, species and size filters and search (P4).
SC5	Full profile from the database	Met	Clicking an animal opens its photo, details, personality and needs, all read from the database (page S14). `test_view_profile_opens_the_animal_with_its_needs` checks the needs shown match the stored record; `test_review3_profile_offers_the_questionnaire_as_a_button` checks the next step is offered (P5).
SC6	Registration validation	Met	Normal, boundary and invalid data for every field, duplicate emails refused (P6).

SC	Criterion	Verdict	Evidence and comment
SC7	Secure accounts	Met	Salted scrypt hashes, one generic error, sign-out clears the session (P7).
SC8	10-question questionnaire, cannot skip	Partly met	10 screens in 4 sections and cannot be skipped (P8). The 3-minute target could not be measured, because the testers were simulated.
SC9	Correct score	Met	The worked example on S20 gives the same percentage by hand and in the program (`test_worked_example_in_the_write_up_matches_the_program`). `test_weights_match_the_design` checks the weights are the designed ones, `test_each_factor_loses_its_weight` removes each factor in turn and checks exactly its weight is lost, and `test_boundaries_count_as_met` checks that equal values (for example 6 hours when 6 are allowed) count as met. The weight-sensitivity and fixed-profile checks below show the ranking does not depend on one exact weight (P9).
SC10	Ranked quickly and safely	Met	Warned animals never above safe ones (T5.7); the top 5 shown, at least 4 as required, with "show all" (T5.9); 500 animals in 0.098 s at worst (P10).
SC11	Each result explains itself	Met	Met, not-met and warning lists on every match, plus the score key (P11).
SC12	Staff CRUD with validation	Met	`test_admin_can_add_edit_and_delete` and `test_staff_can_add_an_animal_and_then_delete_it` (through the real page) add, edit and delete an animal; `test_admin_add_still_validates`, `test_staff_add_form_reports_missing_fields` and `test_invalid_animal_fields_are_rejected` show bad data is refused with a message; `test_image_type_and_size` refuses wrong photo types and photos over 2 MB; `test_delete_removes_animal_and_its_stored_results` shows a delete leaves no stored scores behind (P12). The staff tester wanted Delete to be more visible.
SC13	Adopters cannot use staff functions	Met	Every staff action refused for adopters and signed-out users (P13). My requirements list (8.1) asked for a separate admin login; one login form is used, and the role stored in the database decides what opens. This is simpler and just as secure, because every staff action checks the role itself.
SC14	Reliable storage, scales	Met	Constraints back up validation; 500 animals and 100 users; browse page 1.76 s in Streamlit's test runner (P14).
SC15	Easy to use (SUS ≥ 68), client sign-off	Partly met	Mean SUS 73.8 and every tester above 60, and the AI role-played client signed off; but neither the testers nor the client's reviews were real people.
SC16	Laptop and tablet	Met	No clipped content at 1440 and 1024 px in a browser (P16), which is how the criterion is defined; a phone and tablet render check was also run. Trying it on a real iPad is future development.
Req. 8.1	A separate admin login (requirements list)	Partly met	Staff use the same login form as adopters, and the role stored in the database decides what opens. Every staff action checks the role itself (T2.8), so this is as secure as a separate login, but it is not what requirement 8.1 says. A staff-only login page would be a small change: the same form that refuses non-staff accounts.

Summary: 14 of 16 success criteria met, 2 partly met (SC8, SC15), none unmet. Both are partly met because of how they were tested (no real users), not because a feature is missing. From the requirements list, 8.1 (a separate admin login) is partly met by design choice.

5.2 How the partly met criteria would be fully met

SC	What is missing	How I would meet it
SC8	Timing with real people	Ask five adopters at the rescue to complete the questionnaire while I time them; target under 3 minutes each.
SC15	Real usability testers	Repeat tasks U1 to U5 and the SUS questionnaire with at least five real people from the stakeholder groups (including an older user and a staff member), then compare with the simulated results.

SC	What is missing	How I would meet it
Req. 8.1	Separate staff login	Add a staff login page that uses the same check but refuses adopter accounts, and remove the staff link from the adopter login.

5.3 Usability features evaluated

Feature (from D58)	Evidence of how well it worked	Effective?
Pink palette, Nunito, large photos	Liked by the client in every review and by all four testers	Yes
Contrast and text size	Audit passes; P3 (older user) could read every page	Yes
One consistent menu	Every tester found every page	Yes
One question per screen with progress	Liked in U2; no tester got lost	Yes
Card buttons	Every choice visible at once and easy to tap; the selected state could be stronger (client review 4)	Partly
Next disabled until answered	No incomplete questionnaire stored (T4.6); disabled buttons look faint	Partly
Error under each box	Every tester fixed errors without help in U1; no red outline on the box	Partly
One generic login error	Client agreed it was right for security	Yes
Met / not met / warnings	Lowest task score (U3, on the pre-stage-7 screenshots); clearer after the stage 7 changes, but not re-tested	Partly
"Staff make the final decision"	On home, profile and matches pages	Yes
Tick-box before delete	No accidental deletes; Delete button not visible enough (U4)	Partly
Keyboard focus outline	Audit shows a focus outline on every control	Yes

5.4 Maintenance

- **Modular structure.** All rules live in `pawmatch/` and all screens in `webui/`. A change to how a page looks cannot break the scoring, and the 208 tests show at once if a change to the rules does.
- **One place for each rule.** The weights, the warning factors, the allergy cap and the category bands are constants at the top of `matching.py`; the species, sizes and statuses are lists in `models.py`. Changing a weight is one edit, and the percentage stays 0 to 100 because it is divided by the total.
- **Adding a species.** Issue: the rescue starts taking a new kind of animal. Fix: add it to `SPECIES` in `models.py` and to the CHECK on the animals table, then add demo data; the tests on fixed lists show what else needs updating.
- **Changing a weight.** Issue: the rescue finds, for example, that time alone matters more than the client thought. Fix: change one number in `WEIGHTS`; update `test_weights_match_the_design`, then run the tests and the sensitivity check below to see how the order of matches changes.
- **Database upgrades.** `init_schema` adds missing columns to an older database file automatically (R5), so the rescue never loses records when the program is updated.
- **Comments and names.** Every module and function starts with a comment saying what it does and which criterion it serves; names follow the design's data dictionary.
- **Pinned versions.** Issue: Streamlit and other libraries release new versions that can change how pages behave. Fix: `requirements.txt` fixes the exact versions tested, so nothing changes without warning. Updating is done deliberately: change the version, run the 208 tests, and only deploy if they all pass.

- **Backups.** Issue: a broken laptop or a mistaken delete would lose every record. Fix: all data is one SQLite file (`data/pawmatch.db`), so a daily copy of that file to another drive is a complete backup, and restoring is copying it back.
- **Forgotten passwords.** Issue: PawMatch has no reset email. Fix: staff can delete the account so the adopter registers again; a reset email is future development.
- **Who maintains it.** A small rescue has no programmer. The README explains how to install, run, test and back up the system, so a volunteer with basic computer skills could keep it running, and a student or freelancer could change it safely because the tests protect the rules.

Privacy and personal data

PawMatch stores personal data, so it has to follow UK GDPR: collect only what is needed, keep it safe and remove it when asked.

- **What is stored.** For each user: name, email, age, role and a password hash (`users` table); each questionnaire answer set and the scores worked out from it (`questionnaire_responses` and `match_results`); and sign-in tokens that expire after 12 hours (`sessions`). No address, phone number or payment details are collected (limitation 8).
- **How it is protected.** Passwords are never stored: each is salted and hashed with scrypt (`auth.py`), and a wrong email and a wrong password give the same message (SC7). Every query uses placeholders, so typed text cannot change the SQL (R1). Sign-in tokens are random (`secrets.token_urlsafe`) and are deleted on sign-out.
- **How a user's data is removed.** There is no "delete my account" button yet. If an adopter asks, staff remove the user's row from the database file, and the database deletes their answers, scores and sign-in tokens with it (ON DELETE CASCADE). A self-service delete button is future development (limitation 9).

5.5 Limitations

1. **The client reviews and usability testers were simulated by AI.** They found real problems that were fixed, but they cannot replace real people; the client's own sign-off and real testing are the first step of future development.
2. **The weights are judgement, not data.** They come from the client and the survey. Without records of past adoptions and returns, nobody can prove that 20 points for time alone is better than 15.
3. **Only five yes/no or level-based facts describe an animal's needs.** Real animals have needs the form cannot hold (for example a dog that is fine with cats but not with other dogs); staff have to put these in the personality text, which is not scored.
4. **Streamlit limits the interface.** It cannot outline a single box in red, change its own placeholder and disabled colours, or add a Cancel button that behaves like a normal web form. These were requested in reviews and are not done.
5. **SQLite allows one writer at a time.** That is fine for two or three staff, but would slow down a large rescue with many staff saving at once.

6. **The optional Google sign-in uses a test set-up made by the developer.** The rescue would need its own Google account settings before using it for real; the email and password login works without it.
7. **The sign-in token is in the address.** Staying signed in after a reload works by keeping a random token in the page address. Anyone who is sent that exact address within 12 hours would be signed in as that user, so users should not share the address while signed in. A browser cookie would avoid this, but Streamlit cannot set cookies itself.
8. **No contact details are stored.** I chose not to collect addresses and phone numbers (data minimisation), so the rescue cannot contact an adopter through PawMatch; the adopter has to get in touch.
9. **Users cannot delete their own account.** Erasing a user's data needs staff to edit the database file, which a volunteer may not be confident doing.
10. **Only one site.** The system serves one rescue building; a rescue with several branches could not filter animals or staff by branch.
11. **The live system was not tried on real tablets and phones.** The browser render checks pass at phone and tablet sizes, but nobody has used it on a real device.
12. **The questionnaire has no "save and continue later".** Answers are only stored when all ten are given (T4.6), so an adopter who leaves halfway starts again.

5.6 Future development

Improvement	How it would be done	Addresses
Real usability testing and client sign-off	Five or more real testers, timed, with SUS; client walk-through at the rescue	SC8, SC15, limitation 1
Learn the weights from outcomes	Record whether each adoption succeeded or the animal was returned; after enough adoptions, compare outcomes with scores and adjust weights	Limitation 2
"Good with cats / dogs / rabbits" instead of one yes/no	Three yes/no fields for each animal and a matching question; one more factor in `_factor_checks`	Limitation 3
Register interest in an animal	A button that sends the adopter's details and score to staff, with the adopter's permission, then a staff list of enquiries	Limitation 8, client Q14
Clearer Delete and a Cancel button; red outline on error boxes	Custom CSS for the Delete button now; the other two need a custom Streamlit component or a move to a web framework such as Flask	Limitation 4, U4
Number of matches on the "done" screen	Count results without warnings and show "You have N strong or good matches"	SC11, client review 4
Several branches	Add a branches table and a branch_id on animals and staff; filter by branch	Limitation 10, scope
Real tablet and phone testing	Repeat the usability tasks on an iPad and a phone; the browser render checks already pass at those sizes	SC16, limitation 11
Keep the sign-in in a cookie instead of the address	A small custom Streamlit component that sets a secure, HTTP-only cookie	Limitation 7
PostgreSQL instead of SQLite	Only `db.py` changes, because no other module writes SQL	Limitation 5
"Delete my account" button	A button on the account page that asks for the password again, then deletes the user (their answers and scores go with it)	Limitation 9, privacy
Save answers part way	Store the answers given so far in a draft row and reopen the questionnaire at the first unanswered question	Limitation 12, SC8
Run the stakeholder survey	Send the survey on E6 after a week of real use and compare with the analysis baseline	Limitation 1, SC15

Appendices

Appendix A: Use of AI tools

AI tools were used for the tasks below. Every output was checked before it was used, and nothing an AI produced was accepted as evidence without a test or a measurement behind it.

Task	What the AI did	How it was checked
Client reviews 1 to 6 (S6 to S26)	Role-played the client. It was given only the client interview from the analysis and the screenshots of the stage being reviewed, and answered the review questions.	Each answer was compared with what the client said in the interview. Requests were weighed against the design rather than accepted automatically: for example, address and phone fields were declined for data minimisation. Every change that was made is in stage 7 and is tested.
Usability testers P1 to P4 (E4)	Simulated four tester profiles doing tasks U1 to U5 from screenshots, rating each task and answering the SUS statements.	The SUS scores were calculated by code from the answers (`nea_data.sus_score`). Findings were acted on only where they pointed to something visible in the screenshots. Treated as a limitation (SC15 partly met).
Writing and testing code	An AI coding assistant helped write the code and the tests, working from the original design and code.	Every rule is covered by an automated test written from the design's test plan, and failing runs were saved before each fix (Appendix C). Nothing was kept that did not pass the tests.
Research	Searched for further existing solutions (E4 to E8) and summarised their public pages.	Every source was opened and read; the links are in Appendix B.
Proofreading	Suggested shorter, plainer wording in this write-up.	All technical claims were checked against the code, the tests and the measurements.

The AI did **not** decide the success criteria, the weights or the design; those come from the client interview, the survey and the design section.

Appendix B: References

Ref	Source	Link
E1 to E3	Dogs Trust, RSPCA Find a Pet, Petfinder	See the references on A36
E4	ASPCA Meet Your Match (Canine-ality)	https://www.aspca.org/meet-your-match/canine-ality-quiz Guide: https://www.aspcapro.org/sites/default/files/aspca-canine-ality-guide-2015.pdf
E5	Battersea Dogs and Cats Home	https://www.battersea.org.uk/adopt
E6	Woodgreen Pets Charity	https://woodgreen.org.uk/adoption-and-rehoming
E7	Shelterluv shelter software	https://www.shelterluv.com
E8	OkCupid (question-based matching)	https://www.okcupid.com/
Also read	Blue Cross rehoming	https://www.bluecross.org.uk/rehoming
SUS	Brooke, J. (1996) "SUS: a quick and dirty usability scale", in Usability Evaluation in Industry, Taylor and Francis	The average score of 68 is used as the target
WCAG	W3C Web Content Accessibility Guidelines 2.2 (contrast minimum 4.5:1)	https://www.w3.org/TR/WCAG22/
scrypt	Python `hashlib.scrypt`	https://docs.python.org/3/library/hashlib.html
Passwords	OWASP Password Storage Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
Streamlit	Streamlit documentation, including AppTest	https://docs.streamlit.io

Ref	Source	Link
SQLite	SQLite documentation	https://www.sqlite.org/docs.html

Appendix C: Evidence files

All files are in the project repository under `docs/evidence/`, so every result in this document can be checked or re-run.

File	What it holds
`pytest-junit.xml`	The final run of all 208 automated tests (0 failures)
`stage1-pytest.txt`, `stage2-pytest.txt`, `stage3-pytest.txt`, `stage5-pytest.txt`, `review-pytest.txt`	The test run at the end of each stage
`stage3-fail1-apptest-path.txt`, `stage3-fail2-error-box.txt`, `stage5-fail-design-recheck.txt`, `stage5-fail-regression-auth.txt`, `style-test-before.txt`, `review-fail1-confirm-blank.txt`, `review-fail2-hours-prefilled.txt`	The failing runs shown in the fail, fix and retest boxes, saved before each fix
`rendered-audit-desktop.json`, `rendered-audit-tablet.json`	The browser audit of contrast, text size and keyboard focus on every screen at 1440 and 1024 px
`measurements.json`	Capacity and speed results from `docs/tools/measure_v2.py`
`client-reviews.json`	The client review answers, and the usability testers' ratings and SUS answers

Appendix D: How to install and run

1. Install Python 3.13, then in the `rebuild/` folder run `pip install -r requirements.txt`.
2. To create the first staff account, set `PAWMATCH_ADMIN_EMAIL` and `PAWMATCH_ADMIN_PASSWORD` before starting.
3. Start the system with `streamlit run streamlit_app.py`. The first start creates `data/pawmatch.db` and fills it with demo animals.
4. Run the tests with `pip install -r requirements-dev.txt` and then `pytest`.
5. Back up the system by copying `data/pawmatch.db`.

Appendix E: Program listing

The full listing of the rules layer (`pawmatch/`) follows. This is where every success criterion about data, scoring, security and storage is met. The screens (`webui/`) and the stylesheet (`webui/pawmatch_style.py`, mostly the pink theme carried over from the original design) are in the repository; the important parts are shown in the development section.

File	Purpose
`streamlit_app.py`	Starts the system, prepares the database and opens the page asked for
`pawmatch/models.py`	The data classes and fixed lists (species, sizes, statuses, roles)
`pawmatch/validation.py`	Every validation rule from the design, with clear messages
`pawmatch/auth.py`	Registration, login and password hashing
`pawmatch/matching.py`	The weighted score, warnings, allergy cap, categories and ranking
`pawmatch/db.py`	The only module that runs SQL: schema, upgrades and queries
`pawmatch/services.py`	Staff-only actions, each checking the user's role first
`pawmatch/seed.py`	Demo animals and the first staff account

Code X1: Entry point · streamlit_app.py, lines 1–28

```
"""PawMatch entry point: prepares the database, then runs the page asked for.

The pages and their order are the original app's (app.py, then pages/1_ to 7_). Each page
draws its own menu and checks who may see it (webui/state.py), as in the original, so
Streamlit's own page list is hidden.
"""

import streamlit as st

from pawmatch.seed import prepare
from webui.state import DB_PATH, PAGES

st.set_page_config(page_title="PawMatch", page_icon=" ", layout="wide")


@st.cache_resource
def _prepare_once(path: str) -> None:
    prepare(path)


_prepare_once(str(DB_PATH))

TITLES = {
    "login": "Log in", "home": "Home", "registration": "Create an account", "questionnaire":
"Questionnaire",
    "browse": "Browse animals", "account": "My account", "matches": "My matches", "admin":
"Administrator",
}
pages = [st.Page(PAGES[name], title=title, default=name == "login") for name, title in TITLES.items()]
st.navigation(pages, position="hidden").run()
```

Code X2: Data classes and fixed lists · pawmatch/models.py, lines 1–132

```
"""Domain records and the fixed option lists shared by every layer."""

from __future__ import annotations

from dataclasses import dataclass, field

SPECIES: tuple[str, ...] = ("Dog", "Cat", "Rabbit")
SIZES: tuple[str, ...] = ("Small", "Medium", "Large")
SEXES: tuple[str, ...] = ("Male", "Female", "Unknown")
ROLES: tuple[str, ...] = ("user", "admin")
# Adoption status (the original admin form): only Available animals are shown to adopters and scored.
STATUSES: tuple[str, ...] = ("Available", "Reserved", "Adopted")
NO_PREFERENCE = "No preference"

# Ordinal scales: the stored integer is the level, the label is what users see
# (the original questionnaire's wording: indoor space and activity level).
HOUSING_LEVELS: dict[int, str] = {
    1: "Limited space",
    2: "Moderate space",
    3: "Plenty of space",
}
ACTIVITY_LEVELS: dict[int, str] = {
    1: "Low",
    2: "Moderate",
    3: "High",
}
EXPERIENCE_LEVELS: tuple[str, ...] = (
    "None",
    "Some (family pet growing up)",
    "Experienced owner",
)
# What an animal needs from its adopter, as a level of EXPERIENCE_LEVELS (stage 9, from the
# client interview: "whether they have experience caring for that type of animal").
EXPERIENCE_NEEDED: dict[int, str] = {
    0: "Suits a first-time owner",
    1: "Needs some experience",
    2: "Needs an experienced owner",
}


@dataclass(frozen=True)
class User:
    """A registered account. The password hash never leaves the repository layer."""

    user_id: int
    name: str
    email: str
    age: int
    role: str

    @property
    def is_admin(self) -> bool:
        return self.role == "admin"


```

```python
@dataclass(frozen=True)
class Animal:
    """One animal in the shelter, including the requirements used for matching."""

    animal_id: int
    name: str
    species: str
    breed: str
    age: int
    sex: str
    size: str
    personality: str
    exercise_needs: int
    min_housing_level: int
    garden_required: bool
    child_compatible: bool
    max_hours_alone: int
    good_with_pets: bool = True  # can share a home with other animals (stage 9)
    experience_needed: int = 0  # a key of EXPERIENCE_NEEDED (stage 9)
    medical_notes: str = ""
    image_path: str = ""
    status: str = "Available"
    date_added: str = ""  # set by the database; read-only


@dataclass(frozen=True)
class Answers:
    """A validated questionnaire response."""

    housing_level: int
    garden_access: bool
    has_children: bool
    other_pets: bool
    experience: str
    activity_level: int
    hours_alone: int
    preferred_species: str
    preferred_size: str
    allergies: tuple[str, ...] = ()  # species someone at home is allergic to (stage 9)


@dataclass(frozen=True)
class MatchResult:
    """The score for one animal against one set of answers."""

    animal: Animal
    compatibility: float
    category: str
    bonus: int
    met: list[str] = field(default_factory=list)
    unmet: list[str] = field(default_factory=list)
    warnings: list[str] = field(default_factory=list)

    @property
    def ranking_points(self) -> float:
        """Compatibility plus the preference bonus; used for ordering only."""
        return self.compatibility + self.bonus

    @property
    def suitable(self) -> bool:
        """True when no important (welfare) requirement is unmet."""
        return not self.warnings


@dataclass(frozen=True)
class ShelterStats:
    """Counts for the staff dashboard (criterion 8): what is in the shelter and who is using it."""

    by_species: dict[str, int]
    adopters: int
    staff: int
    questionnaires: int
    by_status: dict[str, int] = field(default_factory=dict)

    @property
    def animals(self) -> int:
        return sum(self.by_species.values())
```

Code X3: Validation rules · pawmatch/validation.py, lines 1–272

```python
"""Input validation. Every function returns (cleaned values, errors by field).

An empty errors dict means the input is valid. Uniqueness of email is checked
by the repository, because it needs the database.
"""

from __future__ import annotations

import re
```

```
from typing import Any, Callable

from .models import (
    ACTIVITY_LEVELS,
    EXPERIENCE_LEVELS,
    EXPERIENCE_NEEDED,
    HOUSING_LEVELS,
    NO_PREFERENCE,
    SEXES,
    SIZES,
    SPECIES,
    STATUSES,
    Answers,
)

Errors = dict[str, str]

NAME_MIN, NAME_MAX = 2, 50
MIN_AGE, MAX_AGE = 18, 120
PASSWORD_MIN = 8
ANIMAL_AGE_MAX = 30
TEXT_MAX = 500
EMAIL_PATTERN = re.compile(r"^[^@\s]+@[^@\s]+\.[A-Za-z]{2,}$")


def _text(value: Any) -> str:
    return "" if value is None else str(value).strip()


def _int(value: Any) -> int | None:
    """Parse a whole number; bools and decimals are rejected."""
    if isinstance(value, bool):
        return None
    if isinstance(value, int):
        return value
    text = _text(value)
    return int(text) if re.fullmatch(r"-?\d+", text) else None


def _yes_no(value: Any) -> bool | None:
    if isinstance(value, bool):
        return value
    return {"yes": True, "no": False, "1": True, "0": False}.get(_text(value).lower())


def _has_letter(text: str) -> bool:
    return any(ch.isalpha() for ch in text)


def _check_name(value: Any, errors: Errors, field: str = "name", allow_digits: bool = False) -> str:
    """Person names: letters, spaces, hyphens, apostrophes. Animal names may also use digits ("R2")."""
    name = _text(value)
    allowed = " '-" + ("0123456789" if allow_digits else "")
    if not name:
        errors[field] = "Enter a name."
    elif not NAME_MIN <= len(name) <= NAME_MAX:
        errors[field] = f"Name must be {NAME_MIN} to {NAME_MAX} characters."
    elif not _has_letter(name) or any(not (ch.isalpha() or ch in allowed) for ch in name):
        errors[field] = "Name can only use letters, spaces, hyphens and apostrophes" + (
            " (and numbers)." if allow_digits else ".")
    return name


def _check_range(value: Any, low: int, high: int, field: str, label: str, errors: Errors) -> int | None:
    number = _int(value)
    if number is None:
        errors[field] = f"{label} is required." if _text(value) == "" else f"{label} must be a whole
number."
    elif not low <= number <= high:
        errors[field] = f"{label} must be between {low} and {high}."
    return number


def _check_choice(value: Any, options: tuple[str, ...], field: str, label: str, errors: Errors) -> str:
    text = _text(value)
    if text not in options:
        errors[field] = f"Choose a {label}."
    return text


def _check_flag(value: Any, field: str, label: str, errors: Errors) -> bool:
    flag = _yes_no(value)
    if flag is None:
        errors[field] = f"Answer yes or no for {label}."
    return bool(flag)


def validate_person(name: Any, email: Any, age: Any) -> tuple[dict[str, Any], Errors]:
    """Name, email and age: shared by sign-up and by finishing a Google account (criteria 5.1, 5.2)."""
    errors: Errors = {}
    clean_name = _check_name(name, errors)
```

```
    clean_email = _text(email).lower()
    if not clean_email:
        errors["email"] = "Enter an email address."
    elif not EMAIL_PATTERN.match(clean_email):
        errors["email"] = "Enter a valid email address, like name@domain.com."
    clean_age = _check_range(age, MIN_AGE, MAX_AGE, "age", "Age", errors)
    if "age" in errors and clean_age is not None and clean_age < MIN_AGE:
        errors["age"] = f"You must be {MIN_AGE} or over to adopt."
    return {"name": clean_name, "email": clean_email, "age": clean_age}, errors


def validate_registration(
    name: Any, email: Any, age: Any, password: Any, confirm: Any
) -> tuple[dict[str, Any], Errors]:
    """Validate a sign-up form (criteria 5.1 and 5.2)."""
    cleaned, errors = validate_person(name, email, age)
    raw_password = "" if password is None else str(password)
    if not raw_password.strip():
        errors["password"] = "Enter a password."
    elif len(raw_password) < PASSWORD_MIN:
        errors["password"] = f"Password must be at least {PASSWORD_MIN} characters."
    raw_confirm = "" if confirm is None else str(confirm)
    if not raw_confirm:
        # Client review 1: a blank confirm box was not flagged when the password was also blank.
        errors["confirm"] = "Type your password again to confirm it."
    elif raw_password.strip() and raw_password != raw_confirm:
        errors["confirm"] = "The two passwords do not match."
    cleaned["password"] = raw_password
    return cleaned, errors


def validate_login(email: Any, password: Any) -> tuple[dict[str, str], Errors]:
    """Presence checks only; whether the details are right is the auth layer's job."""
    errors: Errors = {}
    clean_email = _text(email).lower()
    raw_password = "" if password is None else str(password)
    if not clean_email:
        errors["email"] = "Enter your email address."
    if not raw_password:
        errors["password"] = "Enter your password."
    return {"email": clean_email, "password": raw_password}, errors


def validate_animal(raw: dict[str, Any]) -> tuple[dict[str, Any], Errors]:
    """Validate an admin's add or edit form (criteria 8.2 and 8.3)."""
    errors: Errors = {}
    cleaned: dict[str, Any] = {"name": _check_name(raw.get("name"), errors, allow_digits=True)}
    cleaned["species"] = _check_choice(raw.get("species"), SPECIES, "species", "species", errors)
    breed = _text(raw.get("breed"))
    if not breed:
        errors["breed"] = "Enter a breed (use 'Mixed' if unknown)."
    elif not NAME_MIN <= len(breed) <= NAME_MAX:
        errors["breed"] = f"Breed must be {NAME_MIN} to {NAME_MAX} characters."
    elif not _has_letter(breed):
        errors["breed"] = "Breed must contain letters."
    cleaned["breed"] = breed
    cleaned["age"] = _check_range(raw.get("age"), 1, ANIMAL_AGE_MAX, "age", "Age", errors)
    cleaned["sex"] = _check_choice(raw.get("sex"), SEXES, "sex", "sex", errors)
    cleaned["size"] = _check_choice(raw.get("size"), SIZES, "size", "size", errors)
    personality = _text(raw.get("personality"))
    if not personality:
        errors["personality"] = "Describe the animal's personality."
    elif len(personality) > TEXT_MAX:
        errors["personality"] = f"Personality must be at most {TEXT_MAX} characters."
    cleaned["personality"] = personality
    cleaned["exercise_needs"] = _check_range(raw.get("exercise_needs"), 1, 3, "exercise_needs", "Exercise
needs", errors)
    cleaned["min_housing_level"] = _check_range(
        raw.get("min_housing_level"), 1, 3, "min_housing_level", "Minimum housing level", errors
    )
    cleaned["garden_required"] = _check_flag(raw.get("garden_required"), "garden_required", "garden
required", errors)
    cleaned["child_compatible"] = _check_flag(raw.get("child_compatible"), "child_compatible", "child
compatible", errors)
    cleaned["max_hours_alone"] = _check_range(raw.get("max_hours_alone"), 0, 24, "max_hours_alone", "Hours
alone", errors)
    cleaned["good_with_pets"] = _check_flag(raw.get("good_with_pets"), "good_with_pets", "good with other
pets", errors)
    cleaned["experience_needed"] = _check_range(
        raw.get("experience_needed"), min(EXPERIENCE_NEEDED), max(EXPERIENCE_NEEDED), "experience_needed",
        "Experience needed", errors,
    )
    notes = _text(raw.get("medical_notes"))
    if len(notes) > TEXT_MAX:
        errors["medical_notes"] = f"Medical notes must be at most {TEXT_MAX} characters."
    cleaned["medical_notes"] = notes
    cleaned["image_path"] = _text(raw.get("image_path"))
    # A new record is Available unless staff choose otherwise (the original form's default).
    cleaned["status"] = _check_choice(raw.get("status") or STATUSES[0], STATUSES, "status", "adoption
status", errors)
    return cleaned, errors
```

```
185
186
187 def _q_housing(v: Any, e: Errors) -> int | None:
188     level = _int(v)
189     if level not in HOUSING_LEVELS:
190         e["housing_level"] = "Choose the type of home you live in."
191     return level
192
193
194 def _q_allergies(v: Any, e: Errors) -> tuple[str, ...]:
195     """Species someone at home is allergic to: any of Dog, Cat, Rabbit, or none (an empty answer)."""
196     if v is None:
197         e["allergies"] = "Choose 'No allergies' or the animals someone is allergic to."
198         return ()
199     chosen = [part.strip() for part in v.split(",")] if isinstance(v, str) else list(v)
200     chosen = [part for part in chosen if part]
201     if any(part not in SPECIES for part in chosen) or len(set(chosen)) != len(chosen):
202         e["allergies"] = "Allergies can only be dogs, cats or rabbits, each chosen once."
203         return ()
204     return tuple(species for species in SPECIES if species in chosen)  # always in the same order
205
206
207 def _q_activity(v: Any, e: Errors) -> int | None:
208     level = _int(v)
209     if level not in ACTIVITY_LEVELS:
210         e["activity_level"] = "Choose how active you can be with an animal."
211     return level
212
213
214 # One validator per question, grouped by questionnaire step (criterion 6.2).
215 QUESTION_VALIDATORS: dict[str, Callable[[Any, Errors], Any]] = {
216     "housing_level": _q_housing,
217     "garden_access": lambda v, e: _check_flag(v, "garden_access", "garden access", e),
218     "has_children": lambda v, e: _check_flag(v, "has_children", "children in the home", e),
219     "other_pets": lambda v, e: _check_flag(v, "other_pets", "other pets", e),
220     "allergies": _q_allergies,
221     "experience": lambda v, e: _check_choice(v, EXPERIENCE_LEVELS, "experience", "level of experience",
e),
222     "activity_level": _q_activity,
223     "hours_alone": lambda v, e: _check_range(v, 0, 24, "hours_alone", "Hours alone", e),
224     "preferred_species": lambda v, e: _check_choice(
225         v, SPECIES + (NO_PREFERENCE,), "preferred_species", "species or 'No preference'", e
226     ),
227     "preferred_size": lambda v, e: _check_choice(
228         v, SIZES + (NO_PREFERENCE,), "preferred_size", "size or 'No preference'", e
229     ),
230 }
231
232 QUESTION_STEPS: tuple[tuple[str, tuple[str, ...]], ...] = (
233     ("Your home", ("housing_level", "garden_access")),
234     ("Your household", ("has_children", "other_pets", "allergies")),
235     ("Your lifestyle", ("activity_level", "hours_alone", "experience")),
236     ("Your preferences", ("preferred_species", "preferred_size")),
237 )
238
239
240 def validate_step(step: int, raw: dict[str, Any]) -> Errors:
241     """Validate only the questions on one step, so users can't move on with gaps."""
242     errors: Errors = {}
243     for name in QUESTION_STEPS[step][1]:
244         QUESTION_VALIDATORS[name](raw.get(name), errors)
245     return errors
246
247
248 def validate_question(name: str, value: Any) -> Errors:
249     """Validate one answer, so the one-question-per-screen form stops at a gap or a bad value."""
250     errors: Errors = {}
251     QUESTION_VALIDATORS[name](value, errors)
252     return errors
253
254
255 def validate_answers(raw: dict[str, Any]) -> tuple[Answers | None, Errors]:
256     """Validate the whole questionnaire; returns Answers only when every answer is valid."""
257     errors: Errors = {}
258     cleaned = {name: check(raw.get(name), errors) for name, check in QUESTION_VALIDATORS.items()}
259     return (None, errors) if errors else (Answers(**cleaned), errors)
260
261
262 IMAGE_TYPES = ("png", "jpg", "jpeg")
263 IMAGE_MAX_BYTES = 2 * 1024 * 1024
264
265
266 def validate_image(filename: str, size_bytes: int) -> str | None:
267     """Return an error message for an unsuitable upload, or None if it is acceptable."""
268     if filename.rsplit(".", 1)[-1].lower() not in IMAGE_TYPES:
269         return "Images must be PNG or JPG."
270     if size_bytes > IMAGE_MAX_BYTES:
271         return "Images must be 2 MB or smaller."
272     return None
```

Code X4: Accounts and password hashing · pawmatch/auth.py, lines 1–109

```
"""Registration, login and role checks. Passwords are stored only as salted scrypt hashes.

The optional "Continue with Google" sign-in (Streamlit's st.login, set up in
.streamlit/secrets.toml) only proves who someone is. What they may do is still decided
here: external_login() maps the verified email to a local adopter account, refuses staff
accounts (staff always use their PawMatch password) and asks a newcomer for their age.
"""

from __future__ import annotations

import hashlib
import hmac
import secrets
import sqlite3
from typing import Any

from . import db
from .models import User
from .validation import EMAIL_PATTERN, Errors, validate_login, validate_person, validate_registration

SCRYPT_N, SCRYPT_R, SCRYPT_P = 2**14, 8, 1
LOGIN_FAILED = "Incorrect email or password."
STAFF_USE_PASSWORD = "Staff accounts sign in with their PawMatch email and password."
# Stored instead of a hash for accounts made through Google: it is not in the scrypt format,
# so verify_password() always refuses it and nobody can log in to them with a password.
EXTERNAL_ONLY = "external$oidc"


class AccessDenied(PermissionError):
    """Raised when a user without the right role calls a protected action."""


def hash_password(password: str) -> str:
    """Return 'scrypt$n$r$p$salt$hash' with a fresh 16-byte salt."""
    salt = secrets.token_bytes(16)
    digest = hashlib.scrypt(password.encode(), salt=salt, n=SCRYPT_N, r=SCRYPT_R, p=SCRYPT_P)
    return f"scrypt${SCRYPT_N}${SCRYPT_R}${SCRYPT_P}${salt.hex()}${digest.hex()}"


def verify_password(password: str, stored: str) -> bool:
    """Recompute the hash with the stored salt and compare in constant time."""
    try:
        scheme, n, r, p, salt_hex, digest_hex = stored.split("$")
    except ValueError:
        return False
    if scheme != "scrypt":
        return False
    digest = hashlib.scrypt(password.encode(), salt=bytes.fromhex(salt_hex), n=int(n), r=int(r), p=int(p))
    return hmac.compare_digest(digest.hex(), digest_hex)


def register(
    conn: sqlite3.Connection, name: Any, email: Any, age: Any, password: Any, confirm: Any
) -> tuple[User | None, Errors]:
    """Validate, check the email is unused, then store the account with a hashed password."""
    cleaned, errors = validate_registration(name, email, age, password, confirm)
    if not errors and db.email_exists(conn, cleaned["email"]):
        errors["email"] = "An account with this email already exists."
    if errors:
        return None, errors
    user = db.insert_user(conn, cleaned["name"], cleaned["email"], cleaned["age"],
hash_password(cleaned["password"]))
    return user, {}


def login(conn: sqlite3.Connection, email: Any, password: Any) -> tuple[User | None, Errors]:
    """Return the user for correct details; one generic message otherwise (no account probing)."""
    cleaned, errors = validate_login(email, password)
    if errors:
        return None, errors
    found = db.find_user_with_hash(conn, cleaned["email"])
    if found is None or not verify_password(cleaned["password"], found[1]):
        return None, {"login": LOGIN_FAILED}
    return found[0], {}


def external_login(conn: sqlite3.Connection, email: Any, email_verified: Any) -> tuple[User | None,
Errors]:
    """Match someone signed in by Google (or Clerk) to their PawMatch account.

    Returns (user, {}) for an adopter account, (None, {}) when there is no account yet (the
    page then asks for their name and age), or (None, errors) when the sign-in can't be used.
    """
    clean = "" if email is None else str(email).strip().lower()
    if email_verified is not True:
        return None, {"email": "That account's email address is not verified, so PawMatch can't use it."}
    if not EMAIL_PATTERN.match(clean):
        return None, {"email": "The sign-in did not share a valid email address."}
    found = db.find_user_with_hash(conn, clean)
```

```
    if found is None:
        return None, {}
    if found[0].is_admin:
        return None, {"login": STAFF_USE_PASSWORD}
    return found[0], {}


def finish_external_account(conn: sqlite3.Connection, name: Any, email: Any, age: Any) -> tuple[User | None, Errors]:
    """Create an adopter ('user' role) account for a Google sign-in, after the same checks as sign-up."""
    cleaned, errors = validate_person(name, email, age)
    if not errors and db.email_exists(conn, cleaned["email"]):
        errors["email"] = "An account with this email already exists."
    if errors:
        return None, errors
    return db.insert_user(conn, cleaned["name"], cleaned["email"], cleaned["age"], EXTERNAL_ONLY), {}


def require_admin(user: User | None) -> User:
    """Gate for every staff action (criterion 8.5)."""
    if user is None or not user.is_admin:
        raise AccessDenied("Only shelter staff can manage animal records.")
    return user
```

Code X5: Matching algorithm · pawmatch/matching.py, lines 1–166

```
"""Weighted compatibility scoring and ranking (criterion 7).

Normalisation, precisely:
  compatibility = round(100 * earned_points / 90, 1)
where 90 is the sum of the seven factor weights below. Each factor earns its
full weight when the requirement is met and 0 when it is not, except activity
and experience, which lose half their weight per level of shortfall.

An allergy is not weighed against the other answers. The client asked that an
animal someone at home is allergic to is never shown as a strong match, so it
adds a warning and caps the score at 39% (the Low band).

Species (10) and size (5) preferences never change the percentage. They are
added only to a separate ranking total, so a preferred but unsuitable animal
cannot show a misleadingly high score.
"""

from __future__ import annotations

from collections.abc import Iterable

from .models import EXPERIENCE_LEVELS, EXPERIENCE_NEEDED, HOUSING_LEVELS, NO_PREFERENCE, Animal, Answers, MatchResult

WEIGHTS: dict[str, int] = {
    "hours_alone": 20,
    "children": 15,
    "housing": 15,
    "activity": 12,
    "garden": 10,
    "other_pets": 10,
    "experience": 8,
}
MAX_POINTS = sum(WEIGHTS.values())  # 90
PREFERENCE_BONUS: dict[str, int] = {"species": 10, "size": 5}

# Unmet requirements on these factors are welfare risks, so they raise a warning.
IMPORTANT_FACTORS = frozenset({"hours_alone", "children", "housing", "other_pets"})
# The highest whole-number score in the Low band (Low is below 40).
ALLERGY_CAP = 39.0

CATEGORY_THRESHOLDS: tuple[tuple[float, str], ...] = (
    (80.0, "Strong"),
    (60.0, "Good"),
    (40.0, "Possible"),
    (0.0, "Low"),
)
MIN_RESULTS_SHOWN = 4


def category_for(compatibility: float) -> str:
    """Map a 0-100 percentage to its label."""
    for threshold, label in CATEGORY_THRESHOLDS:
        if compatibility >= threshold:
            return label
    return "Low"


def _factor_checks(animal: Animal, answers: Answers) -> list[tuple[str, float, str]]:
    """Return (factor, fraction of weight earned, explanation) for each welfare factor."""
    activity_gap = max(0, animal.exercise_needs - answers.activity_level)
    level = EXPERIENCE_LEVELS.index(answers.experience) if answers.experience in EXPERIENCE_LEVELS else 0
    experience_gap = max(0, animal.experience_needed - level)
```

```
    return [
        (
            "hours_alone",
            1.0 if answers.hours_alone <= animal.max_hours_alone else 0.0,
            f"Time alone: you said {answers.hours_alone} h a day; "
            f"{animal.name} can be left for up to {animal.max_hours_alone} h.",
        ),
        (
            "children",
            1.0 if animal.child_compatible or not answers.has_children else 0.0,
            "Children: "
            + ("there are children in your home and " if answers.has_children else "no children in your
home; ")
            + f"{animal.name} is {'' if animal.child_compatible else 'not '}suitable for homes with
children.",
        ),
        (
            "housing",
            1.0 if answers.housing_level >= animal.min_housing_level else 0.0,
            f"Space: you have '{HOUSING_LEVELS[answers.housing_level]}'; "
            f"{animal.name} needs at least '{HOUSING_LEVELS[animal.min_housing_level]}'.",
        ),
        (
            "activity",
            max(0.0, 1.0 - activity_gap / 2),
            f"Activity: you can offer level {answers.activity_level} of 3; "
            f"{animal.name} needs level {animal.exercise_needs} of 3.",
        ),
        (
            "garden",
            1.0 if answers.garden_access or not animal.garden_required else 0.0,
            f"Garden: {animal.name} {'needs' if animal.garden_required else 'does not need'} a garden; "
            f"you {'have' if answers.garden_access else 'do not have'} one.",
        ),
        (
            "other_pets",
            1.0 if animal.good_with_pets or not answers.other_pets else 0.0,
            "Other pets: "
            + ("another pet lives with you and " if answers.other_pets else "no other pets live with you;
")
            + f"{animal.name} {'can' if animal.good_with_pets else 'cannot'} share a home with other
animals.",
        ),
        (
            "experience",
            max(0.0, 1.0 - experience_gap / 2),
            f"Experience: you said '{answers.experience}'; "
            f"{animal.name} {EXPERIENCE_NEEDED[animal.experience_needed].lower()}.",
        ),
    ]


def preference_bonus(animal: Animal, answers: Answers) -> int:
    """Ranking-only bonus for the preferred species and size ('No preference' scores 0)."""
    bonus = 0
    if answers.preferred_species not in (NO_PREFERENCE, "") and animal.species ==
answers.preferred_species:
        bonus += PREFERENCE_BONUS["species"]
    if answers.preferred_size not in (NO_PREFERENCE, "") and animal.size == answers.preferred_size:
        bonus += PREFERENCE_BONUS["size"]
    return bonus


def score_animal(animal: Animal, answers: Answers) -> MatchResult:
    """Score one animal: percentage, category, met/unmet lists and warnings."""
    earned = 0.0
    met: list[str] = []
    unmet: list[str] = []
    warnings: list[str] = []
    for factor, fraction, reason in _factor_checks(animal, answers):
        earned += WEIGHTS[factor] * fraction
        if fraction == 1.0:
            met.append(reason)
        else:
            unmet.append(reason)
            if factor in IMPORTANT_FACTORS:
                warnings.append(reason)
    compatibility = round(100 * earned / MAX_POINTS, 1)
    if animal.species in answers.allergies:
        reason = (f"Allergy: someone in your home is allergic to {animal.species.lower()}s; "
                  f"{animal.name} is a {animal.species.lower()}.")
        unmet.insert(0, reason)
        warnings.insert(0, reason)
        compatibility = min(compatibility, ALLERGY_CAP)
    return MatchResult(
        animal=animal,
        compatibility=compatibility,
        category=category_for(compatibility),
        bonus=preference_bonus(animal, answers),
        met=met,
        unmet=unmet,
```

```
        warnings=warnings,
    )


def rank_key(result: MatchResult) -> tuple[int, float, float, str, int]:
    """Suitable animals first, then ranking points, compatibility, name and id (all deterministic)."""
    return (
        0 if result.suitable else 1,
        -result.ranking_points,
        -result.compatibility,
        result.animal.name.lower(),
        result.animal.animal_id,
    )


def rank_animals(animals: Iterable[Animal], answers: Answers) -> list[MatchResult]:
    """Score every animal and return them best first."""
    return sorted((score_animal(a, answers) for a in animals), key=rank_key)
```

Code X6: Staff-only actions · pawmatch/services.py, lines 1–102

```
"""Use cases the pages call. Each one validates, checks permissions, then uses the repository."""

from __future__ import annotations

import sqlite3
from typing import Any

from . import db
from .auth import require_admin
from .matching import rank_animals
from .models import STATUSES, Animal, MatchResult, ShelterStats, User
from .validation import Errors, validate_animal, validate_answers

AVAILABLE = STATUSES[0]
# A profile needs a photo and a description of at least this many characters before staff
# count it as complete (the original admin page's "needs attention" rule).
PROFILE_TEXT_MIN = 80
PROFILE_FIELDS = ("name", "species", "breed", "age", "sex", "size", "personality", "exercise_needs",
                  "min_housing_level", "garden_required", "child_compatible", "max_hours_alone",
                  "good_with_pets", "experience_needed", "medical_notes", "image_path", "status")


def add_animal(conn: sqlite3.Connection, actor: User | None, raw: dict[str, Any]) -> tuple[int | None,
Errors]:
    require_admin(actor)
    values, errors = validate_animal(raw)
    if errors:
        return None, errors
    return db.insert_animal(conn, values), {}


def edit_animal(conn: sqlite3.Connection, actor: User | None, animal_id: int, raw: dict[str, Any]) ->
Errors:
    require_admin(actor)
    values, errors = validate_animal(raw)
    if errors:
        return errors
    if not db.update_animal(conn, animal_id, values):
        return {"animal": "That animal no longer exists."}
    return {}


def remove_animal(conn: sqlite3.Connection, actor: User | None, animal_id: int) -> bool:
    require_admin(actor)
    return db.delete_animal(conn, animal_id)


def submit_questionnaire(
    conn: sqlite3.Connection, user: User, raw: dict[str, Any]
) -> tuple[list[MatchResult], Errors]:
    """Validate the answers, store them, score every animal and store the ranking."""
    answers, errors = validate_answers(raw)
    if answers is None:
        return [], errors
    response_id = db.save_response(conn, user.user_id, answers)
    results = rank_animals(db.list_animals(conn, status=AVAILABLE), answers)  # reserved or adopted: not
scored
    db.save_results(conn, response_id, results)
    return results, {}


def latest_results(conn: sqlite3.Connection, user: User) -> list[MatchResult] | None:
    """The user's most recent stored ranking, or None if they have not taken the questionnaire."""
    latest = db.latest_response(conn, user.user_id)
    return None if latest is None else db.load_results(conn, latest[0])


def browse(conn: sqlite3.Connection, species: str | None, size: str | None, search: str) -> list[Animal]:
```

```
    """What adopters see: available animals only (criterion 3)."""
    return db.list_animals(conn, species=species, size=size, search=search, status=AVAILABLE)


def staff_animals(conn: sqlite3.Connection, actor: User | None, species: str | None = None,
                  status: str | None = None, search: str = "") -> list[Animal]:
    """Every record, whatever its status, for the staff dashboard."""
    require_admin(actor)
    return db.list_animals(conn, species=species, search=search, status=status)


def result_for_animal(conn: sqlite3.Connection, user: User, animal_id: int) -> MatchResult | None:
    """The user's latest score for one animal, for the profile page; None before the questionnaire."""
    return next((r for r in latest_results(conn, user) or [] if r.animal.animal_id == animal_id), None)


def dashboard(conn: sqlite3.Connection, actor: User | None) -> ShelterStats:
    """Staff-only counts for the dashboard."""
    require_admin(actor)
    return db.shelter_stats(conn)


def profile_issues(animal: Animal) -> list[str]:
    """What staff should add before a profile is complete: a photo, and a description adopters can use."""
    issues = []
    if not animal.image_path.strip():
        issues.append("photo")
    if len(animal.personality.strip()) < PROFILE_TEXT_MIN:
        issues.append("description")
    return issues


def profile_completion(animal: Animal) -> int:
    """The percentage of the record's fields that are filled in (a yes/no answer always counts)."""
    filled = sum(1 for name in PROFILE_FIELDS
                 if str(getattr(animal, name)).strip() or isinstance(getattr(animal, name), bool))
    return round(100 * filled / len(PROFILE_FIELDS))
```

Code X7: Database access · pawmatch/db.py, lines 1–345

```
"""SQLite repository. Every query is parameterised; no SQL is built from user input."""

from __future__ import annotations

import json
import secrets
import sqlite3
from collections.abc import Iterator
from contextlib import contextmanager
from datetime import datetime, timedelta, timezone
from pathlib import Path
from typing import Any

from .models import SPECIES, STATUSES, Animal, Answers, MatchResult, ShelterStats, User

SCHEMA = """
CREATE TABLE IF NOT EXISTS users (
    user_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    name          TEXT    NOT NULL,
    email         TEXT    NOT NULL UNIQUE COLLATE NOCASE,
    age           INTEGER NOT NULL CHECK (age >= 18),
    password_hash TEXT    NOT NULL,
    role          TEXT    NOT NULL DEFAULT 'user' CHECK (role IN ('user', 'admin')),
    created_at    TEXT    NOT NULL
);
CREATE TABLE IF NOT EXISTS animals (
    animal_id         INTEGER PRIMARY KEY AUTOINCREMENT,
    name              TEXT    NOT NULL,
    species           TEXT    NOT NULL CHECK (species IN ('Dog', 'Cat', 'Rabbit')),
    breed             TEXT    NOT NULL,
    age               INTEGER NOT NULL CHECK (age BETWEEN 1 AND 30),
    sex               TEXT    NOT NULL CHECK (sex IN ('Male', 'Female', 'Unknown')),
    size              TEXT    NOT NULL CHECK (size IN ('Small', 'Medium', 'Large')),
    personality       TEXT    NOT NULL,
    exercise_needs    INTEGER NOT NULL CHECK (exercise_needs BETWEEN 1 AND 3),
    min_housing_level INTEGER NOT NULL CHECK (min_housing_level BETWEEN 1 AND 3),
    garden_required   INTEGER NOT NULL CHECK (garden_required IN (0, 1)),
    child_compatible  INTEGER NOT NULL CHECK (child_compatible IN (0, 1)),
    max_hours_alone   INTEGER NOT NULL CHECK (max_hours_alone BETWEEN 0 AND 24),
    good_with_pets    INTEGER NOT NULL DEFAULT 1 CHECK (good_with_pets IN (0, 1)),
    experience_needed INTEGER NOT NULL DEFAULT 0 CHECK (experience_needed BETWEEN 0 AND 2),
    medical_notes     TEXT    NOT NULL DEFAULT '',
    image_path        TEXT    NOT NULL DEFAULT '',
    status            TEXT    NOT NULL DEFAULT 'Available' CHECK (status IN ('Available', 'Reserved',
'Adopted')),
    date_added        TEXT    NOT NULL
);
CREATE TABLE IF NOT EXISTS questionnaire_responses (
    response_id       INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    user_id           INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    housing_level     INTEGER NOT NULL CHECK (housing_level BETWEEN 1 AND 3),
    garden_access     INTEGER NOT NULL CHECK (garden_access IN (0, 1)),
    has_children      INTEGER NOT NULL CHECK (has_children IN (0, 1)),
    other_pets        INTEGER NOT NULL CHECK (other_pets IN (0, 1)),
    experience        TEXT    NOT NULL,
    activity_level    INTEGER NOT NULL CHECK (activity_level BETWEEN 1 AND 3),
    hours_alone       INTEGER NOT NULL CHECK (hours_alone BETWEEN 0 AND 24),
    preferred_species TEXT    NOT NULL,
    preferred_size    TEXT    NOT NULL,
    allergies         TEXT    NOT NULL DEFAULT '',
    completed_at      TEXT    NOT NULL
);
CREATE TABLE IF NOT EXISTS match_results (
    match_id      INTEGER PRIMARY KEY AUTOINCREMENT,
    response_id   INTEGER NOT NULL REFERENCES questionnaire_responses(response_id) ON DELETE CASCADE,
    animal_id     INTEGER NOT NULL REFERENCES animals(animal_id) ON DELETE CASCADE,
    rank          INTEGER NOT NULL,
    compatibility REAL    NOT NULL CHECK (compatibility BETWEEN 0 AND 100),
    category      TEXT    NOT NULL CHECK (category IN ('Strong', 'Good', 'Possible', 'Low')),
    bonus         INTEGER NOT NULL,
    met           TEXT    NOT NULL,
    unmet         TEXT    NOT NULL,
    warnings      TEXT    NOT NULL,
    calculated_at TEXT    NOT NULL DEFAULT ''
);
CREATE TABLE IF NOT EXISTS sessions (
    token      TEXT    PRIMARY KEY,
    user_id    INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    method     TEXT    NOT NULL DEFAULT 'password',
    expires_at TEXT    NOT NULL
);
"""

ANIMAL_FIELDS = (
    "name", "species", "breed", "age", "sex", "size", "personality", "exercise_needs",
    "min_housing_level", "garden_required", "child_compatible", "max_hours_alone",
    "good_with_pets", "experience_needed", "medical_notes", "image_path", "status",
)
# Stage 9 columns, added in place to databases made before them (Design, database changes).
STAGE9_COLUMNS = (
    ("animals", "good_with_pets", "INTEGER NOT NULL DEFAULT 1 CHECK (good_with_pets IN (0, 1))"),
    ("animals", "experience_needed", "INTEGER NOT NULL DEFAULT 0 CHECK (experience_needed BETWEEN 0 AND
2)"),
    ("questionnaire_responses", "allergies", "TEXT NOT NULL DEFAULT ''"),
)


def _now() -> str:
    return datetime.now(timezone.utc).isoformat(timespec="seconds")


@contextmanager
def connect(path: str | Path) -> Iterator[sqlite3.Connection]:
    """Open the database, commit on success, roll back on error, always close."""
    conn = sqlite3.connect(path)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()


def init_schema(conn: sqlite3.Connection) -> set[str]:
    """Create any missing tables and add any missing columns; returns the stage 9 columns it added."""
    conn.executescript(SCHEMA)
    # Databases made before stage 5 lack calculated_at (Design, Match Results table): add it in place.
    columns = {row[1] for row in conn.execute("PRAGMA table_info(match_results)")}
    if "calculated_at" not in columns:
        conn.execute("ALTER TABLE match_results ADD COLUMN calculated_at TEXT NOT NULL DEFAULT ''")
    # Databases made before round 2 lack the adoption status: every existing animal starts Available.
    if "status" not in {row[1] for row in conn.execute("PRAGMA table_info(animals)")}:
        conn.execute("ALTER TABLE animals ADD COLUMN status TEXT NOT NULL DEFAULT 'Available' "
                     "CHECK (status IN ('Available', 'Reserved', 'Adopted'))")
    added = set()
    for table, column, definition in STAGE9_COLUMNS:  # fixed names, never user input
        if column not in {row[1] for row in conn.execute(f"PRAGMA table_info({table})")}:
            conn.execute(f"ALTER TABLE {table} ADD COLUMN {column} {definition}")
            added.add(column)
    return added


# ---- sessions (a reload keeps the user signed in) -------------------------

SESSION_HOURS = 12

```

```
139
140 def create_session(conn: sqlite3.Connection, user_id: int, method: str = "password") -> str:
141     """A random token for this browser tab; only the token travels, never the user id."""
142     token = secrets.token_urlsafe(32)
143     expires = (datetime.now(timezone.utc) + timedelta(hours=SESSION_HOURS)).isoformat(timespec="seconds")
144     conn.execute("DELETE FROM sessions WHERE expires_at < ?", (_now(),))
145     conn.execute("INSERT INTO sessions (token, user_id, method, expires_at) VALUES (?, ?, ?, ?)",
146                  (token, user_id, method, expires))
147     return token
148
149
150 def session_user(conn: sqlite3.Connection, token: str) -> tuple[User, str] | None:
151     """The user and sign-in method for an unexpired token, or None."""
152     row = conn.execute(
153         "SELECT u.*, s.method FROM sessions s JOIN users u ON u.user_id = s.user_id "
154         "WHERE s.token = ? AND s.expires_at >= ?", (token, _now())).fetchone()
155     return (_user(row), row["method"]) if row else None
156
157
158 def end_session(conn: sqlite3.Connection, token: str) -> None:
159     conn.execute("DELETE FROM sessions WHERE token = ?", (token,))
160
161
162 # ---- users -----------------------------------------------------------
163
164 def _user(row: sqlite3.Row) -> User:
165     return User(row["user_id"], row["name"], row["email"], row["age"], row["role"])
166
167
168 def email_exists(conn: sqlite3.Connection, email: str) -> bool:
169     return conn.execute("SELECT 1 FROM users WHERE email = ?", (email,)).fetchone() is not None
170
171
172 def insert_user(conn: sqlite3.Connection, name: str, email: str, age: int, password_hash: str, role: str =
"user") -> User:
173     cur = conn.execute(
174         "INSERT INTO users (name, email, age, password_hash, role, created_at) VALUES (?, ?, ?, ?, ?, ?)",
175         (name, email, age, password_hash, role, _now()),
176     )
177     return User(int(cur.lastrowid), name, email, age, role)
178
179
180 def find_user_with_hash(conn: sqlite3.Connection, email: str) -> tuple[User, str] | None:
181     row = conn.execute("SELECT * FROM users WHERE email = ?", (email,)).fetchone()
182     return (_user(row), row["password_hash"]) if row else None
183
184
185 def get_user(conn: sqlite3.Connection, user_id: int) -> User | None:
186     row = conn.execute("SELECT * FROM users WHERE user_id = ?", (user_id,)).fetchone()
187     return _user(row) if row else None
188
189
190 def set_role(conn: sqlite3.Connection, user_id: int, role: str) -> None:
191     conn.execute("UPDATE users SET role = ? WHERE user_id = ?", (role, user_id))
192
193
194 # ---- animals ---------------------------------------------------------
195
196 def _animal(row: sqlite3.Row) -> Animal:
197     data = {k: row[k] for k in ("animal_id",) + ANIMAL_FIELDS + ("date_added",)}
198     data["garden_required"] = bool(data["garden_required"])
199     data["child_compatible"] = bool(data["child_compatible"])
200     data["good_with_pets"] = bool(data["good_with_pets"])
201     return Animal(**data)
202
203
204 def insert_animal(conn: sqlite3.Connection, values: dict[str, Any]) -> int:
205     columns = ", ".join(ANIMAL_FIELDS)  # fixed column names, never user input
206     marks = ", ".join("?" for _ in ANIMAL_FIELDS)
207     cur = conn.execute(
208         f"INSERT INTO animals ({columns}, date_added) VALUES ({marks}, ?)",
209         tuple(values[f] for f in ANIMAL_FIELDS) + (_now(),),
210     )
211     return int(cur.lastrowid)
212
213
214 def update_animal(conn: sqlite3.Connection, animal_id: int, values: dict[str, Any]) -> bool:
215     assignments = ", ".join(f"{f} = ?" for f in ANIMAL_FIELDS)
216     cur = conn.execute(
217         f"UPDATE animals SET {assignments} WHERE animal_id = ?",
218         tuple(values[f] for f in ANIMAL_FIELDS) + (animal_id,),
219     )
220     return cur.rowcount == 1
221
222
223 def delete_animal(conn: sqlite3.Connection, animal_id: int) -> bool:
224     return conn.execute("DELETE FROM animals WHERE animal_id = ?", (animal_id,)).rowcount == 1
225
226
227 def get_animal(conn: sqlite3.Connection, animal_id: int) -> Animal | None:
228     row = conn.execute("SELECT * FROM animals WHERE animal_id = ?", (animal_id,)).fetchone()
```

```
    return _animal(row) if row else None


def list_animals(
    conn: sqlite3.Connection, species: str | None = None, size: str | None = None, search: str = "",
    status: str | None = None,
) -> list[Animal]:
    """Browse query with optional filters; the search term is bound, never concatenated."""
    sql = "SELECT * FROM animals WHERE 1 = 1"
    params: list[Any] = []
    if status:
        sql += " AND status = ?"
        params.append(status)
    if species:
        sql += " AND species = ?"
        params.append(species)
    if size:
        sql += " AND size = ?"
        params.append(size)
    if search.strip():
        sql += " AND (name LIKE ? OR breed LIKE ?)"
        term = f"%{search.strip()}%"
        params += [term, term]
    sql += " ORDER BY species, name"
    return [_animal(r) for r in conn.execute(sql, params)]


def count_animals(conn: sqlite3.Connection) -> int:
    return conn.execute("SELECT COUNT(*) FROM animals").fetchone()[0]


def set_pets_and_experience(conn: sqlite3.Connection, values_by_name: dict[str, tuple[int, int]]) -> int:
    """Give named animals their 'good with pets' and 'experience needed' values; returns rows changed."""
    return sum(
        conn.execute("UPDATE animals SET good_with_pets = ?, experience_needed = ? WHERE name = ?",
                     (pets, experience, name)).rowcount
        for name, (pets, experience) in values_by_name.items()
    )


def use_png_pictures(conn: sqlite3.Connection) -> int:
    """Point round-1 demo animals (.jpg on black) at the transparent .png cut-outs; returns rows
changed."""
    return conn.execute(
        """UPDATE animals SET image_path = substr(image_path, 1, length(image_path) - 4) || '.png'
           WHERE image_path LIKE 'assets/animals/%.jpg'"""
    ).rowcount


def shelter_stats(conn: sqlite3.Connection) -> ShelterStats:
    """Dashboard counts in five aggregate queries; a species or status with no animals still shows as
0."""
    by_species = dict.fromkeys(SPECIES, 0)
    for species, count in conn.execute("SELECT species, COUNT(*) FROM animals GROUP BY species"):
        by_species[species] = count
    roles = dict(conn.execute("SELECT role, COUNT(*) FROM users GROUP BY role").fetchall())
    questionnaires = conn.execute("SELECT COUNT(*) FROM questionnaire_responses").fetchone()[0]
    by_status = dict.fromkeys(STATUSES, 0)
    for status, count in conn.execute("SELECT status, COUNT(*) FROM animals GROUP BY status"):
        by_status[status] = count
    return ShelterStats(by_species, roles.get("user", 0), roles.get("admin", 0), questionnaires,
by_status)


# ---- questionnaire and results -----------------------------------------

def save_response(conn: sqlite3.Connection, user_id: int, answers: Answers) -> int:
    cur = conn.execute(
        """INSERT INTO questionnaire_responses (user_id, housing_level, garden_access, has_children,
               other_pets, experience, activity_level, hours_alone, preferred_species, preferred_size,
               allergies, completed_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        (user_id, answers.housing_level, int(answers.garden_access), int(answers.has_children),
         int(answers.other_pets), answers.experience, answers.activity_level, answers.hours_alone,
         answers.preferred_species, answers.preferred_size, ",".join(answers.allergies), _now()),
    )
    return int(cur.lastrowid)


def latest_response(conn: sqlite3.Connection, user_id: int) -> tuple[int, Answers] | None:
    row = conn.execute(
        "SELECT * FROM questionnaire_responses WHERE user_id = ? ORDER BY response_id DESC LIMIT 1",
        (user_id,),
    ).fetchone()
    if row is None:
        return None
    return row["response_id"], Answers(
        housing_level=row["housing_level"], garden_access=bool(row["garden_access"]),
        has_children=bool(row["has_children"]), other_pets=bool(row["other_pets"]),
        experience=row["experience"], activity_level=row["activity_level"],
hours_alone=row["hours_alone"],
        preferred_species=row["preferred_species"], preferred_size=row["preferred_size"],
```

```
        allergies=tuple(a for a in row["allergies"].split(",") if a),
    )


def save_results(conn: sqlite3.Connection, response_id: int, results: list[MatchResult]) -> None:
    conn.execute("DELETE FROM match_results WHERE response_id = ?", (response_id,))
    calculated_at = _now()  # one timestamp for the whole ranking
    conn.executemany(
        """INSERT INTO match_results (response_id, animal_id, rank, compatibility, category, bonus,
               met, unmet, warnings, calculated_at) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)""",
        [(response_id, r.animal.animal_id, rank, r.compatibility, r.category, r.bonus,
          json.dumps(r.met), json.dumps(r.unmet), json.dumps(r.warnings), calculated_at)
         for rank, r in enumerate(results, start=1)],
    )


def load_results(conn: sqlite3.Connection, response_id: int) -> list[MatchResult]:
    """Stored results joined to the current animal records, in saved rank order."""
    rows = conn.execute(
        """SELECT m.compatibility, m.category, m.bonus, m.met, m.unmet, m.warnings, a.*
           FROM match_results m JOIN animals a ON a.animal_id = m.animal_id
           WHERE m.response_id = ? ORDER BY m.rank""",
        (response_id,),
    ).fetchall()
    return [
        MatchResult(animal=_animal(r), compatibility=r["compatibility"], category=r["category"],
                    bonus=r["bonus"], met=json.loads(r["met"]), unmet=json.loads(r["unmet"]),
                    warnings=json.loads(r["warnings"]))
        for r in rows
    ]
```